LINE: Queueing Analysis Algorithms

User manual for MATLAB

Last revision: July 24, 2026

Contents

1 Introduction **8**
- 1.1 What is LINE? . . . 8
- 1.2 Obtaining the latest release . . . 10
- 1.3 References . . . 10
- 1.4 Contact and credits . . . 11
- 1.5 Copyright and license . . . 11
- 1.6 Acknowledgement . . . 11

2 Getting started **12**
- 2.1 Installation and support . . . 12
  - 2.1.1 Software requirements . . . 12
  - 2.1.2 Documentation . . . 14
  - 2.1.3 Getting help . . . 14
- 2.2 Getting started examples . . . 14
  - 2.2.1 Controlling verbosity . . . 14
  - 2.2.2 Model gallery . . . 15
  - 2.2.3 Tutorials . . . 16
  - 2.2.4 The utilization convention . . . 16

3 Network models **18**
- 3.1 Network object definition . . . 19
  - 3.1.1 Creating a network and its nodes . . . 19
  - 3.1.2 Advanced node parameters . . . 28
  - 3.1.3 Job classes . . . 30
  - 3.1.4 Routing strategies . . . 34
  - 3.1.5 Class switching . . . 39
  - 3.1.6 Service and inter-arrival time processes . . . 41
  - 3.1.7 Finite capacity regions . . . 56
  - 3.1.8 Places and transitions . . . 57

3.1.9 Signals . . . 59
3.2 Internals . . . 60
3.2.1 Representation of the model structure . . . 60
3.2.2 Nodes, stations, and stateful nodes . . . 60
3.3 Debugging and visualization . . . 62

4 Analysis methods **65**
4.1 Performance metrics . . . 65
4.2 Steady-state analysis . . . 65
4.2.1 Station average performance . . . 65
4.2.2 Station response time distribution . . . 68
4.2.3 Station queue-length and response-time moments . . . 70
4.2.4 Loss rates and loss ratios . . . 71
4.2.5 System average performance . . . 72
4.3 Specifying states . . . 73
4.3.1 Station states . . . 73
4.3.2 Network states . . . 76
4.3.3 Initialization of transient classes . . . 76
4.3.4 State space generation . . . 77
4.3.5 State probability analysis . . . 77
4.4 Transient analysis . . . 78
4.4.1 Computing transient averages . . . 78
4.4.2 First passage times into stations . . . 79
4.5 Advanced metrics . . . 80
4.5.1 Fork-join metrics . . . 80
4.5.2 Finite capacity region queue length . . . 80
4.5.3 Age of Information analysis . . . 80
4.5.4 Reward models . . . 81
4.6 Sample path analysis . . . 83
4.7 Sensitivity analysis and numerical optimization . . . 85
4.7.1 Performance sensitivities . . . 85
4.7.2 Symbolic analysis backend . . . 86
4.7.3 Fast parameter update . . . 88
4.7.4 Direct modification of NetworkStruct objects . . . 89
4.7.5 Refreshing a network topology with non-probabilistic routing . . . 90
4.7.6 Saving a network object before a change . . . 90
4.8 Model adaptation . . . 91
4.8.1 Chain aggregation . . . 91
4.8.2 Flow-equivalent server aggregation . . . 91
4.8.3 Convolution-based FES analysis . . . 91

4.8.4 Tagged job models . . . 92
4.8.5 Class removal . . . 92

5 Network solvers **93**
5.1 Overview . . . 93
5.2 Solvers and solution methods . . . 95
5.2.1 AUTO . . . 101
5.2.2 BA . . . 102
5.2.3 CTMC . . . 103
5.2.4 FLD . . . 106
5.2.5 JMT . . . 108
5.2.6 MAM . . . 108
5.2.7 MVA . . . 111
5.2.8 NC . . . 114
5.2.9 QNS . . . 118
5.2.10 SSA . . . 119
5.2.11 LDES . . . 119
5.2.12 UQ . . . 123
5.3 Supported language features and options . . . 124
5.3.1 Solver features . . . 124
5.3.2 State-dependent routing and service . . . 124
5.3.3 Class functions . . . 125
5.3.4 Node types . . . 128
5.3.5 Scheduling strategies . . . 128
5.3.6 Routing strategies . . . 130
5.3.7 Statistical distributions . . . 130
5.3.8 Solver options . . . 130
5.4 Solver maintenance . . . 135

6 Layered network models **137**
6.1 Basics about layered networks . . . 137
6.2 LayeredNetwork object definition . . . 139
6.2.1 Creating a layered network topology . . . 139
6.2.2 FunctionTask . . . 139
6.2.3 Cache tasks . . . 140
6.2.4 Replication and fan-out . . . 141
6.2.5 Describing host demands of entries . . . 142
6.2.6 Activity configuration . . . 145
6.2.7 Debugging and visualization . . . 145
6.3 Internals . . . 147

6.3.1 Representation of the model structure . . . 147
6.3.2 Decomposition into layers . . . 149
6.4 Solvers . . . 150
6.4.1 LQNS . . . 151
6.4.2 LN . . . 152
6.5 Conversion between Network and LayeredNetwork models . . . 156
6.5.1 From Network to LayeredNetwork . . . 156
6.5.2 From LayeredNetwork to Network . . . 156
6.6 Model import and export . . . 157
6.6.1 Ensemble merging . . . 157

7 Random environments **158**
7.1 Environment definition . . . 158
7.1.1 Specifying the environment . . . 158
7.1.2 Specifying system models for each stage . . . 159
7.1.3 Specifying a reset policy . . . 159
7.2 Solvers . . . 160
7.2.1 ENV . . . 160
7.3 Examples . . . 162
7.3.1 Example 1: Fast and slow service . . . 162
7.3.2 Example 2: Breakdown and repair . . . 165
7.3.3 Example 3: Markov-modulated service . . . 167
7.3.4 Example 4: semi-Markov process environments . . . 168
7.3.5 Example 5: MMAP-modulated cache workload . . . 169

8 Parameter Inference **172**
8.1 Quick start . . . 172
8.2 Specifying measurement data . . . 174
8.2.1 Periodic samples and trace data . . . 175
8.2.2 Conditional metrics . . . 176
8.2.3 Configuring and running the estimator . . . 176
8.3 Estimation methods . . . 176
8.4 Inference examples . . . 177
8.5 Layered network parameter identification . . . 178

9 Optimization **179**
9.1 Overview . . . 179
9.2 Requirements . . . 179
9.3 Examples . . . 180
9.3.1 Tutorial 1: Sizing an M/M/c queue . . . 180

9.3.2 Tutorial 2: Exact sizing by bisection . . . 181
9.3.3 Tutorial 3: Continuous service rate tuning . . . 181
9.3.4 Tutorial 4: Load balancing across heterogeneous servers . . . 182
9.3.5 Tutorial 5: Population sizing in a closed network . . . 182
9.3.6 Tutorial 6: Robust sizing under workload scenarios . . . 183
9.3.7 Tutorial 7: Cost-performance tradeoff frontier . . . 184
9.3.8 Tutorial 8: Decomposing a multi-variable problem . . . 184
9.4 Decision variables . . . 185
9.5 Objectives and constraints . . . 186
9.6 Solvers . . . 186
9.6.1 Differential evolution . . . 186
9.6.2 Analytic gradients for product-form networks . . . 187
9.6.3 Exact sizing by bisection . . . 187
9.6.4 Workload scenarios . . . 188
9.6.5 Cost-performance tradeoffs . . . 188
9.7 Decomposition workflows . . . 188
9.8 Results and metrics . . . 189
9.9 Current limitations . . . 189

10 Interoperability **190**
10.1 Model import and export . . . 190
10.1.1 Creating a LINE model using JMT . . . 191
10.1.2 Supported JMT features . . . 191
10.2 JSON model format . . . 192
10.2.1 API . . . 193
10.2.2 Top-level structure . . . 193
10.2.3 Network models . . . 193
10.2.4 Distributions . . . 203
10.2.5 LayeredNetwork models . . . 206
10.2.6 Workflow models . . . 209
10.2.7 Environment models . . . 210
10.2.8 Examples . . . 210
10.2.9 State not carried by the format . . . 213
10.2.10 Serialization conventions . . . 213
10.3 Command-line interface (line-cli) . . . 214
10.3.1 Installation . . . 214
10.3.2 Basic Usage . . . 215
10.3.3 Server Modes . . . 216
10.3.4 Advanced Options . . . 217
10.3.5 Command Reference . . . 218

10.3.6 Examples 218

A Examples **227**

Chapter 1

Introduction

1.1 What is LINE?

LINE is an open-source software package to analyze queueing models via analytical methods and simulation. The tool aims at simplifying the computation of performance and reliability metrics in models of systems such as software applications, business processes, computer networks, and others. LINE decomposes a high-level system model into one or more stochastic models, typically extended queueing networks, that are subsequently analyzed using either numerical algorithms or simulation. The tool covered in this manual is implemented in the MATLAB language (https://sf.net/p/line-solver/code/ci/master/tree/matlab), with selected methods written in Java and C/C++.

A key feature of LINE is that it decouples the model description from the solvers used for its solution. That is, LINE implements model-to-model transformations that automatically translate the model specification into the input format (or data structure) accepted by its solutions algorithms. LINE sends models for evaluation either to native algorithms of external solvers that include Java Modelling Tools (JMT; http://jmt.sf.net), BuTools (http://webspn.hit.bme.hu/~telek/tools/butools/), Q-MAM and SMCSolver (https://win.uantwerpen.be/~vanhoudt/), and LQNS (http://www.sce.carleton.ca/rads/lqns/). Native model solvers are instead based on formalisms and algorithms based on:

- Continuous-time Markov chains (`CTMC`)
- Discrete-event simulation (`LDES`)
- Fluid/mean-field ordinary differential equations (`FLD`)
- Matrix analytic methods (`MAM`)
- Normalizing constant analysis (`NC`)
- Mean-value analysis (`MVA`)

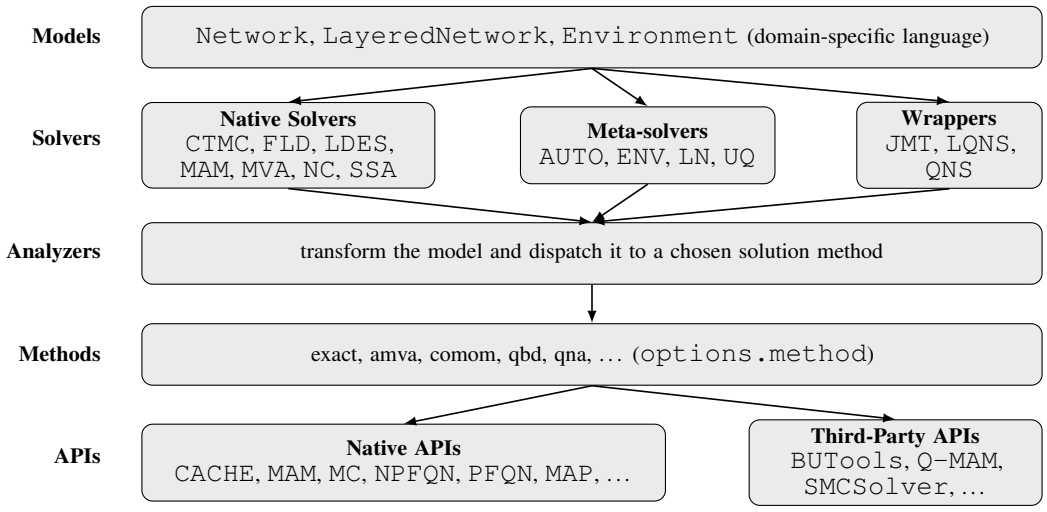

Figure 1.1: Layered internal architecture of LINE. A model is evaluated by a native solver, by a meta-solver that coordinates other solvers, or handed to an external tool through a solver wrapper; each solver runs an analyzer that selects a solution method, and methods are implemented on top of shared numerical APIs.

- Stochastic Simulation Algorithms (`SSA`)

Each solver encodes a general solution paradigm and can implement both exact and approximate analysis methods. For example, the `MVA` solver implements both exact mean value analysis (MVA) and approximate mean value analysis (AMVA). The offered methods typically differ for accuracy, computational cost, and the subset of model features they support. A special solver, called `AUTO`, is supplied that provides an automated selection of the solver to use for a given model.

LINE also includes two meta-solvers that can leverage other solvers to analyze composite models such as layered queueing networks or queueing networks in random environments. The native meta-solvers include:

- Random environment solver (`ENV`)
- Layered network meta-solver (`LN`).

Figure 1.1 summarizes how these components are layered internally. A high-level system model is first handled by a meta-solver, when composite, and then dispatched to one of the network solvers. Each solver exposes an analyzer that selects a concrete solution method, which in turn is built on top of a set of reusable numerical APIs.

LINE can be applied to models specified in the following formats:

- LINE *modeling language*. This is a domain-specific object-oriented language designed to resemble the abstractions available in JMT's queueing network simulator (JSIM). This requires the model direct coding of the system model.

- *Layered queueing network models (LQNS XML format).* LINE is able to solve a sub-class of layered queueing network models, either specified using the LINE modeling language or according to the XML metamodel of the LQNS solver.
- *JMT simulation models (JSIMg, JSIMw formats).* LINE is able to import and solve queueing network models specified using JSIMgraph and JSIMwiz. LINE models can be exported to, and visualized with, JSIMgraph and JSIMwiz.
- *JSON model serialization.* LINE also accepts models supplied in a portable JSON format that conforms to the `line-model.schema.json` specification, enabling model interchange across codebases and with external tools, as described in Section 10.2.

1.2 Obtaining the latest release

This document contains the user manual for the latest version of LINE, which can be obtained from:

http://line-solver.sf.net/

LINE 3.0.x has been tested using MATLAB R2025a and later releases and requires the *Statistics and Machine Learning Toolbox*. Some advanced features also require the *Parallel Computing Toolbox*, the *Global Optimization Toolbox*, the *Optimization Toolbox*, and the *Symbolic Math Toolbox*.

Alternatively, a development version of the codebase can be cloned from Github at https://github.com/imperial-qore/line-solver.

1.3 References

To cite the LINE solver, we recommend to reference:

- G. Casale. "Integrated Performance Evaluation of Extended Queueing Network Models with LINE", in *Proc. of WSC 2020*, ACM Press, Dec 2020.

The following are references for the individual tools used by LINE. Please cite them if your use of LINE focuses on features developed in these tools:

- *BuTools* [45]: G. Horváth and M. Telek. "BuTools 2: A Rich Toolbox for Markovian Performance Evaluation", in *Proc. of VALUETOOLS*, 2017.
- *SMCSolver* [7]: D. Bini, B. Meini, S. Steffé, J. F. Pérez, and B. Van Houdt. "SMCSolver and Q-MAM: Tools for Matrix-Analytic Methods", in *SIGMETRICS Performance Evaluation Review*, 39(4), 2012.
- *MAMSolver* [68]: A. Riska and E. Smirni. "MAMSolver: A Matrix Analytic Methods Tool", in *Proc. of the 12th Int. Conf. on Modelling Techniques and Tools (TOOLS)*, LNCS 2324:205–211, 2002.

- *KPC-Toolbox* [22]: G. Casale, E. Z. Zhang, and E. Smirni. "KPC-Toolbox: Best Recipes for Automatic Trace Fitting Using Markovian Arrival Processes", in *Performance Evaluation*, 67(9):873–896, 2010.
- *JMT* [5]: M. Bertoli, G. Casale, and G. Serazzi. "The JMT Simulator for Performance Evaluation of Non-Product-Form Queueing Networks", in *Proc. of the 40th Annual Simulation Symposium (ANSS)*, 2007.
- *LQNS* [37]: G. Franks, P. Maly, M. Woodside, D. C. Petriu, A. Hubbard, and M. Mroz. "Layered Queueing Network Solver and Simulator User Manual", Carleton University, 2013.
- *Q-MAM* [7]: D. Bini, B. Meini, S. Steffé, J. F. Pérez, and B. Van Houdt. "SMCSolver and Q-MAM: Tools for Matrix-Analytic Methods", in *SIGMETRICS Performance Evaluation Review*, 39(4), 2012.

1.4 Contact and credits

LINE is the collective work of many students and researchers hosted at the QORE Lab (https://qore.doc.ic.ac.uk/) at Imperial College London. Please refer to the AUTHORS files in the codebase for detailed credits to individual project contributors.
Project coordinator: Giuliano Casale, Department of Computing, Imperial College London, 180 Queen's Gate, SW7 2AZ, London, United Kingdom. Web: http://wp.doc.ic.ac.uk/gcasale/

1.5 Copyright and license

Copyright Imperial College London (2012-Present). LINE is freeware and open-source, released under the 3-clause BSD license.
License files of third-party libraries are listed at

https://sf.net/p/line-solver/code/ci/master/tree/matlab/lib/thirdparty

1.6 Acknowledgement

LINE has been partially funded by the European Commission grants FP7-318484 (MODAClouds), H2020-644869 (DICE), H2020-825040 (RADON), and by the EPSRC grant EP/M009211/1 (OptiMAM).

Chapter 2

Getting started

2.1 Installation and support

Quick Start: You can get started with LINE as follows:

1. Obtain the latest release

   - Stable release (zip file): https://sf.net/projects/line-solver/files/latest/download

   Ensure that the files are decompressed in the installation folder.

2. Before running LINE you will always need to add the installation folder and its subfolders to the MATLAB path. In order to do so, start MATLAB and change the active directory to the `matlab/` folder, then run

   ```
   lineStart
   ```

3. LINE is now ready to use. For example, you can run the demonstrators using

   ```
   allExamples
   ```

2.1.1 Software requirements

Certain features of LINE depend on external tools and libraries. The recommended dependencies are:

- MATLAB version 2025a or later, with the following toolboxes:
  - Global Optimization Toolbox
  - Optimization Toolbox

  - Parallel Computing Toolbox
  - Statistics and Machine Learning Toolbox
  - Symbolic Math Toolbox

These libraries are shipped with LINE:

- Java Modelling Tools (http://jmt.sf.net): version 1.2.4 or later. The latest version is automatically downloaded at the first call of the `JMT` solver.
- KPC-Toolbox (https://github.com/kpctoolboxteam/kpc-toolbox): version 0.3.4 or later. This release is already included under the `lib` subfolder.
- MAMSolver (https://www.cs.wm.edu/MAMSolver/): tools for matrix-analytic methods. This release is already included under the `lib` subfolder.
- M3A (https://github.com/imperial-qore/M3A): version 1.0.0. This release is already included under the `lib` subfolder.
- BuTools (https://github.com/ghorvath78/butools): version 2.0 or later. This release is already included under the `lib` subfolder.
- Q-MAM/SMC (https://win.uantwerpen.be/~vanhoudt/tools/QBDfiles.zip). This release is already included under the `lib` subfolder.

Optional dependencies recommended to utilize all features available in LINE are as follows:

- LQNS (https://github.com/layeredqueuing/V6): version 6.2.28 or later. System paths need to be configured such that the `lqns` and `lqnsim` solvers need are available on the command line (i.e., it is required that both commands work from within MATLAB using the `dos` or `unix` commands without need to specify the paths of the executables).
- Symbolic analysis backend (https://hub.docker.com/r/imperialqore/line-sage-rest): a SageMath computer algebra system behind a small REST service, used for symbolic steady-state analysis, exact parametric sensitivities and fluid Jacobians (§4.7.2). It requires Docker and is obtained with

  ```
  docker pull imperialqore/line-sage-rest:latest
  ```

  after which LINE starts and stops the service on its own. Without it, symbolic analysis falls back on the Symbolic Math Toolbox, which is licensed separately.

2.1.2 Documentation

This manual introduces the main concepts to define models in LINE and run its solvers. The document includes in particular several tables that summarize the features currently supported in the modeling language and by individual solvers. Additional resources are as follows:

- MATLAB manual: https://line-solver.sf.net/doc/LINE-matlab.pdf
- Java manual: https://line-solver.sf.net/doc/LINE-java.pdf (also covers usage from Kotlin, which calls the same JAR on the same JVM)
- Python manual: https://line-solver.sf.net/doc/LINE-python.pdf

2.1.3 Getting help

For discussions, bug reports, new feature requests, please create a thread on the Sourceforge forums:

- General discussion: https://sf.net/p/line-solver/discussion/help/
- Bugs and issues: https://sf.net/p/line-solver/tickets/
- Feature requests: https://sf.net/p/line-solver/feature-requests/

2.2 Getting started examples

In this section, we present some examples that illustrate how to use LINE. The relevant scripts are included under the `examples/gettingstarted/` folder. The examples describe one of two main available classes of stochastic models within LINE:

- `Network` models are extended queueing networks. Typical instances are open, closed and mixed queueing networks, possibly including advanced features such as class-switching, finite capacity, priorities, non-exponential distributions, and others. Technical background on these models can be found in books such as [9, 51] or in tutorials such as [4, 50].
- `LayeredNetwork` models are layered queueing networks, i.e., models consisting of layers, each corresponding to a `Network` object, which interact through synchronous and asynchronous calls. Technical background on layered queueing networks can be found in [90].

2.2.1 Controlling verbosity

Solver verbosity may be configured at program start using, e.g.:

```
GlobalConstants.setVerbose(VerboseLevel.DEBUG)
```

The three available verbosity levels are:

- `VerboseLevel.SILENT`: Suppresses all solver output messages
- `VerboseLevel.STD`: Shows standard solver output messages (default)
- `VerboseLevel.DEBUG`: Shows detailed solver output including debug information

2.2.2 Model gallery

LINE includes a collection of classic, commonly occurring, queueing models under the `gallery/` folder. They include single queueing systems (e.g., $M/M/1$, $M/H_2/1$, $D/M/1$, ...), tandem queueing systems, and basic queueing networks. For example, to instantiate and estimate the mean response time for a tandem network of $M/M/1$ queues we may run

```
>> MVA(gallery_mm1_tandem).avgTable()
MVA analysis (method: default, lang: matlab) completed. Runtime: 0.027725 seconds. ...
Iterations: 181.
ans =
    Station     JobClass     QLen     Util    RespT     ResidT    ArvR    Tput
    ________    ________    ______    ____    ______    ______    ____    ____

    mySource    myClass          0       0         0         0     0       1
    Queue1      myClass     8.9992     0.9    8.9992    8.9992     1       1
    Queue2      myClass     8.9992     0.9    8.9992    8.9992     1       1
```

The examples in the gallery may also be used as templates to accelerate the definition of basic models. Example 9 later shows an example of gallery instantiation of a $M/E_2/1$ queue.

Table 2.1 lists the gallery models grouped by category. Each entry is a parameter-free factory available in MATLAB (`gallery_*.m`), in the Java JAR (`Gallery.gallery_*()`), and in Python (`from line_solver import gallery_*`). Layered models return a `LayeredNetwork`, the random-environment model returns an `Environment`, and all others return a `Network`.

Table 2.1: Model gallery entries by category.

Category	Gallery functions
Single queues	`gallery_aphm1`, `gallery_coxm1`, `gallery_detm1`, `gallery_dm1`, `gallery_erlm1`, `gallery_gamm1`, `gallery_hypm1`, `gallery_mapm1`, `gallery_merl1`, `gallery_mhyp1`, `gallery_mm1`, `gallery_mmap1`, `gallery_mpar1`, `gallery_parm1`, `gallery_replayerm1`, `gallery_um1`
Processor sharing	`gallery_erlm1_ps`, `gallery_erlm1ps`, `gallery_mm1_ps`
Multiserver	`gallery_erldk`, `gallery_hyperlk`, `gallery_mapmk`, `gallery_mdk`, `gallery_merlk`, `gallery_mhypk`, `gallery_mmapk`, `gallery_mmk`
Multiclass / priority	`gallery_mm1_multiclass`, `gallery_mm1_prio`, `gallery_mm1_ps_multiclass`, `gallery_mmap1_multiclass`
Tandem / linear	`gallery_hyphyp1_linear`, `gallery_hyphyp1_tandem`, `gallery_merl1_linear`, `gallery_merl1_tandem`, `gallery_mhyp1_linear`, `gallery_mhyp1_tandem`, `gallery_mm1_linear`, `gallery_mm1_tandem`, `gallery_mm1_tandem_multiclass`
Feedback	`gallery_mm1_feedback`, `gallery_mm1_ps_feedback`

Category	Gallery functions
Reentrant lines	`gallery_erlerl1`, `gallery_erlerl1_reentrant`, `gallery_erlm1_reentrant`, `gallery_hyperl1_feedback`, `gallery_hyperl1_reentrant`, `gallery_hyphyp1_reentrant`, `gallery_hypm1_reentrant`, `gallery_lukumar_reentrant`, `gallery_merl1_reentrant`, `gallery_mhyp1_reentrant`, `gallery_mm1_ps_reentrant`, `gallery_mm1_reentrant`
Closed networks	`gallery_cqn`, `gallery_cqn_multiclass`, `gallery_repairmen`
Fork-join	`gallery_fj_closed`, `gallery_fj_open`
Caches	`gallery_cache_lru`, `gallery_cache_routing`
Finite capacity	`gallery_fcr`, `gallery_mm1k`
Layered (LQN)	`gallery_lqn_basic`, `gallery_lqn_workflows`, `gallery_multitier`, `gallery_multitier_storage`
Random environment	`gallery_renv_breakdown`
Random generators	`gallery_qn_random`, `gallery_lqn_random`

The `gallery_qn_random` and `gallery_lqn_random` entries wrap the `NetworkGenerator` and `LayeredNetworkGenerator` random model generators. Both accept an optional `seed` argument (default fixed) so that repeated calls return the same model; `gallery_qn_random` uses a deterministic (cyclic) topology to keep the generated network reproducible under the seed.

2.2.3 Tutorials

The sixteen step-by-step tutorials that introduce the modelling language, from a single M/M/1 queue to layered, cached, random-environment, and Petri-net models, are collected in the LINE primer for this language (`LINE-primer-matlab.pdf`, chapter *Tutorials*). The present manual documents each construct they use in full detail, and the LINE internals manual (`LINE-internals.pdf`) documents the solution algorithms.

2.2.4 The utilization convention

Throughout LINE, `Util` is the mean fraction of a station's service capacity held by jobs of the given class, so that a station's total utilization lies in $[0, 1]$ irrespective of how many servers it owns. Three regimes must be distinguished, and they must not be mixed.

Load-independent stations. The Utilization Law applies,

$$U_{ir} = X_r D_{ir}/c_i = T_{ir}/(\mu_{ir} c_i), \tag{2.1}$$

with c_i the number of servers at station i, D_{ir} the service demand and X_r the class-r throughput. The division by c_i is what keeps a c-server station comparable to a single-server one: an M/M/3 offered $\lambda = 1.2$ with unit mean service reports $U = 0.4$, not 1.2. Infinite-server (delay) stations are the exception and report $U =$ `QLen`, the mean number of jobs in service, which may exceed one.

Load-dependent and class-dependent stations. When a station's rate varies with its occupancy (`setLoadDependence`), the capacity that normalizes the utilization is the effective one, $c_i^{eff} = \max(c_i, \max_n \alpha_i(n))$, since a load-dependent station emulates multiserver capacity while c_i remains one. A model built both ways – as a true c-server station and as a load-dependent station with $\alpha(n) = \min(n, c)$ – therefore reports the same utilization. For class-dependent rates the utilization is normalized the same way, as $U_{ir} = T_{ir}S_{ir}/p_{ir}$, where p_{ir} is the *peak rate scaling* per class that the user declares explicitly when the station is created (`setClassDependence(beta,peakRatePerClass)`, stored in the `NetworkStruct` field `cdscalingpeak`, see Section 3.1.6); a scalar peak is broadcast to every class. This makes the class-dependent utilization follow the same $T_{ir}S_{ir}/c$ convention as an ordinary multiserver station.

Order-independent and pass-and-swap stations. Here $U_{ir} = E[s_{ir}]/c_i$, the mean number of class-r jobs receiving a strictly positive rank rate. Note that s_{ir} counts *jobs*, not servers, so a job served concurrently by several compatible servers counts once; this coincides with the offered load only when a job engages a single server. `CTMC`, `LDES` and `SSA` observe s_{ir} directly on the ordered state. The convention applies only to stations the user declares as `OI` or `PAS`.

Chapter 3

Network models

Throughout this chapter, we discuss the specification of `Network` models, which are extended queueing networks. LINE currently supports open, closed, and mixed networks with non-exponential service and arrivals, and state-dependent routing. All solvers support the computation of basic performance metrics, while some more advanced features are available only in specific solvers. Each `Network` model requires in input a description of the nodes, the network topology, and the characteristics of the jobs that circulate within the network. In output, LINE returns performance and reliability metrics.

The default metrics supported by all solvers are as follows:

- Mean queue-length (`QLen`). This is the mean number of jobs residing at a node when this is observed at a random instant of time.

- Mean utilization (`Util`). For nodes that serve jobs, this is the mean fraction of time the node is busy processing jobs. In both single-server and multi-server nodes, this is a number normalized between 0 and 1, corresponding to 0% and 100%. In infinite-server nodes, the utilization is set by convention equal to the mean queue-length, therefore taking the interpretation of the mean number of jobs in execution at the station.

- Mean response time (`RespT`). This is the mean time a job spends traversing a node within a network. If the node is visited multiple times, the response time is the time spent for a single visit to the node.

- Mean residence time (`ResidT`). This is the total time a job accumulates, on average, to traverse a node within a network. If the node is visited multiple times, the residence time is the time accumulated overall visits to the node prior to returning to the reference station or arriving to a sink.

- Mean throughput (`Tput`). This is the mean departure rate of jobs completed at a resource per time unit. Typically, this matches the mean arrival rate, unless the node switches the class of the jobs in which case the arrival rate of a class may not match its departure rate.

- Mean arrival rate (`ArvR`). This is the actual rate at which jobs of a given class arrive at a station, measured in jobs per unit time. The arrival rate differs from throughput primarily in networks with class-switching mechanisms, where jobs may arrive at a node in one class and depart in a different class. In such cases, the arrival rate for a class reflects the incoming flow before any class transformation occurs, while the throughput represents the outgoing flow after transformation. For nodes without class-switching, the arrival rate and throughput are identical under steady-state conditions due to flow balance. In open networks with probabilistic routing, the arrival rate at a downstream station depends on the throughput of upstream stations weighted by the routing probabilities. State-dependent routing strategies such as join-the-shortest-queue or round-robin introduce additional complexity, as the effective arrival rate at each destination varies dynamically based on system state. The arrival rate metric is particularly useful for validating Little's Law, which relates queue length, arrival rate, and response time, and for analyzing bottleneck behavior where high arrival rates relative to service capacity lead to queue buildup and increased delays.

The metrics above characterize individual nodes. Response times, throughputs and queue lengths are also reported end-to-end, in which case they are called *system* metrics and are indexed by chain rather than by station-class pair:

- `SysRespT`: time a job spends in the network, measured from its arrival at the `Source` for open chains, or from its departure from the reference station for closed chains, until it reaches the `Sink` or returns to the reference station.
- `SysTput`: rate at which such end-to-end visits complete.
- `SysQLen`: total number of jobs present in the network, summed over all stations and classes.

These are obtained with the `avgSys` method, which returns a table, or with `getAvgSys`, which returns the same quantities as arrays.

3.1 Network object definition

3.1.1 Creating a network and its nodes

A queueing network can be described in LINE using the `Network` class constructor with a unique string identifying the model name:

```
model = Network('myModel');
```

The returned object of the `Network` class offers functions to instantiate and manage resource *nodes* (stations, delays, caches, ...) visited by jobs of several types (*classes*).

A node is a resource in the network that can be visited by a job. A node must have a unique name and can either be *stateful* or *stateless*, the latter meaning that the node does not require state variables to determine its

Table 3.1: Nodes available in `Network` models.

Node	Description
`Cache`	A node to switch job classes based on hits/misses in its cache
`ClassSwitch`	A node to switch job classes based on a static probability matrix
`Delay`	A station where jobs spend time without queueing
`Fork`	A node that forks jobs into tasks
`Join`	A node that joins sibling tasks into the original job
`Logger`	A node that logs passage of jobs
`Place`	A node that holds tokens in a stochastic Petri net
`Queue`	A node where jobs queue and receive service
`Router`	A node that routes jobs to other nodes
`Sink`	Exit point for jobs in open classes
`Source`	Entry point for jobs in open classes
`Transition`	A timed or immediate node that fires tokens between places in a stochastic Petri net

state or actions. If jobs visiting a stateful node can be required to spend time in it, the node is also said to be a *station*. A list of nodes available in `Network` models is given in Table 3.1.

We now provide more details on each of the nodes available in `Network` models.

Queue node. A `Queue` specifies a queueing station from its name and scheduling strategy, e.g.

```
queue = Queue(model, 'Queue1', SchedStrategy.FCFS);
```

specifies a first-come first-serve queue. It is alternatively possible to instantiate a queue using the `QueueingStation` constructor, which is merely an alias for `Queue`.

Queueing stations have by default a single server. The `setNumberOfServers` method can be used to instantiate multi-server stations. For heterogeneous servers with different capabilities, use `ServerType` to define server types and `HeteroSchedPolicy` to control which server types handle which job classes.

Valid scheduling strategies are specified within the `SchedStrategy` static class and include: If a strategy requires class weights, these can be specified directly as an argument to the `setService` function or using the `setStrategyParam` function, see later the description of DPS scheduling for an example.

Heterogeneous servers. Queue nodes can be configured with heterogeneous servers that have distinct service rates and job class compatibilities. This feature enables modeling of systems where servers have different capabilities or speeds, such as heterogeneous CPU pools, mixed-skill service desks, or tiered processing resources. The `ServerType` class allows defining server types within a queue, each with its own characteristics. To add a server type, first create a `ServerType` object with a name and the number of servers of that type, then pass it to the `addServerType` method. After creating server types, configure their service rates per job class using the same methods as for homogeneous queues, but targeting specific server types. Optionally, the scheduling policy for heterogeneous servers can be controlled using `HeteroSchedPolicy`, which determines how jobs are assigned to available server types. This allows specifying whether jobs can

Table 3.2: Scheduling strategies available in LINE

Strategy	Code	Description
Discriminatory processor-sharing	DPS	Processor sharing with class-dependent weights
Earliest due date	EDD	Job with earliest deadline is served first (non-preemptive)
Earliest deadline first	EDF	Preemptive variant of EDD: an arriving job with an earlier deadline preempts the in-service job
Foreground-background (Least attained service)	FB / LAS	Preemptive; serves the job with least attained service. FB and LAS are synonyms
First-come first-serve	FCFS	Jobs are served in order of arrival (non-preemptive)
FCFS preemptive-independent	FCFSPI	FCFS where a higher-priority arrival preempts the in-service job and the preempted job restarts service from scratch
FCFS preemptive-resume	FCFSPR	FCFS where a higher-priority arrival preempts the in-service job and the preempted job resumes from where it stopped
Fair sojourn protocol	FSP	Preemptive; serves the job with smallest virtual processor-sharing finish time (Friedman & Henderson). Supported by LDES
Generalized processor-sharing	GPS	Processor sharing with configurable weights per class
Infinite-server	INF	All jobs receive service simultaneously (delay station)
Last-come first-serve	LCFS	Most recently arrived job is served first (non-preemptive)
LCFS preemptive-independent	LCFSPI	LCFS where an arriving job preempts the in-service job and the preempted job restarts service from scratch
LCFS preemptive-resume	LCFSPR	LCFS where an arriving job preempts the in-service job and the preempted job resumes from where it stopped
Longest expected processing time	LEPT	Job with longest expected processing time is served first
Limited processor sharing	LPS	Processor sharing with limit on concurrent jobs
Longest remaining processing time	LRPT	Preemptive scheduling that always serves the job with the largest remaining service requirement; the dual of SRPT
Polling	POLLING	Server cycles through queues in round-robin fashion
Processor-sharing	PS	Server capacity is shared equally among all present jobs
Preemptive shortest job first	PSJF	Preemptive scheduling that prioritises jobs by their original service requirement (not the residual one), avoiding the constant churn of SRPT but still favouring short jobs
Shortest expected processing time	SEPT	Job with shortest expected processing time is served first
Shortest elapsed time first	SETF	Non-preemptive variant of FB/LAS: at each completion the next job in service is the one with least attained service, but it is not preempted by later-arriving jobs
Service in random order	SIRO	Jobs are selected randomly for service
Shortest job first	SJF	Job with shortest service requirement is served first

only be served by compatible server types or whether more sophisticated assignment policies are used. The following example demonstrates a queue with two server types, fast and slow servers, where each type has different service rates.

```
queue = Queue(model, 'HeteroQueue', SchedStrategy.FCFS);
fastType = ServerType('FastServer', 2);
slowType = ServerType('SlowServer', 2);
queue.addServerType(fastType);
```

```
queue.addServerType(slowType);
queue.setHeteroService(jobclass, fastType, Exp(1.0));
queue.setHeteroService(jobclass, slowType, Exp(0.5));
```

This capability is particularly useful for capacity planning studies where resource pools contain servers with varying performance characteristics, or for modeling systems where specialization or skill matching affects service delivery.

When a job is compatible with multiple server types, the `HeteroSchedPolicy` class determines the assignment strategy. The available policies are listed in Table 3.3. The policy is set via

Table 3.3: Heterogeneous server assignment policies (`HeteroSchedPolicy`)

Policy	Description
`ORDER`	Assign to the first compatible server type in definition order (default)
`ALIS`	Assign Longest Idle Server: routes to the server type with the longest accumulated idle time (round-robin with busy servers placed at the back of the priority list)
`ALFS`	Assign Least Flexible Server: among compatible server types prefers the one that can serve the fewest classes (least cross-class flexibility), saving more flexible servers for classes that need them
`FAIRNESS`	Round-robin with fairness sorting: when a server type is used it is moved to the back of the priority list, so all compatible types are exercised before any one repeats
`FSF`	Fastest Server First: assigns the job to the compatible server type with the minimum expected service time
`RAIS`	Random Available Idle Server: selects uniformly at random among compatible server types that have at least one idle server

`queue.setHeteroSchedPolicy(HeteroSchedPolicy.FSF)`. When the model is exported to JSIM XML for use with the `JMT` solver, the policy is serialised using JMT's descriptive string form (e.g. `"ALIS (Assign Longest Idle Server)"`) via `HeteroSchedPolicy.toJMTText`, so the JMT engine recognises it.

Delay node. Delay stations, also called infinite server stations, may be instantiated either as objects of `Queue` class, with the `SchedStrategy.INF` scheduling strategy, or using the following specialized constructor

```
delay = Delay(model, 'ThinkTime');
```

As for queues, for readability it is possible to instantiate delay nodes using the `DelayStation` class which is an alias for the `Delay` class.

Source and Sink nodes. As seen in the M/M/1 getting started example, these nodes are mandatory elements for the specification of open classes. Their constructor only requires a specification of the unique name associated to the nodes:

```
source = Source(model, 'Source');
sink = Sink(model, 'Sink');
```

The Source node supports state-dependent arrival rates via the load-dependent scaling mechanism. Instead of specifying a fixed inter-arrival distribution, the effective arrival rate can vary as a function of the total number of jobs in the system. This is achieved by assigning a load-dependent rate function to the Source station using the lldscaling mechanism, in the same way load-dependent service rates are specified for queue nodes (see the load-dependent service section under Advanced node parameters). When a load-dependent arrival rate is defined, the CTMC and SSA solvers automatically account for state-dependent rates during analysis, enabling accurate modeling of systems where arrival rates diminish as congestion increases (e.g., finite source populations or flow-controlled networks).

Batch arrivals. By default a Source releases one job per arrival epoch. A batch arrival stream is obtained by attaching a discrete batch-size distribution to a class with setArrivalBatch: at each epoch generated by the inter-arrival distribution, the source releases a number of jobs drawn from the batch distribution instead of a single job. The batch size must be at least one, so distributions that can return zero are rejected rather than clamped.

```
source.setArrival(oclass, Geometric(0.2));
source.setArrivalBatch(oclass, Geometric(0.5));
```

The batch distribution is stored in the arrivalbatch field of the network structure and is advertised through the BatchArrival language feature. Batch arrivals are simulated by the LDES solver. Combining a geometric inter-arrival distribution with a geometric batch size under slotted simulation (Section 5.2.11) yields the Geo^X arrival stream, whose analytical reference is qsys_geoxgeo1.

Customer impatience and reneging. Queue nodes support customer impatience, where jobs may abandon the queue if their waiting time becomes excessive. This feature models real-world scenarios such as call centers where customers hang up after waiting too long, web users navigating away from slow-loading pages, or service systems where customers leave due to long queues. The setPatience method on Queue nodes specifies a patience distribution for a given job class. When a job arrives or begins waiting, a patience time is sampled from this distribution. If the job's waiting time exceeds this sampled patience time before service begins, the job abandons the queue and exits the system. The patience distribution can be any standard distribution supported by LINE, such as exponential for memoryless impatience or deterministic for fixed deadlines. The following example demonstrates a queue where jobs of a certain class have exponentially distributed patience with mean 10 time units.

```
queue = Queue(model, 'ServiceQueue', SchedStrategy.FCFS);
queue.setPatience(jobclass, Exp(0.1)); % mean patience = 10
```

This feature is supported by the simulation solvers JMT, LDES and SSA, which track individual job patience times and trigger abandonment events on the sample path, and by the CTMC solver, which encodes reneging, balking and retrial as additional state transitions of the Markov chain. Customer impatience is particularly relevant for performance analysis of systems with quality-of-service constraints, where understanding

abandonment rates and their impact on effective throughput is critical for capacity planning and service level agreement compliance.

Fork and Join nodes. The fork and join nodes are supported by the `JMT` and `LDES` simulators, by the `CTMC` and `SSA` solvers through a native fork-join state representation (closed and multiclass models, including `setTasksPerLink`), and, for mean performance metrics, by the `MVA` and `MAM` analytical solvers (and by `LN` for layered models). The `Fork` splits an incoming job into a set of sibling tasks, sending out one task for each outgoing link. These tasks inherit the class of the original job and are served as normal jobs until they are reassembled at a `Join` station.

Their specification of Fork and Join nodes only requires the name of the node

```
fork = Fork(model, 'Fork');
join = Join(model, 'Join', fork);
```

The number of tasks sent by a `Fork` on each output link can be set using the `setTasksPerLink` method of the `fork` object. To enable effective analytical approximations, presently LINE requires that every join node is bound to a specific fork node, although specific solvers will ignore this information (e.g., JMT).

Also note that the routing probabilities out of the `Fork` node need to be set to 1.0 towards every other node connected to the `Fork`. For example, a `Fork` sending jobs in class 1 to nodes A, B and C, cannot send jobs in class 2 only to A and B: it must send them to all three connected nodes A, B and C. A new fork node visited only by class-2 jobs needs to be created in order to send that class of jobs only to A and B.

Fork nodes also support probabilistic (state-dependent) branching via `RoutingStrategy.PROB`, where different output links carry non-uniform routing probabilities. This is set using `setProbRouting` on the Fork node and is available when using `JMT`. This allows a Fork node to direct tasks to different parallel branches with configurable proportions, enabling asymmetric fork-join models where not all branches are equally loaded.

After splitting a job into tasks, LINE takes the convention that visit counts refer to the average number of passages at the target resources for the original job, scaled by the number of tasks. For example, if a job is split into two tasks at a fork node, each visiting respectively nodes A and B, the average visit count at A and B will be 0.5.

For Fork-Join response time percentiles, `MAM` automatically detects valid FJ topologies (Source $\rightarrow$ Fork $\rightarrow$ K Queues $\rightarrow$ Join $\rightarrow$ Sink) and applies percentile computation as follows

```
solver = MAM(model);
[percRT, percTable] = solver.getPerctRespT([90, 95, 99]);
```

ClassSwitch node. This is a stateless node to change the class of a transiting job based on a static probabilistic policy. For example, it is possible to specify that all jobs belonging to class 1 should become of class 2 with probability 1.0, or that a transiting job of class 2 should become of class 1 with probability 0.3. This example is instantiated as follows

```
cs = ClassSwitch(model, 'ClassSwitchPoint',[0.0, 1.0; 0.3, 0.7]);
```

Cache node. This is a stateful node to store one or more items in a cache of finite size, for which it is possible to specify a replacement policy. The *cache* constructor requires the total cache capacity and the number of items that can be referenced by the jobs in transit, e.g.,

```
cacheNode = Cache(model, 'Cache1', nitems, capacity, ReplacementStrategy.LRU);
```

If the capacity is a scalar integer (e.g., `15`), then it represents the total number of items that can be cached and the value cannot be greater than the number of items. Conversely, if it is a vector of integers (e.g., `[10,5]`) then the node is a list-based cache, where the vector entries specify the capacity of each list. We point to [39] for more details on list-based caches and their replacement policies.

Available replacement policies are specified within the `ReplacementStrategy` static class and include:

- First-in first-out (`ReplacementStrategy.FIFO`)
- Random replacement (`ReplacementStrategy.RR`)
- Least-recently used (`ReplacementStrategy.LRU`)
- Strict first-in first-out (`ReplacementStrategy.SFIFO`)

Upon cache hit or cache miss, a job in transit is switched to a user-specified class. More details are given later in Section 3.1.5.

Beyond basic capacity constraints, cache nodes in LINE support weight-based occupancy policies where each cached item can consume a configurable amount of storage space rather than occupying a single slot. This is useful for modeling caches where objects have heterogeneous sizes, such as web caches storing documents of varying lengths. The FCRWeight and FCRMemOcc mechanisms enable specification of per-item weights counted against the total cache capacity. State-dependent routing exploits detailed cache state information to direct jobs along different paths depending on whether their requested item is present in the cache. When a job requests a cached item (a hit), it is classified into a hit class and routed to a delay node representing fast cache access, while jobs experiencing misses are switched to a miss class and directed to a queue representing slower backend retrieval. Solvers such as CTMC and NC employ stochastic complementation techniques to efficiently analyze cache models by eliminating intermediate Router states and computing effective transition rates between observable cache states.

Router node. This node is able to route jobs according to a specified `RoutingStrategy`, which can either be probabilistic or not (e.g., round-robin). Upon entering a `Router`, a job neither waits nor receives service; it is instead directly forwarded to the next node according to the specified routing strategy. A `Router` can be instantiated as follows:

```
router = Router(model, 'RouterNode');
```

An example of use of this node is given in the round-robin load-balancing tutorial of the LINE primer. Routing strategies need to be specified for each class using the `setRouting` method and valid choices are as follows

- Random routing (`RoutingStrategy.RAND`)
- Round robin (`RoutingStrategy.RROBIN`)
- Weighted round robin (`RoutingStrategy.WRROBIN`)
- Probabilistic routing (`RoutingStrategy.PROB`)
- Join-the-shortest-queue (`RoutingStrategy.JSQ`)
- Shortest queue of d, SQ(d) (`RoutingStrategy.SQ`)
- Reinforcement-learning policy (`RoutingStrategy.RL`), driven by a tabular or linear-approximation value function

The state-dependent strategies (`RROBIN`, `WRROBIN`, `JSQ`, `SQ`, `RL`) are supported by both `SSA` and `LDES`. `SQ` samples d candidate destinations with replacement and dispatches to the shortest among them. For `WRROBIN`, weights are supplied via additional arguments to `setRouting`; the dispatcher then cycles through a list in which each destination appears as many times as its integer weight. For example, assume that `oclass` is a class of jobs. In order to route jobs in this class with equal probabilities to every outgoing link we set

```
router.setRouting(oclass, RoutingStrategy.RAND);
```

It should be noted that `setRouting` is also available for all other nodes such as queueing stations, delays, etc. Therefore, the added value of the `Router` node is the ability to represent certain system elements that centralize the routing logic, such as load balancers.

Logger node. A logger node is a node that closely resembles the logger node available in the JSIMgraph simulator within JMT. Models that include this element can be solved using the `JMT` and `LDES` simulators.

A `Logger` node records information about passing jobs in a `csv` file, such as the timestamp of passage and general information about the jobs. The node can be instantiated as follows

```
logger=Logger(model,'LoggerNode','logfile.csv');
```

The file is written in CSV format, one row per job passage, with a column for each enabled field. The `LogTunnel` section that implements the logging is transparent: a job crosses the logger instantaneously and no queueing delay is introduced, so a logger may be placed anywhere in the topology without perturbing the metrics being measured. Placing several loggers yields a trace of job trajectories from which empirical inter-passage distributions can be estimated offline.

The following methods can be used to specify the information that needs to be stored in the `csv` file

- `setStartTime`: record a timestamp for the wallclock time when the simulation started.
- `setJobID`: record a unique id for the passing job.
- `setJobClass`: record the class of the passing job.
- `setTimestamp`: record a timestamp for the simulated time when the job passed in the logger.
- `setTimeSameClass`: record the time elapsed since the last passage of a job of the same class.
- `setTimeAnyClass`: record the time elapsed since the last passage of a job of any class.

Each method can be called either with a single `true` or `false` argument, to enable or disable the recording of the corresponding information, e.g.

```
logger.setJobClass(true);
```

The routing behavior of jobs can be set up as explained for regular nodes such as queues or delay stations.

Place and Transition nodes. LINE supports stochastic Petri net modeling via `Place` and `Transition` nodes. A `Place` holds tokens representing jobs or resources, while a `Transition` fires when its enabling conditions are met, consuming tokens from input places and producing tokens at output places. Transitions support multiple firing modes, each with its own enabling/inhibiting conditions, firing priorities, and firing rate distributions. Modes allow a single transition to represent different behaviors depending on system state. For example:

```
place = Place(model, 'Buffer');
trans = Transition(model, 'Process');
fastMode = trans.addMode('fast');  % returns Mode object
slowMode = trans.addMode('slow');
trans.setDistribution(fastMode, Exp(2.0));  % fast processing mode
trans.setDistribution(slowMode, Exp(0.5));  % slow processing mode
trans.setEnablingConditions(fastMode, jobclass, place, 1);  % requires 1 token
trans.setEnablingConditions(slowMode, jobclass, place, 1);
```

Each mode of a `Transition` is configured through the per-mode setters of Table 3.4. Their full use is illustrated in the dedicated Petri-net section (§3.1.8); worked models are `spn_basic_open`, `spn_twomodes` and related files in the `examples/` folder.

Queueing places. By default a `Place` is an *ordinary place*: a token deposited by an input transition is immediately available to the output transitions. LINE also supports *queueing places* in the sense of queueing Petri nets (Bause, 1993), where a scheduling station is embedded inside the place. A place becomes a queueing place as soon as a service process is assigned to a token colour with `setService`: an arriving token then enters an embedded queue served under the place's scheduling strategy and, upon service completion,

Table 3.4: Per-mode setters of a `Transition` node

Setter	Effect on the mode
`setDistribution`	Firing time distribution
`setEnablingConditions`	Token multiplicities required on the input arcs
`setInhibitingConditions`	Token-level inhibition arcs
`setFiringPriorities`	Integer priority breaking ties among enabled modes
`setFiringWeights`	Weight randomizing the choice among modes of equal highest priority
`setNumberOfServers`	Per-mode server count; `Inf` gives an infinite-server transition
`setTimingStrategy`	With `TimingStrategy.IMMEDIATE`, marks the mode immediate and replaces its firing distribution by the `Immediate` singleton

moves to a *depository* from which it becomes available to the output transitions. The scheduling strategy is chosen through the optional third constructor argument, and the number of embedded servers through `setNumberOfServers`. For example, a processor-sharing CPU place and an infinite-server think place:

```
cpu = Place(model, 'CPU', SchedStrategy.FCFS);
cpu.setService(jobclass, Exp(2.0));    % embedded FCFS queue, rate 2
cpu.setNumberOfServers(2);             % two embedded servers (M/M/2)
```

The order in which depository tokens become available to the output transitions is governed by the departure discipline (`DepartureDiscipline.NORMAL`, the default, or `DepartureDiscipline.FIFO`), set with `setDepartureDiscipline`. Queueing places are simulated by the `LDES` solver, which supports the `FCFS`, `LCFS`, `SIRO` and `INF` embedded scheduling strategies (single- or multi-server) with exponential, deterministic, immediate and renewal phase-type service. The result reported for the place (its token count) is the total marking, i.e. the sum of the tokens waiting, in service and in the depository. Because a queueing place embeds a service station inside a Petri-net place, it cannot be represented as a single node in `JMT` (whose engine keeps token storage and job service in separate nodes); a queueing-place model is therefore rejected by `SolverJMT` and must be expanded into a place plus an adjacent queueing station to be simulated there.

3.1.2 Advanced node parameters

Scheduling parameters

Upon setting service distributions at a station, one may also specify scheduling parameters such as weights as additional arguments to the `setService` function. For example, if the node implements discriminatory processor sharing (`SchedStrategy.DPS`), the command

```
queue.setService(class2, Cox2.fitMeanAndSCV(0.2,10), 5.0);
```

assigns a weight 5.0 to jobs in class 2. The default weight of a class is 1.0.

Finite buffers

The functions `setCapacity` and `setChainCapacity` of the `Station` class are used to place constraints on the number of jobs, total or for each chain, that can reside within a station. The capacity follows standard Kendall notation where K represents the *total system capacity* (queue + in-service jobs). For an M/M/c/K system with c servers, the buffer holds $K - c$ waiting jobs plus c jobs in service. LINE does not allow one to specify buffer constraints at the level of individual classes unless chains contain a single class, in which case `setChainCapacity` is sufficient for the purpose.

For example,

```
cqn_threeclass_hyperl
delay.setChainCapacity([1,1])
model.refreshCapacity()
```

creates an example model with two chains and three classes (specified in `cqn_threeclass_hyperl.m`) and requires the second station to accept a maximum of one job in each chain. Note that if we were to ask for a higher capacity, such as `setChainCapacity([1,7])`, which exceeds the total job population in chain 2, LINE would have automatically reduced the value 7 to the chain 2 job population (2). This automatic correction ensures that functions that analyze the state space of the model do not generate unreachable states.

The `refreshCapacity` function updates the buffer parameterizations, performing appropriate sanity checks. Since `cqn_threeclass_hyperl` has already invoked a solver prior to our changes, the requested modifications are materially applied by LINE to the network only after calling an appropriate `refreshStruct` function, see the sensitivity analysis section. If the buffer capacity changes were made before the first solver invocation on the model, then there would not be the need for a `refreshCapacity` call, since the internal representation of the `Network` object used by the solvers is still to be created.

Cyclic polling

In the polling scheduling policy, the server cyclically visits the input buffer of each job class, processing jobs from that queue for a finite amount of time before switching to the next buffer. This scheduling policy further requires to specify the polling type at the station, chosen among:

- Exhaustive (`PollingType.EXHAUSTIVE`), where the server moves to the next input buffer only after finishing all jobs in the current buffer, including new arrivals during the service period;
- Gated (`PollingType.GATED`), where the number of served jobs for a buffer equals the jobs therein at the instant where the server switched to processing that buffer. Thus, newly arrived jobs during the service period will be processed at the next period.
- K-Limited (`PollingType.KLIMITED`), where the number of jobs processed is a constant K specified by the user. If the queue empties before k jobs have been served, the server will move on and attend the next buffer.

To specify a polling type at a queue, we may write for instance

```
queue.setPollingType(PollingType.EXHAUSTIVE)
```

or for K-Limited with $K = 2$

```
queue.setPollingType(PollingType.KLIMITED, 2)
```

A polling station cannot at the same time be the blocking station of a true blocking-after-service (`BAS`) relationship: the two features both require the same per-station state slot, so a model that declares `BAS` blocking at a polling station is rejected with an explicit error rather than solved. Blocking-after-service at ordinary (non-polling) stations is fully supported.

3.1.3 Job classes

Jobs travel within the network placing service demands at the stations. The demand placed by a job at a station depends on the class of the job. Jobs in *open classes* arrive from the external world and, upon completing the visit, leave the network. Jobs in *closed classes* start within the network and are forbidden to ever leave it, perpetually cycling among the nodes.

Open classes

The constructor for an open class only requires the class name and the creation of special nodes called `Source` and `Sink`

```
source = Source(model, 'Source');
sink = Sink(model, 'Sink');
```

Sources are special stations holding an infinite pool of jobs and representing the external world. Sinks are nodes that route a departing job back into this infinite pool, i.e., into the source. Note that a network can include at most a single `Source` and a single `Sink`.

Once source and sink are instantiated in the model, it is possible to instantiate open classes using

```
class1 = OpenClass(model, 'Class1');
```

LINE does not require explicitly associating source and sink with the open classes in their constructors, as this is done automatically. However, the LINE language requires explicitly creating these nodes since the routing topology needs to indicate the arrival and departure points of jobs in open classes. However, if the network does not include open classes, the user will not need to instantiate a `Source` and a `Sink`.

Closed classes

To create a closed class, we need instead to indicate the number of jobs that start in that class (e.g., 5 jobs) and the *reference station* for that class (e.g., `queue`), i.e.:

```
class2 = ClosedClass(model, 'Class2', 5, queue);
```

The reference station indicates a point in the network used to calculate certain performance indexes, called *system performance indexes*. The end-to-end response time for a job in an open class to traverse the system is an example of a system performance index (system response time). The reference station of an open class is always automatically set by LINE to be the Source. Conversely, the reference station needs to be indicated explicitly in the constructor for closed classes since the point at which a class job completes execution depends on the semantics of the model.

LINE also supports a special class of jobs, called *self-looping jobs*, which perpetually loop at the reference station, remaining in their class. The following example shows the syntax to specify a self-looping job, which is identical to closed classes but there is no need later to specify routing information.

```
model = Network('model');
% Block 1: nodes
delay = Delay(model, 'Delay');
queue = Queue(model, 'Queue1', SchedStrategy.FCFS);
% Block 2: classes
cclass = ClosedClass(model, 'Class1', 10, delay, 0);
slclass = SelfLoopingClass(model, 'SLC', 1, queue, 0);
delay.setService(cclass, Exp(1.0));
queue.setService(cclass, Exp(1.5));
queue.setService(slclass, Exp(1.5));
% Block 3: topology
P = model.initRoutingMatrix;
P{cclass} = [0.7,0.3;1.0,0];
model.link(P);
```

Note that any routing information specified for the self-looping class will be ignored.

Mixed models

LINE also accepts models where a user has instantiated both open and closed classes. The only requirement is that, if two classes communicate by means of a class-switching mechanism, then the two classes must either be all closed or all open. In other words, classes in the same chain must either be both closed or open. Furthermore, for all closed classes in the same chain, it is required for the reference station to be the same.

The `FCFSPRPRIO` (FCFS preemptive-resume with priority) strategy combines priority-based scheduling with preemptive-resume semantics. When a higher-priority job arrives at a queue using this strategy, it preempts the currently running lower-priority job. The preempted job retains its accumulated service progress and resumes from where it was interrupted once all higher-priority jobs have been served. This is suitable for modeling systems such as interrupt-driven processors or prioritized packet queues. The `FCFSPRPRIO` strategy is supported by the CTMC, LDES, JMT, and SSA solvers.

The `SRPT` (Shortest Remaining Processing Time) strategy is a preemptive, non-priority discipline that always serves the job with the least remaining service requirement. When a new job arrives with a shorter

Table 3.5: Priority scheduling strategies available in LINE

Strategy	Code	Description
FCFS with priority	FCFSPRIO	FCFS within priority classes, non-preemptive
FCFS preemptive-resume with priority	FCFSPRPRIO	FCFS with preemptive-resume priority
FCFS preemptive-independent with priority	FCFSPIPRIO	FCFS with preemptive-independent priority
LCFS with priority	LCFSPRIO	LCFS within priority classes, non-preemptive
LCFS preemptive-resume with priority	LCFSPRPRIO	LCFS with preemptive-resume priority
LCFS preemptive-independent with priority	LCFSPIPRIO	LCFS with preemptive-independent priority
Processor-sharing with priority	PSPRIO	PS within priority classes
Discriminatory PS with priority	DPSPRIO	DPS within priority classes
Generalized PS with priority	GPSPRIO	GPS within priority classes
SRPT with priority	SRPTPRIO	Shortest remaining processing time within priority classes

remaining time than the job currently in service, the server preempts the current job and begins serving the new arrival. SRPT is known to minimize mean response time in single-server queues. In LINE, SRPT is supported by the LDES and JMT solvers. The priority variant SRPTPRIO applies SRPT within priority groups: jobs are first partitioned by priority level, and within each level the SRPT discipline is used to select the next job.

```
queue_pr = Queue(model, 'PreemptQueue', SchedStrategy.FCFSPRPRIO);
queue_srpt = Queue(model, 'SRPTQueue', SchedStrategy.SRPT);
```

The FSP (Fair Sojourn Protocol) strategy, introduced by Friedman & Henderson, is a preemptive single-server discipline that ranks jobs by their *virtual processor-sharing finish time*: the time at which each job would complete under a hypothetical processor-sharing shadow running over the same set of jobs. The server serves the job whose virtual finish time is smallest. FSP has two properties that distinguish it from SRPT and from PS: (i) on every sample path, no job is delayed relative to PS, so FSP pareto-dominates PS; (ii) its mean response time is close to that of SRPT without permitting starvation of large jobs. In single-server single-class M/G/1, FSP and SRPT produce the same scheduling decisions; they differ in multi-server settings, where the virtual-PS shadow accounts for per-job rates that depend on the number of jobs and the number of servers. In LINE, FSP is currently supported by the LDES solver.

```
queue_fsp = Queue(model, 'FSPQueue', SchedStrategy.FSP);
```

Class priorities

If a class has a priority, **with 0 representing the highest priority**, this can be specified as an additional argument to both OpenClass and ClosedClass, e.g.,

```
class2 = ClosedClass(model, 'Class2', 5, queue, 0);
```

Important: Class priorities are only effective when a station uses a priority scheduling policy (see Table 3.5). If no priority scheduling policy is explicitly assigned to a station, class priorities are ignored. In `Network` models, priorities are intended as hard priorities. Weight-based policies such as DPS and GPS may be used, as an alternative, to prevent starvation of jobs in low-priority classes.

Class switching

In LINE, jobs can switch their class while they travel between nodes (including self-loops on the same node). For example, this feature can be used to model queueing properties such as re-entrant lines in which a job visiting a station a second time may require a different average service demand than at its first visit.

A chain defines the set of reachable classes for a job that starts in the given class r and over time changes class. Since class switching in LINE does not allow a closed class to become open, and vice-versa, chains can themselves be classified into *open chains* and *closed chains*, depending on the classes that compose them.

Jobs in open classes can only switch to another open class. Similarly, jobs in closed classes can only switch to a closed class. Thus, class switching from open to closed classes (or vice-versa) is forbidden. More details about class-switching are given in Section 3.1.5.

Reference station

Before we have shown that the specification of classes requires choosing a reference station. In LINE, reference stations are properties of chains, thus if two closed classes belong to the same chain they must have the same reference station. This avoids ambiguities in the definition of the completion point for jobs within a chain.

For example, the system throughput for a chain is defined as the sum of the arrival rates at the reference station for all classes in that chain. That is, the solver counts a return to the reference station as a completion of the visit to the system. In the case of open chains, the reference station is always the `Source` and the system throughput corresponds to the rate at which jobs arrive at the sink `Sink`, which may be seen as the arrival rate seen by the infinite pool of jobs in the external world. If there is no class switching, each chain contains a single class, thus per-chain and per-class performance indexes will be identical.

Reference class

Occasionally, it is possible to encounter situations where a job needs to change class while remaining inside the same station. In this case, LINE modifies the network automatically to introduce a class-switching node for the job to route out of the station and immediately return to it in the new class.

One complication of the approach is that, by departing the node and returning to it, the job visits the station one additional time, affecting the visit count to the station and therefore performance metrics such as the residence time. To cope with this issue, LINE offers a method for the class objects, called `setReferenceClass`, that allows users to specify whether the visit of that class to the reference station should be considered upon computing the residence times across the network for the chain to which the class belongs. By default, all classes traversing the reference station are used in the visit count calculation.

Table 3.6: Signal types available in `Network` models.

SignalType	Description
`NEGATIVE`	Removes a random number of jobs from the queue according to a removal distribution (default: 1 job)
`CATASTROPHE`	Removes all jobs from the queue upon arrival
`REPLY`	Triggers a reply action for synchronous call semantics

Networks with signals

Networks with signals are supported by the `LDES` (LINE Discrete Event Simulator) and `SSA` simulators, by the `CTMC` solver, which encodes signal arrivals and the induced job removals directly in the state-transition rates, and by the `MAM` (matrix-analytic methods) solver. Table 3.6 summarizes the signal types available in LINE.

All signal classes are created using the `Signal` class with a required `SignalType` parameter. A `Signal` class can be configured with:

- **Removal distribution**: A discrete distribution specifying how many jobs to remove (default: 1 job). An alternative to the default is a `Geometric` distribution for batch removals.
- **Removal policy**: Determines which jobs are selected for removal when multiple jobs are present:
  - `RemovalPolicy.RANDOM` – Select jobs uniformly at random (default)
  - `RemovalPolicy.FCFS` – Remove the oldest job in the station first
  - `RemovalPolicy.LCFS` – Remove the newest job in the station first

The following example shows how to create a negative signal class:

```
negSignal = Signal(model, 'NegativeCustomer', SignalType.NEGATIVE);
negSignal.setRemovalDistribution(Geometric(0.5)); % batch removal
negSignal.setRemovalPolicy(RemovalPolicy.LCFS);   % remove newest first
source.setArrival(negSignal, Exp(0.1));
```

A catastrophe signal removes all jobs from a queue. Use `SignalType.CATASTROPHE`:

```
disaster = Signal(model, 'Disaster', SignalType.CATASTROPHE);
source.setArrival(disaster, Exp(0.01)); % rare catastrophic events
```

3.1.4 Routing strategies

Probabilistic routing

Jobs travel between nodes according to the network topology and a routing strategy. Typically a queueing network will use a probabilistic routing strategy (`RoutingStrategy.PROB`), which requires specifying

routing probabilities among the nodes. The simplest way to specify a large routing topology is to define the routing probability matrix for each class, followed by a call to the `link` function. This function will automatically add certain nodes to the network to ensure the correct class switching for jobs moving between stations (`ClassSwitch` elements).

In the running case, we may instantiate a routing topology as follows:

```
P = model.initRoutingMatrix;
P{class1}(source,queue) = 1.0;
P{class1}(queue,[queue,delay]) = [0.3,0.7]; % self-loop with probability 0.3
P{class1}(delay,sink) = 1.0;
P{class2}(delay,queue) = 1.0; % note: closed class jobs start at delay
P{class2}(queue,delay) = 1.0;
model.link(P);
```

When used as arguments to a cell array or matrix, class, and node objects will be replaced by a corresponding numerical index. Normally, the indexing of classes and nodes matches the order in which they are instantiated in the model and one can therefore specify the routing matrices using this property. In this case, we would have

```
P = model.initRoutingMatrix;
P{class1} = [0,1,0,0;   % row: source
             0,.3,.7,0; % row: queue
             0,0,0,1;   % row: delay
             0,0,0,0];  % row: sink
P{class2} = [0,0,0,0;
             0,0,1,0;
             0,1,0,0;
             0,0,0,0];
model.link(P);
```

Where needed, the `getClassIndex` and `getNodeIndex` functions return the numerical index associated with a node name, for example `model.getNodeIndex('Delay')`. Class and node names in a network *must be unique*. The list of names already assigned to nodes in the network can be obtained with the `getClassNames`, `getStationNames`, and `getNodeNames` functions of the `Network` class.

It is also important to note that the routing matrix in the last example is specified between *nodes*, instead of between just stations or stateful nodes, which means that for example elements such as the Sink need to be explicitly considered in the routing matrix. The only exception is that `ClassSwitch` elements do not need to be explicitly instantiated in the routing matrix, provided that one uses the `link` function to instantiate the topology. Note that the routing matrix assigned to a model can be printed on the screen in a human-readable format using the `printRoutingMatrix` function, e.g.,

```
>> model.printRoutingMatrix
Delay [Class1] => Queue1 [Class1] : Pr=1.000000
Delay [Class2] => Queue1 [Class2] : Pr=0.001000
Queue1 [Class1] => Queue1 [Class1] : Pr=0.300000
```

```
Queue1 [Class1] => Source [Class1] : Pr=0.700000
Queue1 [Class2] => Source [Class2] : Pr=1.000000
Source [Class1] => Sink [Class1] : Pr=1.000000
Source [Class2] => Queue1 [Class2] : Pr=1.000000
Sink [Class2] => Source [Class2] : Pr=1.000000
```

Other routing strategies

The above routing specification style is only for models with probabilistic routing strategies between every pair of nodes. A different style should be used for scheduling policies that do not require to explicit routing probabilities, as in the case of state-dependent routing. Currently supported strategies include:

- Round robin (`RoutingStrategy.RROBIN`). This is a deterministic strategy that sends jobs to outgoing links in a cyclic order.
- Random routing (`RoutingStrategy.RAND`). This is equivalent to a standard probabilistic strategy that for each class assigns identical values to the routing probabilities of all outgoing links. When a target is invalid its probability is kept to zero, e.g., random routing will not send a job in a closed class to a sink.
- Join-the-Shortest-Queue (`RoutingStrategy.JSQ`). This is a non-probabilistic strategy that sends jobs to the destination with the smallest total number of jobs in it (either queueing or receiving service). If multiple stations have the same total number of jobs, then the destination is chosen at random with equal probability.
- Weighted round robin (`RoutingStrategy.WRROBIN`). Routes jobs cyclically through a list in which each destination appears as many times as its integer weight, so destinations with higher weights are visited proportionally more often. Configure each weight with a per-destination call `router.setRouting(class, RoutingStrategy.WRROBIN, dest, weight)`; with all weights equal to one this collapses to ordinary round-robin.
- Shortest queue of d, SQ(d) (`RoutingStrategy.SQ`, formerly `KCHOICES`). Samples d candidate destinations uniformly at random *with replacement* from the set of available outgoing links and dispatches the job to the one with the shortest queue, breaking ties by first occurrence in the candidate list (Vvedenskaya, Dobrushin and Karpelevich, 1996; Mitzenmacher, 1996). The parameter d defaults to 2 if not specified. Dispatcher memory is not supported: variants that retain state between routing decisions, such as the one-memory-choice scheme of Mitzenmacher, Prabhakar and Shah (FOCS 2002) or the SQ(d, N) scheme of Anselmi and Dufour, are outside the scope of LINE. JMT's `withMemory` flag selects the latter; LINE always exports `withMemory=false` and warns when importing a JSIM model in which it is set. To set a specific value of d use `setRouting(class, RoutingStrategy.SQ, d)`.

- Event-based routing (`RoutingStrategy.FIRING`). This strategy is used internally by `Transition` nodes in stochastic Petri net models and routes tokens to output places according to the firing outcomes configured for each mode of the transition. It is set automatically when specifying Petri net firing outcomes and is not intended for direct assignment by the user.

- Reinforcement learning (`RoutingStrategy.RL`). This is a state-dependent routing strategy that uses a learned value function to make routing decisions based on current queue lengths. The value function can be either a tabular lookup table (for small state spaces) or a linear/quadratic function approximation (for larger state spaces). When the current state exceeds the learned state space, the strategy falls back to JSQ (Join-the-Shortest-Queue) routing. The RL routing policy is trained offline using temporal difference (TD) learning via the `rl_env` and `rl_td_agent` API classes, then applied to the network model. To use RL routing:

  ```
  node.setRouting(jobClass, RoutingStrategy.RL, valueFunction, {actionNodes, ...
  stateSize});
  ```

  where `valueFunction` is the learned value function (tabular array or regression coefficients), `actionNodes` is a vector of node indices where RL routing decisions are needed, and `stateSize` controls the state truncation (`0` for tabular, `>0` for function approximation). The strategy is evaluated by simulators that compute state-dependent routing (`SSA`, `LDES`). When the dispatching node is not in `actionNodes`, or the current per-queue occupancy falls outside the learned value function support, the closure falls back to JSQ.

- Disabled routing (`RoutingStrategy.DISABLED`, internal value -1). This pseudo-strategy explicitly marks a (node, class) pair as inactive and prevents jobs of that class from being dispatched out of the node. It is used internally by the model linker to suppress routing for classes that should not visit a given node, and is not normally set by user code.

For the above policies, the function `addLink` should be first used to specify pairs of connected nodes

```
model.addLink(queue, queue); %self-loop
model.addLink(queue, delay);
```

Then an appropriate routing strategy should be selected at every node, e.g.,

```
queue.setRouting(class1,RoutingStrategy.RROBIN);
```

assigns round robin among all outgoing links from the `queue` node.

A model could also include both classes with probabilistic routing strategies and classes that use round-robin or other non-probabilistic strategies. To instantiate routing probabilities in such situations one should then use, e.g.,

```
queue.setRouting(class1,RoutingStrategy.PROB);
queue.setProbRouting(class1, queue, 0.7)
```

```
queue.setProbRouting(class1, delay, 0.3)
```

where `setProbRouting` assigns the routing probabilities to the two links.

Routing probabilities for Source and Sink nodes

In the presence of open classes, and in mixed models with both open and closed classes, one needs only to specify the routing probabilities *out* of the source. The probabilities out of the sink can all be set to zero for all classes and destinations (including self-loops). The solver will take care of adjusting these inputs to create a valid routing table.

Simplified definition of tandem and cyclic topologies

Tandem networks are open queueing networks with a serial topology. LINE provides functions that ease the definition of tandem networks of stations with exponential service times. For example, the getting started Example 1 on the M/M/1 queue illustrates a simplified way to specify a serial routing topology, i.e.,

```
model.link(Network.serialRouting(source,queue,sink));
```

In a similar fashion, we can also rapidly instantiate a tandem network consisting of stations with PS and INF scheduling as follows

```
lambda = [10,20]; % lambda(r) - arrival rate of class r
D = [11,12; 21,22]; % D(i,r) - class-r demand at station i (PS)
Z = [91,92; 93,94]; % Z(i,r)  - class-r demand at station i (INF)
modelPsInf = Network.tandemPsInf(lambda,D,Z)
```

The above snippet instantiates an open network with two queueing stations (PS), two delay stations (INF), and exponential distributions with the given inter-arrival rates and mean service times. The `Network.tandemPs`, `Network.tandemFcfs`, and `Network.tandemFcfsInf` functions provide static constructors for networks with other combinations of scheduling policies, namely only PS, only FCFS, or FCFS and INF.

A tandem network with closed classes is instead called a cyclic network. Similar to tandem networks, LINE offers a set of static constructors:

- `Network.cyclicPs`: cyclic network of PS queues
- `Network.cyclicPsInf`: cyclic network of PS queues and delay stations
- `Network.cyclicFcfs`: cyclic network of FCFS queues
- `Network.cyclicFcfsInf`: cyclic network of FCFS queues and delay stations

These functions only require replacing the arrival rate vector `A` by a vector `N` specifying the job populations for each of the closed classes, e.g.,

```
N = [10,20]; % N(r) - closed population in class r
D = [11,12; 21,22]; % D(i,r) - class-r demand at station i (PS)
modelPsInf = Network.cyclicPs(N,D)
```

Simplified definition of cluster topologies

A cluster is a network in which jobs flow from an arrival point through a dispatcher (modelled by a `Router`) that distributes them across M parallel servers, after which they leave the system. LINE provides static constructors that build the open topology `Source` $\rightarrow$ `Dispatcher` $\rightarrow$ `Server[1..`M`]` $\rightarrow$ `Sink` and its closed counterpart `Think (Delay)` $\rightarrow$ `Dispatcher` $\rightarrow$ `Server[1..`M`]` $\rightarrow$ `Think`. Service times at the servers are exponential and the dispatcher routes jobs according to `RoutingStrategy.RAND` by default.

- `Network.clusterPs`: open cluster with PS queues
- `Network.clusterFcfs`: open cluster with FCFS queues, optionally multi-server
- `Network.cluster`: open cluster with user-supplied per-server scheduling strategies
- `Network.clusterClosed`: closed cluster with a per-class think delay

For example, an open PS cluster and a closed FCFS cluster can be created as follows:

```
lambda = [10,20];           % arrival rates per class
D = [11,12; 21,22];         % D(i,r) - class-r demand at server i
modelOpen = Network.clusterPs(lambda, D);

N = [2,1];                  % closed populations
Z = [5,5];                  % per-class think times
S = [2;1];                  % multiplicity of each server
strategy = {SchedStrategy.FCFS, SchedStrategy.FCFS};
modelClosed = Network.clusterClosed(N, Z, D, strategy, S);
```

For richer configuration, LINE also exposes a chainable `Cluster` builder with setters `setDispatching`, `setScheduling`, `setStationServers`, and `setClosed`, together with helpers `compareDispatching`, `compareScheduling`, `sweepArrivalRate`, and `sweepNumStations` for parametric studies. Worked examples are available under `examples/basic/cluster/` (e.g. `cl_basic`, `cl_closed`, `cl_compare`, `cl_multiclass`, `cl_sweep`).

3.1.5 Class switching

Depending on the specified probabilities, a job will be able to switch its class only among a subset of the available classes. Each subset is called a *chain*. Chains are computed in LINE as the weakly connected components of the routing probability matrix of the network when this is seen as an undirected graph.

The function `model.getChains()` produces the list of chains for the model, inclusive of a list of their composing classes.

The definition of class switching in a model is integrated in the specification of the routing between stations as described in the next subsection.

Probabilistic class switching

In models with class switching and probabilistic routing at all nodes, a routing matrix is required for each possible pair of source and target classes. For instance, suppose that in the previous example the job in the closed class `class2` switches into a new closed class (`class3`) while visiting the `queue` node. We can specify this routing strategy as follows:

```
class3 = ClosedClass(model, 'Class3', 0, queue, 0);

P = model.initRoutingMatrix;
P{class1,class1}(source, queue) = 1.0;
P{class1,class1}(queue, [queue,delay]) = [0.3,0.7];
P{class1,class1}(delay, sink) = 1.0;
P{class2,class3}(delay, queue) = 1.0;
P{class3,class2}(queue, delay) = 1.0;
model.link(P);
```

where `P{r,s}` is the routing matrix for jobs switching from class `r` to `s`. That is, `P{r,s}(i,j)` is the probability that a job in class `r` departs node `i` routing into node `j` as a job of class `s`.

Importantly, LINE assumes that a job switches class an instant *after* leaving a station, thus the performance metrics of a class at the node refer to the class that jobs had upon arrival to that node.

Class switching with non-probabilistic routing strategies

In the presence of non-probabilistic routing strategies, such as round-robin or join-the-shortest-queue, one may need to manually specify the details of the class switching mechanism. This can be done through addition to the network topology of `ClassSwitch` nodes. ClassS68.481 0 Td 2-31032 .

```
C = csnode.initClassSwitchMatrix;
C(jobclass{1},jobclass{2}) = 1.0;
C(jobclass{2},jobclass{2}) = 1.0;
csnode.setClassSwitchingMatrix(C);
```

Note that for a network with M stations, up to M^2 `ClassSwitch` elements may be required to implement class-switching across all possible links, including self-loops.

Cache-based class-switching

An advanced feature of LINE available for example within the `Cache` node, is that the class-switching decision can dynamically depend on the state of the node (e.g., cache hit/cache miss). However, in order to statically determine chains, LINE requires that every class-switching node declares the pair of classes that can potentially communicate with each other via a switch. This is called the *class-switching mask* and it is automatically computed. The boolean matrix returned by the `model.getClassSwitchingMask` function provides this mask, which has an entry in row r and column s set to true only if jobs in class r can switch into class s at some node in the network.

Upon cache hit or cache miss, a job in transit is switched to a user-specified class, as specified by the `setHitClass` and `setMissClass`, so that it can be routed to a different destination based on whether it found the item in the cache or not. The `setRead` function allows the user to specify a discrete distribution (e.g., `Zipf`, `DiscreteSampler`) for the frequency at which an item is requested. For example,

```
refModel = Zipf(0.5,nitems);
cacheNode.setRead(initClass, refModel);
cacheNode.setHitClass(initClass, hitClass);
cacheNode.setMissClass(initClass, missClass);
```

Here `initClass`, `hitClass`, and `missClass` can be either open or closed instantiated as usual with the `OpenClass` or `ClosedClass` constructors.

3.1.6 Service and inter-arrival time processes

A number of statistical distributions are available to specify job service times at the stations and inter-arrival times from the Source station. The class `Markovian` offers distributions that are analytically tractable, which are defined using absorbing Markov chains consisting of one or more states (*phases*) and called phase-type distributions. They include as special cases the distributions shown in Table 3.7.

Discrete-time modelling with DMAP. LINE supports discrete-time modelling through the `DMAP` (Discrete-time Markovian Arrival Process) class. Unlike the continuous-time `MAP`, where $D_0 + D_1$ is an infinitesimal generator, in a `DMAP` the matrix $D_0 + D_1$ is a (row-)stochastic transition matrix. A `DMAP` is constructed as follows:

```
dmap = DMAP(D0, D1);
```

Table 3.7: Phase-type distributions available in LINE

Distribution	Description
Exponential	`Exp`(λ), where λ is the rate of the exponential
n-phase Erlang	`Erlang`(α, n), where α is the rate of each of the n exponential phases
2-phase hyper-exponential	`HyperExp`$(p, \lambda_1, \lambda_2)$, that returns an exponential with rate λ_1 with probability p, and an exponential with rate λ_2 otherwise
n-phase hyper-exponential	`HyperExp`(p, λ), that builds a n-phase hyper-exponential from a rate vector $\lambda = [\lambda_1, \ldots, \lambda_n]$ and phase selection probabilities $p = [p_1, \ldots, p_n]$
2-phase Coxian	`Coxian`(μ_1, μ_2, ϕ_1), which assigns phases μ_1 and μ_2 to the two rates, and completion probability from phase 1 equal to ϕ_1 (the probability from phase 2 is $\phi_2 = 1.0$)
n-phase Coxian	`Coxian`(μ, ϕ), which builds an arbitrary Coxian distribution from a vector $\mu = [\mu_1, \ldots, \mu_n]$ of n rates and a completion probability vector $\phi = [\phi_1, \ldots, \phi_n]$ with $\phi_n = 1.0$
n-phase acyclic phase-type	`APH`(α, T), which defines an acyclic phase-type distribution with initial probability vector $\alpha = [\alpha_1, \ldots, \alpha_n]$ and transient generator T
n-phase matrix exponential	`ME`(α, A), which defines a matrix exponential distribution with initial vector $\alpha = [\alpha_1, \ldots, \alpha_n]$ (possibly non-normalized) and matrix parameter A. This generalizes phase-type distributions by allowing non-stochastic initial vectors [8]
n-phase concentrated matrix exponential	`CME`(m, n), a subclass of `ME` defining the matrix exponential distribution of odd order $n = 2k + 1$ with mean m whose squared coefficient of variation is minimal for that order [46]. Its SCV decays as $O(1/k^2)$, hence well below the bound $1/n$ attainable by a phase-type distribution of the same order, which makes it an inexpensive representation of a near-deterministic timing. Available orders are returned by `CME.getSupportedOrders()`, and `CME.fitMeanAndSCV`(m, c^2) returns the lowest order meeting a target SCV
Rational arrival process	`RAP`(H_0, H_1), which defines a correlated arrival process with hidden transition matrix H_0 and visible transition matrix H_1, where $H_0 + H_1$ forms a generator matrix. This generalizes the Markovian arrival process (MAP) [3]
Markovian arrival process	`MAP`(D_0, D_1), which defines a point process with phase-dependent arrival rates via matrices D_0 (hidden transitions) and D_1 (arrivals)
Markov-modulated Poisson process	`MMPP2`$(\lambda_1, \lambda_2, \sigma_1, \sigma_2)$, a two-phase MMPP with arrival rates λ_i and switching rates σ_i
Batch Markovian arrival process	`BMAP`(D), which generalizes MAP to allow batch arrivals via a set of arrival matrices $D = \{D_0, D_1, \ldots, D_k\}$
General phase-type	`PH`(α, T), which defines a phase-type distribution with initial probability vector α and transient generator T, allowing cyclic phase transitions
Markov-modulated deterministic process	`MMDP`(Q, R), which defines a fluid process with deterministic flow rates modulated by a background CTMC with generator Q and diagonal rate matrix R. This is the deterministic analogue of MMPP and is supported by the Markovian fluid queue solver (`FLD` with method `mfq`)
2-state Markov-modulated deterministic process	`MMDP2`$(r_0, r_1, \sigma_0, \sigma_1)$, a two-state MMDP with deterministic rates r_0 and r_1 in the two phases, and switching rates σ_0 (from state 0 to 1) and σ_1 (from state 1 to 0)
Marked Markovian arrival process	`MarkedMAP`(D, K), which defines a Markovian arrival process with K marking (color) types represented via the M3A format $D = \{D_0, D_1, D_{11}, \ldots, D_{1K}\}$. Supports multi-class arrival flows with correlated inter-arrival times and class assignment
Marked Markov-modulated Poisson process	`MarkedMMPP`(D, K), a restriction of `MarkedMAP` to diagonal D_{1k} matrices, i.e., a MMPP with K arrival classes. Represents a Poisson arrival stream partitioned into K distinguishable job types modulated by a shared background Markov chain
Discrete-time Markovian arrival process	`DMAP`(D_0, D_1), the discrete-time counterpart of `MAP`. Matrices D_0 (no-arrival transitions) and D_1 (arrival transitions) are sub-stochastic with $D_0 + D_1$ having row sums equal to 1. Useful for modelling slotted or frame-based arrival processes

A random two-phase DMAP can be generated with `DMAP.rand()`. The `DMAP` class inherits statistical accessors from its continuous-time counterpart (`getMean`, `getSCV`, `sample`). In addition, LINE provides

distance measures between two DMAPs (or MAPs) to quantify their dissimilarity:

- `dmap_dist(A,B,L)`: squared L_2 distance of lag-L joint probability mass functions.
- `dmap_dist_lag1(A,B)`: efficient lag-1 specialization via a discrete Lyapunov equation.
- `dmap_dist_acf(A,B)`: squared L_2 distance of the autocorrelation functions.

Analogous functions `map_dist`, `map_dist_lag1`, and `map_dist_acf` are available for continuous-time MAPs. These distance measures are described in [43].

MAP and BMAP analysis

The `MAP` class exposes a number of inspection methods to assess the temporal characteristics of a Markovian arrival process. Given a `MAP` instance, the following methods are commonly used:

- `getRate()` returns the mean arrival rate λ.
- `getACF(lags)` returns the autocorrelation function of the inter-arrival times at the requested lags.
- `getACFDecay()` returns the asymptotic (and optionally interpolated) decay rate of the autocorrelation function.
- `getIDC()` returns the asymptotic index of dispersion for counts; passing a timescale t returns the finite-horizon counterpart.
- `toTimeReversed()` returns the time-reversed MAP, useful in departure-process and reversed-time arguments.
- `MAP.rand(order)` produces a random MAP of the specified phase order.

For example, to inspect the burstiness of a MAP one may write

```
m = MAP.rand(3);
lambda = m.getRate();
acf = m.getACF(1:10);
idc = m.getIDC();
```

The Batch Markovian Arrival Process `BMAP` extends `MAP` to allow batch arrivals via the cell array $D = \{D_0, D_1, \ldots, D_K\}$, where D_k is the rate matrix for transitions producing batches of size k. The most useful inspection methods are:

- `getMaxBatchSize()` returns the largest batch size K supported.
- `getMeanBatchSize()` returns the average batch size $E[B]$ weighted by the batch arrival rates.
- `getBatchRates()` returns a vector of per-batch-size arrival rates.

- `[X,B]=bmap.sample(n)` draws n batches, returning inter-batch times X and batch sizes B.
- `BMAP.rand(order,maxBatchSize)` generates a random BMAP of given phase order and maximum batch size.

A typical inspection of a BMAP combines its scalar summaries with sampling, e.g.,

```
bmap = BMAP.rand(2, 4);
maxK = bmap.getMaxBatchSize();
EB   = bmap.getMeanBatchSize();
[X,B] = bmap.sample(1000);
```

A BMAP can also be constructed deterministically from an underlying MAP and a batch-size probability mass function via `BMAP.fromMAPWithBatchPMF(D0,D1,batchSizes,pmf)`. For example, given mean $\mu = 0.2$ and squared coefficient of variation SCV=10, where SCV=variance/μ^2, we can assign to a node a 2-phase Coxian service time distribution with these moments as

```
queue.setService(class2, Cox2.fitMeanAndSCV(0.2,10.0));
```

where `Cox2` is a static class to fit 2-phase `Coxian` distributions. Inter-arrival time distributions can be instantiated in a similar way, using `setArrival` instead of `setService` on the `Source` node. For example, if the `Source` is node 3 we may assign the inter-arrival times of class 2 to be exponential with mean 0.1 as follows

```
source.setArrival(class2, Exp.fitMean(0.1));
```

Is it also possible to plot the structure of a phase-type distribution using the `plot` instance method of the `Markovian` class.

Non-Markovian distributions are also available, but typically they can restrict the available solvers to the `JMT` simulator. They include the distributions shown in Table 3.8.

When non-Markovian distributions are used with the `CTMC`, `SSA`, or `MAM` solvers, they are automatically converted to phase-type (PH) distributions using a Bernstein polynomial approximation. A warning message is issued when this conversion occurs. The number of phases used in the approximation can be controlled via `options.config.bernstein` (default: 20 phases).

Table 3.9 shows the internal parameter storage format for each non-Markovian distribution. These parameters are stored in the `proc{ist}{r}` field of the network structure.

Lastly, we discuss two special distributions. The `Disabled` distribution can be used to explicitly forbid a class to receive service at a station. This may be useful to declare in models with sparse routing matrices to debug the model specification. Performance metrics for disabled classes will be set to `NaN`.

Conversely, the `Immediate` class can be used to specify instantaneous service (zero service time). Normally, solvers will replace zero service times with small positive values (ε =`GlobalConstants.FineTol`).

Table 3.8: Non-Markovian distributions available in LINE

Distribution	**Description**
Deterministic	`Det`(μ) assigns probability 1.0 to the value μ
Uniform	`Uniform`(a, b) assigns uniform probability $1/(b-a)$ to the interval $[a, b]$
Gamma	`Gamma`(α, k) assigns a gamma density with shape α and scale k
Pareto	`Pareto`(α, k) assigns a Pareto density with shape α and scale k
Weibull	`Weibull`(r, α) assigns a Weibull density with shape r and scale α
Lognormal	`Lognormal`(μ, σ) assigns a lognormal density with parameters μ and σ
Zipf	`Zipf`(s, N) assigns power-law probabilities $p(k) \propto k^{-s}$ for $k = 1, \ldots, N$
Trace-based distributions (replay empirical data from files)	
Replayer	`Replayer`(filename) reads interarrival or service times cyclically from a CSV file
Trace	`Trace`(data) wraps an empirical dataset as a distribution
EmpiricalCDF	`EmpiricalCDF`(x,F) defines a distribution from CDF values F at points x

Table 3.9: Non-Markovian distribution parameter formats in `proc{ist}{r}`

Distribution	**Format**
Gamma	{shape (α), scale (β)}
Weibull	{shape (r), scale (α)}
Lognormal	{μ, σ}
Pareto	{shape (α), scale (k)}
Uniform	{min (a), max (b)}
Det	{t}

Fitting a distribution

The `fitMeanAndSCV` function is available for all distributions that inherit from the `Markovian` class. This function provides exact or approximate matching of the first two moments, depending on the theoretical constraints imposed by the distribution. For example, an Erlang distribution with SCV=0.75 does not exist, because in a n-phase Erlang it must be SCV=$1/n$. In a case like this, `Erlang.fitMeanAndSCV(1,0.75)` will return the closest approximation, e.g., a 2-phase Erlang (SCV=0.5) with unit mean. The Erlang distribution also offers a function `fitMeanAndOrder`(μ, n), which instantiates a n-phase Erlang with given mean μ.

In distributions that are uniquely determined by more than two moments, `fitMeanAndSCV` chooses a particular assignment of the residual degrees of freedom other than mean and SCV. For example, `HyperExp` depends on three parameters, therefore it is insufficient to specify mean and SCV to identify the distribution. Thus, `HyperExp.fitMeanAndSCV` automatically chooses to return a probability of selecting phase 1 equal to 0.99. Compared to other choices, this particular assignment corresponds to a higher probability mass in the tail of the distribution. `HyperExp.fitMeanAndSCVBalanced` instead assigns p in a two-phase hyper-exponential distribution so that $p/\mu_1 = (1-p)/\mu_2$.

Some distributions admit no closed-form two-moment inverse and are fitted by an approximation, in which case the mean is matched exactly and the SCV only approximately. This is the case for `Weibull`, whose 7 10.909r0(parameter)4576(is)09rob3(tmined)-652(from)4276(the)427J(muu(is)09raset)44(.)0386(976,F))4

scale then being chosen so that the mean is exact. The relation was published for $1 \leq k \leq 10$, i.e. SCV ≤ 1, where the realized SCV is within about 3% of the request; outside that range the error grows quickly, reaching 12% at SCV $= 2$ and 48% at SCV $= 4$. Use a phase-type distribution instead when the second moment must be matched exactly at high variability.

Three-moment fitting for APH and PH. The acyclic phase-type (`APH`) and general phase-type (`PH`) classes additionally expose static factory methods that fit a distribution from the first three moments, providing finer control over higher-order behaviour such as burstiness and skewness of the service time. For both `APH` and `PH`, three equivalent entry points are available:

- `APH.fit(MEAN,SCV,SKEW)` fits the distribution from the first three standard moments (mean, squared coefficient of variation and skewness).
- `APH.fitCentral(MEAN,VAR,SKEW)` accepts the first three central moments (mean, variance and skewness).
- `APH.fitRawMoments(m1,m2,m3)` accepts the first three raw moments $E[X^k]$ for $k = 1, 2, 3$.

Each variant returns an `APH` (resp. `PH`) instance whose parameters reproduce the requested moments, falling back to an exponential or immediate distribution when the requested mean is below `GlobalConstants.FineTol`. For example,

```
aph = APH.fit(1.0, 4.0, 8.0); % mean=1, SCV=4, skewness=8
queue.setService(class1, aph);
```

The `Coxian` class supports the same three-moment family of methods through its inherited APH interface. Both `APH` and `PH` also offer `fitMeanAndSCV(MEAN,SCV)` for two-moment fitting.

Inspecting and sampling a distribution

To verify that the fitted distribution has the expected mean and SCV it is possible to use the `getMean` and `getSCV` functions, e.g.,

```
>> dist = Exp(1);
>> dist.getMean
ans =
     1
>> dist.getSCV
ans =
     1
```

Moreover, the `sample` function can be used to generate values from the obtained distribution, e.g. we can generate 3 samples as

Table 3.10: Discrete distributions available in LINE

Distribution	Description
Bernoulli	`Bernoulli`(p), single trial with success probability $p \in [0,1]$. Mean $= p$, support $\{0,1\}$
Binomial	`Binomial`(n,p), number of successes in n independent Bernoulli trials. Mean $= np$, support $\{0,\ldots,n\}$
Poisson	`Poisson`(λ), count of events occurring in a fixed interval at average rate λ. Mean $=$ Var $= \lambda$, support $\{0,1,2,\ldots\}$
DiscreteUniform	`DiscreteUniform`(a,b), uniform probability $1/(b-a+1)$ on the integers $\{a,a+1,\ldots,b\}$
Geometric	`Geometric`(p), number of Bernoulli trials needed to obtain the first success. Mean $= 1/p$, support $\{1,2,\ldots\}$
DiscreteSampler	`DiscreteSampler`(p,x), fully user-specified discrete distribution. Probability vector p assigns mass p_i to value x_i. If x is omitted it defaults to $1{:}n$ where n is the length of p

```
>> dist.sample(3)
ans =
    0.2049
    0.0989
    2.0637
```

The `evalCDF` and `evalCDFInterval` functions return the cumulative distribution function at the specified point or within a range, e.g.,

```
>> dist.evalCDFInterval(2,5)
ans =
    0.1286
>> dist.evalCDF(5)-dist.evalCDF(2)
ans =
    0.1286
```

For more advanced uses, the distributions of the `Markovian` class also offer the possibility to obtain the standard (D_0, D_1) representation used in the theory of Markovian arrival processes by means of the `getRepresentation` function [9]. The result will be a cell array where element $k+1$ corresponds to matrix D_k.

Discrete distributions

In addition to continuous service- and inter-arrival-time distributions, LINE provides a family of discrete distributions through the `DiscreteDistribution` class hierarchy. These distributions take values on a (possibly finite) integer support and are primarily used as count or popularity models, e.g. to specify the number of items requested from a cache, the number of arrivals per slot, or batch sizes. Table 3.10 summarizes the discrete distributions currently available.

All discrete distributions inherit the same inspection interface as the continuous ones: `getMean`, `getSCV` and `sample(n)` return the first two moments and draw n pseudo-random values. In addition, `evalPMF(k)` returns the probability mass at the integer k, and `evalCDF(k)` returns the cumulative probability up to k.

A typical use case for `DiscreteSampler` is to specify the popularity profile of items in a `CacheTask` or `ItemEntry`. The example below assigns a non-uniform popularity to four items so that item 1 is requested with probability 0.5 and items 2–4 share the remaining mass:

Class dependence (`setClassDependence`, stored as `cdscaling`) is the *product-form* case of the QD-AMVA model [19]: the demand factorizes as $D_{i,r}(n_i) = \theta_{i,r}\,\beta_{i,r}(n_{i,r})$, where the scaling $\beta_{i,r}$ read by class r depends only on its own marginal count $n_{i,r}$ (or, equivalently, on the total n_i, which also preserves product form). The handle returns the per-class vector $[\beta_{i,1}(n_i), \ldots, \beta_{i,R}(n_i)]$, of which the solver selects class r; a scalar return is broadcast to all classes. For example

```
queue.setClassDependence(@(ni) [min(ni(1),c), 1], [c 1]);
```

gives class-1 jobs a multiserver-type scaling in their own count while class-2 jobs are always single-server.

Joint dependence (`setJointDependence`, stored as `jdscaling`) is the *non-product-form* case: the handle $\eta_i(n_i)$ may read the joint population vector arbitrarily – for instance a scalar rate driven by a foreign class marginal, $\min(n_{i,1}, c)$ shared across all classes – which violates the QD-AMVA product-form condition. Such models are solved by the same solvers as an approximation, with no exactness or uniqueness guarantee:

```
queue.setJointDependence(@(ni) min(ni(1),c), c);
```

In both cases the second argument is the required peak rate scaling per class (a scalar is broadcast to all classes) that normalizes utilization as $U = T\,S/\text{peak}$. The solver implicitly assumes the handles are smooth and defined also for fractional values of $n_{i,r}$.

Table 3.11 summarizes the queue-dependent service mechanisms by what the handle reads and returns, the field that stores them, and whether the resulting demands are product form.

Argument read	Return	Setter	Field	Prod. form
total $n_i = \sum_r n_{i,r}$	scalar (shared)	`setLoadDependence`	`lldscaling`	yes
own marginal $n_{i,r}$	per-class vector	`setClassDependence`	`cdscaling`	yes
total n_i	per-class vector	`setClassDependence`	`cdscaling`	yes
joint $[n_{i,1}, \ldots, n_{i,R}]$	scalar (shared)	`setJointDependence`	`jdscaling`	no
joint $[n_{i,1}, \ldots, n_{i,R}]$	per-class vector	`setJointDependence`	`jdscaling`	no

Table 3.11: Queue-dependent service mechanisms. `setLoadDependence` takes a numeric vector $\alpha(n_i)$; `setClassDependence` and `setJointDependence` take a function handle of the joint population and a required per-class peak rate. The product-form column indicates whether the BCMP recurrence of [19] is preserved. A scalar handle that reads only the total n_i is equivalent to `lldscaling` and should be declared through `setLoadDependence`.

Per-class dependence and tabulated scalings

The class-dependence function of §3.1.6 may also be made *per class*: instead of returning a single scaling shared by every class, $\beta_i(n_i)$ may return the vector $[\beta_{i,1}(n_i), \ldots, \beta_{i,R}(n_i)]$, so that each class r receives its own scaling at the same joint population $n_i = [n_{i,1}, \ldots, n_{i,R}]$. This is the chain-dependent service rate $\mu_{r,i}(n_i)$ of Sauer [73], and it is what allows a station to serve different classes at genuinely different rates in the same state, rather than merely at different points of a shared function.

Earlier releases exposed this through separate *limited joint dependence* (LJD) and *limited joint class dependence* (LJCD) lookup tables. Those are now subsumed by the handle forms: a scaling that is constant across classes is the scalar (or all-equal) case, and a tabulated dependence is obtained by wrapping the table in the handle. Three fields remain (Table 3.11): `lldscaling` (dependence on the total station population), `cdscaling` (product-form per-class dependence, §3.1.6), and `jdscaling` (non-product-form joint dependence). A per-class *tabulated* scaling whose entry r reads only its own marginal is product form and belongs in `cdscaling`; a table that reads foreign marginals is joint and belongs in `jdscaling`.

A dependence obtained from measurements, or defined only over a bounded population range, is expressed by having the handle index a table and clamp the population to the range it was tabulated on:

```
% Two-class model with cutoffs [2, 2]: the table has (2+1)*(2+1)=9 entries
cutoffs = [2, 2];
scalingClass1 = [1.0, 1.0, 0.9, 1.0, 0.8, 0.7, 0.9, 0.7, 0.5];
scalingClass2 = [1.0, 1.0, 0.9, 1.0, 0.8, 0.7, 0.9, 0.7, 0.5];
% beta_i(n) returns [beta_{i,1}(n), beta_{i,2}(n)]; ljd_linearize maps the
% clamped population vector to the linearized table index
beta = @(n) [scalingClass1(ljd_linearize(min(n,cutoffs), cutoffs)), ...
             scalingClass2(ljd_linearize(min(n,cutoffs), cutoffs))];
% peakRatePerClass is the per-class peak of the tabulated scaling
peakRatePerClass = [max(scalingClass1), max(scalingClass2)];
queue.setClassDependence(beta, peakRatePerClass);
```

Here $\text{idx} = 1 + n_1 + n_2(N_1{+}1) + n_3(N_1{+}1)(N_2{+}1) + \cdots$ is the linearized population index used by `ljd_linearize`, with $n_r \in [0, N_r]$; populations beyond the cutoffs are clamped by the `min`.

Per-class dependence is used by the MVA solver during the mean value analysis iteration, and by the `NC` solver, which solves such models exactly by the multichain convolution of §4.8.3 (Sauer's eq. (40) recurrence). It is useful for systems whose server efficiency depends on the mix of job classes present, such as heterogeneous workloads competing for a shared resource. Note that a rate function supplied this way is not automatically product form; Sauer gives the necessary and sufficient condition $\mu_{i,m}(n)\,\mu_{j,m}(n-e_i) = \mu_{j,m}(n)\,\mu_{i,m}(n-e_j)$ for all classes $i \neq j$, and requires that jobs of all classes can be in service simultaneously, which excludes FCFS.

At a station with limited class dependence, utilization is reported as the normalized busy-capacity fraction $U_{i,r} = T_{i,r}S_{i,r}/p_{i,r}$, where $T_{i,r}$ is the class-r throughput, $S_{i,r}$ the nominal mean service time before application of the scaling handle, and $p_{i,r}$ the *peak rate scaling* per class. The peak is declared explicitly by the user when the station is created, as the required second argument of `setClassDependence(beta,peakRatePerClass)` (a scalar is broadcast to every class), and is stored in the network structure as `cdscalingpeak` (an `nstations`×`nclasses` array). This makes class-dependent utilization follow the same $T\,S/c$ convention as ordinary multiserver stations and the $U/\max(\texttt{lldscaling})$ convention used for load-dependent stations. Requiring an explicit declaration – rather than auto-computing a lattice peak – avoids enumerating the population lattice and avoids the ambiguity, in multiclass models, of a handle peak that carries the $|n_i|/n_{i,r}$ processor-sharing cancellation factor rather than a physical service capacity.

Switchover times

In multiclass models, queueing stations alternate over time the processing of jobs of different classes. In some real-world situations, overheads may arise when a server needs to be reconfigured to process a different class of service. Such overheads are referred to as *switchover times*.

LINE supports switchover times in queueing stations. For example, to configure the overhead to begin processing *jobclass2* jobs after *jobclass1* we may write

```
queue.setSwitchover(jobclass1, jobclass2, Exp(1));
```

A special case arises when the station uses (deterministic) polling scheduling. In this situation, the switchover time specifies the time for moving from the input buffer of class i to the input buffer of class $i+1$ mod R, where R is the total number of input classes to the queue. Using this fact, we need only to specify the switchover time after each class, e.g., the switchover time to begin processing *jobclass2* jobs after *jobclass1* is set as

```
queue.setSwitchover(jobclass1, Exp(1));
```

Impatience (customer abandonment)

LINE supports customer impatience through configurable patience distributions. When a job arrives at a queue, it draws a patience time from the specified distribution. If the job has not entered service before its patience expires, it reneges (abandons) the queue and is routed to the next station or sink. Impatience can be configured at queue-level or class-level:

```
queue.setPatience(jobclass, Exp(0.5))  % Mean patience = 2.0
jobclass.setPatience(Det(5.0))  % Fixed timeout = 5.0 everywhere
```

When both global and queue-specific patience are configured, the queue-specific setting takes precedence. Impatience supports all standard distributions except modulated processes (BMAP, MAP, MMPP2).

Retrial queues

LINE supports retrial queues, where customers that find all servers busy upon arrival are redirected to an orbit and retry after a random delay. This mechanism is common in telecommunication systems, call centers, and cloud computing platforms where blocked requests are not dropped but instead reattempt access after a backoff period.

A retrial queue is configured by calling `setRetrial` on a queue node, specifying the retrial delay distribution and optionally a maximum number of retrial attempts. Setting maximum attempts to -1 (the default) allows unlimited retries. Additionally, orbit impatience can be configured using `setOrbitImpatience`, causing customers in the orbit to abandon after a random time if they have not been served.

```
queue.setRetrial(jobclass, Exp(0.5));           % Unlimited retries, exp delay
queue.setRetrial(jobclass, Erlang(0.3, 2), 3);  % Max 3 attempts, Erlang delay
queue.setOrbitImpatience(jobclass, Exp(0.008));  % Orbit abandonment rate
```

The retrial mechanism in LINE models a BMAP/PH/N/N bufferless queue, where the queue capacity equals the number of servers. Arriving customers that find all servers busy are sent to a virtual orbit, from which they independently retry at a per-customer rate α. The analysis is based on a level-dependent quasi-birth-death (QBD) process that tracks the number of customers in orbit across truncated levels. The underlying algorithm supports BMAP arrivals, phase-type service distributions, orbit impatience (per-customer abandonment rate γ), and batch rejection probabilities. Retrial queues are analyzed by the `MAM` solver through the level-dependent QBD described above, and by the `CTMC` and `SSA` solvers, which represent the orbit population as an explicit state component. The retrial delay distribution must not be a modulated process (BMAP, MAP, MMPP2).

The matrix-analytic route additionally requires every arrival and service process of the retrial station to admit a Markovian (D_0, D_1) representation. A deterministic distribution, a trace replayer, or a non-homogeneous rate schedule has no such representation and is rejected with an explicit error instead of being passed to the QBD engine; use `CTMC`, `SSA` or `LDES` for those models. The orbit truncation level is chosen adaptively: the QBD is solved at increasing levels until the stationary mass at the top level falls below the target tolerance, so heavy-tailed service (large squared coefficient of variation) no longer truncates the orbit prematurely. The level can still be pinned explicitly through the `MaxLevel` option when a fixed truncation is wanted.

First-class orbit declaration. The idiom above requires the modeller to know that a retrial station is made bufferless by setting its capacity equal to the number of servers. The method `setOrbit` declares the same station directly: it removes the waiting room, installs the retrial delay, and records the retrial policy and the orbit capacity in a single call. Its signature is `setOrbit(class, retrialDistribution, policy, maxOrbit)`, where `policy` is `RetrialPolicy.LINEAR` (default), under which each orbiting job retries at its own rate ν so that the aggregate retrial rate in an orbit of n jobs is $n\nu$ (the classical Falin-Templeton retrial queue), or `RetrialPolicy.CONSTANT`, under which the orbit retries as a whole at rate ν whenever it is non-empty, modelling a single retrial controller shared by the orbit. The argument `maxOrbit` is the orbit capacity: -1 (default) leaves the orbit unbounded, while a non-negative value makes the orbit finite, so that a job finding it full is lost. The current configuration is read back with `getOrbit`.

```
queue.setNumberOfServers(1);
queue.setOrbit(jobclass, Exp(1.0));  % Unbounded orbit, linear policy
queue.setOrbit(jobclass, Exp(1.0), RetrialPolicy.CONSTANT, 20);  % Finite orbit
[nu, policy, maxOrbit] = queue.getOrbit(jobclass);
```

Note that the finite orbit declared by `maxOrbit` is a property of the model, not a truncation of the solution: it changes the loss probability seen by the arriving jobs, whereas the QBD truncation level discussed above is an accuracy control that leaves the model unchanged.

Orbit metric. Since a retrial station has no waiting room, its queue-length metric `QLen` reports the whole station population, that is, the jobs in service plus the jobs orbiting. The mean orbit length is reported directly by `getAvgOrbit`, which returns an (*nstations* $\times$ *nclasses*) matrix whose entries are zero at stations that are not retrial queues, and by the labelled variant `getAvgOrbitTable`. The in-service population is obtained from the station throughput by Little's law applied to the servers alone, $E[\text{in service}] = X\,E[S]$, which holds for any service distribution and any number of servers, so the reported orbit length is exact whenever `QLen` and `Tput` are.

```
solver = SolverMAM(model);
ON = solver.getAvgOrbit();  % Mean orbit length per (station, class)
solver.getAvgOrbitTable()
```

Server breakdowns

A station whose server is unreliable is declared with `setBreakdown` on a queue node. The server then alternates between an UP and a DOWN status: while up it fails after the failure distribution, while down it is restored after the repair distribution. This is the native form of the failure-repair model, in which the solver expands the joint (queue, server status) chain, as opposed to the random-environment construction of Chapter 7, which places the UP and DOWN behaviours in separate stages of an ensemble.

The semantics are as follows. The failure clock runs whenever the server is up, whether or not a job is in service, so a station can fail while idle. Arrivals are unaffected by the server status and keep queueing, subject to the station capacity, while the server is down. A job in service when the server fails is not lost: it stays at the station and, service being memoryless in the supported case, resumes on repair. An optional third argument gives the service distribution used while the server is down, either a single distribution applied to every class or a per-class list; omitting it means that the server does not serve at all while down, which is the usual breakdown model. The configuration is read back with `getBreakdown`.

```
queue.setBreakdown(Exp(0.0001), Exp(0.1));  % Fail at 1e-4, repair at 0.1
queue.setBreakdown(Exp(0.0001), Exp(0.1), Exp(0.5));  % Degraded service if down
```

Only exponential failure and repair distributions are currently expanded into the joint chain; a non-exponential failure or repair process needs its own phase in that chain and is rejected with an explicit error rather than silently approximated, in which case the random-environment ensemble of Chapter 7 remains available. Breakdowns are registered as the `Breakdown` language feature, so a solver that does not build the joint chain rejects the model instead of solving it breakdown-free; the always-up solution would understate the queue length and overstate the throughput by the whole unavailability. The `CTMC` solver supports the feature and yields both the steady-state and, through its transient interface, the time-dependent solution.

Balking

Balking models customer behaviour where an arriving job inspects the state of the queue and decides whether to join or to leave the system immediately. Unlike reneging (see 3.1.6), where a job joins the queue and abandons after waiting too long, a balking customer never enters the queue at all. Typical applications include web users that close a tab when they see a long buffering bar, callers that hang up when an automated message reports a long expected wait, and admission-control schemes that reject requests on the basis of an instantaneous load measurement.

A balking policy is configured per class on a queue node by calling `setBalking`, passing a `BalkingStrategy` constant and a list of thresholds. Each threshold is a triple `(minJobs, maxJobs, probability)`: when the queue length falls in the closed integer range $[\texttt{minJobs}, \texttt{maxJobs}]$ at the time of arrival, the customer balks with the specified probability. Thresholds are evaluated in order and the first matching range determines the balking probability; an unbounded upper limit is expressed as ∞ (MATLAB), `Integer.MAX_VALUE` (Java/Kotlin) or `float('inf')` (Python).

Three strategies are supported, listed in Table 3.12.

Table 3.12: Balking strategies available in LINE

Strategy	Description
`BalkingStrategy.QUEUE_LENGTH`	Balking decision is taken from the current queue length. The threshold list assigns a balking probability to each range of queue lengths.
`BalkingStrategy.EXPECTED_WAIT`	Balking decision is based on the expected waiting time, estimated from the current queue length and the per-class service rate. The customer balks when the estimated wait exceeds a tolerance threshold.
`BalkingStrategy.COMBINED`	Both queue-length and expected-wait conditions are evaluated; the customer balks when either condition triggers (logical OR). Useful when modelling customers that are sensitive to both visible queue length and perceived waiting time.

The following example configures a single class to balk with probability 0.3 when the queue holds 5 to 10 jobs, and with probability 1 (certain refusal) when 11 or more jobs are present.

and `Drop` apply to per-station capacity in all four codebases, while `BAS`/`BBS`/`ReServiceOnRejection` are blocking semantics typically used in conjunction with finite buffers and routing to downstream finite-capacity destinations. The `Retrial` and `RetrialWithLimit` values are normally not set explicitly: they are assigned automatically when `setRetrial` is called on a queue node (see 3.1.6).

Table 3.13: Drop strategies available in LINE

Strategy	Description
`DropStrategy.Queue`	Default policy. Jobs that find the station full wait in an external (logical) waiting queue until capacity becomes available. Selected with `DropStrategy.Queue` or by passing `false` to the boolean overload of `setDropRule` on `Region`.
`DropStrategy.Drop`	Jobs that find the station full are rejected and leave the system permanently. Selected with the MATLAB constant `DropStrategy.DROP` or by passing `true` to the boolean overload of `setDropRule` on `Region`.
`DropStrategy.BlockingAfterService` (BAS)	Blocking-after-service. A job completing service waits at the server until the downstream destination has free capacity, blocking that server in the meantime. Selected via `DropStrategy.BAS` (MATLAB constant).
`DropStrategy.BlockingBeforeService` (BBS)	Blocking-before-service. A job is held back from entering service while the downstream destination is full, so that no service capacity is consumed for jobs that cannot complete. Selected via `DropStrategy.BBS` (MATLAB constant).
`DropStrategy.ReServiceOnRejection`	Re-service on rejection. A job rejected by a full downstream station is returned to the originating server and served again. Selected via `DropStrategy.RSRD` (MATLAB constant).
`DropStrategy.Retrial`	Job moves to a virtual orbit and retries after a random delay; the number of retry attempts is unlimited. Set automatically by `setRetrial(jobclass, delay)`.
`DropStrategy.RetrialWithLimit`	As `Retrial`, but with a finite cap on the number of attempts; once the cap is exhausted the job is dropped. Set automatically by `setRetrial(jobclass, delay, maxAttempts)`.

The next listing assigns different drop rules to two classes sharing the same finite-capacity queue: jobs of the first class are rejected on overflow, while jobs of the second class are admitted to the system-level waiting queue.

```
queue.setDropRule(class1, DropStrategy.DROP);
queue.setDropRule(class2, DropStrategy.Queue);
```

The same `setDropRule` API is exposed by the `Region` class for finite capacity regions (see 3.1.7); on `Region` the boolean overload `setDropRule(jobclass, true)`/`setDropRule(jobclass, false)` is also accepted, with `true` mapping to `DropStrategy.Drop` and `false` to `DropStrategy.Queue`. Solver support for the more advanced strategies depends on the chosen backend: simulators (`JMT`, `LDES`, `SSA`) handle the full range of blocking semantics, while analytical solvers typically require `Drop` or `Queue`.

Temporal dependent processes

It is sometimes useful to specify the statistical properties of a *time series* of service or inter-arrival times, as in the case of systems with short- and long-range dependent workloads. When the model is stochastic, we refer to these as situations where one specifies a *process*, as opposed to only specifying the *distribution* of

the service or inter-arrival times. In LINE processes include the 2-state Markov-modulated Poisson process (`MMPP2`) and empirical traces read from files (`Replayer`).

For the latter, LINE assumes that empirical traces are supplied as text files (ASCII), formatted as a column of numbers. Once specified, the `Replayer` object can be used as any other distribution. This means that it is possible to run a simulation of the model with the specified trace. However, analytical solvers will require tractable distributions from the `Markovian` class.

3.1.7 Finite capacity regions

A Finite Capacity Region (FCR) provides a mechanism to constrain the total number of jobs that can simultaneously reside within a designated group of stations in a queueing network. Unlike per-station buffer limits, which restrict occupancy at individual queues, an FCR imposes a collective constraint across multiple stations, limiting the aggregate number of jobs across all stations in the region. When the region reaches its capacity, incoming jobs are either dropped or blocked depending on the configured drop strategy. This modeling capability is particularly useful for representing systems with admission control policies, finite buffers spanning multiple processing stages, resource constraints that affect groups of servers, or loss networks where jobs are rejected when the system lacks sufficient capacity.

An FCR is created using the `addRegion` method of the `Network` class, which takes as input a cell array or list of stations belonging to the region. The maximum occupancy across all stations in the region is set via `setGlobalMaxJobs`, which specifies the total number of jobs allowed in the region. The drop behavior for each class is configured using `setDropRule`, which determines whether arriving jobs are dropped when the region is at capacity.

```
fcr = model.addRegion({queue1, queue2});
fcr.setGlobalMaxJobs(K);
fcr.setDropRule(jobclass1, true);  % true = drop jobs when region full
```

Two primary drop strategies are available. The `DropStrategy.Drop` strategy causes jobs to be permanently lost when they arrive at a full region, modeling systems where arrivals are rejected or abandoned. The `DropStrategy.Queue` strategy causes jobs to wait in a virtual queue until capacity becomes available, modeling systems with admission control or backpressure mechanisms. In addition to global capacity constraints, per-class capacity limits can be specified within a region to enforce class-specific occupancy restrictions.

A special case of interest is an FCR containing a single queue with dropping enabled. This configuration behaves identically to an M/M/1/K queue, where K is the maximum capacity including both jobs in service and jobs waiting in the buffer. This equivalence provides a convenient way to model loss systems and finite-buffer queues. The examples `fcr_mm1kdrop`, `fcr_mm1waitq`, `fcr_constraints`, and `fcr_oqndrop` demonstrate various applications of finite capacity regions, including single-queue finite buffers, multi-station regions with different drop policies, and open queueing networks with job loss.

Linear admission constraints. Beyond global and per-class job caps, an FCR can be constrained by an arbitrary set of linear admission inequalities of the form $A\mathbf{n} \leq \mathbf{b}$, where $\mathbf{n} = (n_1, \ldots, n_R)$ is the per-class job count vector inside the region, n_r denoting the number of class-r jobs currently residing in the region and R the number of job classes. Constraints are configured via `setLinearConstraints(A, b)` on the region object. The example `fcr_lincon` illustrates this feature.

```
fcr = model.addRegion({queue1, queue2});
fcr.setLinearConstraints([1 1; 2 1], [10; 15]); % n1+n2<=10, 2n1+n2<=15
```

The constraint pair is accessible programmatically via `getLinearConstraints()` on the region object, which returns the pair (A, b), and is surfaced in the network struct as `sn.regionlincon{f}`, a single cell whose row f holds A and b as its two elements (`sn.regionlincon{f,1}`$= A$ and `sn.regionlincon{f,2}`$= b$). Linear admission constraints are honoured by the `LDES` simulator across all four codebases.

Two further per-region metrics are available with the `JMT` and `LDES` solvers, and are useful when classes consume unequal shares of a pooled resource, as in cache or storage models. `FCRWeight` (`MetricType.FCRWeight`) is the time-average total weight of the in-region jobs, summing the per-class weight set with `setClassWeight`; with the default weight 1 it coincides with the number of customers. `FCRMemOcc` (`MetricType.FCRMemOcc`) is the time-average memory occupation, summing the per-class footprint set with `setClassSize`.

In `JMT` the two map to the native region measures `FCR Capacity` and `FCR Memory`; `LDES` accumulates the same time-weighted quantities directly. They are returned per region and per class as `WeightNfcr` and `MemOccNfcr`, whose row sums are the region aggregates.

3.1.8 Places and transitions

LINE supports the specification of stochastic Petri net (SPN) models through dedicated `Place` and `Transition` node types, providing an alternative modeling paradigm to queueing networks. In an SPN, the system state is represented by the distribution of tokens across places, and the system dynamics are governed by transition firings that consume tokens from input places and produce tokens in output places. This formalism is particularly well-suited for modeling systems with synchronization, resource sharing, and complex control flow that may be difficult to express naturally in the queueing network framework.

A `Place` is a stateful node that holds tokens, which correspond to jobs in the queueing network interpretation. Places can have capacity limits and initial token populations. A `Transition` is a node that defines one or more firing modes, each representing a distinct way the transition can fire. Each mode has its own timing distribution, which governs the delay between enabling and firing, its own enabling condition, which specifies the number of tokens required in each input place, and its own firing outcome, which specifies the number of tokens deposited in each output place when the transition fires. The ability to define multiple modes on a single transition allows modeling of complex state-dependent behaviors and routing decisions.

Internal sections of `Place` and `Transition` nodes. Like every node in LINE, a `Place` and a `Transition` are decomposed into three internal sections, listed in Table 3.14, that determine how tokens are accepted, processed and dispatched. The sections are instantiated by the constructors and are not manipulated directly; they are described here because they name the entries appearing in the `nodeparam` tables of §3.1.2.

Table 3.14: Internal sections of `Place` and `Transition` nodes

of the Petri net formalism. The examples `spn_basic_open`, `spn_twomodes`, `spn_inhibiting`, and `spn_closed_fourplaces` demonstrate various applications of stochastic Petri nets, including open and closed systems, multi-mode transitions for state-dependent routing, inhibiting arcs for resource constraints, and complex token flows.

3.1.9 Signals

LINE supports the modeling of networks with signals through the `Signal` class. Signals extend traditional queueing networks by introducing negative customers, which have the distinctive property of removing positive customers from queues upon arrival rather than joining them. This mechanism provides a natural way to model phenomena such as job cancellations, cache invalidations, service interrupts, or resets in computing systems. The interaction between positive and negative customers creates rich dynamics that capture feedback control, load regulation, and catastrophic events in networked systems.

A signal is defined using the `Signal` constructor, which takes a model reference, a descriptive name, and a signal type that specifies the behavior of the signal class. The `SignalType.NEGATIVE` type creates a negative customer class with the semantics that each arrival at a queue removes one positive customer from that queue. If the queue is empty when a negative customer arrives, the negative customer has no effect and is simply absorbed.

```
negClass = Signal(model, 'Negative', SignalType.NEGATIVE);
source.setArrival(negClass, Exp(lambda_neg));
```

In this example, a negative customer class is created and assigned an arrival process at a source node. When negative customers arrive at queues in the network, they reduce the queue populations by removing positive customers. The routing of negative customers is specified using the same mechanisms as for ordinary job classes, allowing complex patterns of cancellation and control.

Models with signals can be analyzed using the `MAM` solver with the INAP, INAPPLUS, or INAPINF methods, which exploit the product-form structure of certain signal network configurations through the RCAT (Reversed Compound Agent Theorem) framework. Under specific conditions on the arrival and service processes, networks with signals admit product-form solutions that allow efficient exact analysis without state-space explosion. This capability makes it possible to analyze large-scale signal networks with multiple positive and negative customer classes while maintaining tractability. For open signal networks with birth-death or catastrophe structure, the INAPINF method [55] solves each isolated component on its infinite state space by a matrix-geometric decomposition, avoiding the truncation error of the finite-state INAP iteration.

```
options = SolverOptions();
options.method = 'inap';
solver = MAM(model, options);
[QN, UN, RN, TN] = solver.getAvg();
```

The example `ag_gnetwork` demonstrates the construction and analysis of a network with signals featuring both positive customers and negative arrivals, illustrating how signal classes interact with ordinary job classes to produce non-trivial equilibrium behavior. Networks with signals are particularly useful for modeling systems with dynamic load control, self-regulation, or reset mechanisms, where the conventional queueing network paradigm does not naturally capture the feedback effects of cancellations and interruptions.

3.2 Internals

In this section, we discuss the internal representation of the `Network` objects used within the LINE solvers. By applying changes directly to this internal representation it is possible to considerably speed up the sequential evaluation of models.

3.2.1 Representation of the model structure

For efficiency reasons, once a user requests to solve a `Network`, LINE calls internally generate a static representation of the network structure using the `refreshStruct` function. This function returns a representation object that is then passed on to the chosen solver to parameterize the analysis.

The representation used within LINE is the `NetworkStruct` class, which describes an extended multi-class queueing network with class-switching and acyclic phase-type (APH) service times. APH generalizes known distributions such as Coxian, Erlang, Hyper-Exponential, and Exponential. The representation can be obtained as follows

```
sn = model.getStruct()
```

The field-by-field reference of `NetworkStruct` is given in the LINE internals manual. It covers the static properties that carry the parameterization given at model definition time, the computed properties derived from them by `refreshStruct`, and the `nodeparam` entries of `Cache` and `Transition` nodes, tabulated for all three codebases side by side (`LINE-internals.pdf`, appendix *The `NetworkStruct` reference*). The remainder of this section describes how the structure is organized rather than what each field contains.

3.2.2 Nodes, stations, and stateful nodes

As shown in the tables, internally to LINE there is an explicit differentiation between properties of nodes, stations, and stateful nodes. This distinction has impact in particular over routing and class-switching mechanisms, and also allows solvers to better differentiate between different kinds of nodes.

The three categories form a nested hierarchy. Every element of a `Network` is a *node*, the most general category, uniquely identified by a node index in $1, \ldots,$ `nnodes`. A *stateful node* is a node that carries local state variables needed to determine its behaviour; stateful nodes are separately indexed in $1, \ldots,$ `nstateful` and flagged by the `isstateful` field. A *station* is a stateful node in which jobs may additionally spend time (sojourn), so that it contributes to queueing and response-time metrics; stations are separately indexed in

$1, \ldots,$ `nstations` and flagged by the `isstation` field. Hence every station is a stateful node, and every stateful node is a node, but not conversely. For example, a `Sink` is a plain stateless node, since jobs cannot sojourn in it and no state variable is required; a `Cache` is a stateful node but not a station, since it must track the cached items yet passage through it is instantaneous; and a `Queue` is a station, since jobs sojourn in its buffer and servers. A `Source` is likewise treated as a station, because the inter-arrival time of jobs from the external world is modelled as time spent in the source, although this time is not counted as part of the system response time. Table 3.15 classifies the nodes available in `Network` models according to these three categories.

Table 3.15: Classification of `Network` nodes as stateful nodes and stations

Node	Stateful	Station	Category
`Source`	yes	yes	station
`Queue`	yes	yes	station
`Delay`	yes	yes	station
`Join`	yes	yes	station
`Place`	yes	yes	station
`Router`	yes	no	stateful node
`Cache`	yes	no	stateful node
`Transition`	yes	no	stateful node
`ClassSwitch`	no	no	stateless node
`Fork`	no	no	stateless node
`Logger`	no	no	stateless node
`Sink`	no	no	stateless node

In some cases, one may want to access some properties of nodes that are contained in `NetworkStruct` fields that are however referenced by station or stateful node index. To help this and similar situations, the `NetworkStruct` class also provides static methods to quickly convert the indexing of nodes, stations, and stateful nodes, which is used in referencing its data structures:

- `nodeToStateful`
- `nodeToStation`
- `stationToNode`
- `stationToStateful`
- `statefulToNode`

As an example, we can determine the portion of the `nodevisits` field that refers to stateful nodes in chain $c = 1$ as follows

```
c = 1;
V = zeros(sn.nstateful,1);
sn = model.getStruct(); % NetworkStruct object
```

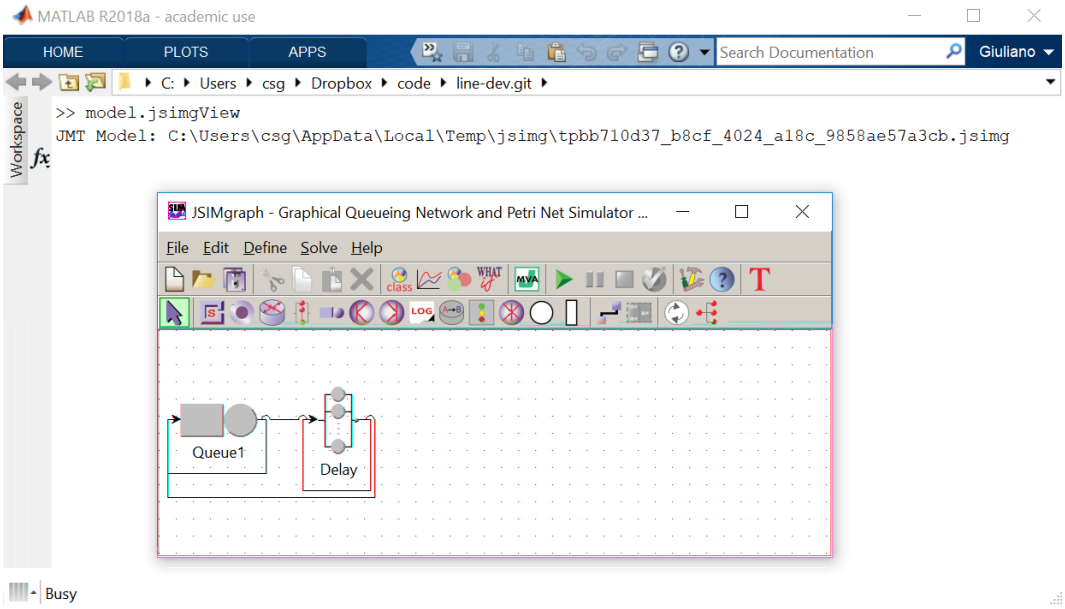

Figure 3.1: `jsimgView` function

```
for ind=1:sn.nnodes
    if sn.isstateful(ind)
        isf = sn.nodeToStateful(ind);
        V(isf,1) = sn.nodevisits{c}(ind,2);
    end
end
```

3.3 Debugging and visualization

JSIMgraph is the graphical simulation environment of the JMT suite. LINE can export models to this environment for visualization purposes using the command

```
model.jsimgView
```

An example is shown in Figure 3.1. Using a related function, `jsimwView`, it is also possible to export the model to the JSIMwiz environment, which offers a wizard-based interface.

Another way to debug a LINE model is to transform it into a graph object, e.g.

```
G = model.getGraph();
plot(G,'EdgeLabel',G.Edges.Weight,'Layout','Layered')
```

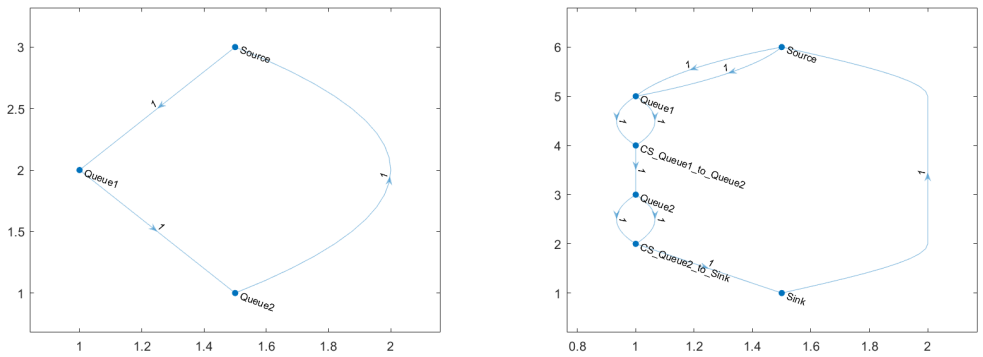

Figure 3.2: `getGraph` function: station topology (left) and node topology (right) for a 2-class tandem queueing network with class-switching.

plots a graph of the network topology in term of stations only. In a similar manner, the following variant of the same command shows the model in terms of nodes, which corresponds to the internal representation within LINE.

```
[~,H] = model.getGraph();
plot(H,'EdgeLabel',H.Edges.Weight,'Layout','Layered')
```

The latter example is invoked by default if we type

```
plot(model)
```

which also adds automatic node coloring to highlight the class-switch nodes.

Figure 3.2 shows the difference between the two commands for an open queueing network with two classes and class-switching. Weights on the edges correspond to routing probabilities. In the station topology on the left, note that since the Sink node is not a station, departures to the Sink are drawn as returns to the Source. The node topology on the right, illustrates all nodes, including `ClassSwitch` nodes that are automatically added by LINE to apply the class-switching routing strategy. Double arcs between nodes indicate that both classes are routed to the destination.

Furthermore, the graph properties concisely summarize the key features of the network

```
>> G.Nodes
ans =
  2x5 table
      Name            Type            Sched     Jobs     ClosedClass1
    ________    ________________    ______    _____    ____________
    'Delay'     'Delay'             'inf'      5            1
    'Queue1'    'Queue'             'ps'       0            2
>> G.Edges
```

```
ans =
  3x4 table
          EndNodes           Weight    Rate        Class
    ____________________     ______    ____    _____________

    'Delay'     'Delay'       0.7        1     'ClosedClass1'
    'Delay'     'Queue1'      0.3        1     'ClosedClass1'
    'Queue1'    'Delay'         1      0.5     'ClosedClass1'
```

Here, `Edge.Weight` is the routing probability between the nodes, whereas `Edge.Rate` is the service rate of the node appearing in the first column under `EndNodes`.

Chapter 4

Analysis methods

4.1 Performance metrics

As discussed earlier, LINE supports a set of steady-state and transient performance metrics. Table 4.1 summarizes the definition of the associated random variables. For each metric, one or more analysis types may be available, which are extensively discussed in the next sections.

4.2 Steady-state analysis

4.2.1 Station average performance

LINE decouples network specification from its solution, allowing to evaluate the same model with multiple solvers. Model analysis is carried out in LINE according to the following general steps:

Step 1: Definition of the model. This proceeds as explained in the previous chapters.

Step 2: Instantiation of the solver(s). A solver is an instance of the `Solver` class. LINE offers multiple solvers, which can be configured through a set of common and individual solver options. For example,

```
solver = JMT(model);
```

returns a handle to a simulation-based solver based on JMT, configured with default options.

Step 3: Solution. Finally, this step solves the network and retrieves the concrete values for the performance indexes of interest. This may be done as follows, e.g.,

```
% QN(i,r): mean queue-length of class r at station i
QN = solver.avgQLen()
% UN(i,r): utilization of class r at station i
UN = solver.avgUtil()
```

Table 4.1: Performance metrics

Metric	**Acronym**	**Description**
Queue-length	`QLen`	Number of jobs of class r (or chain-c) residing at a node i
Utilization	`Util`	Utilization of class-r (or chain-c) jobs at node i, scaled in [0,1] for multi-server nodes, equal to `QLen` at infinite server nodes
Response time	`RespT`	Time that a class-r (or chain-c) jobs spends for a single visit at node i
Residence time	`ResidT`	Cumulative time that a class-r (or chain-c) jobs spends across all visits at node i
Arrival rate	`ArvR`	Arrival rate of class-r (or chain-c) jobs at node i
Throughput	`Tput`	Throughput of class-r (or chain-c) jobs at node i
System Response time	`SysRespT`	For an open chain c, this is the time from leaving the source to arriving at the sink for *any* class in the chain. For a closed chain c, this is the interval of time between two successive visits to the reference station in any two *completing classes* within the chain.
System Throughput	`SysTput`	For an open chain c, this is the departure rate towards the sink for *any* class in the chain. For a closed chain c, this is the rate of arrival of *completing classes* in the chain at the reference station.
System queue length	`SysQLen`	Total mean number of jobs in chain c across all stations. For a closed chain, equal to the total number of jobs in the chain. For an open chain, related to `SysRespT` and `SysTput` through Little's law: $\texttt{SysQLen}(c) = \texttt{SysTput}(c) \cdot \texttt{SysRespT}(c)$.
Fork-join queue length	`FJQLen`	Number of jobs present within a fork-join scope, counting from the moment they are dispatched at the fork until synchronization is completed at the join. Measured per fork-join pair.
Fork-join response time	`FJRespT`	Time from fork dispatch to completion of synchronization at the join, measuring the end-to-end latency of the parallel section across all spawned branches.
Marginal queue-length distribution	`ProbMarg`	Probability mass function $P(n_{i;r} = k)$ of the number of class-r jobs at station i, marginalized over all other classes and stations. Returned by `getProbMarg`.

```
% RN(i,r): mean response time of class r at station i (per visit)
RN = solver.avgRespT()
% TN(i,r): mean throughput of class r at station i
TN = solver.avgTput()
% AN(i,r): mean arrival rate of class r at station i
AN = solver.avgArvR()
% WN(i,r): mean residence time of class r at station i (summed on visits)
WN = solver.avgResidT()
```

Alternatively, all the above metrics may be obtained in a single method call as

```
[QN,UN,RN,TN,AN,WN] = solver.avg()
```

In the methods above, LINE assigns station and class indexes (e.g., i, r) in order of creation in order of creation of the corresponding station and class objects. However, large models may be easier

per-class queue-lengths inside the chain). Use this view to study models where the chain decomposition is more meaningful than the per-class breakdown.

- `getAvgNodeTable`: returns one row per (*node*, *class*) pair. Unlike the station-level table, the node-level table also reports metrics for non-station nodes such as `Source`, `Sink`, `Router`, `ClassSwitch`, `Fork`, and `Join`; this is the recommended view to inspect, for example, throughputs at sinks or arrival rates at routers.

- `getAvgSysTable`: returns one row per chain, with the system-level response time `SysRespT` and system-level throughput `SysTput` as observed at the reference station of the chain. This view summarizes end-to-end performance and is the natural complement of `avgSys`.

- `getAvgCacheTable`: returns the cache hit/miss summary for each `Cache` node, with one row per (*node*, *class*) reporting the hit and miss probabilities, plus per-list (per-level) rows where an exact algorithm resolves the per-level occupancy. This is the recommended view to inspect caching behaviour.

- `getAvgOrbitTable`: returns the mean orbit length of each retrial station, with one row per (*station*, *class*) pair reporting the `Orbit` column. Because a retrial station is bufferless, its `QLen` counts the jobs in service together with the jobs orbiting; this view separates the orbiting population, which would otherwise have to be recovered as `QLen` minus the in-service population. The table is empty for models without retrial stations (see 3.1.6).

- `getAvgItemTable`: returns the item-level cache occupancy for each `Cache` node, with one row per (*node*, *item*, *list*) reporting the list capacity `ListCap` and the steady-state probability `Prob` that the item resides in that list. The per-item distribution is taken from the exact product-form cache algorithm, so the per-list columns of each row form a proper distribution; the table is empty for solvers that do not compute a per-item distribution.

Method aliases. MATLAB and Java expose progressively shorter aliases for the methods used most often during interactive analysis, all forwarding to the same canonical implementation. Table 4.2 lists the most common ones. The same short-alias convention (`get`-prefix dropped, then `Table` shortened to `T`, then reduced to initials) extends to the per-chain and per-node-chain variants of Section 4 (e.g., `getAvgQLenChain` → `avgQLenChain`) and to the ensemble-solver methods used by `LN`, `UQ`, and `ENV` (e.g., `getEnsembleAvgTables` → `ensembleAvgTables`).

4.2.2 Station response time distribution

Several solvers (`CTMC`, `FLD`, `JMT`, `NC`, and `MAM`) support the computation of response time distributions for individual classes through the `getCdfRespT` function. The function returns the response time distribution for every station and class. For example, the following code plots the cumulative distribution function at steady-state for class 1 jobs when they visit station 2:

Table 4.2: Common method aliases

Canonical method	Short alias(es)	Description
`getAvgTable`	`avgTable`, `avgT`, `aT`	Average metrics table per (station, class); default view, matches `avg`
`getAvgNodeTable`	`avgNodeTable`, `avgNodeT`, `aNT`	Average metrics table per (node, class); includes non-station nodes
`getAvgChainTable`	`avgChainTable`, `avgChainT`, `aCT`	Average metrics table per (station, chain)
`getAvgNodeChainTable`	`avgNodeChainTable`, `avgNodeChainT`, `aNCT`	Average metrics table per (node, chain)
`getAvgSysTable`	`avgSysTable`, `avgSysT`, `aST`	System-level response time and throughput per chain
`getAvg`	`avg`	`QLen`, `Util`, `RespT`, `Tput`, `ArvR`, `ResidT` matrices per (station, class)
`getAvgChain`	`avgChain`	Chain-aggregated counterpart of `avg`
`getAvgSys`	`avgSys`	System-level counterpart of `avg`
`getCdfRespT`	`cdfRespT`	Response time cumulative distribution function
`getTranAvg`	`tranAvg`	Transient (time-dependent) average trajectories
`getProb`	`prob`	Steady-state marginal state probability
`getMomentTable`	`momentT`, `mT`	Queue-length (and FCFS response-time) moments per (station, class)
`getMomentChainTable`	`momentChainT`, `mCT`	Queue-length moments per (station, chain)
`getMomentStationTable`	`momentStationT`, `mST`	Total-queue-length moments per station
`getSensitivityTable`	`sensitivityT`, `sT`	Derivatives of mean measures w.r.t. service rate per (station, class)
`getAvgCacheTable`	`cacheAvgT`, `aCaT`	Cache hit/miss summary per (node, class)
`getAvgItemTable`	`itemAvgT`, `aIT`	Per-item cache hit probabilities
`getAvgOrbitTable`	`orbitAvgT`, `aOT`	Retrial-orbit occupancy per (node, class)
`getAvgLossTable`	`lossAvgT`, `aLT`	Loss rate and loss ratio per (station, class)
`getAvgRegionLossTable`	`regionLossAvgT`, `aRLT`	Loss rate and loss ratio per finite-capacity region (LDES, JMT)

```
solver = FLD(model);
FC = solver.cdfRespT();
plot(FC{2,1}(:,2),FC{2,1}(:,1)); xlabel('t'); ylabel('Pr(RespT<t)');
```

4.2.3 Station queue-length and response-time moments

The tables seen so far report mean values. For product-form networks LINE can also return the *higher moments* of the queue lengths and of the response times, exactly and without simulation, through three methods that differ only in how they group the job classes.

- `getMomentTable` gives one row per station and class, reporting `QLen`, `QLenVar`, `QLenSCV` and, at FCFS stations, `RespT`, `RespTVar` and `RespTSCV`.
- `getMomentChainTable` gives one row per station and chain, for the queue length of all the classes of a chain taken together.
- `getMomentStationTable` gives one row per station, for the total queue length $Q_i = \sum_r n_{i,r}$.

All three are the same computation under a different grouping of the classes. Scaling the service demands of a set T of classes at a station and differentiating the mean queue length produces the moments of $\sum$

percent. The algorithm is a property of the solver object, set at construction, and not an argument of the table call; the Linearizer approximates the per-station totals only, so a per-chain grouping of more than one chain requires an exact method.

The queue-length moments are available for closed, mixed and open models; the third moment and the response-time moments require a closed model. A second output argument returns the underlying structure, which carries the full covariance matrices, including the covariances between different stations that the tables do not display.

These three tables are marginal: every row describes one queue length in isolation. The *joint* law of two or more queue lengths, and any moment family other than the raw and central ones, are reached instead from the API layer, with `pfqn_qlen_joint_moments`. Given the service demands, the population and the think times of a closed product-form model, it returns the whole joint array over any chosen set of (station, class) coordinates: the survival probabilities, the binomial, factorial, upward-factorial, raw, central and cumulant moments, together with the mean vector and the covariance matrix. The entry of multi-order $(1, 1)$ of the cumulant array is the covariance of the two coordinates, so a cross-station or cross-class covariance that the tables do not display is read off directly. The computation is exact and rests on normalizing constants only, so any of the algorithms of the `NC` solver can supply them; the moment families themselves are converted by the `moment` package described in the API reference.

4.2.4 Loss rates and loss ratios

In models where jobs can be dropped – a station with finite capacity, blocking, or reneging (impatience) – it is useful to quantify how much offered traffic is actually lost. The `getAvgLossTable` method returns one row per station-class pair that receives offered traffic, reporting the offered arrival rate (`ArvR`), the carried throughput (`Tput`), the loss rate and the loss ratio. The loss rate is the difference `ArvR−Tput`, i.e. the rate of jobs of that class dropped at the station, and the loss ratio is `LossRate/ArvR`, the fraction of offered jobs that are lost. For a single finite-capacity queue the loss ratio equals the blocking probability. Only pairs with `ArvR`> 0 are listed, which excludes the source (whose offered arrival rate is zero); a lossless station has `ArvR=Tput` and therefore a zero loss rate and ratio.

For example, an `M/M/1/2` queue with arrival rate $\lambda = 1.5$ and service rate $\mu = 1$ loses a fraction $\rho^2(1-\rho)/(1-\rho^3) \approx 0.4737$ of its arrivals, where $\rho = \lambda/\mu$:

```
queue.setCapacity(2);
lossTable = CTMC(model).getAvgLossTable   % alias: aLT, lossAvgT
```

The loss rate is derived from the same offered `ArvR` and carried `Tput` columns reported by `getAvgTable`, so the two tables are always consistent. A solver that does not model dropping returns `ArvR=Tput` and hence a zero loss rate.

A quorum (partial) fork-join is the one exception to the `ArvR−Tput` identity. When a join fires as soon as a quorum of k out of n forked siblings has arrived, the remaining $n - k$ siblings are discarded, but the join throughput is counted in *parent* units and its offered rate in *sibling* units, so `ArvR−Tput` at the join measures the synchronisation multiplicity, not the loss. On the join row the loss rate is therefore the directly measured

sibling-discard rate (from the LDES simulation), and the loss ratio is that rate divided by the offered sibling rate ArvR. A standard (STD) join discards nothing and reports a zero loss rate.

Loss that occurs at a finite-capacity *region* boundary (a region with the DROP rule, refusing entry when full) is reported separately by getAvgRegionLossTable, one row per region and class, with the same ArvR/Tput/LossRate/LossRatio columns; here the offered rate is the carried throughput plus the region drop rate. The LDES simulation measures region drops directly (tracking refusals at the region boundary), while JMT reconstructs the region drop rate by flow balance: it exposes no region drop measure and its region throughput is the carried (admitted) rate, so the offered rate is computed as the rate at which stations outside the region route jobs across its boundary (a quantity unaffected by the drop), and the drop rate is offered minus carried. The table is empty for solvers that report neither. Because JMT reports region metrics as class aggregates, the aggregate region loss is exact while its split across classes inherits the same approximation used for the other region metrics. For a region that wraps a single station the region drop also surfaces in that station's row of getAvgLossTable, so the two views agree.

4.2.5 System average performance

LINE also allows users to analyze models for end-to-end performance indexes such a system throughput or system response time. However, in models with class switching the notion of system-wide metrics can be ambiguous. For example, consider a job that enters the network in one class and departs the network in another class. In this situation one may attribute system response time to either the arriving class or the departing one, or attempt to partition it proportionally to the time spent by the job within each class. In general, the right semantics depends on the aim of the study.

LINE tackles this issue by supporting only the computation of system performance indexes *by chain*, instead than by class. In this way, since a job switching from a class to another remains by definition in the same chain, there is no ambiguity in attributing the system metrics to the chain. The solver functions avgSys and avgSysTable return system response time and system throughput per chain as observed: (i) upon arrival to the sink, for open classes; (ii) upon arrival to the reference station, for closed classes. The system queue length (SysQLen) is related to these two quantities through Littleś law and can be obtained as their product; for closed chains, SysQLen is simply equal to the fixed number of jobs in the chain.

In some cases, it is possible that a chain visits multiple times the reference station before the job completes. This also affects the definition of the system averages, since one may want to avoid counting each visit as a completion of the visit to the system. In such cases, LINE allows users to specify which classes of the chain can complete at the reference station. For example, in the code below we require that a job visits reference station 1 twice, in classes 1 and 2, but completes at the reference station only when arriving in class 2. Therefore, the system response time will be counted between successive passages in class 2.

```
class1 = ClosedClass(model, 'ClosedClass1', 1, queue, 0);
class2 = ClosedClass(model, 'ClosedClass2', 0, queue, 0);

class1.completes = false;
```

```
P = cell(2); % 2-classes model
P{1,1} = [0,1; 0,0]; % routing within class 1 (no switching)
P{1,2} = [0,0; 1,0]; % routing from class 1 into class 2
P{2,1} = [0,0; 1,0]; % routing within class 2 (no switching)
P{2,2} = [0,1; 0,0]; % routing from class 2 into class 2

model.link(P);
```

Note that the `completes` property of a class always refers to the reference station for the chain.

4.3 Specifying states

In some analyses it is important to specify the state of the network, for example to assign the initial position of the jobs in a transient analysis. We thus discuss the support in LINE for state modeling.

4.3.1 Station states

We begin by explaining how to specify a state s_0 for a station. For example, it is not supported for shortest job first (`SchedStrategy.SJF`) scheduling, in which state must include the service time samples for the jobs and it is therefore a continuous quantity.

Suppose that the network has R classes and that service distributions are phase-type, i.e., that they inherit from `Markovian`. Let K_r be the number of phases for the service distribution in class r at a given station. Then, we define three types of state variables:

- c_j: class of the job waiting in position $j \leq b$ of the buffer, out of the b currently occupied positions. If $b = 0$, then the state vector is indicated with a single empty element $c_1 = 0$.
- k_j: service phase of the job waiting in position $j \leq b$ of the buffer, out of the b currently occupied positions.
- n_r: total number of jobs of class r in the station
- b_r: total number of jobs of class r in the station's buffer
- s_{rk}: total number of jobs of class r running in phase k in the server

Here, by phase we mean the number of states of a distribution of class `Markovian`. If the distribution is not Markovian, then there is a single phase. With these definitions, the table below illustrates how to specify in LINE a valid state for a station depending on its scheduling strategy. There, S is the number of servers of the queueing station. All state variables are non-negative integers. The `SchedStrategy.EXT` policy is used for the `Source` node, which may be seen as a special station with an infinite pool of jobs sitting in the buffer and a dedicated server for each class $r = 1, ..., R$.

States can be manually specified or enumerated automatically. LINE library functions for handling and generating states are as follows:

Table 4.3: State descriptors for Markovian scheduling policies

Sched. strategy	Station state vector	State condition
EXT	$[\texttt{Inf}, s_{11}, ..., s_{1K_1}, ..., s_{R1}, ..., s_{RK_R}]$	$\sum_k s_{rk} = 1, \forall r$
FCFS, FCFSPRIO, LCFS	$[c_b, ..., c_1, s_{11}, ..., s_{1K_1}, ..., s_{R1}, ..., s_{RK_R}]$	$\sum_r \sum_k s_{rk} = S$
LCFSPR, LCFSPI	$[c_b, k_b, ..., c_1, k_1, s_{11}, ..., s_{1K_1}, ..., s_{R1}, ..., s_{RK_R}]$	$\sum_r \sum_k s_{rk} = S$
SEPT, SIRO	$[b_1, ..., b_R, s_{11}, ..., s_{1K_1}, ..., s_{R1}, ..., s_{RK_R}]$	$\sum_r \sum_k s_{rk} = S$
PS, DPS, GPS, INF	$[s_{11}, ..., s_{1K_1}, ..., s_{R1}, ..., s_{RK_R}]$	None

- `State.fromMarginal`: enumerates all states that have the same marginal state $[n_1, n_2, ..., n_R]$.
- `State.fromMarginalAndRunning`: restricts the output of `State.fromMarginal` to states with given number of running jobs, irrespectively of the service phase in which they currently run.
- `State.fromMarginalAndStarted`: restricts the output of `State.fromMarginal` to states with given number of running jobs, all assumed to be in service phase $k = 1$.
- `State.fromMarginalBounds`: similar to `State.fromMarginal`, but produces valid states between given minimum and maximum value of the number of resident jobs.
- `State.toMarginal`: extracts marginal statistics from a state, such as the total number of jobs in a given class that are running at the station in a certain phase.

Note that if a function call returns an empty state (`[]`), this should be interpreted as an indication that no valid state exists that meets the required criteria. Often, this is because the state supplied in input is invalid.

Example

We consider the example network in `cqn_multiserver`. We look at the state of station 2 (the `Queue1` station), which is a multi-server FCFS station with 3 servers. There are 4 classes all having exponential service times except class 2 that has Erlang-2 service times. We are interested to states with 2 running jobs in class 1 and 1 in class 2, and with 2 jobs, respectively of classes 3 and 4, waiting in the buffer. We can automatically generate this state space, which we store in the `space` variable, as:

```
>> cqn_multiserver;
>> space = State.fromMarginalAndRunning(model,node{2},[2,1,1,1],[2,1,0,0])
space =
     4     3     2     1     0     0     0
     4     3     2     0     1     0     0
     3     4     2     1     0     0     0
     3     4     2     0     1     0     0
```

Here, each row of `space` corresponds to a valid state. The argument `[2,1,1,1]` gives the number of jobs in the node for the 4 classes, while `[2,1,0,0]` gives the number of running jobs in each class. This station has

four valid states, differing on whether the class-2 job runs in the first or in the second phase of the Erlang-2 and on the relative position of the jobs of class 3 and 4 in the waiting buffer.

To obtain states where the jobs have just started running, we can instead use

```
>> space = State.fromMarginalAndStarted(model,node{2},[2,1,1,1],[2,1,0,0])
space =
     4     3     2     1     0     0     0
     3     4     2     1     0     0     0
```

We see that the above state space restricted the one obtained with `State.fromMarginalAndRunning` to states where the job in class 1 is always in the first phase.

If we instead remove the specification of the running jobs, we can use `State.fromMarginal` to generate all possible combinations of states depending on the class and phase of the running jobs. In the example, this returns a space of 20 possible states.

```
>> space = State.fromMarginal(model,node{2},[2,1,1,1])
space =
     4     3     2     1     0     0     0
     4     3     2     0     1     0     0
     4     2     2     0     0     1     0
     4     1     1     1     0     1     0
     4     1     1     0     1     1     0
     3     4     2     1     0     0     0
     3     4     2     0     1     0     0
     3     2     2     0     0     0     1
     3     1     1     1     0     0     1
     3     1     1     0     1     0     1
     2     4     2     0     0     1     0
     2     3     2     0     0     0     1
     2     1     1     0     0     1     1
     1     4     1     1     0     1     0
     1     4     1     0     1     1     0
     1     3     1     1     0     0     1
     1     3     1     0     1     0     1
     1     2     1     0     0     1     1
     1     1     0     1     0     1     1
     1     1     0     0     1     1     1
```

Assigning a prior to an initial state

It is possible to assign the initial state to a station using the `setState` function on that station's object. LINE offers the possibility to specify a prior probability on the initial states, so that if multiple states have a non-zero prior, then the solver will need to solve an independent model using each one of those initial states, and then carry out a weighting of the results according to the prior probabilities. The default is to assign a probability 1.0 to the *first* specified state. The functions `setStatePrior` and `getStatePrior` can be used

to check and change the prior probabilities for the initial states specified for a station or stateful node.

4.3.2 Network states

A collection of states that are valid for each station is not necessarily valid for the network as a whole. For example, if the sum of jobs of a closed class exceeds the population of the class, then the network state would be invalid. To identify these situations, LINE requires to specify the initial state of a network using functions supplied by the `Network` class. These functions are `initFromMarginal`, `initFromMarginalAndRunning`, and `initFromMarginalAndStarted`. They require a matrix `n` with elements (i, r) specifying the total number of resident class-r jobs at node i and the latter two require a matrix `s` with elements (i, r) with the number of running (or started) class-r jobs at node i. The user can also manually verify if the supplied network state is going to be valid using `State.isValid`.

It is also possible to request LINE to automatically identify a valid initial state, which is done using the `initDefault` function available in the `Network` class. This is going to select a state where:

- no jobs in open classes are present in the network;
- jobs in closed classes all start at their reference stations;
- the servers at each reference station are occupied by jobs of in class order, i.e., jobs in the firstly created class are assigned to the server, then spare server are allocated to jobs in the second class, and so forth;
- service or arrival processes are initialized in phase 1 for each job;
- if the scheduling strategy requires it, jobs are ordered in the buffer by class, with the firstly created class at the head and the lastly created class at the tail of the buffer.

The `initFromAvgQLen` method is a wrapper for `initFromMarginal` to initialize the system as close as possible to the average steady-state distribution of the network. Since averages are typically not integer-valued, this function rounds the average values to the nearest integer and adjusts the result to ensure feasibility of the initialization.

4.3.3 Initialization of transient classes

Because of class-switching, it is possible that a class r with a non-empty population at time $t = 0$ becomes empty at some positive time $t' > t$ without ever being visited again by any job. LINE allows users to place jobs in transient classes and therefore it will not trigger an error in the presence of this situation. If a user wishes to prohibit the use of a class at a station, it is sufficient to specify that the corresponding service process uses the `Disabled` distribution.

Certain solvers may incur problems in identifying that a class is transient and in setting to zero its steady-state measures. For example, the `JMT` solver uses a heuristic whereby a class is considered transient if it has fewer samples than jobs initially placed in the corresponding chain the class belongs to. For such classes, `JMT` will set the values of steady-state performance indexes to zero.

4.3.4 State space generation

As discussed in Example 3, the state space of a model can be obtained by either invoking `model.stateSpace()` or `solver.stateSpace()` on an instant of the CTMC solver, where the latter returns the state space cached during the solution of the CTMC.

LINE supports two state space generation methods, configurable using the option `options.config.state_space_gen` of the CTMC solver. Details may be found in Table 5.2.3.

4.3.5 State probability analysis

Besides mean values, solvers that build an explicit state representation can return the stationary probability of the network states. Table 4.4 lists the available accessors, which differ in whether the state is global or local to a station, and in whether the phase information of non-exponential service is retained or aggregated away.

Table 4.4: State probability accessors

Method	Returns
`getProbSys`	Probability of each global state of the network
`getProbSysAggr`	The same, aggregated by class over all stations, so that phase and buffer-order detail is summed out
`getProb`	Probability mass function of the state of one station
`getProbAggr`	The same, aggregated by class: the joint per-class queue-length distribution at that station
`getProbMarg`	Marginal queue-length distribution $P(n_{i;r} = k)$ of a single class r at station i, other classes marginalized out (`MVA`, `MAM`). Under `NC` the method takes only a station and returns the class-aggregated distribution at it

Reading these outputs requires the state encoding of Section 4.3.4: a state is a vector of the number of jobs of each class at each station, extended with the phase of each in-service job when the service process is non-exponential, and with the buffer order under FCFS scheduling, which is why FCFS state spaces are larger than the corresponding processor-sharing or infinite-server ones. The example files `statepr_aggr` and `statepr_allprobs_fcfs` show how to aggregate over classes and stations and how to enumerate all states of a FCFS network, from which quantities such as $P(\text{station empty})$ or $P(\text{total population} > k)$ follow by summation.

For example, the marginal queue-length distribution of class `cclass1` at `queue1` can be obtained as follows:

```
solver = MVA(model);
Pmarg = solver.getProbMarg(queue1, cclass1);
% Pmarg(k+1) = P(n_{queue1, cclass1} = k)
```

Transient state probabilities

For solvers that expose the underlying state space, LINE also provides transient counterparts of the steady-state probability methods, giving the distribution over network states at a finite time horizon (controlled

by the `timespan` option). These are useful for analyzing warm-up behavior, ramp-up of arrival rates, or short-term reliability questions. Per-station transient probabilities are returned by `getTranProb` (full state) and `getTranProbAggr` (aggregated by class), while system-level joint transient probabilities are returned by `getTranProbSys` and `getTranProbSysAggr`. All four methods are currently supported by the `CTMC` solver. `JMT` additionally provides the aggregated form (`getTranProbAggr`).

The following example computes the per-class transient probabilities at `queue1` from the default initial state up to time $t = 1$:

```
options = SolverOptions();
options.timespan = [0, 1];
solver = CTMC(model, options);
[Pi_t, SSnode_a] = solver.getTranProbAggr(queue1);
% Pi_t(k,:) gives the probability vector at the k-th time step
% SSnode_a lists the corresponding aggregated marginal states
```

4.4 Transient analysis

So far, we have seen how to compute steady-state average performance indexes, which are given by

$$E[n] = \lim_{t \to +\infty} E[n(t)]$$

where $n(t)$ is an arbitrary performance index, e.g., the queue-length of a given class at time t.

We now consider instead the computation of the quantity $E[n(t)|s_0]$, which is the *transient average* of the performance index, conditional on a given initial system state s_0. Compared to $n(t)$, this quantity averages the system state at time t across all possible evolutions of the system from state s_0 during the t time units, weighted by their probability. In other words, we observe all possible stochastic evolutions of the system from state s_0 for t time units, recording the final values of $n(t)$ in each trajectory, and finally average the recorded values at time t to obtain $E[n(t)|s_0]$.

4.4.1 Computing transient averages

The computation of transient metrics proceeds similarly to the steady-state case. We first obtain the handles for transient averages:

```
model = gallery_cqn(2) % closed single class queueing network with 2 stations
[Qt,Ut,Tt] = model.getTranHandles();
```

After solving the model, we will be able to retrieve *both* steady-state and transient averages as follows

```
[QNt,UNt,TNt] = CTMC(model,'timespan',[0,1]).tranAvg(Qt,Ut,Tt);
plot(QNt{1,1}.t, QNt{1,1}.metric)
```

The transient average queue-length at node i for class r is stored within `QNt` in row i and column r.

Note that the above code specifies a maximum time t for the output time series. This can be done using the `timespan` solver option. This applies also to average metrics. In the following example, the first model is solved at steady-state, while the second model reports averages at time $t = 1$ after initialization

```
>> CTMC(model).avgTable
State space size: 21 states.
CTMC analysis (method: default) completed in 0.187991 seconds.
ans =
  3x7 table
    Station    JobClass     QLen       Util      RespT     ResidT      Tput
    _______    ________    _______    _______    ______    ______    _______
    Delay 1     Class1     0.62015    0.62015         2         2    0.31008
    Queue 1     Class1      1.3018    0.62015    4.1984    4.1984    0.31008
    Queue 2     Class1       3.078    0.93023    9.9267    9.9267    0.31008
>> CTMC(model,'timespan',[0,1]).avgTable
State space size: 21 states.
CTMC analysis completed in 0.118398 sec
ans =
  3x7 table
    Station    JobClass     QLen       Util      RespT    ResidT      Tput
    _______    ________    _______    _______    _____    ______    ________
    Delay 1     Class1      3.0615     3.0615        0         0      1.5308
    Queue 1     Class1      1.6775    0.84395        0         0     0.42197
    Queue 2     Class1     0.26098    0.23425        0         0    0.078083
```

4.4.2 First passage times into stations

When the model is in a transient, the average state seen upon arrival to a station changes over time. That is, in a transient, successive visits by a job may experience different response time distributions. The function `getTranCdfRespT`, implemented by `JMT` offers the possibility to obtain this distribution given the initial state specified for the model. As time passes, this distribution will converge to the steady-state one computed by solvers equipped with the function `getCdfRespT`.

However, in some cases one prefers to replace the notion of response time distribution in transient by the one of *first passage time*, i.e., the distribution of the time to complete the *first visit* to the station under consideration. The function `getTranCdfPassT` provides this distribution, assuming as initial state the one specified for the model, e.g., using `setState` or `initDefault`. This function is available in `FLD` and `JMT` and has a similar syntax as `getCdfRespT`. A steady-state passage CDF is currently provided only by `FLD` (as `getCdfPT`); the generic `getCdfPassT` dispatcher is reserved for future implementations and currently raises an error on all solvers.

4.5 Advanced metrics

This section collects analyses that go beyond the standard per-station and system-wide performance indexes. They include metrics specific to fork-join structures and finite capacity regions, the Age of Information for status-update systems, and user-defined reward models evaluated over the underlying Markov chain.

4.5.1 Fork-join metrics

Two metrics report a fork-join section as a whole rather than station by station. `FJQLen` is the mean number of jobs in the section, counted from dispatch at the fork to synchronization at the join, and `FJRespT` is the corresponding mean sojourn time, that is the time for all sibling tasks of a job to complete and be joined. Both therefore aggregate over every station lying between a fork and its join, which the ordinary per-station metrics cannot express. They are appended to the `getAvgTable` output whenever the model contains a fork-join section, and are computed by the `MVA`, `JMT` and `LDES` solvers.

4.5.2 Finite capacity region queue length

When a station is part of a Finite Capacity Region (FCR) with `Queue` drop strategy, the reported queue length (`QLen`) includes both jobs currently inside the station and jobs that are blocked waiting to enter the FCR. This semantic ensures consistency with blocking queueing network models where blocked jobs are logically associated with their destination queue. For example, if an FCR with capacity 2 contains a queue with 2 jobs, and 1 additional job is blocked waiting to enter, the reported queue length will be 3. This convention is consistent with JMT's handling of FCR blocking.

4.5.3 Age of Information analysis

`FLD` supports the computation of Age of Information (AoI) metrics for single-queue open systems with capacity constraints. AoI measures the freshness of information in status update systems, defined as the time elapsed since the last successfully received update was generated. When the model has a valid AoI topology (Source $\rightarrow$ Queue $\rightarrow$ Sink with one open class, one queue, one server, and queue capacity 1 or 2), the `mfq` method automatically switches to the AoI MFQ analysis and computes exact AoI and Peak AoI distributions. The `getAvgAoI` method returns the mean and variance of both AoI and Peak AoI, while `getCdfAoI` returns their cumulative distribution functions:

```
model = Network('AoI_Example');
source = Source(model, 'Source');
queue = Queue(model, 'Queue', SchedStrategy.FCFS);
queue.setNumberOfServers(1);
queue.setCapacity(1);  % bufferless
sink = Sink(model, 'Sink');
jobclass = OpenClass(model, 'Updates');
source.setArrival(jobclass, Exp(1));
```

```
queue.setService(jobclass, Erlang(2, 2));
model.link(Network.serialRouting(source, queue, sink));

solver = FLD(model, 'method', 'mfq');
[AoI, PAoI, aoiTable] = solver.getAvgAoI();
[AoI_cdf, PAoI_cdf] = solver.getCdfAoI();
```

The solver supports FCFS, LCFS, and preemptive LCFS scheduling strategies, mapping them to appropriate preemption/replacement policies for the underlying MFQ analysis.

4.5.4 Reward models

LINE supports the definition of custom reward functions on network models, enabling the computation of application-specific performance metrics beyond the standard measures of queue length, utilization, and response time. Rewards associate a numerical value with each state of the underlying continuous-time Markov chain, allowing analysts to define metrics that capture domain-specific objectives such as energy consumption, revenue, or composite performance indicators. The reward framework supports both steady-state analysis, which computes the expected reward in equilibrium, and transient analysis, which tracks the evolution of reward values over time. This feature is available through the `CTMC` solver, which constructs the Markov chain representation of the network and evaluates reward functions over the state space.

Reward functions are defined using the `setReward` method on the `Network` object. This method takes two arguments: a string name for the reward, which is used to identify it in the results, and a function that maps a state accessor object to a scalar reward value. The state accessor provides the method `at`, which takes a node and a job class as arguments and returns the number of jobs of that class at that node in the current state.

```
model.setReward('QueueLength', @(state) state.at(queue, oclass));
```

This example defines a reward equal to the number of jobs of class `oclass` at the `queue` node. More complex reward functions can be constructed by combining state information from multiple nodes and classes, or by applying arbitrary mathematical operations to state variables.

To simplify common use cases, LINE provides pre-built reward templates through the `Reward` class. The `Reward.utilization` method creates a reward function that returns an indicator variable, taking the value 1 when the specified node is serving a job of the specified class and 0 otherwise. The `Reward.blocking` method creates a reward function that returns an indicator for buffer-full conditions, taking the value 1 when the node's buffer is at capacity and 0 otherwise.

```
model.setReward('Utilization', Reward.utilization(queue, oclass));
model.setReward('BlockingProb', Reward.blocking(queue));
```

The complete set of pre-built templates is summarized in Table 4.5. Each template is a factory that returns a reward function suitable for `setReward`.

Table 4.5: Reward function templates in the `Reward` class (MATLAB method names shown, with the Python-native name in parentheses where it differs). In Java, the same metrics are written directly as `RewardFunction` lambdas.

Template	Arguments	Description
`queueLength` (`queue_length`)	`node`, [`jobclass`]	Number of jobs at `node`, totalled across classes or restricted to `jobclass` when supplied.
`utilization`	`node`, [`jobclass`]	Fraction of busy servers at `node`, computed as $\min(\text{jobs}, \texttt{nservers})$; per-class when `jobclass` is supplied.
`blocking`	`node`	Indicator equal to 1 when `node` is at capacity (buffer full) and 0 otherwise.
`custom`	`fn`	Returns the user-defined reward function `fn` unchanged, documenting the intent that it is a custom reward.

Once reward functions are defined on the model, they can be evaluated by creating a `CTMC` solver instance and calling the appropriate analysis methods. For steady-state analysis, the `getAvgReward` method returns a vector of expected reward values in equilibrium, along with a cell array of reward names. For transient analysis, the `getTranReward` method returns, for each defined reward, the time-indexed trajectory of the expected reward $E[r(X(t))]$, together with the vector of time points and the reward names. Here $X(t)$ is the system state at time t and the expectation is taken over the transient distribution of the Markov chain, starting from the model's initial state; a finite `timespan` must therefore be supplied to the solver.

```
solver = CTMC(model, 'timespan', [0, T]);
[R, names] = solver.getAvgReward();      % steady-state expected rewards
[Rt, t, names] = solver.getTranReward(); % transient E[r(X(t))] trajectories
```

The transient trajectories can be plotted to visualize how each reward relaxes from its initial value towards the steady-state equilibrium. For example, the following snippet overlays the transient expected rewards on their steady-state values, producing Figure 4.1.

```
[Rt, ~, names] = CTMC(model, 'timespan', [0, 5]).getTranReward();
[R, ~] = CTMC(model).getAvgReward();
figure; hold on;
for r = 1:numel(names)
    plot(Rt{r}.t, Rt{r}.metric);   % transient E[r(X(t))]
    yline(R(r), '--');             % steady-state reference
end
xlabel('Time t'); ylabel('E[r(X(t))]'); legend(names);
```

The reward framework is particularly useful for defining objective functions in optimization problems, tracking custom performance indicators that combine multiple system states, or analyzing transient behavior of derived metrics. The examples `rewardModel_mm1k`, `rewardModel_templates`, and `rewardModel_aggregation` demonstrate various applications of reward models, including simple state-based rewards, the use of template functions, and the aggregation of reward values across multiple nodes or classes.

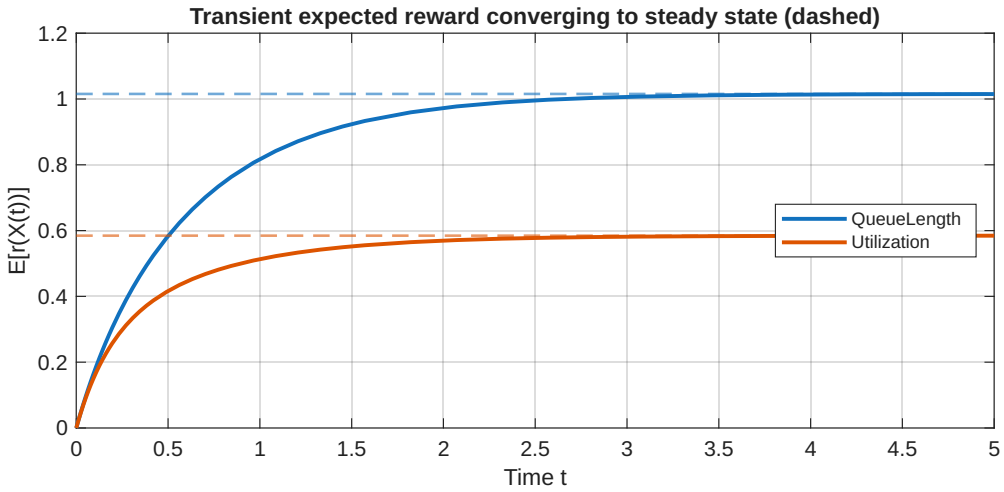

Figure 4.1: Transient expected reward $E[r(X(t))]$ for the queue length and utilization of an $M/M/1/3$ queue, converging over time to the steady-state values (dashed) computed by `getAvgReward`.

4.6 Sample path analysis

Sample path analysis differs in nature from the transient analysis of Section 4.4. Transient analysis returns the *expected* value $E[X(t)]$ of a performance index, obtained by averaging the system state at time t over all possible stochastic evolutions from the initial state, weighted by their probability. Sample path analysis instead returns a single *realization* of the process $X(t)$, that is, one particular trajectory produced by a specific sequence of random events, rather than an average over trajectories. Repeating the analysis with a different random seed yields a different sample path, and averaging many such realizations at a fixed time t recovers the transient average $E[X(t)]$.

With LINE is also possible to obtain a particular sample path from the stochastic process underlying the queueing network. The following functions are available for this purpose:

- `sample`: returns a data structure including the time-varying state of a given stateful node, labelled with information about the events that changed the node state.
- `sampleAggr`: returns a data structure similar to the one provided by `sample`, but where the state is aggregate to count the number of jobs in each class at the node.
- `sampleSys`: similar to the `sample` function, but returns the state of every stateful node in the model.
- `sampleSysAggr`: similar to the `sampleAggr` function, but returns the aggregated state of every stateful node in the model.

It is worth noting that the `JMT` solver only supports `sampleAggr` since the simulator does not offer a simple way to extra detailed data such as phase change information in the service process. This information is instead available with the `SSA` solver.

For example, the following command extract a sample path consisting of 10 samples for a $APH(2)/M/1$ queue:

```
>> model=gallery_aphm1; samplePath = JMT(model).sampleAggr(model.nodes{2},1e1); ...
[samplePath.t, samplePath.state]
JMT Model: /tmp/jsimg/tpa52644b6_17ce_46ee_9c09_8da7c2d1d3c6.jsimg
ans =
         0         0
    0.3255    1.0000
    0.3562         0
    1.1478    1.0000
    1.2220         0
    1.8727    1.0000
    2.6076    2.0000
    2.6862    3.0000
    3.4737    2.0000
    3.7663    3.0000
    4.2110    2.0000
```

In the example, `samplePath.t` refers to the time since initialization at which the node 2 (here the $APH(2)/M/1$ queueing station) enters the state shown in the second column.

If we repeat the same experiment with the `SSA` solver and using the `sampleSys` function, we now have the full state space of the model, including both the source and the queueing station:

```
>> model=gallery_aphm1; samplePath = SSA(model).sampleSys(10); [samplePath.t, ...
samplePath.state{1}, samplePath.state{2}]

SSA analysis (method: default) completed in 0.011636 seconds.
ans =
    0.4776       Inf    1.0000         0         0         0
    0.5843       Inf         0    1.0000         0         0
    1.2043       Inf    1.0000         0         0    1.0000
    1.2952       Inf    1.0000         0         0         0
    1.3196       Inf         0    1.0000         0         0
    1.5724       Inf    1.0000         0         0    1.0000
    1.5975       Inf         0    1.0000         0    1.0000
    1.6129       Inf         0    1.0000         0         0
    1.6226       Inf    1.0000         0         0    1.0000
    2.5289       Inf    1.0000         0         0         0
```

4.7 Sensitivity analysis and numerical optimization

Frequently, performance and reliability analysis requires to change one or more model parameters to see the sensitivity of the results or to optimize some goal function. In order to do this efficiently, we have discussed before the internal representation of the `Network` objects used within the LINE solvers. By applying changes directly to this internal representation it is possible to considerably speed-up the sequential evaluation of models as discussed next.

4.7.1 Performance sensitivities

Before resorting to a numerical sweep, it is worth asking whether the derivative of a performance measure can be obtained directly. For product-form models it can, exactly and at negligible cost, since the derivatives of the mean measures are available in closed form from the same recursion that produces the means themselves. Outside that class the derivative is still available, but as a finite difference on the predictions of the solver at hand.

The `getSensitivityTable` method returns one row per station and class, giving the derivative of that row's mean measures with respect to that station-class service *rate*: `dTput_dRate`, `dRespT_dRate`, `dQLen_dRate` and `dUtil_dRate`. A second output returns the underlying structure.

```
sensTable = SolverMVA(model).getSensitivityTable
sensTable = SolverCTMC(model).getSensitivityTable('scheme','central')
```

Two branches produce these numbers, and the method selects between them. The *analytic* branch is exact to machine precision and costs less than a single extra solve. Closed product-form networks are differentiated through the mean value analysis recursion; open product-form networks admit a closed-form expression, because their stations decouple and only the own service rate of a station affects its measures. Rate derivatives follow from the demand derivatives by the chain rule, $\partial(\cdot)/\partial\mu_{i,r} = -(L_{i,r}/\mu_{i,r})\,\partial(\cdot)/\partial L_{i,r}$, since $L_{i,r} = v_{i,r}/\mu_{i,r}$. It is reached only from the solvers that evaluate that recursion, `SolverMVA` and `SolverNC`, and only for single-server stations plus an optional delay, mixed open-and-closed models being excluded.

The *finite-difference* branch covers everything else, and in particular the non-product-form models that the analytic expressions cannot reach. The service process at a station and class is rate-scaled by $1 + h$, a pure time scaling that preserves the shape of the distribution and moves only its rate, the calling solver is re-run with its own options, and the difference quotient is formed. The derivative is therefore the derivative of *that* solver's mapping, approximations included: a finite difference on an approximate mean value analysis reports how that approximation responds, not how the exact model does. The cost is $1 + MR$ solves for the default forward scheme and $2MR$ for the central one.

The branch is chosen by the `method` option, which is `'auto'` by default and may be forced to `'exact'` or `'fd'`; `'exact'` raises an error where it does not apply, rather than silently approximating. The relative step is set by `step`, whose default is 10^{-4}, and the scheme by `scheme`, either `'forward'` or `'central'`. The branch actually taken is reported alongside the table.

With a simulation solver the difference quotient is only as good as the pairing of the two runs: the base and perturbed runs reuse the same options and hence the same seed, so that common random numbers cancel most of the Monte Carlo error, and the default step is raised to 10^{-2}, since a variance of order σ^2/h^2 would otherwise swamp the signal. A seed left unset is fixed before the sweep starts rather than drawn per run, so the whole sweep is a deterministic function of it and repeating it reproduces the table exactly. What remains is a sampling error that the pairing does not remove, and it decays only with the run length: on a closed two-station model the estimates sit within a few tens of percent of the analytic derivatives at $2 \cdot 10^4$ samples and within about one percent at $2 \cdot 10^5$. Read a simulated sensitivity for its sign and its ranking across stations before reading it for its value.

The analytic branch differentiates at the level of chains, not of classes, which is what a product-form model is solved at: a chain carries the population and the arrival rate, and a class is a share of it. The chain demands $D_c(i) = \sum_{r \in c} D_{i,r}$, populations and think times are formed first, the differentiated recursion is evaluated on them, and the per-class measures are recomposed from the chain ones, a class throughput at a station being $X_c v_{i,r}$ and a per-visit response time $Q_{i,r}/T_{i,r}$. Class switching is therefore supported, the whole chain population sitting on the reference class being immaterial once the aggregation is done.

A layered network is differentiated layer by layer. `SolverLN` owns no recursion of its own, so its table is the concatenation of the layer tables with a leading `Layer` column, obtained by delegating to each layer solver with the options given. A layer submodel is chain-based, the callers being modelled as switching class, which the chain aggregation above handles, so the layers normally take the analytic branch. These entries are partial derivatives *within* a layer, taken with the parameters that the fixed point produced held fixed: perturbing a host demand also moves the think times, populations and service rates of the other layers through the fixed-point map, and that indirect term is not included. They attribute a bottleneck inside a layer; they do not predict the effect of a parameter change on the solved layered model.

```
solver = SolverLN(model, @(layer) SolverMVA(layer));
sensTable = solver.getSensitivityTable
sensTable.Properties.UserData.method   % 'exact', 'fd', or 'mixed'
```

Second moments, rather than derivatives, are reported by the moment tables described in Section 4.2.3; both rest on the same differentiated recursion, since a queue-length covariance in a product-form network is itself a demand derivative of a mean queue length.

4.7.2 Symbolic analysis backend

Some analyses are exact only if carried out in symbolic form: the stationary distribution of a Markov chain as a rational function of its transition rates, the derivative of that distribution with respect to a rate, or the Jacobian of a mean-field drift. LINE delegates this work to SageMath [82], packaged as the `imperialqore/line-sage-rest` service (§2.1). One engine serves all three implementations, so a symbolic result computed from MATLAB, Java or Python is returned in the same normal form and the three can be compared.

Configuring the backend. The service is selected by `options.config.symbolic` and resolved in the following order:

1. an explicit service URL, e.g. `'http://hostname:8080'`;
2. the `LINE_SAGE_URL` environment variable;
3. a service already listening on port 8085 or 8080, identified through its `/api/v1/info` endpoint rather than by the port alone;
4. a container started by LINE on a free port, from a locally available image;
5. nothing, in which case the native engine is used.

The accepted values are `'auto'` (the default: use the native engine, and the service only where there is none), `'sage'` (always use the service), `'none'` (never use it), or a URL. A second option, `options.config.symbolic_timeout`, bounds each request on both the client and the server side; the default is 300 seconds. The bound matters because a symbolic solve grows superpolynomially in the number of states, so it is meant for small chains.

Symbolic steady-state distribution. Given the symbolic generator of §5.2.3, in which each event filtration is scaled by a symbol $x_1, \ldots, x_E$, the method below returns the stationary distribution as a vector of rational functions of those symbols, together with its numerator vector and common denominator.

```
options = SolverOptions('CTMC');
options.config.symbolic = 'sage';
solver = SolverCTMC(model, options);
[pi, num, den] = solver.getSymbolicSolution();
```

With the Symbolic Math Toolbox installed the result is returned as `sym` objects, and otherwise as character strings, so that symbolic analysis remains available without that toolbox. The same option also switches the engine used by `ctmc_solve` on a symbolic generator.

Expressions are exchanged as plain infix strings and numeric coefficients are read as exact rationals, so no rounding is introduced by the exchange. The symbols are numbered by the order in which events are enumerated, which may differ between implementations: two symbolic results should therefore be compared by substituting each event's own rate and comparing numbers, not by comparing the expressions as text.

Exact parametric sensitivities. The `getSensitivity` method accepts a third argument selecting how the derivative is taken. The default, `'fd'`, differences the whole solve and is accurate to $O(h^2)$. With `'symbolic'`, the stationary distribution is differentiated exactly with respect to each rate symbol and the chain rule

$$\frac{d\pi}{d\theta} = \sum_{e} \frac{\partial \pi}{\partial x_e} \frac{dx_e}{d\theta}$$

leaves only the rate map $x_e(\theta)$ to be differenced. That map is affine in the common cases, such as a rate set to θ or scaled by it, and the method verifies as much before using a wide difference step, so the result is exact up to floating-point evaluation. On a two-station closed model the measured error falls from $3 \cdot 10^{-11}$ to $3 \cdot 10^{-14}$.

```
param = struct('name','mu','value',2.0, ...
    'set', @(m,v) m.getNodeByName('Queue1').setService(m.getClasses{1}, Exp(v)));
[S, SS, dpi, pi] = solver.getSensitivity(param, reward, 'symbolic');
```

A parameter that reshapes an event, rather than scaling it, invalidates the chain rule; such a parameter is refused with an explanatory error rather than approximated, and `'fd'` should be used for it.

Fluid drift and Jacobian. The mean-field drift of §5.2.4 can also be exported as expressions and differentiated, which yields the Jacobian that distinguishes a stable fixed point from a limit cycle and gives the local rate of convergence, neither of which follows from integrating the ODEs.

```
options = SolverOptions('FLD');
options.method = 'matrix';
options.config.pstar = 8;          % p-norm smoothing: a differentiable drift
options.config.symbolic = 'sage';
solver = SolverFLD(model, options);
[rhs, vars] = solver.getSymbolicDrift(options);
J = solver.getJacobian(options);
```

Only a differentiable drift can be exported. The `default`, `matrix`, `closing` and `statedep` methods scale service rates by $\min(n_i, S_i)$, which has no derivative at $n_i = S_i$; these are refused, naming the offending term, instead of being answered with a one-sided derivative at the very point where the regime changes. The differentiable alternatives are the p-norm smoothing, enabled by setting `options.config.pstar`, and the `softmin` method.

4.7.3 Fast parameter update

Successive invocations of `getStruct()` will return a cached copy of the `NetworkStruct` representation, unless the user has called `model.refreshStruct()` or `model.reset()` in-between the invocations. The `refreshStruct` function regenerates the internal representation, while `reset` destroys it, together with all other representations and cached results stored in the `Network` object. In the case of `reset`, the internal data structure will be regenerated at the next `refreshStruct()` or `getStruct()` call.

The performance cost of updating the representation can be significant, as some of the structure array field require a dedicated algorithm to compute. For example, finding the chains in the model requires an analysis of the weakly connected components of the network routing matrix. For this reason, the `Network` class provides several functions to selectively refresh only part of the `NetworkStruct` representation, once the modification has been applied to the objects (e.g., stations, classes, ...) used to define the network. These functions are as follows:

Table 4.6: Functions for direct modification of `NetworkStruct` objects

Function	Description
`sn_set_service`	Sets service rate at a specific station and class. Optional parameters: squared coefficient of variation (default 1.0) and auto-refresh flag.
`sn_set_arrival`	Sets arrival rate for a class at the Source station. Automatically locates the Source and updates its rate.
`sn_set_population`	Sets the number of jobs for a closed class. Automatically recalculates `nclosedjobs`.
`sn_set_servers`	Sets the number of servers at a station.
`sn_set_priority`	Sets the priority for a class.
`sn_set_routing_prob`	Sets a routing probability between two stateful node-class pairs.
`sn_set_routing`	Sets the full routing matrix.
`sn_set_fork_fanout`	Sets fork fanout parameters.
`sn_set_service_batch`	Sets service parameters for batch processing.

- `refreshCapacity`: this function should be called after changing buffer capacities, as it updates the `capacity` and `classcapacity` fields.
- `refreshChains`: this function should be used after changing the routing topology, as it refreshes the `rt`, `chains`, `nchains`, and `visits` fields.
- `refreshPriorities`: this function updates class priorities in the `classprio` field.
- `refreshScheduling`: updates the `sched`, and `schedparam` fields.
- `refreshProcesses`: updates the `mu`, `phi`, `phases`, `rates` and `scv` fields. This function should also be called after updating the inter-arrival distribution at a `Source`.

For example, suppose we wish to update the service time distribution for class-1 at node 1 to be exponential with unit rate. This can be done efficiently as follows:

```
queue.setService(class1, Exp(1.0));
model.refreshProcesses;
```

4.7.4 Direct modification of NetworkStruct objects

For maximum performance in optimization loops, LINE provides low-level `sn_set_*` functions that directly modify `NetworkStruct` objects without requiring a `Network` object. These functions bypass the high-level API and operate directly on the internal data structure, making them ideal for sensitivity analysis and numerical optimization where many parameter variations must be evaluated quickly.

The available functions are listed in Table 4.6.

For example, suppose we wish to vary the service rate at station 1 for class 1 in an optimization loop:

```
sn = model.getStruct();  % Get NetworkStruct once
for rate = 0.5:0.1:2.0
```

```
    sn = sn_set_service(sn, 1, 1, rate);
    % Solve directly with sn
    result = solver.solve(sn);
end
```

4.7.5 Refreshing a network topology with non-probabilistic routing

The `resetNetwork` function should be used before changing a network topology with non-probabilistic routing. It will destroy by default all class switching nodes. This can be avoided if the function is called as, e.g., `model.resetNetwork(false)`. The default behavior is though shown in the next example

```
>> model = Network('model');
node{1} = ClassSwitch(model,'CSNode',[0,1;0,1]);
node{2} = Queue(model, 'Queue1', SchedStrategy.FCFS);
>> model.getNodes
ans =
  2x1 cell array
    {1x1 ClassSwitch}
    {1x1 Queue}
>> model.resetNetwork
ans =
  1x1 cell array
    {1x1 Queue}
```

As shown, `resetNetwork` updates the station indexes and the revised list of nodes that compose the topology is obtained as a return parameter. To avoid stations to change index, one may simply create `ClassSwitch` nodes as last before solving the model. This node list can be employed as usual to reinstantiate new stations or `ClassSwitch` nodes. The `addLink`, `setRouting`, and possibly the `setProbRouting` functions will also need to be re-applied as described in the previous sections.

4.7.6 Saving a network object before a change

The Network object, and its inner objects that describe the network elements, are always passed by reference. The `copy` function should be used to clone LINE objects, for example before modifying a parameter for a sensitivity analysis. This function recursively clones all objects in the model, therefore creating an independent copy of the network. For example, consider the following code

```
modelByRef = model; modelByRef.setName('myModel1');
modelByCopy = model.copy; modelByCopy.setName('myModel2');
```

Using the `getName` function it is then possible to verify that `model` has now name `myModel1`, since the first assignment was by reference. Conversely, `modelByCopy.setName` did not affect the original `model` since this is a clone of the original network.

4.8 Model adaptation

The `ModelAdapter` class provides static methods for transforming and adapting `Network` models. These methods are useful for model reduction, response time analysis, and preprocessing operations.

4.8.1 Chain aggregation

The `ModelAdapter.aggregateChains` function transforms a multi-class model with K classes organized into C chains into a model with C classes, preserving chain population, arrival rates, service demands, and routing while reducing state space when $C \ll K$. The function returns the aggregated model, an aggregation factors matrix α satisfying $\sum_{r \in c} \alpha(i, r) = 1$, and deaggregation information for transforming chain-level results back to class-level metrics.

```
[chainModel, alpha, deaggInfo] = ModelAdapter.aggregateChains(model);
```

The same transformation is reachable from the model itself, which is the recommended form:

```
[chainModel, alpha, deaggInfo] = model.aggregateChains();
```

Class switching is eliminated by the transformation, since it is internal to a chain. The aggregation is exact for product-form models and approximate otherwise, because a single service process fitted on the chain mean and squared coefficient of variation replaces the per-class ones weighted by α. Chain-level metrics are mapped back to class level with `sn_deaggregate_chain_results`, passing the chain queue lengths and utilizations as empty: splitting the chain queue length by α alone ignores the per-class service times, whereas the response-time branch rescales them by S_{ir}/S_{ic}.

4.8.2 Flow-equivalent server aggregation

The `ModelAdapter.aggregateFES` function replaces a subset of stations in a closed product-form queueing network with a single Flow-Equivalent Server (FES). The FES has state-dependent service rates where the rate for each class equals the throughput of that class in an isolated subnetwork consisting only of the aggregated stations. This technique reduces the model size while preserving exact throughput and utilization metrics.

```
[fesModel, fesStation, deaggInfo] = ModelAdapter.aggregateFES(model, {queue2, ...
queue3});
```

4.8.3 Convolution-based FES analysis

The FES station produced by `aggregateFES` stores its throughput lookup table as a *per-class dependence* function (§3.1.6). For each aggregated station the per-class throughput $X_r(n)$ is tabulated over all population vectors up to a per-class cutoff, so that the exact dependence of throughput on the joint class population is

preserved; the table is then wrapped as the handle $\beta_{i,r}(n) = X_r(n)$, i.e. Sauer's chain-dependent service rate $\mu_{r,i}(n)$ [73], and stored in the `NetworkStruct` field `cdscaling{ist}`. The population is clamped to the cutoffs the table was built on, so the rate saturates beyond the tabulated range.

When a model contains one or more class-dependent stations, the `NC` solver uses the multichain convolution algorithm (`pfqn_conv`) to compute exact normalizing constants and performance metrics, building each station factor $X_m(n)$ by Sauer's recurrence $X_m(n) = (u_{km}/\mu_{km}(n))\, X_m(n - e_k)$ [73]. The convolution is invoked automatically by the `NC` solver when a class-dependence function is detected and `method='exact'` is requested; no explicit user action is required beyond creating the FES model via `aggregateFES` with appropriate cutoffs.

4.8.4 Tagged job models

The `ModelAdapter.tagChain` function creates a tagged job model for response time distribution analysis. It isolates a single job from a specified chain by creating duplicate tagged classes, enabling the computation of per-job response time distributions using methods such as passage time analysis.

```
[taggedModel, taggedJob] = ModelAdapter.tagChain(model, chain, jobclass);
```

4.8.5 Class removal

The `ModelAdapter.removeClass` function removes a specified job class from a model, updating all station configurations, routing matrices, and class-dependent parameters accordingly. This is useful for analyzing submodels or simplifying complex multi-class networks. The removal is delegated to the nodes, each of which drops its own per-class configuration (service and arrival processes, class capacities, scheduling parameters, routing strategies, and the class-switching matrix of a `ClassSwitch`). Removing the last remaining class, or any class of a model containing a `Cache`, is rejected: the cache item state is indexed by class and cannot be re-indexed on the node alone.

Two variants are available. The model method removes the class in place, whereas `ModelAdapter` returns a copy and leaves the original model solvable, which is the form to use for ablation studies or a per-class decomposition:

```
model.removeClass(jobclass);                % in place
newmodel = ModelAdapter.removeClass(model, jobclass);  % on a copy
```

Chapter 5

Network solvers

5.1 Overview

Solvers analyze objects of class Network to return average, transient, distributions, or state probability metrics. A solver can implement one or more *methods*, which although featuring a similar overall solution strategy, they can differ significantly from each other in the way this strategy is actually implemented and on whether the final solution is exact or approximate.

A 'method' flag can be passed upon invoking a solver to specify the solution method that should be used. For example, the following invocations are identical:

```
MVA(model,'exact').avgTable()
MVA(model,'method','exact').avgTable()
opt = MVA.defaultOptions; opt.method = 'exact'; MVA(model,opt).avgTable()
```

In what follows, we describe the general characteristics and supported model features for each solver available in LINE and their methods.

Available solvers

The following Network solvers are available within LINE 3.0.x:

- `AUTO`: This solver uses an algorithm to select the best solution method for the model under consideration, among those offered by the other solvers. Analytical solvers are always preferred to simulation-based solvers. This solver is implemented by the `AUTO` class.
- `CTMC`: This is a solver that returns the exact values of the performance metrics by explicit generation of the continuous-time Markov chain (CTMC) underpinning the model. As the CTMC typically incurs state-space explosion, this solver can successfully analyze only small models. The CTMC solver is the only method offered within LINE that can return an exact solution on all Markovian models, all other solvers are either approximate or are simulators. This solver is implemented by the `CTMC` class.

- `FLD`: This solver analyzes the model by means of an approximate fluid model, leveraging a representation of the queueing network as a system of ordinary differential equations (ODEs). The fluid model is approximate, but if the servers are all PS or INF, it can be shown to become exact in the limit where the number of users and the number of servers in each node grow to infinity [61]. This solver is implemented by the `FLD` class.

- `JMT`: This is a solver that uses a model-to-model transformation to export the LINE representation into a JMT simulation (JSIM) or analytical (JMVA) models [5]. The JSIM simulation solver can analyze also non-Markovian models, in particular those involving deterministic or Pareto distributions, or empirical traces. This solver is implemented by the `JMT` class.

- `LDES`: This is the LINE Discrete Event Simulator, a native simulation solver built on the SSJ library. Like `JMT`, it analyzes the model by discrete-event simulation and supports non-Markovian distributions and a broad range of scheduling, routing, and blocking features, but it runs in-process (or through a standalone native binary) without requiring an external tool. This solver is implemented by the `LDES` class.

- `MAM`: This is a matrix-analytic method solver, which relies on quasi-birth death (QBD) processes to analyze open queueing systems. This solver is implemented by the `MAM` class.

- `MVA`: This is a solver based on approximate and exact mean-value analysis. This solver is typically the fastest and offers very good accuracy in a number of situations, in particular models where stations have a single-server. Besides mean-value analysis, the `MVA` solver also hosts LINE's closed-form analytical decomposition methods for open networks, namely the Queueing Network Analyzer (`qna`) [87] and its index-of-dispersion counterpart, the Robust Queueing Network Analyzer (`rqna`) [88], which characterizes each flow by its full index of dispersion for counts to capture bursty, non-renewal traffic and bounds the mean workload at each queue by robust queueing. In the near-immediate feedback elimination of `rqna`, our implementation keeps the strictly higher-utilization stations as explicit queues and collapses only the equal- or lower-utilization ones, so that the network traffic rates and the variability injected by upstream bottlenecks are preserved. This solver is implemented by the `MVA` class.

- `NC`: This solver uses a combination of methods based on the normalizing constant of state probability to solve a model. The underpinning algorithm are particularly useful to compute marginal and joint state probabilities in queueing network models. This solver is implemented by the `NC` class.

- `SSA`: This is a Stochastic Simulation Algorithms based on the CTMC representation of the model. Contrary to the `JMT` simulator, which has online estimators for all the performance metrics, `SSA` estimates only the probability distribution of the system states, indirectly deriving the metrics after the simulation is completed. Moreover, the `SSA` execution can more efficiently parallelized on multi-core machines. Moreover, it is possible to retrieve the evolution over time of each node state, including quantities that are not loggable in `JMT`, e.g., the active phase of a service or arrival distribution. This solver is implemented by the `SSA` class.

- `QNS`: This is a dedicated wrapper Network solver for the `qnsolver` utility distributed within LQNS. This allows users to evaluate product-form models using the MVA algorithms implemented within LQNS. The available options specify the multiserver handling algorithm: Conway's multiserver approximation (`conway`) [28], Rolia's multiserver (`rolia`) [71], load-dependent MVA by Reiser-Lavenberg (`reiser`) [67], and Zhou-Woodside's multiserver approximation (`zhou`) [93]. This solver is implemented by the `QNS` class.

5.2 Solvers and solution methods

We now describe the solution methods available within the Network solvers. Table 5.1 provides a global summary, while Figure 5.1 organizes the same methods by class of stochastic model. Some of the listed methods (e.g., `mg1`) are not associated to a specific solver, as they do not fall in one of the reference formalisms. A solver that runs these methods can be instantiated as follows, e.g.:

```
>> solver = LINE.load('mg1',model);
>> solver.avgTable();
```

Note that the `LINE.load` notation can also be used to instantiate a custom solver pre-configured with the specified method. For example

```
>> solver = LINE.load('ctmc',model);
```

runs the `CTMC` solver with default options. Solver-specific methods can be specified by appending their name to the method option, e.g. this command creates the `CTMC` solver with `gpu` method enabled:

```
>> solver = LINE.load('ctmc.gpu',model);
```

Bound analysis (`SolverBA`). Asymptotic and hierarchical performance bounds are provided by a dedicated `SolverBA` solver, to which the bound methods formerly served by `SolverMVA` (`aba`, `bjb`, `gb`, `pb`, `sb`, `mwba`) have been moved. Each method returns an optimistic (`.upper`) or pessimistic (`.lower`) throughput and queue-length bound, and the two sides of a family bracket the exact solution. Beyond the noniterative families, `SolverBA` offers level-parameterized hierarchies that tighten monotonically toward the exact solution as the `level` option is increased: the performance bound hierarchy `pbh` and its iterative proportional and balanced variants `pbk`/`bjbk`, the convolutional bound hierarchy `cbh`, the multiserver disaggregation bounds `ssd`, the multiclass composite upper bound `cub` (upper-only, with its multiclass balanced-job lower seed exposed as `mbjb`), the successively-improving bounds `sib`, and the load-dependent BCMP lower bound `ldbcmp`. A further family, `lr.upper`/`lr.lower` (bare `lr` meaning `lr.upper`), computes Linear Reduction bounds for single-class closed networks by optimising a linear utilisation functional over a linear-reduction polytope with a pure linear program (`linprog`, HiGHS, or simplex according to the codebase). Note that `lr` is *not* an alias of the QRF method `qrf.mmi.linear`, whose name refers only to its explicit constraint representation

and whose objective is a nonlinear entropy. The convenience method `getBounds` returns the lower/upper bracket for the current family. Since bounds require only the service demands and populations, `SolverBA` accepts any closed product-form-parameterized model. Separately, closed networks with non-exponential (Coxian) FCFS service can be approximated by Marie's aggregation-decomposition, available as the `marie` method of `SolverMVA`.

Table 5.1: Solution methods for `Network` solvers.

Solver	Method	Description	Refs.
CTMC	default	Solution based on global balance	[9, § 2.1.2]
LDES	default	LINE Discrete Event Simulator using SSJ library	–
FLUID	default	ODE-based mean field approximations	[62, 72]
FLUID	matrix	Alias for the `default` method	[62, 72]
FLUID	closing	Fluid with closing method for open classes	[9, p. 507]
FLUID	statedep	Kurtz's mean field ODEs for closed models	[62]
FLUID	pnorm	ODE approximation using p-norm smoothing for the processor-share constraint	[72]
FLUID	softmin	Smoothed `statedep` with *softmin* replacing *min* functions	–
FLUID	diffusion	Diffusion approximation via Euler-Maruyama SDEs	[9, § 10.1.1]
FLUID	mfq	Markovian fluid queue for single-queue systems	[44]
FLUID	rmf	Refined mean field approximation for multi-list cache analysis	[39]
JMT	default	Alias for the `jsim` method	–
JMT	jmva	Alias for the `jmva.mva` method	–
JMT	jmva.mva	Exact MVA in JMVA	[67]
JMT	jmva.recal	Exact RECAL algorithm in JMVA	[29]
JMT	jmva.comom	Exact CoMoM algorithm in JMVA	[12]
JMT	jmva.amva	Approximate MVA, alias for `jmva.bs`.	–
JMT	jmva.aql	AQL algorithm in JMVA	[91]
JMT	jmva.bs	Bard-Schweitzer algorithm in JMVA	[9, § 9.1.1]
JMT	jmva.chow	Chow algorithm in JMVA	[25]
JMT	jmva.dmlin	De Souza-Muntz Linearizer in JMVA	[31]
JMT	jmva.lin	Linearizer algorithm in JMVA	[23]
JMT	jmva.ls	Logistic sampling in JMVA	[13]
JMT	jsim	Exact discrete-event simulation in JSIM	[5]
JMT	replication	Transient simulation via independent replications	–
MAM	default	Matrix-analytic solution of structured QBDs	[45]
MAM	dec.source	Decomposition with arrivals as from the source	–
MAM	dec.poisson	Decomposition based on Poisson arrival flows	–
MAM	mna	Decomposition based on MNA method	[52]
MAM	inap	Iterative Numerical Approximation Procedure via RCAT	[16]
MAM	inapplus	Improved INAP with weighted rates (no normalization)	[16]
MAM	inapinf	INAP with matrix-geometric infinite-state components	[55]
MVA	default	Approximate MVA, same as `qd` option	–
MVA	amva	Approximate MVA, same as `qd` option	–
MVA	bs	Bard-Schweitzer approximate MVA	[9, § 9.1.1]
MVA	lin	Linearizer approximate MVA	[23]
MVA	marie	Marie aggregation-decomposition, closed FCFS non-exponential (Coxian) service	–
MVA	qd	Queue-dependent approximate MVA	[19]
MVA	qdlin	Queue-dependent Linearizer approximate MVA	–
MVA	exact	Exact solution, method depends on model	–
MVA	mva	Alias for the `mva.amva` method	[67], [11]
MVA	sqd	Smith Queue Decomposition, single-chain closed blocking-after-service (BAS) networks; offered only when the model is BAS	–

Continued on next page

Table 5.1 – Solution methods for `Network` solvers. *Continued from previous page*

Solver	Method	Description	Refs.
MVA	qna	Queueing Network Analyzer (two-moment decomposition) - open	[87]
MVA	rqna	Robust Queueing Network Analyzer (indices of dispersion) - open	[88]
MVA	sum	Summation method (SUM/ESUM); closing method for open and mixed models	[9, § 9.2, § 10.1.4.4, § 10.1.5]
MVA	esum	Alias for the `sum` method	[9, § 10.1.4.4]
BA	aba.upper	Asymptotic bound analysis (upper bounds)	[9, § 9.4]
BA	aba.lower	Asymptotic bound analysis (lower bounds)	[9, § 9.4]
BA	mwba.upper	Majumdar-Woodside robust box bounds, multiclass NBUE (upper bounds)	[54]
BA	mwba.lower	Majumdar-Woodside robust box bounds, multiclass NBUE (lower bounds)	[54]
BA	bjb.upper	Balanced job bounds (upper bounds)	[18, Table 3]
BA	bjb.lower	Balanced job bounds (lower bounds)	[18, Table 3]
BA	gb.upper	Geometric square-root bounds (upper bounds)	[18]
BA	gb.lower	Geometric square-root bounds (lower bounds)	[18]
BA	pb.upper	Proportional bounds (upper bounds)	[18, Table3]
BA	pb.lower	Proportional bounds (lower bounds)	[18, Table3]
BA	sb.upper	Simple bounds (upper bounds, Thm. 3.2, $n = 3$)	[41, Table3]
BA	sb.lower	Simple bounds (lower bounds, Eq. 1.6)	[41, Table3]
BA	pbh.upper/lower	Performance bound hierarchy, level-parameterized (Eager-Sevcik)	–
BA	pbk.upper/lower	Iterative proportional bounds PB(k) with delay	–
BA	bjbk.upper/lower	Iterative balanced job bounds BJB(k) with delay	–
BA	cbh.upper/lower	Convolutional bound hierarchy, level-parameterized (Dowdy et al.)	–
BA	ssd.upper/lower	Multiserver disaggregation bounds (Suri-Dallery)	–
BA	cub.upper	Composite upper bound, multiclass, upper-only (Kerola)	–
BA	mbjb.lower	Multiclass balanced job bounds lower (Kerola eq. 10; seeds `cub`)	–
BA	sib.upper/lower	Successively-improving bounds, level-parameterized (Srinivasan; delay-free)	–
BA	ldbcmp.lower	Load-dependent BCMP closed-open equivalence lower bound (Anselmi-Cremonesi)	–
BA	lr.upper/lower	Linear Reduction bounds, pure LP (`linprog`/HiGHS/simplex), single-class closed; bare `lr` means `lr.upper`	–
BA	qr / qrf.mmi	Quadratic Reduction Framework, NLP-based (mutual-information objective), single-class closed PH service; single-server stations only, delay stations rejected	–
BA	qrf.mmi.linear	QRF with an explicit linear constraint representation; the objective is still the nonlinear MEM entropy, so this is an NLP and is *not* the same as `lr`	–
BA	qrf.mem/mmi.ld	Further QRF reduction sub-methods (maximum entropy, load-dependent); NLP-based	–
BA	qrf.bas.*/rsrd	QRF blocking-after-service and RS-RD reduction bounds; LP-based	–
MVA	gig1.allen	Allen-Cunneen formula - GI/G/1	[9, § 6.3.4]
MVA	gig1.heyman	Heyman formula - GI/G/1	–
MVA	gig1.kingman	Kingman upper bound- GI/G/1	[9, § 6.3.6]
MVA	gig1.klb	Kramer-Langenbach-Belz formula - GI/G/1	[9, § 6.3.4]
MVA	gig1.kobayashi	Kobayashi diffusion approximation - GI/G/1	[9, § 10.1.1]
MVA	gig1.marchal	Marchal formula - GI/G/1	[9, § 10.1.3]
MVA	gigk	Kingman approximation - GI/G/k	
MVA	mg1	Pollaczek–Khinchine formula - M/G/1	[9, § 3.3.1]
MVA	mg1.fb	Feedback/LAS scheduling - M/G/1/FB	[89]
MVA	mg1.lrpt	Longest remaining PT - M/G/1/LRPT	[89]
MVA	mg1.psjf	Preemptive SJF - M/G/1/PSJF	[89]
MVA	mg1.srpt	Shortest remaining PT - M/G/1/SRPT	[89]

Continued on next page

Table 5.1 – Solution methods for `Network` solvers. *Continued from previous page*

Solver	Method	Description	Refs.
MVA	mg1.setf	Shortest elapsed time first - M/G/1/SETF	[60]
MVA	mm1	Exact formula - M/M/1	[9, § 6.2.1]
MVA	mmk	Exact formula - M/M/k (Erlang-C)	
NC	default	Automated choice of solution method	–
NC	exact	Automated choice of exact solution method.	–
NC	ca	Multiclass convolution algorithm (exact)	–
NC	clw	Choudhury-Leung-Whitt generating function inversion (exact); limited load-dependent stations handled via Bertozzi-McKenna transforms	[24], [6]
NC	comom	Class-oriented method of moments for hommogeneous models (exact)	[12]
NC	cub	Grundmann-Moeller cubature rules	[13]
NC	mva	Product of throughputs on MVA lattice (exact)	[66, Eq. (47)]
NC	imci	Improved Monte carlo integration sampler	[85]
NC	is	Importance sampling over job orderings; closed models only, load-independent, load-dependent	[27]
NC	kt	Knessl-Tier asymptotic expansion	[47]
NC	le	Logistic asymptotic expansion	[13]
NC	ls	Logistic sampling	[13]
NC	nrl	Norlund-Rice integral with logit transformation	[17]
NC	nrp	Norlund-Rice integral with probit transformation	[17]
NC	panacea	Panacea asymptotic expansion	[57], [70]
NC	rd	Reduction heuristic	[17]
NC	sampling	Automated selection of sampling method	–
NC	mem	Maximum Entropy Method for open ($GE/GE/1$, $GE/GE/c$, $GE/GE/\infty$ building blocks), closed (pseudo-open decomposition plus convolution) and mixed queueing networks; auto-selected by `default` for non-Markovian open models	[49]
NC	mem.blocking	Finite station buffers in a single-class open network: censored $GE/GE/c/0$; N stations under the loss rule, plus the holding-node expansion of transfer blocking (BAS); explicit request only	[49, 80]
SSA	default	Alias for `nrm` if the model supports it, otherwise `serial`.	–
SSA	nrm	Next reaction method for population models (e.g., PS/INF scheduling).	[2]
SSA	serial	CTMC stochastic simulation on a single core	[40]
SSA	para	Parallel simulations (independent replicas)	–

MAM method selection. The MAM solver offers several decomposition approaches for networks with non-exponential service. The `dec.mna` method uses the MNA decomposition, which is generally the most accurate for networks with Markovian arrival processes. The `dec.source` method approximates arrivals at each station using the distribution observed at the source, while `dec.poisson` replaces all arrival flows with Poisson processes. The `inapplus` method provides an improved iterative approximation that avoids the normalization step of the standard `inap` method, and is recommended for models where RCAT convergence is slow. The `inapinf` method [55] solves each isolated open component directly on its infinite state space by a matrix-geometric decomposition, yielding the exact geometric marginal without state-space truncation; it is preferred over `inap` for open G-networks with birth-death or catastrophe structure, where truncation would otherwise bias the queue-length estimates.

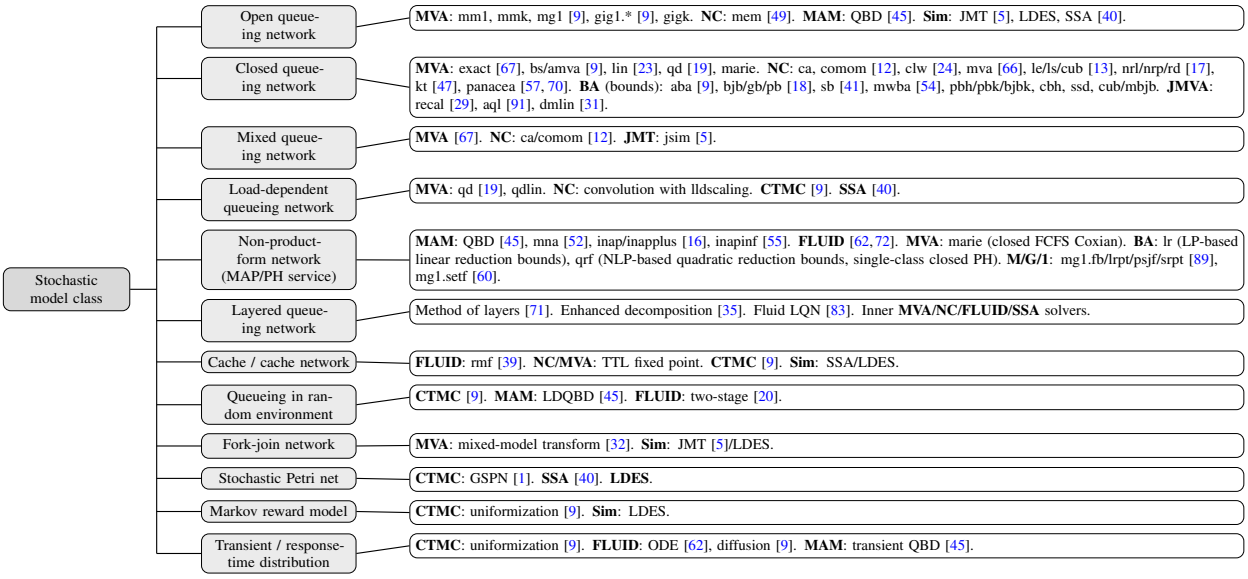

Figure 5.1: Taxonomy of solution techniques available in LINE for each class of stochastic model. Each leaf lists the short method name (as accepted by `options.method`) with its bibliographic reference; entries without a reference denote simulation backends or aliases. See Table 5.1 for the solver exposing each method.

FLUID method selection. The default FLUID method (`matrix`) formulates a set of ordinary differential equations (ODEs) that describe the mean-field dynamics of the network. The `statedep` method uses Kurtz's mean-field theorem, which is exact for state-dependent models in the large population limit. The `softmin` variant replaces the *min* function with a smooth approximation, improving numerical stability of the ODE integration at the cost of a small approximation error. The `mfq` method solves Markovian fluid queues and is applicable to single-queue systems with MAP arrivals. The `diffusion` method applies Euler-Maruyama discretization to model stochastic fluctuations around the mean field. The `rmf` method uses the refined mean field approximation [39] to analyze multi-list caches with RANDOM(m) replacement, providing $O(1/N)$ corrections to mean field fixed points via Lyapunov equations; it is automatically selected when the model contains Cache nodes.

NC method selection. The normalizing constant solver provides both exact and approximate methods. For exact computation, `ca` (convolution algorithm) and `comom` (class-oriented method of moments) are available, with `comom` being generally faster for models with many classes. Among the approximate methods, `nr.logit` and `nr.probit` use Norlund-Rice integrals with different variable transformations and are suitable for large populations. The `rd` method applies a reduction heuristic that recursively simplifies the network. The `cub` method employs cubature rules for numerical integration of the normalizing constant. The `mem.blocking` variant extends the same building block to finite station buffers, which no other `NC` method can represent: a capacitated station becomes a censored $GE/GE/c/0; N$ queue, exact for $M/M/1/N$ and $M/M/c/N$, and blocking after service is handled by the holding-node transformation of Tahilramani, Manjunath and Bose that restores work conservation. For open networks, the `mem` method applies the Maximum Entropy Method of Kouvatsos to estimate the joint queue-length distribution via a queue-by-queue decomposition into $GE/GE/1$, $GE/GE/c$ and $GE/GE/\infty$ building blocks; it is exact for $M/G/1$, $M/M/c$ and infinite-server stations and captures the propagation of non-exponential variability (including immediate feedback) through the network. The `default` method automatically selects `mem` through a benchmark-calibrated heuristic: open networks within its feature set whenever any arrival or service process has a non-unit squared coefficient of variation (except bursty external arrivals at low load, where the GE bound is loose), and closed networks that are non-product-form due to hypoexponential services or class-dependent FCFS rates; purely exponential models and hyperexponential closed FCFS models are instead solved by the native normalizing-constant path. For closed networks, `mem` implements the two-stage pseudo-open decomposition and convolution algorithm of Kouvatsos on $G/G/1$ and $G/G/\infty$ building blocks; it conserves class populations exactly and reduces to the exact BCMP solution on Markovian single-class networks, but it is selected only on explicit request since the default path is exact for product-form closed models. Mixed open/closed models compose the two algorithms with product-form-style conditioning (capacity reduction of the closed subnetwork by the open load, inflation of the open queue lengths by the closed occupancy), exactly recovering mixed MVA in the product-form limit.

SSA method selection. The `serial` method implements Gillespie's stochastic simulation algorithm on a single core, while `para` runs independent replicas in parallel for faster convergence. The `nrm` (next reaction

method) offers significantly faster simulation for population models with processor-sharing or infinite-server scheduling, as it avoids recomputing all transition rates after each event.

5.2.1 AUTO

The AUTO class (aliases: SolverAuto, SolverAUTO, LINE) provides interfaces to the core solution functions (e.g., avg, ...) that dynamically bind to one of the other solvers implemented in LINE (CTMC, NC, ...). It is often difficult to identify the best solver without some performance results on the model, for example to determine if it operates in light, moderate, or heavy-load regime.

The selection is therefore driven by structural features extracted from sn: the proportions of FCFS, processor-sharing and delay stations, the presence of class-switching nodes, the mean number of servers per queue, the number of jobs per chain, and the service process types. These features are normalized and combined into ratios that a decision rule maps to a solver. Optionally, a classifier trained offline on a corpus of models and stored in ONNX format may be applied to the same feature vector instead of the rule, returning one of MVA, NC, CTMC or JMT.

When the selected solver does not implement the operation requested (for instance getTranAvg, which not every solver provides), AUTO falls back along the list of feasible solvers ordered by increasing average runtime. Since the list ends with the simulators, a solution strategy always exists. Direct instantiation of a solver remains preferable once the appropriate one is known, as it avoids the feature-extraction step.

Selection methods. The constructor of AUTO (SolverAuto) accepts an options.method argument that fixes how the back-end solver is chosen. Categorical options trigger a single back-end consistent with the requested intent:

- options.method='default' or 'heur': heuristic auto-selection. LINE instantiates a list of feasible solvers ordered by typical execution time and dispatches the requested function to the first one that supports the model (and the operation, e.g. getTranAvg). For Network models the candidate list is, in increasing average runtime, MVA, NC, MAM, FLD, JMT, SSA, CTMC, LDES; for LayeredNetwork models it is LQNS followed by LN wrappers around NC, MVA, MAM, FLD; for Environment models it is ENV wrapping MVA, NC, or FLD.

- options.method='sim': best simulation back-end. Routes to JMT (Network), LQNS (LayeredNetwork), or ENV wrapping JMT (Environment).

- options.method='exact': best exact analytical back-end. Routes to NC for closed Network models, to CTMC for non-closed ones, to LN wrapping NC for layered models, and to ENV wrapping NC for random environments.

- options.method='fast': fast approximate back-end (MVA or its LN/ENV wrapping).

- options.method='accurate': accurate approximate back-end (FLD or its LN/ENV wrapping).

- Per-solver delegations: passing the lowercase solver name forces the corresponding back-end with its default method. Recognised values are `mam`, `mva`, `nc`, `fluid`, `jmt`, `ssa`, `ctmc`, `ldes`, `env`, `ln`, `lqns`.

5.2.2 BA

The `BA` class (alias: `SolverBA`) computes performance *bounds* for closed queueing networks. It differs from the other solvers in what it returns: rather than a point estimate, each method returns one guaranteed side of an interval containing the exact solution, so that the `.lower` and `.upper` members of a family bracket the true value. This is the property that makes bounds useful inside optimization loops and admission tests, where an approximation of unknown sign cannot be used safely. Because every bound is a function of the service demands, think times and populations alone, and never of the service distribution beyond its mean, `BA` is inexpensive and its feature set is deliberately restricted to closed product-form-parameterized models.

The available families are listed in Table 5.1. The asymptotic families, the bound hierarchies parameterized by `options.level`, and the reduction bounds `lr` and `qr`/`qrf.*` are described, together with their algorithms and bibliographic sources, in the LINE internals manual (`LINE-internals.pdf`, Chapter `BA`).

Two methods are one-sided by construction: `cub` yields an upper bound only, while `mbjb` and `ldbcmp` yield lower bounds only. Several families also carry structural restrictions beyond the feature set, and reject a model rather than return a value that would not be a bound: `sb` and `sib` do not admit infinite-server stations, and `lr` requires a single-class closed model with single-server stations.

The normal way to read a bound is `getBoundsTable`, the counterpart of `getAvgTable`. It evaluates both sides of the family of the selected method and reports the bracket per station and class, with columns `Qlower`, `Qupper`, `Tlower` and `Tupper`. It follows the labelling of `getAvgTable` and likewise omits disabled station-class pairs.

```
>> SolverBA(model,'method','gb.upper').getBoundsTable()
    Station    JobClass    Qlower     Qupper    Tlower     Tupper
    _______    ________    ______     ______    ______     ______
     Think        C          1.131    1.2557    0.56551    0.62785
     Q1           C        0.70532    1.5213    0.56551    0.62785
     Q2           C         1.5077     4.187    0.56551    0.62785
```

For a one-sided family the absent side is reported as `NaN` rather than as zero, so that the absence of a bound stays distinguishable from a bound whose value happens to be zero. Requesting `cub`, which is upper-only, on the same model yields `NaN` throughout the `Qlower` and `Tlower` columns.

The programmatic accessor is `getBounds`, which returns the same information as a raw bracket rather than a formatted table, and is the form to use inside a loop. The relationship between the two mirrors that between `getAvg` and `getAvgTable`.

```
b = SolverBA(model,'method','bjb.upper').getBounds();
b.Tlower, b.Tupper    % throughput bracket
b.Qlower, b.Qupper    % queue-length bracket
```

Both accessors inherit the option set of the solver they are called on, so `options.level` applies to the bracket as a whole. On the Eager-Sevcik hierarchy `pbh` the bracket width falls monotonically as the level rises and reaches zero once `level` equals the population, giving a direct accuracy-cost dial: on the three-station model above with $N = 5$ the width contracts from 0.050063 at level 1 to 0.031329 at level 2 and to 0 at level 5, where both sides equal the exact throughput 0.599076. A worked illustration is given in `tut15_bound_analysis`.

The reduction bounds are available in all three codebases. This includes the nonlinear `qr`/`qrf.*` family, which was formerly withheld in native Python; the tokens `qr`, `qrf.mmi`, `qrf.mem`, `qrf.mmi.ld` and `qrf.mmi.linear` now appear in `listValidMethods` everywhere. The blocking variants `qrf.bas.mmi` and `qrf.bas.mem` remain unavailable in native Python and raise an explicit error there.

Two restrictions apply to the whole `qrf.*` family in every codebase. The model must be closed and single-class, and it must not contain a delay station: the underlying formulation gives every station a single server, so a delay would be silently replaced by a queue rather than approximated, and the solver therefore rejects such a model instead of returning a value. Models with think time should use `MVA` or `NC`, or one of the classical bound families, all of which admit infinite-server stations.

A note on precision when comparing across codebases. The linear program methods in MATLAB and native Python reproduce the reference formulation to about 10^{-11}, whereas the Java implementation solves them with a sparse operator-splitting method and attains about 10^{-5}. The Java values are valid bounds, but they should not be compared digit for digit against the other two.

5.2.3 CTMC

The `CTMC` class solves the model by first generating the infinitesimal generator of the Network and then calling an appropriate solver. Steady-state analysis is carried out by solving the global balance equations defined by the infinitesimal generator. If the `keep` option is set to `true`, the solver will save the infinitesimal generator in a temporary file and its location will be shown to the user.

Transient analysis is carried out by numerically solving Kolmogorov's forward equations using MATLAB's ODE solvers. The range of integration is controlled by the `timespan` option. The ODE solver choice is the same as for `FLD`.

As an alternative to numerical integration, the transient distribution can be obtained by uniformization (randomization), in which the continuous-time chain is replaced by a discrete-time chain observed at the epochs of a Poisson process of rate $q \geq \max_i |Q_{ii}|$ and the transient vector is the Poisson-weighted sum of the powers of the uniformized transition matrix. A naive implementation of this sum overflows or underflows as soon as qt becomes large, because it forms e^{-qt} and $(qt)^k/k!$ explicitly. LINE therefore computes the Poisson weights and the left and right truncation points with the Fox-Glynn algorithm, including Jansen's correction to the right tail estimate, so that neither quantity is ever formed and no splitting of the time horizon is required. The routine is exposed through the Markov-chain API as `ctmc_foxglynn`, and all uniformization callers inside the solver are routed through it. Its tolerance defaults to 10^{-12} and an optional cap on the right truncation point is available for very long horizons.

The CTMC solver refuses by default to solve models whose state space is predicted not to fit in memory.

Rather than comparing against a fixed state count, the decision is taken by a hardware-aware pre-gate, `ctmc_memory_gate`, which compares the predicted peak footprint of the sparse factorization against a fraction (by default 0.6) of the memory actually available on the host. The per-state cost is calibrated once per machine by profiling a local sparse LU factorization, and the fitted power law is cached. The `force` option needs to be set to `true` to bypass this control. In models with infinite states, such as networks with open classes, the `cutoff` option should be used to reduce the CTMC to a finite process. If specified as a scalar value, `cutoff` is the maximum number of jobs that a class can place at an arbitrary station. More generally, a matrix assignment of `cutoff` indicates to LINE that `cutoff` has in row i and column r the maximum number of jobs of class r that can be placed at station i.

The following methods are available for the `CTMC` solver:

- `options.method='default'`: Direct solution of the global balance equations.
- `options.method='gpu'`: GPU-accelerated solution of the global balance equations.

Quadratic Reduction Framework (QRF) methods. For single-class closed networks with phase-type service, the CTMC solver also dispatches a family of approximation algorithms from the Quadratic Reduction Framework (QRF) library [15], selected through the method names `qrf.mmi`, `qrf.mem`, `qrf.mmi.ld`, `qrf.mmi.linear`, `qrf.bas.mmi`, `qrf.bas.mem`, `qrf.bas` and `qrf.rsrd`. The QRF dispatch is accessible only when the model has $K = 1$ class and a finite total population. The algorithms behind these methods are documented in the LINE internals manual (`LINE-internals.pdf`, Chapters `CTMC` and `BA`). The blocking variants `qrf.bas.*` and `qrf.rsrd` require the blocking topology to be supplied in `options.config.qrf_params`, and the load-dependent variants read `options.config.qrf_alpha`; a missing mandatory field aborts the solver with an error. The two configuration entries used by the QRF dispatcher are:

- `options.config.qrf_params`: optional struct describing the blocking configuration of `qrf.bas.*` variants. Fields: `f` (blocking-event index), `MR` (number of resources), `BB` (per-station buffer capacities), `F` (capacity vector), `MM`, `MM1`, `ZZ`, `ZM` (auxiliary index sets needed by the BAS engine). Defaults to an empty struct.
- `options.config.qrf_alpha`: optional $M \times N$ matrix of load-dependent scaling factors used by `qrf.mmi.ld` and `qrf.mmi.linear`. Row i, column n scales the service rate of station i when n jobs are present. Defaults to a matrix of ones.

The MATLAB QRF dispatch additionally requires the Optimization Toolbox: the `qrf_noblo_*` methods call `fmincon`, while `qrf_bas` and `qrf_rsrd` call `linprog`.

The CTMC solver also provides access to the underlying infinitesimal generator and its decomposition into per-event filtration matrices via the `getGenerator` method:

```
solver = CTMC(model);
[infGen, eventFilt] = solver.getGenerator();
```

Here `infGen` is the infinitesimal generator matrix and `eventFilt` is a list of sparse matrices, one per event type, encoding the transitions associated with each event (arrival, departure, phase change, etc.).

Symbolic generator. The generator can also be built symbolically, with each event filtration normalized by its smallest positive rate and scaled by a symbol $x_1, \ldots, x_E$, one per event:

```
[infGen, eventFilt, syncInfo, stateSpace] = solver.getSymbolicGenerator();
```

No computer algebra is needed to assemble it, since the generator is linear in the symbols; substituting each symbol with its event's smallest positive rate recovers the numerical generator exactly. Solving with the symbolic generator does require a computer algebra system, and is described in §4.7.2.

Router elimination via stochastic complementation. When the network contains `Router` nodes (e.g., as part of a cache topology with state-dependent routing, see §3.1.5), the immediate transitions they introduce are eliminated automatically by *stochastic complementation* [58] before the chain is solved, so that per-class arrival and throughput metrics at downstream stations remain correct. The construction is described in the LINE internals manual (`LINE-internals.pdf`, Chapter `CTMC`). The lower-level primitives are exposed through the API functions `Ctmc_stochcomp` (continuous time) and `Dtmc_stochcomp` (discrete time) and may be used directly when working with custom Markov chains.

Details on the additional configuration options of the CTMC solver is given in the next table.

Table 5.2: `CTMC` configuration options (MATLAB)

Option	Value	Description
`options.config.hide_immediate`	`logical`	If true, immediate transitions are hidden from the CTMC.
`options.config.state_space_gen`	`'reachable'`	Direct state space enumeration from initial state.
`options.config.state_space_gen`	`'full'`	Direct state space enumeration from all possible initial states.
`options.timestep`	`double`	Fixed time interval for transient analysis. If specified, generates equally-spaced time points instead of adaptive stepping.

Reward analysis

The CTMC solver supports reward-based analysis through custom reward functions defined on network states. Users can define state-dependent reward functions that map a state vector and the network structure to a scalar reward value. Multiple rewards can be defined by calling `setReward` multiple times with different names. The solver computes rewards using value iteration with uniformization. Results are retrieved via `solver.getAvgReward()`, which returns steady-state expected rewards, or `solver.getTranReward()`, which returns the transient expected-reward trajectories $E[r(X(t))]$ over time. Example applications include computing expected queue lengths, utilization, blocking probabilities, or custom cost functions.

Reward functions are defined using `model.setReward(name, rewardFn)`, where `name` is a string identifier and `rewardFn` is a function handle `@(state, sn)` that maps a state vector and the network structure to a scalar reward value. See `rewardModel_mm1k.m` in the `rewardModel` examples folder.

5.2.4 FLD

This solver implements fluid/mean-field approximation methods, based on the system of fluid ordinary differential equations for INF-PS queueing networks presented in [62]. The latter is based on Kurtz's mean-field approximation theory. The fluid ODEs are normally solved with the `'NonNegative'` ODE solver option enabled. Four types of ODE solvers are used: *fast* or *accurate*, the former only if `options.iter_tol`$> 10^{-3}$, and *stiff* or *non-stiff*, depending on the value of `options.stiff`. The default choice of solver is stored in the following static functions:

- `Solver.accurateStiffOdeSolver`, set to MATLAB's `ode15s`.
- `Solver.accurateOdeSolver`, set to `ode113`.
- `Solver.fastStiffOdeSolver`, set to `ode23s`.
- `Solver.fastOdeSolver`, set to `ode23`.

As an alternative to the built-in MATLAB ODE solvers, LINE includes a MEX-compiled port of the LSODA algorithm, which automatically switches between Adams (non-stiff) and BDF (stiff) integration methods. The MEX solver can be built with `build_lsoda_mex` (one-time compilation) and is used internally by the `FLD` solver when available. It provides performance comparable to native C code while retaining full MATLAB interoperability.

ODE variables corresponding to an infinite number of jobs, as in the job pool of a source station, or to jobs in a disabled class are not included in the solution vector. These rules apply also to the `options.init_sol` vector.

Nodes with zero or near-zero service time (e.g., `Router`, `ClassSwitch`) are treated as immediate nodes and are eliminated from the fluid equations before integration; FCFS stations are mapped into equivalent PS stations with a mixture service process. Both transformations are described in the LINE internals manual (`LINE-internals.pdf`, Chapter `FLD`).

The following methods are available for the `FLD` solver:

- `options.method='default'` or `'matrix'`: matrix formulation of the fluid ODEs [72], for closed and open networks.
- `options.method='closing'` or `'statedep'`: State-dependent fluid solver with closing approximations. Useful for models with DPS scheduling.
- `options.method='pnorm'`: p-norm smoothing of the processor-share constraint [72].
- `options.method='diffusion'`: diffusion approximation for closed multiclass BCMP networks. Presently, this method supports only product-form closed networks.
- `options.method='mfq'` (Markovian Fluid Queue): exact steady-state analysis of single-queue open networks with phase-type arrivals and service [45]. Restricted to a Source $\rightarrow$ Queue $\rightarrow$ Sink topology; closed networks and networks with multiple queues are not supported.

- `options.method='rmf'` (Refined Mean Field): analysis of multi-list caches with RANDOM(m) replacement [39], in steady state and transient. This method is automatically selected as the default when the model contains `Cache` nodes.

- `options.method='tbi'` (Trajectory-Based Iteration): cell-decomposed solution of the closing fluid ODEs for large closed networks [77]. It is configured by `options.config.tbi_tol` (default 10^{-3}), `options.config.tbi_iter_max` (default 50), `options.config.tbi_sweep` (`'gauss-seidel'` by default, `'jacobi'` alternatively), `options.config.tbi_parallel` (MATLAB only, requires the Parallel Computing Toolbox), and by the cell partition, given either as `options.config.tbi_cells`, a cell array of disjoint station index vectors covering the station set, or computed automatically with `options.config.tbi_cellsize` stations per cell on average (default 5); see `examples/advanced/largeScale/largescale_tbi.m` for an illustration. It supports closed models only and does not support caching stations or immediate-node elimination.

The algorithms behind these methods are documented in the LINE internals manual (`LINE-internals.pdf`, Chapter `FLD`).

Exporting the ODE system. The system of ODEs integrated by the mean-field methods (`default`/`matrix`, `pnorm`, `closing`, `statedep`, `softmin`) can be exported in symbolic form as a standalone LaTeX document using the `exportODEs` method. The export is both human readable, since the compiled document shows the equations together with a legend of the state variables, and machine readable, since the document header carries the state-variable mapping and event rates as structured comment lines (`% STATE` and `% EVENT`). Each state variable x_s represents the mean number of jobs of a class in a service phase at a station. Two notations are available:

- `'scalar'` (default): one expanded ODE per state variable, written in terms of station-level auxiliary functions such as the total station mass $n_i(x)$ and the capacity scaling $g_i(x) = \min(n_i(x), S_i)/n_i(x)$;

- `'matrix'`: the compact matrix form, $d\mathbf{x}/dt = W^{\top}\theta(\mathbf{x}) + \boldsymbol{\lambda}$ for the `matrix`/`pnorm` methods (per Ruuskanen et al. [72], with θ the smoothed service vector and $\boldsymbol{\lambda}$ the exogenous arrival rates), and $d\mathbf{x}/dt = J\,r(\mathbf{x})$ for the `closing`/`statedep`/`softmin` methods, where J is the stoichiometry (jump) matrix and $r(\mathbf{x})$ the vector of event rate functions.

The exported document also reports the initial condition $\mathbf{x}(0)$ and remarks on the numerical regularizations applied by the solver. The exported system reflects the method set in the solver options; methods that are not ODE-based (`diffusion`, `mfq`, `rmf`) are not supported by this export.

```
solver = FLD(model, 'method', 'closing');
tex = solver.exportODEs('model_odes.tex', 'scalar');
```

Passing an empty file name returns the LaTeX source without writing a file; the second output returns the structural description of the ODE system (state metadata, stoichiometry, rate coefficients).

5.2.5 JMT

The class is a wrapper for the JMT and consists of a model-to-model transformation from the Network data structure into the JMT's input XML formats (either .jsimg or .jmva) and a corresponding parser for JMT's results. Upon first invocation, the JMT JAR archive will be searched in the MATLAB path and if unavailable automatically downloaded.

This solver offers several methods. The default method is the JSIM solver ('jsim' method), which runs JMT's discrete-event simulator. For parallel simulation, the solver supports both serial and parallel execution methods. The alternative method is the JMVA analytical solver ('jmva' method), which is applicable only to queueing network models that admit a product-form solution. This can be verified calling model.hasProductFormSolution prior to running the JMVA solver.

For transient analysis, the 'replication' method runs independent simulation replications and interpolates the resulting sample paths to compute transient averages. This method is automatically selected when the timespan option is set to a finite interval (e.g., [0, T] for some finite T) and the method is set to 'default'.

In the transformation to JSIM, artificial nodes will be automatically added to the routing table to represent class-switching nodes used in the simulator to specify the switching rules. One such class-switching node is defined for every ordered pair of stations (i, j) such that jobs change class in transit from i to j.

5.2.6 MAM

This is a basic solver for some Markovian open queueing systems that can be analyzed using matrix analytic methods. The core solver is based on the BU tools library for matrix-analytic methods [45]. The solution of open queueuing networks is based on traffic decomposition methods that compute the arrival process at each queue resulting from the superposition of multiple source streams.

The getCdfRespT method computes the response time distribution using matrix-analytic techniques. For processor-sharing (PS) queues with MAP arrivals, it uses the algorithm from Masuyama and Takine [56]. The number of CDF points computed can be controlled via options.config.num_cdf_pts (default: 200).

The following methods are available for the MAM solver:

- options.method='default' or 'dec.source' (Arrival Stream Decomposition): the default decomposition, applicable to open, closed, and mixed queueing networks.
- options.method='mna' (Matrix Network Analyzer) [52]: more accurate than dec.source; recommended for networks with Markovian arrival processes, and restricted to FCFS and HOL scheduling.
- options.method='dec.poisson' (Poisson Arrival Approximation): the fastest but least accurate decomposition, suitable for exploratory analysis or as a baseline.
- options.method='inap' (INAP): Iterative Numerical Approximation Procedure [16] based on the Reversed Compound Agent Theorem (RCAT).

- `options.method='inapplus'` (Improved INAP) [16]: recommended when RCAT convergence of `inap` is slow.
- `options.method='inapinf'` (Matrix-Geometric INAP) [55]: preferred over `inap` for open signal networks with birth-death or catastrophe structure, since it avoids state-space truncation.
- `options.method='dec.mmap'` (ETAQA Departure Decomposition) [69]: computes MAP approximations of the departure processes; when the required state space exceeds `space_max` or the queue is unstable, the solver falls back to a scaled service process.

The algorithms behind these methods are documented in the LINE internals manual (`LINE-internals.pdf`, Chapter `MAM`).

The `dec.mmap` method exposes additional configuration parameters via `options.config`:

- `etaqa_trunc` (default: 8): QBD truncation level for the ETAQA computation; higher values yield more accurate departure processes at increased computational cost.
- `space_max` (default: 128): maximum state-space size $(\texttt{etaqa_trunc}+1)\times n_a \times n_s$ beyond which ETAQA is skipped and a scaled service process is used instead.
- `merge` (default: `'super'`): strategy for merging multiple marked MAP streams via superposition.
- `compress` (default: `'mixture.order1'`): compression method to reduce the order of superposed MMAPs before downstream analysis.

Finite-buffer stations. When an FCFS station declares a finite capacity K (Kendall's K, the job in service included, see §3.1.2), the `MAM` solver analyses that station as a loss system instead of an infinite-buffer queue, and reports the blocking probability as a reduction of the class throughputs. A finite buffer takes precedence over the specialised infinite-buffer closed forms for PH/M/1, D/M/c and MAP/D/c stations, which are only defined for $K=\infty$ and would otherwise silently discard K and return a lossless queue. Three cases are distinguished, in order of preference:

- *Exact M/M/c/K*. If the aggregate arrival process at the station is Poisson and every class holds an exponential service law of common rate, the station is an M/M/c/K queue and is solved exactly in closed form by `qsys_mmck`.
- *MMAP[R]/G/1/K*. Otherwise, if the station has a single server, the marked arrival process and the finite buffer are resolved jointly by `qsys_mmapg1k`, which embeds the queue at departure epochs following the transform-free approach of Niu and Cooper [59] generalised to MAP arrivals. The embedded chain yields, by Markov renewal reward, the time-stationary joint law of buffer level and arrival phase, so a class-r job, which leaves phase i of the arrival MMAP at rate $(D_1^{(r)}\mathbf{1})_i$, receives its own loss ratio $L_r=(\boldsymbol{\pi}_K D_1^{(r)}\mathbf{1})/\lambda_r$, where $\boldsymbol{\pi}_K$ is the phase-resolved full-buffer probability vector. Two classes of equal arrival rate but different interarrival variability or autocorrelation are therefore

told apart, which is the effect studied by Chydzinski [26]. Neither independence between classes nor a PASTA argument is invoked, the phase resolution of π_K doing the work instead. This branch is *exact* whenever the service law is class-independent, which is the MMAP$[R]$/G/1/K model proper; the service law passed to it is the arrival-weighted phase-type mixture of the per-class laws built by `mam_svc_mixture`, which reproduces the common law exactly when all classes share one. When classes hold *different* service laws the mixture is an approximation, because it makes the service time of a job independent of the class that job belongs to, whereas under FCFS the job in service and the buffer content are class-resolved; the resulting error grows with the spread of the per-class service rates. The related function `qsys_mapg1k_perflow` covers the setting of [26], in which the flows are supplied as N separate MAPs rather than as a single marked MMAP, and `qsys_mapg1k` exposes the underlying single-flow MAP/G/1/K analysis, whose service time enters exactly rather than being fitted to a phase-type law.

- *Truncate-and-renormalize.* In the remaining cases, `mam_truncate_renorm` truncates the QBD of the infinite-buffer station at level K and renormalizes its stationary distribution, taking the residual mass as the blocking probability. This is an approximation, and since it yields a single blocking probability shared by all classes, the per-class loss it induces is proportional to the class arrival rates and hence blind to differences in interarrival variability across classes.

Level-Dependent QBD analysis

Beyond the decomposition methods discussed above, the `MAM` solver embeds an exact engine for Level-Dependent Quasi-Birth-Death (LDQBD) processes, which generalise QBDs by allowing the local generator at each level n to depend on n [64]; the construction is described in the LINE internals manual (`LINE-internals.pdf`, Chapter `MAM`). The truncation level is chosen automatically based on the traffic intensity. Convergence tolerance is controlled through `options.tol` and the maximum number of iterations through `options.iter_max`.

The LDQBD engine is invoked under two circumstances. First, when the user requests it explicitly through

```
solver = MAM(model, 'method', 'ldqbd');
AvgTable = solver.getAvgTable();
```

This explicit form is restricted to single-class networks with a single FCFS queue, and covers two regimes. In the *closed* regime the network consists of one infinite-server delay station and one queue with a finite population N: level n is the number of jobs at the queue, $N - n$ is the number at the delay, and the per-level arrival rate is $(N - n)\lambda$. In the *open* regime the network consists of one `Source` and one queue fed by a Poisson stream of constant rate λ: level n is again the number of jobs at the queue, but the level index must be truncated, and the truncation point is taken from `options.cutoff` or chosen automatically so that the residual tail probability is negligible. Both regimes share the same block-tridiagonal generator and differ only in the per-level arrival rate and in the top level. The engine constructs scalar QBD blocks for exponential

service and matrix QBD blocks of phase-type for non-exponential service, and handles multi-server and load-dependent stations through a service rate $\min(n, c)\mu$ or its phase-type generalisation. The analysis is exact for exponential service at any number of servers and for phase-type service at a single server; for phase-type service with $c > 1$ servers the c parallel servers are collapsed into a single phase-type process scaled by $\min(n, c)$, which is an approximation because it ignores the phase of each individual busy server.

Second, the LDQBD engine is invoked automatically by the retrial queue solver. Retrial queues in LINE model a BMAP/PH/N/N bufferless system as already discussed in §3.1.6 and described at [45]: at each level n (number of customers in orbit) the generator includes the BMAP arrival, the per-customer retrial rate α, the orbit impatience rate γ, and the batch rejection probability p. The level dependence arises because the orbit retrial rate is $n\alpha$ when there are n customers in orbit. No special method selector is required: when `MAM` detects a retrial topology it dispatches to the LDQBD-based retrial solver. Performance metrics returned in this case include the mean orbit length, the throughput, the server utilisation, and the orbit blocking probability.

A transient counterpart of the LDQBD engine is provided through `solver_mam_ldqbd_transient.m`. It computes time-dependent state probabilities and per-level marginal distributions over a user-supplied `timespan` by integrating Kolmogorov's forward equation on the level-truncated generator. This routine is invoked automatically when `getTranAvg` is called on a model that the steady-state LDQBD engine would otherwise handle.

Fork-join configuration. For models containing fork-join sections, the `MAM` solver exposes two additional configuration knobs:

- `options.config.fj_accuracy` is the numeric accuracy parameter C of the fork-join approximation (default `100`); larger values yield progressively tighter approximations of the fork-join response-time fit at the cost of additional runtime.
- `options.config.fj_tmode` selects how the fork-join T matrix is computed. Valid values are `'NARE'` (the default, based on a non-symmetric algebraic Riccati equation solver) and `'Sylves'` (based on a Sylvester-equation solver).

5.2.7 MVA

The solver offers approximate mean value analysis (AMVA) (`options.method='default'`), but also exact MVA algorithms (`options.method='exact'`). The default AMVA solver is based on Linearizer [23], unless there are two or less jobs in total within closed classes, in which case the solver runs the Bard-Schweitzer algorithm [74].

AMVA submethod catalogue. Beyond the `default`/`exact` entry points, the `MVA` solver exposes a catalogue of AMVA variants reported by `listValidMethods`. They can all be selected through `options.method`; the `amva.`-prefixed names are aliases of the corresponding short names. The most commonly used variants are:

- `amva.lin` (alias `lin`): Linearizer [23], the default AMVA for closed multi-class product-form models.
- `amva.bs` (alias `bs`): Bard-Schweitzer approximate MVA [74], automatically selected when the closed populations are small.
- `amva.qd` (alias `qd`): queue-dependent AMVA [19] for limited load-dependent stations; this is the engine behind `default` when load dependence is detected.
- `amva.qdlin` (alias `qdlin`): queue-dependent Linearizer, recommended for load-dependent multiclass models.
- `amva.qli`: Wang-Sevcik queue-line approximation [84], an alternative to Linearizer for moderate populations.
- `amva.aql`: aggregate queue-length approximation [91].
- `amva.qdaql`: queue-dependent variant of AQL.
- `amva.fli`: Wang-Sevcik fraction-line approximation [84], complementary to `amva.qli`.
- `amva.ab` (alias `ab`): Akyildiz-Bolch approximate MVA for networks with multi-server stations.
- `amva.schmidt` (alias `schmidt`) and `amva.schmidt-ext` (alias `schmidt-ext`): Schmidt and Schmidt-extended approximations [79], designed for multi-server FCFS stations.
- `egflin` and `gflin`: extended-/generalised-feedback Linearizer variants used internally for specialised topologies.
- `sqni`: single-queue numerical iteration, used when the model is reducible to a single-queue network.
- `qna`: Whitt's Queueing Network Analyzer [9, § 6.7], applicable to open networks with general inter-arrival and service distributions.
- `rqna`: Robust Queueing Network Analyzer [88] for single-class open networks, which captures bursty non-renewal (e.g., MMPP/MAP) arrivals. The method is available in the MATLAB, Java, and Python implementations and returns matching results across them. When requested explicitly it applies to any single-class open network, including a single Source-Queue-Sink queue. Under the `default` method it is auto-selected for multi-station bursty open networks, whereas a single-queue Source-Queue-Sink model is routed to the two-station queueing-system formulas (`gm1`/`gig1.*`); in either case the arrival autocorrelation is not silently discarded.
- `sqd`: Smith Queue Decomposition for closed single-chain networks with blocking-after-service (BAS) finite buffers; see the paragraph below.

For single-class closed models the solver additionally exposes the bound families `aba.upper`/`aba.lower` (asymptotic bounds), `bjb.upper`/`bjb.lower` (balanced job bounds), `gb.*` (geometric square-root bounds), `pb.*` (proportional bounds), and `sb.*` (simple bounds). For *multiclass* closed models the `mwba.upper`/`mwba.lower` methods compute the Majumdar-Woodside robust box bounds [54], which are distribution-insensitive per-class throughput bounds valid for any NBUE service distribution and arbitrary routing; the same bounds are exposed for layered networks by `SolverLN`. For two-station open single-class networks it dispatches the `mm1`/`mmk`/`mg1`/`gig1.*`/`gigk.*` formulas; see Table 5.1 for the complete list. The algorithms behind all these methods, and the policy by which `default` selects among them, are documented in the LINE internals manual (`LINE-internals.pdf`, Chapter `MVA`).

Warm start. All AMVA submethods (`bs`, `aql`, `lin`/`gflin`/`egflin`, and the multi-server `conway`/`linearizerms`) accept an initial queue-length solution through `options.init_sol`, used as the starting point of the fixed-point iteration instead of the default uniform N/M guess. This is exploited by `SolverLN`, which passes each layer's previous-iteration solution forward, and by `initFromSolver`, which seeds a solver from an auxiliary solver's steady-state placement.

`initFromSolver` is available on every network solver and is not specific to simulation. The auxiliary solver is used to compute the steady-state distribution of the model, from which an integer job placement is derived: with `CTMC` the placement is the mode of the exact stationary distribution over the aggregate state space, and with any other solver it is the vector of steady-state mean queue lengths rounded so that each closed-class population is conserved. The placement is then applied as the model initial state, which the state-driven solvers honour in their own way: `FLD` starts the ODE integration from it, `SSA` and `LDES` start the simulated trajectory from it, `JMT` preloads the stations with it, and the AMVA submethods above use it as the starting iterate. Because the placement is written into the model object, it is visible to any other solver instance constructed on the same model.

Polling stations. When the model contains queues with `POLLING` scheduling, `MVA` dispatches a dedicated polling analyzer implementing the `stationtime` method for cyclic polling with exhaustive, gated, and limited service.

Blocking-after-service networks. When the model is a closed single-chain network with blocking-after-service (BAS) finite buffers (the `DropStrategy.BlockingAfterService` semantics of §3.1.6), the `default` method automatically dispatches the `sqd` submethod (Smith Queue Decomposition). Multi-chain BAS models are not supported by this method and return empty results.

Extended queueing models are handled as follows:

- Non-exponential service times in FCFS nodes are handled only in the single-server case via the method selected in the `options.config.highvar` setting. By default high variance is ignored, as the FCFS solver tends to produce good result in closed models also without specialized corrections. It is alternatively possible to handle high variance either using the Diffusion-M/G/k interpolation from [14],

casted with weights $a_i = b_i = 10^{-8}$, or using the high-variance MVA (HV-MVA) corrections proposed in [10, 65]. The multi-server extension is ongoing; we point to the NC solver for a version already available.

- Multi-servers are dealt with using the methods listed in Table 5.3 for the `options.config.multiserver` option. These are coupled with a modification of the Rolia-Sevcik correction [71], where in light-load the Rolia-Sevcik correction is treated as if there was a single server.
- Non-preemptive are dealt with using the methods listed in Table 5.3 for the configuration option `options.config.np_priority`. The solver feature in particular AMVA-CL and the shadow server methods [33].
- DPS queues are analyzed with a standard method similar to the biased processor sharing approximation reported in [51, §11.4]. Here, an arriving job of class r sees a queue-length in class $s \neq r$ scaled by the correction factor w_s/w_r, where w_s is the weight of class s.
- Limited load-dependence (intended here as other than multi-server) and class-dependence are handled through the correction factors proposed in [19]. If a station is both limited load-dependent and multi-server, then if the `softmin` method is chosen the solver will suitably combine the softmin term and the limited load-dependent correcting factors. Moreover, iterative queue-length corrections such as those applied by the AQL and Linearizer methods are also applied to these terms. Limited load-dependence (queue-dependent AMVA or QD-AMVA) is handled through correction factors.
- Fork-join networks are assumed to feature a direct acyclic graph (DAG) in-between forks and joins. They are analyzed by iteratively transforming the sibling tasks into jobs belonging to independent classes, using the algorithm specified in `options.config.fork_join`; the transformations are described in the LINE internals manual (`LINE-internals.pdf`, Chapter `MVA`).

Solver-specific configuration options are reported in Table 5.3.

5.2.8 NC

The `NC` class implements a family of solution algorithms based on the normalizing constant of state probability of product-form queueing networks. Contrary to the other solvers, this method typicallly maps the problem to certain multidimensional integrals, allowing the use of numerical methods such as MonteCarlo sampling and asymptotic expansions in their approximation. The `NC` solver is the recommended back-end for the analysis of load-dependent stations specified through `setLoadDependence` (see §3.1.6); the `nrl`, `nrp`, and `rd` methods listed below are designed for this case and remain compatible with the probability metrics `getProb`, `getProbAggr`, and `getProbNormConstAggr`.

For cache models, the `NC` solver supports an importance sampling method (`'sampling'`) that uses Monte Carlo techniques to estimate cache hit probabilities and miss rates. This method is particularly useful for large cache configurations where exact methods become computationally expensive. The sampling method can be enabled by setting the `method` option:

Table 5.3: MVA configuration options (MATLAB)

Option	Value	Description
options.config.multiserver	'default'	Equals 'softmin' at PS queues and 'seidmann' at FCFS queues.
options.config.multiserver	'seidmann'	Seidmann's decomposition [75].
options.config.multiserver	'softmin'	QD-AMVA's softmin approximation [19].
options.config.multiserver	'suri'	Suri et al.'s multiserver approximation [79].
options.config.np_priority	'default'	Non-preemptive priority handling. Equals 'cl'.
options.config.np_priority	'cl'	Chandy-Lakshmi [33].
options.config.np_priority	'shadow'	Sevcik's shadow server [76].
options.config.highvar	'default'	Ignored - no correction applied.
options.config.highvar	'interp'	Diffusion-M/G/k interpolation from [14].
options.config.highvar	'hvmva'	High-variance MVA as in [10], extended to multiclass as [34, Eq. 3.21].
options.config.fork_join	'default'	Equals 'mmt'.
options.config.fork_join	'mmt'	Mixed-model transformation [32]
options.config.fork_join	'ht'	Heidelberger-Trivedi [42]

```
NC(model, 'method', 'sampling', 'samples', 100000).avgTable()
```

The following deterministic methods are available for the NC solver:

- options.method='cub' (Cubature): Numerical integration of the normalizing constant using Grundmann-Müller cubature rules over the simplex of job populations [13]. This method provides high accuracy for small to medium population networks but becomes computationally expensive as the population grows.

- options.method='nrl' (Norlund-Rice Logistic): Approximates the normalizing constant by applying a Laplace (saddle-point) approximation with a logistic sigmoid transformation of the integration variables [17]. Suitable for load-dependent networks with moderate to large populations.

- options.method='nrp' (Norlund-Rice Probit): Similar to nrl but uses the normal cumulative distribution function (probit transform) instead of the logistic sigmoid [17]. Offers an alternative parametrization of the saddle-point approximation for load-dependent networks.

- options.method='rd' (Reduction): An approximation method for load-dependent networks that decomposes the service rate functions into a gamma correction factor applied to the effective network demands [17]. This heuristic enables fast steady-state analysis of state-dependent queueing systems.

- options.method='mem' (Maximum Entropy): Solves open, closed and mixed queueing networks with general (GE-type) arrival and service processes by the Maximum Entropy Method of Kouvatsos [49]; the algorithm is described in the LINE internals manual (LINE-internals.pdf, Chapter NC). The method is exact for $M/G/1$, $M/M/c$ and infinite-server stations. Supported features: Source, Queue, Delay and Sink nodes; FCFS, PS, SIRO, LCFS, LCFS-PR and INF scheduling

(priority disciplines, class switching and reducible routing with absorbing self-loops are rejected); multiserver stations (open models only), while mixed models are restricted to single-server and infinite-server stations. Convergence is controlled by `options.config.mem_tol` (default 10^{-6}), `options.config.mem_maxiter` (default 1000) and `options.config.mem_verbose`. MEM is used only on explicit request: `options.method='default'` always uses the native normalizing-constant path and never selects MEM automatically.

MEM is also the only `NC` method that represents a *finite station buffer*. Every other method is product-form and has no notion of a buffer, so a model carrying a binding `setCapacity` is rejected outright rather than solved as if the buffer were unbounded. On a single-class open model, MEM instead solves each capacitated station as a censored $GE/GE/c/0; N$ queue, the maximum-entropy building block of [49], Section 4.1, whose queue-length distribution coincides with the global balance solution and reduces to the exact $M/M/1/N$ and $M/M/c/N$ distributions when the arrival and service processes are Poisson and exponential. Because a GE arrival stream is a batch process, the probability that an arrival is refused is not the time-stationary probability of a full buffer: it is evaluated per stream from the same station solution, so the external arrivals and the flow from each upstream station each see their own blocking probability.

Two drop rules are supported at a capacitated station. Under `DropStrategy.DROP`, the default that LINE assigns to a finite buffer reachable by an open class, a refused job is lost; the loss probabilities then enter the flow balance and the reported throughputs are carried, not offered, rates. Under `DropStrategy.BAS` a job that completes service upstream and finds the buffer full is held in the upstream server, which cannot serve anyone else until room appears (blocking after service, also called transfer blocking). Transfer blocking is not work conserving, so the network is first transformed into a work-conserving one by the holding-node construction of Tahilramani, Manjunath and Bose [80]: a $GE/GE/\infty$ holding node is inserted on every routing pair with a capacitated destination, absorbing the blocked job so that the upstream server is released, with a service time equal to the residual life of the minimum of the service times in progress at the destination, inflated geometrically because the released job may find the destination full again. The upstream service time is inflated by the same blocking probability so that the jobs queued behind the blocked one still see the server as busy, and the mean queue length reported for the upstream station adds back the holding-node occupancy, the blocked job being physically resident in that station. External arrivals are lost when they find the buffer full whatever the drop rule, there being no upstream server to hold them in, which is also what `SolverCTMC` does.

The finite-buffer path is reported under the method name `mem.blocking`, so that `solver.citations()` returns [80] alongside [49]. It is restricted to single-class open models with FCFS capacitated stations, the censored building block being single class, and it requires every arrival and service process at a capacitated station to have a squared coefficient of variation of at least one, the GE distribution being undefined below that; a hypo-exponential process is rejected rather than approximated. Accuracy follows the published behaviour of the decomposition: exact under the loss rule with Poisson arrivals, and within a few percent under transfer blocking whenever the buffers are

not so tight that the blocking correlation between neighbouring stations dominates.

The `NC` solver also exposes a wider catalogue of asymptotic, sampling, and exact normalising-constant routines reported by `listValidMethods`:

- `options.method='exact'`: automatic selection of an exact normalising-constant algorithm. Closed networks dispatch to `solver_ncld_analyzer` when load dependence is present and to `solver_nc_analyzer` otherwise; open networks dispatch to the open-network back-end.
- `options.method='ca'`: multiclass convolution algorithm computing the normalising constant exactly [9, § 9.1].
- `options.method='comom'`: class-oriented method of moments [12], an exact algorithm with polynomial complexity in the number of classes; recommended for closed homogeneous models with many classes.
- `options.method='imci'`: improved Monte-Carlo integration sampler for the normalising constant [85].
- `options.method='ls'`: logistic sampling estimator [13]; like `imci` it is a Monte-Carlo method but uses a logistic change of variables that improves the effective sample size for moderate-to-large populations.
- `options.method='is'`: importance sampling over job orderings, for closed models only (open and mixed models are rejected). Unlike `imci` and `ls`, which sample the multidimensional integral representation of the normalising constant, this estimator samples the *orderings* of the jobs [27]; the estimator is described in the LINE internals manual (`LINE-internals.pdf`, Chapter `NC`). The number of samples and the random seed are set through the `samples` and `seed` options. The selector `sampling` chooses automatically among `imci`, `ls` and the McKenna-Mitra sampler.
- `options.method='le'`: logistic asymptotic expansion of the normalising constant [13]; the deterministic counterpart of `ls`.
- `options.method='kt'`: Knessl-Tier asymptotic expansion [47] for closed networks in heavy traffic.
- `options.method='gm'`: Grundmann-Möller cubature variant complementary to `cub`.
- `options.method='gleint'`: Gauss-Legendre integration of the normalising-constant integral.
- `options.method='mmint2'`: McKenna-Mitra integral [57], restricted to two-station closed models.
- `options.method='clw'`: Choudhury-Leung-Whitt numerical inversion of the generating function of the normalising constant [24]; exact up to a controllable aliasing error for closed networks with single-server and infinite-server queues, and efficient when a few chain populations are large. On

load-dependent models, the method inverts the generating function built from the Bertozzi-McKenna per-station transforms [6], which cover limited load-dependent rates such as multiserver stations; general load-dependent rates are truncated at the total population without loss of exactness.

- `options.method='panacea'`: Panacea asymptotic expansion [57, 70] for normally-loaded closed networks.

- `options.method='propfair'`: proportionally-fair throughput approximation, a fast heuristic that scales each class throughput by the proportionally-fair allocation derived from the demand matrix.

5.2.9 QNS

The `QNS` class is a wrapper around the standalone `qnsolver` command-line tool distributed with the LQNS suite. It performs a model-to-model transformation from the Network data structure into a JMVA-format input file (`.jmva`), invokes `qnsolver` as an external process, and parses the resulting per-station chain throughputs, queue-lengths, response times, and utilisations back into the LINE result tables. `QNS` is recommended for product-form closed networks with multi-server FCFS stations, where it offers several well-known multi-server approximations not implemented natively by `MVA`.

For non-product-form models, the MATLAB version of `QNS` performs an automatic conversion to a layered queueing network via `QN2LQN` and then dispatches the analysis to `LQNS`. The JAR and Python versions invoke `qnsolver` directly without this fallback, so for non-product-form workloads they may return less accurate results. The `qnsolver` executable is located on the system `PATH`; if it is missing, the call to `QNS` aborts.

The following methods are available for the `QNS` solver:

- `options.method='default'`: when multi-server FCFS stations are present, dispatches to the Rolia-Sevcik approximation (`rolia`); otherwise calls `qnsolver` with its built-in default product-form solution.

- `options.method='conway'`: Conway's multi-server MVA approximation, invoked through the `-mconway` flag of `qnsolver`.

- `options.method='reiser'`: Reiser's multi-server MVA approximation, invoked through `-mreiser`.

- `options.method='rolia'`: Rolia-Sevcik multi-server approximation [71], invoked through `-mrolia`.

- `options.method='zhou'`: Zhou's multi-server approximation, invoked through `-mzhou`.

- `options.method='suri'`: Suri et al.'s multi-server approximation [79], exposed for compatibility with `MVA`'s `config.multiserver=suri` option.

- `options.method='schmidt'`: Schmidt's multi-server approximation, exposed for compatibility with MVA's `config.multiserver=schmidt` option.

The selected method is reflected in the actual back-end command line, e.g. `qnsolver -l model.jmva -mconway -o result.jmva`, so users can replicate QNS runs by hand if needed.

5.2.10 SSA

The SSA class is a basic stochastic simulator for continuous-time Markov chains. It reuses some of the methods that underpin CTMC to generate the network state space and subsequently simulates the state dynamics by probabilistically choosing one among the possible events that can incur in the system, according to the state spaces of each of node in the network. The sampling algorithms and their state representation are described in the LINE internals manual (`LINE-internals.pdf`, Chapter SSA).

The state space is not generated upfront, but typically stored during the simulation, starting from the initial state. If the initialization of a station generates multiple possible initial states, SSA initializes the model using the first state found. The list of initial states for each station can be obtained using the `getState` function of the `Network` class.

The SSA solver offers two general-purpose methods: `'serial'` and `'para'`. The serial method runs on a single core, while the parallel methods run on multicore via MATLAB's `spmd` command. The `'default'` method resolves to `'nrm'` on models that support it (see below) and to `'serial'` otherwise.

SSA solver also implements the much faster next reaction method (`'nrm'`, see [2]). This is available for closed queueing models with exponential service under the INF and PS disciplines, as well as buffered FCFS and LCFS stations through integer (inverse-CDF) routing that preserves the per-buffer job order. The `'nrm'` method is selected by default on such models. Moreover, by setting `options.config.state_space_gen` to `'full'` it is possible to explicitly generate during the simulation the simulated state space and its steady-state probability.

SSA accepts a warmup-discard option `options.config.warmupfrac` (default `0.0`, disabled) that drops the leading fraction of collected samples before computing per-station means, mitigating transient bias on open networks; values of 0.1–0.3 typically bring open-network averages within $\sim$1% of LDES (which uses MSER-5 truncation). SSA also supports the full set of state-dependent dispatching strategies via a routing callback invoked at each event firing — RAND, PROB, RROBIN, WRROBIN, JSQ, SQ, and RL (tabular or quadratic linear value-function approximation), with the SQ marginals matching the with-replacement, first-occurrence tie-break used by LDES and cached per $(d, \text{ndest}, n[\cdot])$ to amortise enumeration cost.

5.2.11 LDES

The LDES solver is the LINE Discrete Event Simulator, a native discrete-event simulation engine built on the SSJ library and shipped inside the canonical JAR. All three codebases drive the same engine, so a given model and seed produce identical sample paths and hence identical estimates. MATLAB drives it as a fully JSON-mediated subprocess: `linemodel_save` writes `model.json`, the engine is invoked as `solve model.json -o result.json`, and the resulting JSON is parsed back. The engine is the native

`common/ldes` binary, falling back to `java -jar common/ldes.jar`; no in-process object marshalling takes place.

The model features admitted by the solver are those declared by its feature set (`SolverLDES.getFeatureSet`), summarized in the tables of Section 5.3: the node types of Table 3.1, the scheduling strategies of Tables 3.2 and 3.5, the routing strategies of Section 5.3.6, and the distributions of Tables 3.7 and 3.8. Two behaviours are specific to the simulator. A `BMAP` assigned as a service process is interpreted as a batch service process, in which the server completes at each service epoch a batch whose size is drawn from the BMAP batch-size distribution and truncated by the number of jobs present, so that the station behaves as an M/BMSP/1 queue; the same `BMAP` assigned at a `Source` produces batch arrivals instead. Finite-buffer stations are honoured on arrival according to the per-station `DropStrategy` (`Drop` discards the job, `BlockingAfterService` and `BlockingBeforeService` block the upstream server), as are network-level `FiniteCapacityRegion` constraints (global capacity, per-class capacity, and linear admission inequalities).

The accuracy of the estimates is governed by the simulation length and by the transient filter and confidence-interval estimators listed in Table 5.4. The solver is validated against `JMT` by a two-sample t-test at the 1% significance level.

Warm start. Passing an auxiliary solver to the constructor, `SolverLDES(model, SolverMVA(model))`, or calling `initFromSolver` on an existing instance, seeds the initial state from the auxiliary solver's steady state. With `CTMC` the initial state is the mode of the exact stationary distribution over the aggregate state space; with any other solver the steady-state mean queue lengths are rounded to an integer placement that conserves each closed-class population. The placement is stored in `options.init_sol` as a station-major vector $[n_{1,1}, \ldots, n_{1,K}, \ldots, n_{M,K}]$ and forwarded through the `--initsol` flag of the engine. Since the simulation then starts approximately in steady state, warmup removal is disabled (`tranfilter` set to `fixed` with `warmupfrac` $= 0$) so that every sample contributes to the estimators. Warm starts remove the initialization bias and reduce the number of samples needed to reach a target accuracy, most visibly in slowly-mixing models such as closed networks with nearly balanced bottlenecks (see `examples/advanced/initState/ldes_warmstart.m`).

Table 5.4 lists the configuration options. Defaults are those of the engine and are shared by all codebases.

Table 5.4: `LDES` configuration options

Option	Default	Description
Run length and reproducibility		
`options.samples`	$2 \cdot 10^5$	Service-completion events to simulate (engine flag `-s`).
`options.seed`	random	Random number generator seed (`--seed`).
`options.method`	`'default'`	Simulation method; only `default` is defined.
`options.timeout`	`Inf`	Wall-clock budget in seconds (`--maxtime`).
Transient (warmup) filter		
`options.config.tranfilter`	`mser5`	Warmup detection: `mser5` (MSER-5), `fixed` (fixed fraction), `none`.
`options.config.mserbatch`	5	MSER batch size; used only when `tranfilter` is `mser5`.

Continued on next page

Table 5.4 – *Continued from previous page*

Option	Default	Description
`options.config.warmupfrac`	0.2	Fraction discarded; used only when `tranfilter` is `fixed`.
Confidence intervals		
`options.config.cimethod`	`obm`	Estimator: `obm` (overlapping batch means), `bm` (batch means), `spectral` (Heidelberger-Welch), `none`.
`options.config.obmoverlap`	0.5	Overlap fraction of `obm`; 0 reduces it to `bm`.
`options.config.ciminbatch`	10	Minimum batch size; the batch size used is $\max(\texttt{ciminbatch}, \sqrt{n})$ on n observations.
`options.config.ciminobs`	100	Minimum post-warmup observations below which no interval is reported.
`options.config.spectral lowfreqfrac`	0.25	Fraction of lowest frequencies in the log-periodogram regression; used only when `cimethod` is `spectral`.
Convergence-based stopping		
`options.config.cnvgon`	`false`	Stop as soon as the relative precision of every metric falls below `cnvgtol`.
`options.config.cnvgtol`	0.05	Target relative precision (CI half-width over mean).
`options.config.cnvgbatch`	20	Minimum batches before the first convergence check.
`options.config.cnvgchk`	0 (auto)	Events between checks; 0 selects `samples`/50.
Discrete-time (slotted) mode, see §5.2.11		
`options.config.slotted`	`false`	Advance time on a discrete lattice instead of continuously.
`options.config.slotlength`	1.0	Slot length in model time units; meaningful only when `slotted` is set.
Replication and execution		
`options.config.replications`	1	Independent replications, each with its own RNG stream; intervals then use the cross-replication variance.
`options.config.numthreads`	engine default	Threads used when `replications` > 1.
`options.init_sol`	empty	Warm-start placement, station-major vector (`--initsol`); set by `initFromSolver`.
`options.rest_url`	empty	Base URL of an LDES REST server; when set the model is POSTed to `<url>/api/v1/solve` instead of being solved locally.

Remark 1 *Every option in the table has a command-line flag, so the subprocess clients (MATLAB and native Python) forward all of them; there is no in-process-only subset. A flag is emitted only when its value departs from the default listed above, so a default run produces a minimal command line.*

Discrete-time (slotted) simulation

By default the `LDES` engine advances on a continuous time scale. Setting `options.config.slotted` switches it to a discrete (slotted) time scale, in which time advances in integer multiples of a slot of length `options.config.slotlength`. In slotted mode every sampled interarrival and service time must fall on the slot lattice $\{d, 2d, \ldots\}$, where d is the slot length. A sample that does not lie on the lattice is reported as an error rather than rounded, so that a mis-specified distribution is detected instead of being silently altered. Within a slot, simultaneous events are ordered by a fixed phase: service completions first, then internal movement between stations, then external arrivals, then bookkeeping. Non-homogeneous Poisson arrivals and the processor-sharing family of scheduling strategies are incompatible with a slotted clock and are rejected.

The canonical discrete-time model is the Geo/Geo/1 queue: a `Source` with a `Geometric` interarrival distribution feeding an `FCFS Queue` with a `Geometric` service distribution, simulated with a slot length

equal to one. Its analytical counterpart is the `qsys_geogeo1` function of the `qsys` API. When the source additionally releases a batch of jobs at each arrival epoch (Section 3.1.1), the resulting stream is the Geo^X arrival process and the reference formulas are provided by `qsys_geoxgeo1` and `qsys_geoxgeo1_moments`. Slotted mode is available in all three codebases and is reachable from the MATLAB wrapper through the options above, which are forwarded to the engine as the `--slotted` and `--slotlength` command-line flags.

Batch service processes

When a `BMAP` (batch Markovian arrival process) is assigned as the *service* distribution of a station, the `LDES` engine interprets it as a batch service process (BMSP): at a service epoch the server removes and completes a batch of jobs whose size is drawn from the BMAP batch-size distribution, truncated by the number of jobs currently present. The resulting station is an M/BMSP/1 queue. This is distinct from a BMAP used at a `Source`, which produces batch arrivals.

The LDES command-line interface

The `LDES` engine is also distributed as a standalone executable, either as the native binary `common/ldes` or as `common/ldes.jar`. The MATLAB and Python wrappers are thin JSON-mediated clients of this executable: they serialize the model to `model.json`, invoke `solve model.json -o result.json`, and parse the returned metrics. The same interface can be driven directly. The principal flags are summarized below.

Table 5.5: `LDES` command-line flags

Flag	Description
`--samples`	Number of simulation events to execute.
`--maxevents`	Hard cap on the number of simulated events.
`--maxtime`	Hard cap on simulated time.
`--seed`	Random number generator seed.
`--method`	Simulation algorithm selector.
`--cnvgon`, `--cnvgtol`	Enable convergence-based stopping and set its relative tolerance.
`--tranfilter`	Transient (warmup) filter: `mser5`, `fixed` or `none`.
`--warmupfrac`	Warmup fraction discarded under the `fixed` filter.
`--cimethod`	Confidence interval estimator.
`--replications`	Number of independent replications; drives the parallel analyzer.
`--numthreads`	Number of worker threads used for replications.
`--slotted`	Run on a discrete (slotted) time scale.
`--slotlength`	Slot length in model time units; implies `--slotted`.
`--timespan`	Time span for transient analysis.
`--initsol`	Warm-start initial state, as a station-major population vector.
`--trajectory`	Export the simulated sample path.
`--export-histogram`	Export the state histogram, which backs reward, `getProb` and sampling queries.

5.2.12 UQ

The UQ (uncertainty quantification) solver is a meta-solver that extends LINE's capabilities to handle models with parameter uncertainty. In many practical scenarios, model parameters such as service rates or routing probabilities are not known precisely, but are instead characterized by a set of plausible alternatives with associated likelihoods. Rather than requiring the analyst to manually explore each alternative, the UQ solver automates this process by accepting parameters specified as probability distributions over discrete alternatives.

When a model contains parameters defined using the Prior distribution class, which encodes a discrete set of alternative parameter values along with their prior probabilities, the UQ solver detects these uncertain parameters and expands the model into a family of concrete instances. Each instance corresponds to one combination of parameter values from the alternatives. The solver then applies a user-specified inner solver to each instance, obtaining performance metrics for all alternatives. Finally, it aggregates the results using the prior probabilities as weights, producing expected performance metrics that account for the parameter uncertainty.

The constructor for the UQ solver takes two arguments: the model containing uncertain parameters, and a solver factory that specifies which analysis method should be applied to each concrete instance. The solver factory can be any of LINE's standard solvers such as MVA, CTMC, or JMT.

```
post = UQ(model, @MVA);
```

The UQ solver provides several methods for retrieving results. The getAvgTable method returns the prior-weighted expected values of all performance metrics, effectively averaging over the parameter uncertainty. The getPosteriorTable method returns the complete set of results for each alternative, allowing the analyst to examine how performance varies across different parameter values. Additionally, the getPosteriorDist method takes a metric type, node, and class as arguments and returns the posterior distribution of that metric as an EmpiricalCDF object, which can be used to compute quantiles or probabilities.

To specify parameter uncertainty, the analyst creates a Prior distribution by providing a list of alternative distributions and their corresponding probabilities. This prior distribution can then be used in place of a standard distribution when configuring model parameters such as service times.

```
prior = Prior({Exp(1), Exp(2), Exp(3)}, [0.3, 0.5, 0.2]);
queue.setService(jobClass, prior);
```

In this example, the service time at the queue is uncertain, with three possible mean service rates having probabilities 0.3, 0.5, and 0.2 respectively. The UQ solver will analyze all three scenarios and provide both individual results and weighted averages. This capability is particularly useful for sensitivity analysis, robust design under uncertainty, and understanding the range of possible performance outcomes when input parameters cannot be measured precisely.

5.3 Supported language features and options

5.3.1 Solver features

Once a model is specified, it is possible to use the `getUsedLangFeatures` function to obtain a list of the features of a model. For example, the following conditional statement checks if the model contains a FCFS node

```
if (model.getUsedLangFeatures.list.SchedStrategy_FCFS)
...
```

Every LINE solver implements the `support` to check if it supports all language features used in a certain model

```
>> JMT.supports(model)
ans =
  logical
   1
```

A supported feature set is a necessary but not always sufficient condition for a method to run. The `BA` solver is the clearest example: its feature set is deliberately narrow, admitting closed product-form-parameterized models only, because bounds are computed from the service demands and populations alone. Several of its methods then impose further structural restrictions that the feature set cannot express, such as the absence of infinite-server stations for `sb` and `sib`, or a single-class single-server model for `lr`. Those methods raise an error on an inadmissible model rather than returning a value that would not be a valid bound. Consequently `supports` may accept a model for which a particular `BA` method is still unavailable, and `listValidMethods` reports the method names rather than their per-model admissibility.

5.3.2 State-dependent routing and service

A model is state-dependent when its routing decisions or its service rates are functions of the current state. On the routing side the strategies are those of Section 5.3.6: `RROBIN` cycles through the destinations in a fixed order, `WRROBIN` does so through a list in which each destination appears as many times as its integer weight, and `JSQ` sends each job to the destination holding the fewest jobs. On the service side, `setLoadDependence` scales the station rate with its occupancy, which is how congestion effects, speed scaling and batch service are represented.

Support differs by solver. `CTMC` handles both families exactly, by carrying the dispatcher state and the occupancy-dependent rates in the state descriptor; the simulators `JMT`, `LDES` and `SSA` evaluate them on the sample path; `MVA` and `NC` handle load-dependent rates through their queue-dependent algorithms but relax state-dependent routing to its probabilistic counterpart. The example below dispatches jobs round-robin from a `Router` to three parallel queues.

```
router = Router(model, 'Dispatcher');
```

```
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);
queue3 = Queue(model, 'Queue3', SchedStrategy.FCFS);
router.setRouting(jobclass, RoutingStrategy.RROBIN, ...
    {queue1, queue2, queue3});
```

Examples demonstrating state-dependent routing can be found in the examples directory, including models that use RROBIN for load distribution and JSQ for dynamic load balancing.

5.3.3 Class functions

The table below lists the steady-state and transient analysis functions implemented by the Network solvers. Since the features of the LINE solver are the union of the features of the other solvers, in what follows it will be omitted from the description.

Table 5.6: Solver support for average performance metrics

		Network Solver							
Function	**Regime**	CTMC	LDES	FLD	JMT	MAM	MVA	NC	SSA
avg	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgTable	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgChainTable	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgNode	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgNodeTable	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgNodeChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgNodeChainTable	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgSys	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgSysTable	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgArvR	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgArvRChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgNodeArvRChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgQLen	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgQLenChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgNodeQLenChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgRespT	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgRespTChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgNodeRespTChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgSysRespT	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgTput	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgTputChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgNodeTputChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgSysTput	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgUtil	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgUtilChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
avgNodeUtilChain	Steady-state	✓	✓	✓	✓	✓	✓	✓	✓
getTranAvg	Transient	✓	✓	✓	✓	✓			

Table 5.7: Solver support for advanced metrics

		Network Solver							
Function	**Regime**	CTMC	LDES	FLD	JMT	MAM	MVA	NC	SSA
getCdfRespT	Steady-state	✓	✓	✓	✓	✓			
getAvgAoI	Steady-state			✓					
getCdfAoI	Steady-state			✓					
getProb	Steady-state	✓	✓						
getProbAggr	Steady-state	✓	✓	✓	✓		✓	✓	
getProbMarg	Steady-state					✓	✓	✓	
getProbSys	Steady-state	✓	✓						✓
getProbSysAggr	Steady-state	✓	✓		✓			✓	
getProbNormConstAggr	Steady-state				✓		✓	✓	
getCdfPassT	Steady-state			✓		✓	✓		
getAvgReward	Steady-state	✓	✓						
getTranCdfPassT	Transient		✓	✓					
getTranCdfRespT	Transient		✓		✓				
getTranProb	Transient	✓	✓						
getTranProbAggr	Transient	✓	✓						
getTranProbSys	Transient	✓	✓						
getTranProbSysAggr	Transient	✓	✓						
getTranReward	Transient	✓	✓						
sample	Transient		✓						✓
sampleAggr	Transient		✓		✓				✓
sampleSys	Transient		✓						✓
sampleSysAggr	Transient		✓		✓				✓

The functions listed above with the `Table` suffix (e.g., `avgTable`) provide results in tabular format corresponding to the corresponding core function (e.g., `avg`). The features of the core functions are as follows:

- `avg`: returns the mean queue-length, utilization, mean response time (for one visit), and throughput for each station and class.
- `avgChain`: returns the mean queue-length, utilization, mean response time (for one visit), and throughput for every station and chain.
- `avgSys`: returns the system response time and system throughput, as seen as the reference node, by chain.
- `getCdfRespT`: returns the distribution of response times (for one visit) for the stations at steady-state.
- `getCdfPassT`: returns the distribution of *passage* (first-visit) times at steady-state for the stations of interest, complementing the transient counterpart `getTranCdfPassT`.
- `getProbMarg`: returns the marginal queue-length distribution $P(n_{i,r} = k)$ for a single class r at a station i, as opposed to the joint per-class distribution returned by `getProbAggr`.

- `getAvgAoI`: returns the mean and variance of Age of Information (AoI) and Peak AoI for single-queue open systems with capacity constraints.
- `getCdfAoI`: returns the cumulative distribution function of AoI and Peak AoI.
- `avgNode`: behaves similarly to `avg`, but returns performance metrics for each node and class. For example, throughputs at the sinks can be obtained with this method.
- `getProb`: returns state probabilities at equilibrium at a given station.
- `getProbAggr`: returns marginal state probabilities for jobs of different classes at a given station.
- `getProbSys`: returns joint probabilities for a given system state.
- `getProbSysAggr`: returns joint probabilities for jobs of different classes at all stations.
- `getProbNormConstAggr`: returns the normalizing constant of the state probabilities for the model.
- `getTranAvg`: returns transient mean queue length, utilization and throughput for every station and chain from a given initial state.
- `getTranCdfPassT`: returns the distribution of first passage times in transient regime.
- `getTranCdfRespT`: returns the distribution of response times in transient regime.
- `sample`: returns the transient marginal state for a station from a given initial state.
- `sampleAggr`: returns the transient marginal state for jobs of different classes at a given station from a given initial state.
- `sampleSys`: returns the transient marginal system state for a station from a given initial state.
- `sampleSysAggr`: returns the transient marginal system state for jobs of different classes at a given station from a given initial state.
- `getAvgReward`: returns the steady-state expected value $E[r] = \sum_s \pi(s)\, r(s)$ of each Markov reward function r defined via `setReward`. The `CTMC` solver computes it exactly from the stationary distribution, while the `LDES` solver estimates it by time-integrating the reward over the exact joint state along the simulated sample path, so it is correct also for nonlinear rewards.
- `getTranReward`: returns the transient expected reward $E[r(X(t))]$ over time for each reward function. The `CTMC` solver uses the transient distribution from uniformization, while the `LDES` solver returns the reward trajectory sampled along the simulated path.

5.3.4 Node types

The table below shows the node types supported by the different solvers. It should be noted that the `FLD` solver is capable of handling Sink and Source nodes, but due to low accuracy when run on open models this feature is disabled in the current release.

Table 5.8: Solver support for Network nodes

	Network Solver							
Strategy	CTMC	LDES	FLD	JMT	MAM	MVA	NC	SSA
Cache	✓	✓	✓	✓		✓	✓	✓
ClassSwitch	✓	✓	✓	✓	✓	✓	✓	✓
Delay	✓	✓	✓	✓	✓	✓	✓	✓
Fork	✓	✓		✓	✓	✓		✓
Join	✓	✓		✓	✓	✓		✓
Queue	✓	✓	✓	✓	✓	✓	✓	✓
Sink	✓	✓	✓	✓	✓	✓	✓	✓
Source	✓	✓	✓	✓	✓	✓	✓	✓

5.3.5 Scheduling strategies

The table below shows the supported scheduling strategies within LINE queueing stations. Each strategy belongs to a policy class:

- preemptive resume (`SchedStrategyType.PR`)
- preemptive non-resume, i.e. restart with an independent resample (`SchedStrategyType.PNR`)
- non-preemptive (`SchedStrategyType.NP`)
- non-preemptive priority (`SchedStrategyType.NPPrio`)
- preemptive-resume priority (`SchedStrategyType.PRPrio`).

The `SchedStrategyType.PR` policy class includes scheduling strategies like `LCFSPR`, where a preempted job resumes service from the point it was interrupted. In contrast, the `SchedStrategyType.PNR` policy class includes strategies like `LCFSPI` (Last-Come-First-Served Preemptive-restart Independent), where a preempted job restarts from scratch with a new service time sample that is independent of the past.

The table primarily refeers to invocation of the `avg` methods. Specialized methods, such as transient or response time distribution analysis, may be available only for a subset of the scheduling strategies supported by a solver.

The table lists both classical and specialized disciplines; the definitions of the latter are recalled in Table 3.2. Two entries are worth isolating because they are not scheduling disciplines at all: `SQ` and `RL` are *routing* strategies (Section 5.3.6), `SQ` dispatching each job to the shortest of d randomly sampled queues

Table 5.9: Solver support for scheduling strategies

		Network Solver							
Strategy	**Class**	CTMC	LDES	FLD	JMT	MAM	MVA	NC	SSA
FCFS	NP	✓	✓	✓	✓	✓	✓	✓	✓
INF	NP	✓	✓	✓	✓	✓	✓	✓	✓
SIRO	NP	✓	✓		✓		✓	✓	✓
SEPT	NP	✓	✓		✓				✓
SRPT	NP		✓		✓				
FSP	NP		✓						
SJF	NP		✓		✓				
FCFSPRIO	NPPrio	✓	✓		✓		✓		✓
EDD	NPPrio		✓		✓				
EDF	PRPrio		✓		✓				
PS	PR	✓	✓	✓	✓	✓	✓	✓	✓
DPS	PR	✓	✓	✓	✓		✓		✓
GPS	PR	✓	✓		✓				✓
PSPRIO	PRPrio	✓	✓		✓				✓
DPSPRIO	PRPrio	✓	✓		✓				✓
GPSPRIO	PRPrio	✓	✓		✓				✓
HOL	NPPrio	✓	✓	✓	✓	✓	✓		✓
LCFS	NP	✓	✓	✓	✓		✓	✓	✓
LCFSPR	PR	✓	✓	✓	✓		✓	✓	✓
LPS	PR	✓	✓		✓				✓
POLLING	NP	✓	✓		✓		✓		✓
PAS	NP	✓	✓						✓
OI	NP	✓	✓						✓

and RL following a policy learned offline. The specialized disciplines are supported mainly by CTMC, LDES, JMT and SSA, as the feature table below records. Worked models are in the examples folder, notably the exhaustive and gated polling examples and the DPS examples with class-dependent weights.

5.3.6 Routing strategies

The table below summarises which routing strategies are honoured by each network solver. Probabilistic strategies (RAND, PROB) reduce to the entries of the routing matrix, so every solver supports them. State-dependent strategies (RROBIN, WRROBIN, JSQ, SQ, RL) require either an explicit dispatcher in the simulator (LDES, JMT) or a per-event marginal-probability callback (SSA). Analytical solvers (MVA, NC, MAM, FLD) approximate non-product-form policies by their probabilistic relaxation and are not appropriate for dispatching-sensitive analyses; CTMC handles RROBIN, WRROBIN, JSQ and SQ exactly, by augmenting the state descriptor with the dispatcher state, at the cost of an enlarged state space.

Table 5.10: Solver support for routing strategies

	Network Solver							
Strategy	CTMC	LDES	FLD	JMT	MAM	MVA	NC	SSA
RAND	✓	✓	✓	✓	✓	✓	✓	✓
PROB	✓	✓	✓	✓	✓	✓	✓	✓
RROBIN	✓	✓		✓				✓
WRROBIN	✓	✓		✓				✓
JSQ	✓	✓		✓				✓
SQ	✓	✓		✓				✓
RL		✓						✓
DISABLED	✓	✓	✓	✓	✓	✓	✓	✓

5.3.7 Statistical distributions

The table below summarizes the current level of support for arrival and service distributions within each solver. Replayer represents an empirical trace read from a file, which will be either replayed as-is by the JMT solver, or fitted automatically to a Cox by the other solvers. Note that JMT requires that the last row of the trace must be a number, *not* an empty row.

5.3.8 Solver options

Table 5.12 summarizes the main options available within the LINE solvers and their default values. Solver options are encoded in LINE in a structure array that is internally passed to the solution algorithms. The global defaults for the solvers can be manually adjusted by editing the `lineDefaults.m` file in the root folder.

This can be specified as an argument to the constructor of the solver. For example, the following two constructor invocations are identical

Table 5.11: Solver support for statistical distributions

	Network Solver							
Distribution	CTMC	LDES	FLD	JMT	MAM	MVA	NC	SSA
APH	✓	✓		✓	✓	✓	✓	✓
BMAP					✓			
Coxian	✓	✓	✓	✓	✓	✓	✓	✓
Cox2		✓	✓	✓				
Exp	✓	✓	✓	✓	✓	✓	✓	✓
Erlang	✓	✓	✓	✓	✓	✓	✓	✓
HyperExp	✓	✓	✓	✓	✓	✓	✓	✓
MAP	✓	✓		✓	✓			✓
ME		✓		✓	✓			
MMPP2	✓	✓		✓	✓			✓
PH	✓	✓		✓				✓
RAP		✓		✓	✓			
Disabled	✓	✓	✓	✓	✓	✓	✓	✓
Det		✓		✓				
Gamma		✓		✓				
Lognormal		✓		✓				
Pareto		✓		✓				
Replayer		✓		✓				
Uniform		✓		✓				
Weibull		✓		✓				

```
s = JMT(model)
opt = JMT.defaultOptions; s = JMT(model, opt)
```

Modifiers to the default options can either be specified directly in the `options` data structure, or alternatively be specified as argument pairs to the constructor, i.e., the following two invocations are equivalent

```
s = JMT(model,'samples',1e6)
opt = JMT.defaultOptions; opt.samples=1e6; s = JMT(model, opt)
```

Available solver options are as follows:

- `cache` (`logical`) if set to `true` the solver after the first invocation will return the same result upon subsequent calls, without solving again the model. This option is `true` by default. Caching can be bypassed using the `refresh` methods (see Section 4.7).
- `config` (`struct`) this is data structure to pass solver-specific configuration options to customize the execution of particular methods.
- `cutoff` (`integer` ≥ 1) requires to ignore states where stations have more than the specified number of jobs. A finite cutoff is required to analyze open classes using the CTMC solver; it defaults to 10 and is auto-adjusted for open and mixed models.

- `force` (`logical`) requires the solver to proceed with analyzing the model. This bypasses checks and therefore can result in the solver either failing or requiring an excessive amount of resources from the system.
- `iter_max` (`integer` ≥ 1) controls the maximum number of iterations that a solver can use, where applicable. If `iter_max=` n, this option forces the `FLD` solver to compute the ODEs over the timespan $t \in [0, 10n/\mu^{\min}]$, where $\mu^{\min}$ is the slowest service rate in the model. For the `MVA` solver this option instead regulates the number of successive substitutions allowed in the fixed-point iteration.
- `iter_tol` (`double`) controls the numerical tolerance used to convergence of iterative methods. In the `FLD` solver this option regulates both the absolute and relative tolerance of the ODE solver.
- `level` (`integer` ≥ 1, `BA` only) sets the depth of the bound hierarchies `pbh`, `cbh`, `pbk`, `bjbk` and `sib`, and defaults to 2

- `config.variates` (`string`, `LDES` only) specifies the variance reduction method for simulation. Valid values are `'none'` (no variance reduction, default), `'antithetic'` (antithetic variates using synchronized 1-U method to generate paired samples with negative correlation), `'control'` (control variates using mean-based correction that applies post-hoc corrections based on deviation of sampled means from known theoretical means), or `'both'` (combined antithetic and control variates). Default is `'none'`.

- `events` (`integer` ≥ 1, `SSA` and `LDES`) event budget of the discrete-event simulation solvers: the number of reaction firings simulated by `SSA`, and the number of service completion events simulated by `LDES`. When set, it overrides `samples`, which remains accepted as a deprecated alias for the event budget; when unset (default), the solvers fall back to `samples` (10^4 events for `SSA`, $2 \cdot 10^5$ for `LDES`).

- `keep` (`logical`) determines if the model-to-model transformations store on file their intermediate outputs. In particular, if `verbose`≥ 1 then the location of the `.jsimg` models sent to JMT will be printed on screen.

- `lang` (`string`) determines the codebase used to run the chosen method: `'matlab'` (the default), `'java'`, or `'python'`. Under `'java'` the model is marshalled to the JLINE Java classes and the analysis runs inside `jline.jar`; the Java codebase does not support every method available in MATLAB, but it is faster. Under `'python'` the model is marshalled instead to the native (pure Python, no JVM) `line_solver` package and the analysis runs in-process through MATLAB's `py.*` interface. This backend is wired for `MVA`, `NC`, `CTMC`, `MAM`, `FLD`, `SSA`, `AUTO`, `LN` and `ENV`; `JMT` rejects it, since there is no native-Python JMT. It requires MATLAB's `pyenv` to point at a CPython interpreter where `line_solver` is importable, and it pins the BLAS thread count to one, because running NumPy in-process alongside MATLAB's own worker threads oversubscribes the machine and can stall the session on heavy algorithms. Most models are marshalled element by element; `Cache`, stochastic Petri net and `Environment` models instead round-trip through the JSON model format, and layered models round-trip through the LQNX interchange. Two capabilities are intentionally not bridged, because they have no native counterpart: reinforcement-learning routing and transient analysis of layered models.

- `method` (`string`) configures the internal algorithm used to solve the model.

- `samples` (`integer` ≥ 1) statistical sample budget of the simulation- and sampling-based solvers; its unit is solver-dependent. For `JMT` it is the number of samples collected *for each* performance index, with a required minimum of $5 \cdot 10^3$. For `NC`, it is the number of Monte Carlo draws used by the sampling-based normalizing-constant methods. For `SSA` and `LDES`, it is a deprecated alias of `events` (see above) and is used only when `events` is not set.

- `seed` (`integer` ≥ 1) controls the seed used by the pseudo-random number generators. For example, simulation-based solvers will give identical results across invocations only if called with the same `seed`.

- `stiff` (`logical`) requires the solver to use a stiff ODE solver.

- `timeout` (`double`, seconds, default `Inf`) sets a cooperative wall-clock budget measured from solver launch. The budget is polled at checkpoints inside the long-running phases of the analysis, in particular during state-space generation and inside iterative analyzers. When the budget is exhausted the solver raises an error rather than returning a partially converged or partially enumerated result, so that a timed-out run cannot be mistaken for a completed one. The default `Inf` disables the check.

- `timespan` (`real interval`) requires the transient solver to produce a solution in the specified temporal range. If the value is set to $[\texttt{Inf}, \texttt{Inf}]$ the solver will only return a steady-state solution. For the `FLD` solver and in simulation, this setting has the same computational cost of $[0, \texttt{Inf}]$, therefore the latter is used as default for this solver.

- `timestep` (`double`) controls the fixed time interval for transient analysis in the `CTMC` solver. When specified, the solver generates equally-spaced time points instead of using adaptive time stepping. If not specified or set to empty, the solver uses adaptive ODE time stepping. This option only affects transient analysis and is ignored for steady-state computations.

- `tol` default numerical tolerance for all uses other than the ones where `iter_tol` is used.

- `tranfilter` (`string`, `LDES` only) specifies the transient (warmup) detection method. Valid values are `'mser5'` (MSER-5 automatic truncation, default), `'fixed'` (fixed fraction warmup removal), or `'none'` (no transient filtering). The `SSA` solver does not expose adaptive transient detection but supports a configurable warmup discard via `warmupfrac` (see below).

- `mserbatch` (`integer` ≥ 1, `LDES` only) batch size for the MSER transient detection algorithm. Only used when `tranfilter='mser5'`. Default is 5 (standard MSER-5).

- `warmupfrac` (`real` $\in [0, 1)$, `SSA` and `LDES`) fraction of collected samples discarded as warmup before computing steady-state averages. For `LDES` this is used only when `tranfilter='fixed'` (default 0.2); for `SSA` it controls the initial-sample discard directly (default 0.0, i.e. disabled). Values of 0.1–0.3 are typical on open networks.

- `obmoverlap` (`real` $\in [0, 1]$, `LDES` only) overlap fraction for overlapping batch means confidence intervals. A value of 0.5 means 50% overlap (standard OBM). Only used when `cimethod='obm'`. Default is 0.5.

- `spectralLowFreqFrac` (`real` $\in (0, 1]$, `LDES` only) fraction of the lowest periodogram frequencies used for the log-periodogram regression in the Heidelberger–Welch spectral method. A smaller value concentrates the regression near frequency zero, giving a more local estimate of the spectral density; a larger value uses more frequencies for a more stable but potentially biased estimate. Only used when `cimethod='spectral'`. Default is 0.25.

- `cnvgon` (`logical`, `LDES` only) enables convergence-based stopping. When enabled, simulation stops when the confidence interval half-width relative to the mean falls below the convergence tolerance for all metrics. Default is `false`.

- `cnvgtol` (`real` $\in (0, 1)$, `LDES` only) convergence tolerance threshold. Simulation stops when (CI half-width/mean) $<$ `cnvgtol` for all metrics. A value of 0.05 means 5% relative precision. Default is 0.05.
- `cnvgbatch` (`integer` ≥ 1, `LDES` only) minimum number of batches required before checking for convergence. More batches provide more reliable confidence interval estimates. Default is 20.
- `cnvgchk` (`integer` ≥ 0, `LDES` only) number of events between convergence checks. A value of 0 means auto-calculate as `samples`$/50$. Default is 0 (auto).
- `verbose` controls the verbosity level of the solver. Supported levels are 0 for silent, 1 for standard verbosity, 2 for debugging.

5.4 Solver maintenance

The following best practices can be helpful in maintaining the LINE installation:

- To install a new release of JMT, it is necessary to delete (or overwrite) the `JMT.jar` file under the `'JMT'` folder. This forces LINE to download the latest version of the JMT executable.
- To remove temporary by-products of the JMT solver it is recommended to periodically run the `jmtCleanTempDir` script. This is more important when using the `'keep'` option, which stores on disk the temporary `.jsimg` and `.jsimw` models sent to JMT.

Table 5.12: Default values of the LINE solver options and their default assignments (MATLAB)

	Solver default							
Option	MVA	CTMC	LDES	FLD	JMT	MAM	NC	SSA
cache	true	true	true	true	true	true	true	true
confint			0.95		false			false
config								
cutoff		10						
force	false	false	false	false	false	false	false	false
keep			true		true			
init_sol	[]			[]				
iter_max	10^3			200				
iter_tol	10^{-6}			10^{-4}		10^{-4}		
method	'default'	'default'	'default'	'default'	'default'	'default'	'default'	'default'
lang	'matlab'	'matlab'	'matlab'	'matlab'	'matlab'	'matlab'	'matlab'	'matlab'
samples			$2 \cdot 10^5$		10^4			10^4
events			unset					unset
seed	rand	rand	rand	rand	rand	rand	rand	rand
stiff				true				
timeout	Inf	Inf	Inf	Inf	Inf	Inf	Inf	Inf
timespan		[Inf,Inf]	[Inf,Inf]	[0,Inf]	[Inf,Inf]		[Inf,Inf]	[0,Inf]
tol		10^{-4}		10^{-4}				
verbose	1	1	1	1	1	1	1	1

Chapter 6

Layered network models

In this chapter, we present the de nition of the LayeredNetwork class, which encodes the support in LINE for a class of generalized layered stochastic networks. In their basic form, these models are called layered queueing networks (LQNs) and differ from regular queueing networks as servers, in order to process jobs, can issue synchronous and asynchronous calls among each others. We point to [35] and to the LQNS user manual for an introduction [36]. Contrary to the original LQNs, layered networks in LINE can also include non-queueing servers, such as caches, hence they may be conceptualized as more general layered stochastic networks.

The topology of call dependencies in a layered network makes it possible to partition the model into a set of layers, each consisting of a subset of the servers. Each of these layers is then solved in isolation, updating with an iterative procedure its parameters and performance metrics until the layers solutions jointly converge to a consistent solution.

6.1 Basics about layered networks

Layered network models describe a collection of resources called tasks, each representing for example a software server, that run on resources called host processors. Classes of service exposed by a task are called entries. Each entry is an endpoint at which a task can be invoked; for example, if a task represents a web server then its web pages may be described as different entries.

A special task, called the reference task, is used to represent a group of system users. In this case, the host processor for a reference task can either be real, as in the case of users that are themselves software systems, or ctitious, as in the case of human users.

Each entry can be speci ed by a work ow of operations called activities, typically organized as a directed acyclic graph. The time demand that each activity places at the underpinning host processor is called its demand, and it is a random variable with a user-speci ed distribution.

Activity graphs may include calls to entries exposed by other tasks. This is an abstraction of the calls that distributed system components have among themselves. Calls can be synchronous, asynchronous, or

Table 5.11: Solver support for statistical distributions

	Network Solver							
Distribution	CTMC	LDES	FLD	JMT	MAM	MVA	NC	SSA
APH	✓	✓		✓	✓	✓	✓	✓
BMAP					✓			
Coxian	✓	✓	✓	✓	✓	✓	✓	✓
Cox2		✓	✓	✓				
Exp	✓	✓	✓	✓	✓	✓	✓	✓
Erlang	✓	✓	✓	✓	✓	✓	✓	✓
HyperExp	✓	✓	✓	✓	✓	✓	✓	✓
MAP	✓	✓		✓	✓			✓
ME		✓		✓	✓			
MMPP2	✓	✓		✓	✓			✓
PH	✓	✓		✓				✓
RAP		✓		✓	✓			
Disabled	✓	✓	✓	✓	✓	✓	✓	✓
Det		✓		✓				
Gamma		✓		✓				
Lognormal		✓		✓				
Pareto		✓		✓				
Replayer		✓		✓				
Uniform		✓		✓				
Weibull		✓		✓				

```
s = JMT(model)
opt = JMT.defaultOptions; s = JMT(model, opt)
```

Modifiers to the default options can either be specified directly in the `options` data structure, or alternatively be specified as argument pairs to the constructor, i.e., the following two invocations are equivalent

```
s = JMT(model,'samples',1e6)
opt = JMT.defaultOptions; opt.samples=1e6; s = JMT(model, opt)
```

Available solver options are as follows:

- `cache` (`logical`) if set to `true` the solver after the first invocation will return the same result upon subsequent calls, without solving again the model. This option is `true` by default. Caching can be bypassed using the `refresh` methods (see Section 4.7).
- `config` (`struct`) this is data structure to pass solver-specific configuration options to customize the execution of particular methods.
- `cutoff` (`integer` ≥ 1) requires to ignore states where stations have more than the specified number of jobs. A finite cutoff is required to analyze open classes using the CTMC solver; it defaults to 10 and is auto-adjusted for open and mixed models.

replica of the called task receives calls from `value` replicas of the source task. The product of fan-in and fan-out values must be consistent with the replica counts of the two tasks involved in each call. The `lqn_sockshop` example in the `examples/basic/layeredModel/` folder demonstrates a microservice-style topology in which an edge task `T1` is replicated $2\times$ and front-end / back-end tasks declare fan-in and fan-out values around it:

```
P{2} = Processor(model, 'P2_1', 1, SchedStrategy.PS, 0.1);
P{2}.setReplication(2);
T{2} = Task(model, 'T1', 24, SchedStrategy.FCFS).on(P{2});
T{2}.setFanOut('T2', 1);
T{3} = Task(model, 'T2', 21, SchedStrategy.FCFS).on(P{3});
T{3}.setFanIn('T1', 1);
T{3}.setFanOut('T3', 1);
```

Fan-out destinations and fan-in sources are referenced by task name, so the corresponding tasks must already exist (or be created later in the model). Multiple fan-out destinations may be specified by repeated calls to `setFanOut`; each call appends a new `(destination, value)` pair.

6.2.5 Describing host demands of entries

The demands placed by an entry on the underpinning host (also called in layered queueing networks the *host demand*) is described in terms of execution of one or more activities. Although in tools such as LQNS activities can be associated to either entries or tasks, LINE supports only the more general of the two options, i.e., the definition of activities at the level of tasks. In this case:

- Every task defines a collection of activities.
- Every entry needs to specify an initial activity where the execution of the entry starts (the activity is said to be "bound to the entry") and a replying activity, which upon completion terminates the execution of the entry.

For example, in our running example, we may now associate an activity to each entry as follows:

```
A1 = Activity(model, 'A1', Exp(1.0)).on(T1).boundTo(E1).synchCall(E2,3.5);
A2 = Activity(model, 'A2', Exp(2.0)).on(T2).boundTo(E2).repliesTo(E2);
```

Here, `A1` is a task activity for `T1`, acts as initial activity for `E1`, consumes an exponentially distributed time on the processor underpinning `T1`, and requires on average 3.5 synchronous calls to `E2` to complete. Each call to entry `E2` is served by the activity `A2`, with a demand on the processor hosting `T2` given by an exponential distribution with rate $\lambda = 2.0$.

Activity graphs

Often, it is useful to structure the sequence of activities carried out by an entry in a graph. Activity graphs can be characterized by precedence relationships of the following kinds:

- *sequence*: two activities are executed sequentially, one after each other. This is implemented through the `ActivityPrecedence.Serial` construct.
- *loop*: an activity is repeated a number of times. This is implemented in `ActivityPrecedence.Loop`.
- *and-fork*: a serial execution is forked into concurrent activities. This can be materialized using the `ActivityPrecedence.AndFork` construct.
- *or-fork*: the server chooses probabilistically which activity to execute next among a set of alternatives. This is implemented in `ActivityPrecedence.OrFork`.
- *and-join*: concurrent activities are joined into a single serial execution. This is implemented in `ActivityPrecedence.AndJoin`.
- *or-join*: merge point for alternative activities that may execute in parallel after a *or-fork*. This is implemented in `ActivityPrecedence.OrJoin`.
- *cache-access*: split point for cache hit/cache miss results in an activity graph. This is implemented in `ActivityPrecedence.CacheAccess`. For usage examples, see `cache_replc_lru` and `cache_compare_replc` in the `examples/` folder.

A composite example showing fork/join precedences and loops is given in `lqn_workflows` in the `examples/` folder.

An AND-fork precedence spawns several execution paths that proceed independently until they converge at the matching AND-join, which blocks until every spawned branch has completed and then resumes a single thread of control. A branch may contain any sequence of activities, including processor demands, synchronous calls to lower-layer entries, and nested fork-join structures. The construct differs from the fork-join of ordinary queueing networks in that activities are bound to tasks and processors, so contention arises at both levels; and because branches may call entries in different layers, the response time of the forking task is the maximum over branch completion times, each of which depends in turn on lower-layer response times. An entry is therefore complete only when all activities in its activity graph, including every branch of an AND-fork, have synchronized.

Solvers evaluate these structures by treating each fork-to-join branch as a subnetwork, estimating its completion-time distribution, and deriving the distribution of the maximum, which is the synchronization delay at the join. `LN` approximates that maximum by moment matching on matrix-geometric branch representations, whereas `LQNS` decomposes the model layer by layer and refines the parameterization iteratively. The `lqn_workflows` example exercises several AND-fork/AND-join pairs spanning multiple layers.

For instance, we may replace in the running example the specification of the activities underpinning a call to `E2` as

```
A20 = Activity(model, 'A20', Exp(1.0)).on(T2).boundTo(E2);
A21 = Activity(model, 'A21', Erlang.fitMeanAndOrder(1.0,2)).on(T2);
A22 = Activity(model, 'A22', Exp(1.0)).on(T2).repliesTo(E2);
T2.addPrecedence(ActivityPrecedence.Serial(A20, A21, A22));
```

such that a call to `E2` serially executes `A20`, `A21`, and `A22`

This precedence is meaningful only when `preAct` is bound to an `ItemEntry` of a `CacheTask`; see Section 6.2.3 for the cache modeling primitives.

6.2.6 Activity configuration

Beyond the host demand specified at construction time, individual activities expose a number of methods that influence how they participate in the activity graph and in the analysis.

Two-phase activities (`setPhase`). The two-phase model allows part of an activity's execution to be carried out after a synchronous reply has been sent to the caller. Activities are by default in phase 1 (executed before the reply) and can be moved to phase 2 (executed after the reply, while the caller proceeds in parallel) using `A.setPhase(2)`. Only phase values of 1 or 2 are accepted. Phase-2 activities show up in the layered network struct under the `actthink_*` fields.

Per-activity think time (`setThinkTime`). An activity can carry an additional think-time component, separate from the host demand and from the task-level think time, modeling for example a deferred operation. The method `A.setThinkTime(d)` accepts either a numeric mean (which produces an exponential distribution) or any `Distribution` object. A zero or negligible mean is automatically replaced by an `Immediate` distribution.

Asynchronous calls (`asynchCall`). In addition to `synchCall`, an activity can issue non-blocking calls to entries of other tasks via `A.asynchCall(E, m)`. The destination `E` can be passed either as an `Entry` object or by name; the second argument `m` is the mean number of calls per activity execution and defaults to 1.0. Multiple distinct destinations can be added by repeated calls, but a single destination cannot be registered twice – to model multiple calls to the same entry, increase `m` accordingly.

Open arrivals at entries (`Entry.setArrival`). An entry can be turned into an open arrival point by attaching an arrival distribution. The method `E.setArrival(dist)` accepts any `Distribution` object, e.g., `Exp(2.5)` for Poisson arrivals at rate 2.5 or `Erlang(2,0.4)` for an Erlang-2 stream. Open-arrival entries surface in the layered network struct through the `arrival_*` fields and let the entry serve external workload alongside (or instead of) calls received from a reference task.

6.2.7 Debugging and visualization

The structure of a `LayeredNetwork` object can be graphically visualized as follows

```
plot(model)
```

An example of the result is shown in the next figure. The figure shows two processors (`P1` and `P2`), two tasks (`T1` and `T2`), and three entries (`E1`, `E2`, and `E3`) with their associated activities. Both dependencies and

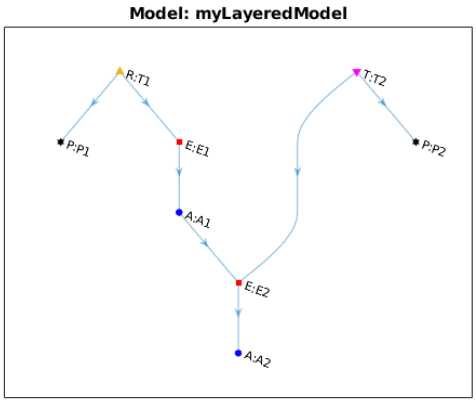

Figure 6.1: `LayeredNetwork.plot` method

calls are both shown as directed arcs, with the edge weight on call arcs corresponding to the average number of calls to the target entry. For example, `A1` calls `E3` on average 2.0 times. In this figure, by clicking with the mouse on a node MATLAB will display, using a data tip, some relevant properties of the node such as scheduling or multiplicity.

When invoked as

```
plot(model,true)
```

The `jsimgView` and `jsimwView` methods can be used to visualize in JMT each layer. This can be done by first calling the `getLayers` method to obtain a cell array consisting of the Network objects, each one corresponding to a layer, and then invoking the `jsimgView` and `jsimwView` methods on the desired layer. This is discussed in more details in the next section.

Lastly, we note a number of specification issues that trigger errors in the LQN definition:

Error Type	**Error Type**
Activity in REF task replies	Entry called both synchronously and asynchronously
Entry on task calls itself	Repeated definition of parent task
Entry on task calls entry on the same task	Invalid .on() argument for an activity
Cycle in activity graph	Invalid .on() argument for a task
Unsupported replyTo	Repeated synch calls
Activity with boundTo specification	Repeated asynch calls

6.3 Internals

6.3.1 Representation of the model structure

It is possible to access the internal representation of a `LayeredNetwork` model in a similar way as for Network objects, i.e.:

```
lqn = model.getStruct();
```

The return `lqn` structure, of class `LayeredNetworkStruct`, contains all the information about the specified model. It relies on relative and absolute indexing for the elements of the `LayeredNetwork`.

- A *relative* index is a number between 1 and the number of similar elements in the model, e.g., for a model with 3 tasks, the relative index t of a task would be a number in $[1, 3]$.
- An *absolute* index is a number between 1 and the total number of elements (of any kind, except calls) in the model, e.g., for a model with 2 hosts, 3 tasks, 5 entries, and 8 activities, the total number of elements is `nidx`$= 18$ and last activity a may have an absolute index `aidx`$= 18$ and a relative index `a`$= 8$.
- The difference between the relative and the absolute index of an element is referred to as *shift*, e.g., in the previous example `ashift`$= 18 - 8 = 10$.
- Absolute and relative indexing for calls and hosts are identical, call index `cidx` ranges in $[1, \texttt{ncalls}]$ and host index `hidx` ranges in $[1, \texttt{nhosts}]$.

Using the above convention, the internal representation of the model is described in Table 6.1. As in the examples above, relative and absolute indexes are differentiated by using the suffix `idx` in the latter (e.g., `a` vs. `aidx`). This indexing style is used throughout the codebase as well.

Table 6.1: `LayeredNetworkStruct` static properties

Field	Type	Description
`nidx`	`integer`	Total number of `LayeredNetwork` elements
`nhosts`	`integer`	Number of `Hosts` or `Processor` elements
`ntasks`	`integer`	Number of `Tasks` elements
`nentries`	`integer`	Number of `Entry` elements
`nacts`	`integer`	Number of `Activity` elements
`ncalls`	`integer`	Number of calls issued by `Activity` elements
`hshift`	`integer`	For host h, the value h+`hshift` returns its absolute index in 1...`nidx`
`tshift`	`integer`	For task t, the value t+`tshift` returns its absolute index in 1...`nidx`
`eshift`	`integer`	For entry e, the value e+`eshift` returns its absolute index in 1...`nidx`
`ashift`	`integer`	For activity a, the value a+`ashift` returns its absolute index in 1...`nidx`
`cshift`	`integer`	For call c, the value c+`cshift` returns its absolute index in 1...`ncalls`
`tasksof`$\{hidx\}$	`array`	Array of absolute indexes for tasks on host with absolute index *hidx*
`entriesof`$\{tidx\}$	`array`	Array of absolute indexes for entries on task with absolute index *tidx*
`actsof`$\{eidx\}$	`array`	Array of absolute indexes for activities reachable from entry with absolute index *eidx*

actsof{$tidx$}	array	Array of absolute indexes for activities on task with absolute index *tidx*
callsof{$aidx$}	array	Array of absolute indexes for calls on activity with absolute index *aidx*
hostdem{idx}	object	*(Deprecated)* Object for the host demand distribution for element *idx*
hostdem_type(idx)	integer	ProcessType id for host demand distribution for element *idx*
hostdem_params{idx}	array	Parameters array for host demand distribution for element *idx*
hostdem_mean(idx)	double	Mean of host demand distribution for element *idx*
hostdem_scv(idx)	double	Squared coefficient of variation for host demand distribution for element *idx*
hostdem_proc{idx}	cell	Process representation (PH/MAP matrices) for host demand for element *idx*
think{idx}	object	*(Deprecated)* Object for the think time distribution for element *idx*
think_type(idx)	integer	ProcessType id for think time distribution for element *idx*
think_params{idx}	array	Parameters array for think time distribution for element *idx*
think_mean(idx)	double	Mean of think time distribution for element *idx*
think_scv(idx)	double	Squared coefficient of variation for think time distribution for element *idx*
think_proc{idx}	cell	Process representation (PH/MAP matrices) for think time for element *idx*
actthink{idx}	object	*(Deprecated)* Object for the activity think time distribution for element *idx*
actthink_type(idx)	integer	ProcessType id for activity think time distribution for element *idx*
actthink_params{idx}	array	Parameters array for activity think time distribution for element *idx*
actthink_mean(idx)	double	Mean of activity think time distribution for element *idx*
actthink_scv(idx)	double	Squared coefficient of variation for activity think time distribution for element *idx*
actthink_proc{idx}	cell	Process representation (PH/MAP matrices) for activity think time for element *idx*
sched(idx)	integer	SchedStrategy scheduling policy id for a host or task with absolute index *idx*
names{idx}	char	Name of the element with absolute index *idx*
hashnames{idx}	char	Same as names{idx} with type prefix ("P:" for host, "R:" for reference task, "T:" for task, "C:" for cache task, "E:" for entry, "I:" for item entry, "A:" for activity)
mult(idx)	integer	Multiplicity for a host or task with absolute index *idx*
repl(idx)	integer	Replication factor for a host or task with absolute index *idx*
type{idx}	integer	LayeredNetworkElement id for an element with absolute index *idx*
nitems(idx)	integer	Total number of items in CacheTask or ItemEntry *idx*
itemcap{idx}(l)	integer	CacheTask *idx* capacity for cache list l
replacestrat($tidx$)	integer	ReplacementStrategy id for cache task with absolute index *tidx*
itemproc(idx)	object	*(Deprecated)* DiscreteDistrib object for for the item popularity in ItemEntry *idx*
itemproc_type(idx)	integer	ProcessType id for item popularity distribution for element *idx*
itemproc_params{idx}	array	Parameters array for item popularity distribution for element *idx*
itemproc_mean(idx)	double	Mean of item popularity distribution for element *idx*
itemproc_scv(idx)	double	Squared coefficient of variation for item popularity distribution for element *idx*
itemproc_proc{idx}	cell	Process representation for item popularity for element *idx*
calltype($cidx$)	integer	CallType id for call *cidx*
callpair($cidx, j$)	integer	$j = 1$ is the activity *aidx* issuing call *cidx*, $j = 2$ is the entry *eidx* called by *cidx*
callproc($cidx$)	object	*(Deprecated)* DiscreteDistrib object for the number of calls with absolute index *cidx*
callproc_type($cidx$)	integer	ProcessType id for call count distribution for call *cidx*
callproc_params{$cidx$}	array	Parameters array for call count distribution for call *cidx*
callproc_mean($cidx$)	double	Mean of call count distribution for call *cidx*
callproc_scv($cidx$)	double	Squared coefficient of variation for call count distribution for call *cidx*

`callproc_proc`$\{cidx\}$	`cell`	Process representation for call count for call *cidx*
`callnames`$\{cidx\}$	`char`	Automatically assigned name to call with absolute index *cidx*
`callhashnames`$\{cidx\}$	`char`	Same as `callnames`$\{cidx\}$ but with type prefix for source and destination
`actpretype`$(aidx)$	`integer`	`ActivityPrecedenceType` id preceding the activity with absolute index *aidx*
`actposttype`$(aidx)$	`integer`	`ActivityPrecedenceType` id after the activity with absolute index *aidx*
`graph`(idx_i, idx_j)	`float`	$\neq 0$ if idx_i "runs on", "calls" or "precedes" idx_j. Values are by default 1.0 unless they specify a mean number of calls, a probability, fork fan-out or join fan-in degrees
`dag`(idx_i, idx_j)	`float`	Similar to `graph`(idx_i, idx_j) but loops do not form cycles and entry-task edges are flipped direction at non-reference tasks to aid multiplicity analysis.
`parent`(idx)	`integer`	Return absolute index value of the parent node for the node whose absolute index value is idx
`replygraph`$(aidx, eidx)$	`logical`	True if activity with absolute index *aidx* on entry *eidx* replies ending the entry call
`taskgraph`$(tidx_i, tidx_j)$	`logical`	True if host or task idx_i is a caller of host or task idx_j
`iscache`$(tidx)$	`logical`	True if task *tidx* is a `CacheTask`
`iscaller`(idx_i, idx_j)	`logical`	True if task or activity idx_i calls task or entry idx_j
`issynccaller`(idx_i, idx_j)	`logical`	True if task or activity idx_i issues a synchronous call to task or entry idx_j
`isasynccaller`(idx_i, idx_j)	`logical`	

loop to the servers [71]. This artificial delay is used to model the inter-arrival time between calls issued by that client.

The current version of LINE adopts SRVN-type layering [36], whereby a layer corresponds to one and only one resource, either a processor or a task. The `getLayers` method returns a cell array consisting of the Network objects corresponding to each layer

```
layers = model.getLayers()
```

The decomposition is performed through the `LN` solver described later.

Within each layer, classes are used to model the time a job spends in a given activity or call, with synchronous calls being modeled by classed with label including an arrow, e.g., `'AS1=>E3'` is a closed class used represent synchronous calls from activity `AS1` to entry `E3`, whereas `'AS1->E3'` denotes an asynchronous call. Artificial delays and reference nodes are modelled as a delay station named `'Clients'`, whereas the task or processor assigned to the layer is modelled as the other node in the layer.

6.4 Solvers

LINE offers two solvers for the solution of a `LayeredNetwork` model consisting in its own native solver (`LN`) and a wrapper (`LQNS`) to the LQNS solver [36]. The latter requires a distribution of LQNS to be available on the operating system command line.

`LQNS` is an external tool developed by the Real-time and Distributed Systems Group at Carleton University. The wrapper writes an XML file conforming to the LQN metamodel, runs the `lqns` (or `lqsim`) executable as a subprocess, and parses the XML output back into LINE metrics. Its solution methods, and the convergence tolerance and iteration cap that govern them, are selected through command-line flags set from the solver options.

The two layered solvers differ in where the decomposition happens. `LN` converts the layered model into a set of ordinary queueing networks in which cross-layer synchronization appears as artificial delay stations, then iterates a network solver until the delays are mutually consistent; this makes the whole LINE solver portfolio, and features such as sensitivity analysis and optimization, available to layered models. `LQNS` instead applies layered algorithms directly to the hierarchy without that intermediate conversion, which converges faster on deeply nested models and supports constructs, notably quorum joins, that the conversion does not yet cover. On standard layered models the two agree; divergences are confined to combinations of scheduling disciplines, replication and activity precedences.

The solution methods available for `LayeredNetwork` models are similar to those for Network objects. For example, the `avgTable` can be used to obtain a full set of mean performance indexes for the model, e.g.,

```
>> AvgTable = LQNS(model).avgTable
AvgTable =
  8x6 table
    Node      NodeType        QLen         Util       RespT       Tput
    ____    _____________    ________    _________    _____    ________
```

```
    'P1'    'Processor'        NaN    0.071429     NaN         NaN
    'T1'    'Task'         0.28571    0.071429     NaN     0.071429
    'E1'    'Entry'        0.28571    0.071429       4     0.071429
    'A1'    'Activity'     0.28571    0.071429       4     0.071429
    'P2'    'Processor'        NaN     0.21429     NaN         NaN
    'T2'    'Task'         0.21429     0.21429     NaN      0.21429
    'E2'    'Entry'        0.21429     0.21429       1      0.21429
    'A2'    'Activity'     0.21429     0.21429       1      0.21429
```

Note that in the above table, some performance indexes are marked as `NaN` because they are not defined in a layered queueing network. Further, compared to the `avgTable` method in Network objects, `LayeredNetwork` do not have an explicit differentiation between stations and classes, since in a layer a task may either act as a server station or a client class.

The main challenge in solving layered queueing networks through analytical methods is that the parameterization of the artificial delays depends on the steady-state performance of the other layers, thus causing a cyclic dependence between input parameters and solutions across the layers. Depending on the solver in use, such issue can be addressed in a different way, but in general a decomposition into layers will remain parametric on a set of response times, throughputs and utilizations.

This issue can be resolved through solvers that, starting from an initial guess, cyclically analyze the layers and update their artificial delays on the basis of the results of these analyses. Both `LN` and `LQNS` implement this solution method. Normally, after a number of iterations the model converges to a steady-state solution, where the parameterization of the artificial delays does not change after additional iterations.

6.4.1 LQNS

The LQNS wrapper operates by first transforming the specification into a valid LQNS XML file. Subsequently, LQNS calls the solver and parses the results from disks in order to present them to the user in the appropriate LINE tables or vectors. The `options.method` can be used to configure the LQNS execution as follows:

- `options.method='default'` or `'lqns'`: LQNS analytical solver with default settings.
- `options.method='exactmva'`: the solver will execute the standard LQNS analytical solver with the exact MVA method.
- `options.method='srvn'`: LQNS analytical solver with SRVN layering.
- `options.method='srvn.exactmva'`: the solver will execute the standard LQNS analytical solver with SRVN layering and the exact MVA method.
- `options.method='sim'` or `'lqsim'`: LQSIM simulator, with simulation length specified via the `samples` field (i.e., with parameter `-A options.samples`).
- `options.method='lqnsdefault'`: alias of `default` retained for backward compatibility with earlier scripts.

Upon invocation, the `lqns` or `lqsim` commands will be searched for in the system path. On Unix hosts, if no native binary is found, LINE transparently falls back to running the solver inside a local Docker image. This dispatch is governed by `options.config.container`: the empty default uses the native binary and falls back to Docker only when `lqns` is missing, the value `'auto'` (or `true`) prefers Docker whenever a local `imperialqore/lqns` image exists, and an explicit image name selects that image. Running inside a container both pins a reproducible `lqns` version and sidesteps the C++ standard-library version clash that can otherwise occur when the native binary is launched from a MATLAB session. When Docker is used the model is staged under the user's home directory rather than the system temporary directory, because a confined Docker installation cannot bind-mount the latter. This local-container path is distinct from `options.config.remote`, which dispatches instead to a remote `lqns-rest` service.

The pragma string passed to `lqns` can be customised through the `options.config.multiserver` entry, which forwards a `-Pmultiserver=...` pragma selecting the LQNS multi-server approximation (`'conway'`, `'reiser'`, `'rolia'`, `'zhou'`, `'suri'`, or `'schmidt'`; the `'default'` value maps to `'rolia'`). The value is passed verbatim to the back-end and therefore accepts any multi-server pragma supported by the installed LQNS version.

Remote Docker execution

As an alternative to local installation of the LQNS tools, LINE supports remote execution via a Docker container running the `lqns-rest` API. This is useful when LQNS binaries are not available locally or when running on systems where compilation is difficult. To enable remote execution, configure the following options:

- `options.config.remote = true`: Enable remote execution via REST API.
- `options.config.remote_url = 'http://localhost:8080'`: URL of the lqns-rest server (default port is 8080).

The Docker container can be started using the `imperialqore/line-lqns-rest` image extended with the REST API. When remote execution is enabled, the model is serialized to LQNX format and sent via HTTP POST to the remote server, which executes the solver and returns the results. This approach supports all solver methods (`lqns`, `lqsim`, etc.) and pragmas available in the local execution mode.

6.4.2 LN

The native `LN` solver iteratively applies the layer updates until convergence of the steady-state measures; the layering, the parameter updates, and the convergence control are described in the LINE internals manual (`LINE-internals.pdf`, Chapter `LN`). Since updates are parametric on the solution of each layer, `LN` can apply any of the Network solvers described in the solvers chapter to the analysis of individual layers, as illustrated in the following example for the `MVA` solver

```
options = LN.defaultOptions;
mvaopt = MVA.defaultOptions;
LN(model, @(layer) MVA(layer, mvaopt), options).avgTable
```

Options parameters may also be omitted. The `LN` method converges when the maximum relative change of mean response times across layers from the last iteration is less than `options.iter_tol`.

Methods supported by the LN solver include:

- `options.method='default'`: default recursive solution based on mean values
- `options.method='moment3'`: solution by recursive 3-moment approximation of response time distributions. This method matches the first three moments (mean, variance, skewness) of response times at each layer by fitting acyclic phase-type distributions, improving accuracy for non-exponential service demands compared to the default mean-value method.
- `options.method='mol'`: Method of Layers iteration with nested inner/outer loops for task and host layers.

Relaxation and MoL controls. The convergence behaviour of the layered iteration is governed by a family of `options.config` entries:

- `options.config.relax` (default `'fixed'`) selects the under-relaxation mode used when blending the new and previous iterates: `'none'` (no relaxation), `'fixed'` (constant factor), `'adaptive'` (factor adjusted from the recent error history), or `'auto'`.
- `options.config.relax_factor` (default 0.5) is the relaxation factor $\omega \in (0, 1]$ applied in the `'fixed'` mode. Smaller values yield more conservative updates and improve convergence on stiff models.
- `options.config.relax_min` (default 0.1) and `options.config.relax_history` (default 5): respectively the lower clamp on the relaxation factor and the error-history window used by the `'adaptive'` mode.
- `options.config.mol_task_inner_max` (default 50) and `options.config.mol_task_inner_tol` (default 10^{-4}): the maximum number of task (inner) iterations per host (outer) iteration and the corresponding task-layer convergence tolerance for the `mol` method.
- `options.config.mol_host_outer_tol` (default 10^{-4}) and `options.config.mol_min_steps` (default 2): the host (outer) layer convergence tolerance and the minimum number of outer iterations executed before host convergence is checked.
- `options.config.stochiter` (default `'auto'`) selects how the layered fixed point is driven when the layer solvers return noisy estimates, as they do when a layer is analysed by simulation. The value

'off' keeps the plain Picard iteration, 'rm' switches to Robbins-Monro stochastic approximation, in which successive iterates are averaged with a decaying step size so that the noise is filtered rather than propagated, 'crn' additionally couples the layer solvers with common random numbers, and 'auto' selects between them according to whether the model is classified as stochastic. The schedule is controlled by options.config.stochiter_alpha (default 0.6), the step-decay exponent, which must lie in $(0.5, 1]$ for the averaged iterate to converge, options.config.stochiter_a0 (default 1.0), the initial step after burn-in, options.config.stochiter_burnin (default 5), the number of Picard iterations executed before the step starts decaying, and options.config.stochiter_conseq (default 3), the number of consecutive sub-tolerance iterations required before the iteration is declared converged. The last of these matters because a single sub-tolerance iterate is not evidence of convergence when the residual is itself a random variable.

- options.config.layer_init (default none) selects the initialization of the layer throughput state. Setting it to 'bound' seeds each element's initial throughput with the geometric mean $\sqrt{X^- X^+}$ of the Majumdar-Woodside robust box bounds (algorithm lqn_boxbounds). The initialization is correctness-preserving; for closed-chain layered models the outer iteration re-derives the throughputs from the first solve, so the converged solution is unchanged.

Layer-wise sensitivities. SolverLN also reports the derivatives of the layer measures with respect to the service rates, by delegating to the layer solvers and concatenating their tables under a leading Layer column; see Section 4.7.1 for the branch selection, the options, and the caveat that these are partial derivatives within a layer rather than total derivatives of the layered model.

Layered box bounds. As a fast, non-iterative alternative to the full decomposition, SolverLN exposes the throughput bound methods options.method='mwba.upper' and 'mwba.lower' (algorithm lqn_boxbounds). These build a closed multiclass queueing network whose stations are the processors and whose classes are the reference-task call chains, and apply the Majumdar-Woodside robust box bounds [54] to return per-task upper and lower throughput bounds. This generalizes the classical LQN Type-1 throughput bound $X \leq N/(Z + D)$ (computed, e.g., by lqns -b) with a utilization-based upper bound and a contention-aware lower bound.

Transient analysis. SolverLN supports transient analysis of a layered model through getTranAvg, which returns the time-varying mean queue lengths, utilizations and throughputs of every layer in the same block-diagonal layout (station by class, per layer) used by the steady-state results. Two schemes are available. The decoupled scheme freezes the inter-layer demands at the converged fixed point and then integrates each layer's transient independently, which is inexpensive but ignores the fact that a layer's demand on another layer itself varies over the transient. The coupled scheme reconciles the layers by waveform relaxation: each layer's fluid transient is driven by the time-varying demand trajectories taken from the other layers' most recent transients, and the sweep is repeated until the trajectories stop changing in the sup-norm over time. The coupled channels are the task think times, which enter as a client delay, and the synchronous-call service

demands, which enter at the caller's client station; intra-layer host service is held at its equilibrium value. The time-varying demands are injected into each layer's closing ODE through a per-event rate multiplier. The first coupled sweep uses the frozen equilibrium demands and therefore reproduces the decoupled result exactly, and at convergence every layer relaxes to its own fixed point, so the endpoint of the transient agrees with `getAvg`.

State transfer and solver switching

The `LN` solver supports transferring its internal convergence state between solver instances, enabling hybrid solving schemes where a fast solver provides initial estimates that a more accurate solver then refines. This avoids restarting the iterative decomposition from scratch when switching solvers.

Three methods are provided:

- `get_state()`: exports the current solver state, including service time processes, throughputs, utilizations, and relaxation parameters.
- `set_state(state)`: imports a previously exported state into a new solver instance, allowing it to continue from the previous solution.
- `update_solver(solverFactory)`: replaces the per-layer solver while preserving the current convergence state.

A typical use case is to obtain a quick initial solution with `MVA` and then refine it with a simulation-based solver:

```
% Fast initial analysis with MVA
solver1 = LN(model, @(m) MVA(m));
T_fast = solver1.getAvgTable;
state = solver1.get_state();

% Refine with NC solver, continuing from MVA solution
solver2 = LN(model, @(m) NC(m));
solver2.set_state(state);
T_refined = solver2.getAvgTable;

% Alternatively, switch solver in-place
solver = LN(model, @(m) MVA(m));
solver.getAvgTable;
solver.update_solver(@(m) NC(m));
T_switched = solver.getAvgTable;
```

6.5 Conversion between Network and `LayeredNetwork` models

LINE provides two converters that bridge the `Network` (queueing network) and `LayeredNetwork` (layered queueing network) abstractions. These converters are useful when one wishes to apply a layered solver to an ordinary queueing network, or vice versa to analyse a layered model with a flat queueing network solver.

6.5.1 From Network to `LayeredNetwork`

The function `QN2LQN` maps a closed queueing network into a layered queueing network. Each closed chain is modelled by a reference task whose entries replicate the chain's visit pattern, and each queueing or delay station is mapped onto a processor hosting the corresponding task. Routing probabilities and class switching are reproduced through OR-fork activity precedences. Open classes (`Source`/`Sink`) are not currently supported by this converter and the input model must be closed. The conversion is invoked as

```
lqnmodel = QN2LQN(model);
AvgTable = LQNS(lqnmodel).getAvgTable();
```

The `QN2LQN` converter is invoked automatically by the `QNS` solver in MATLAB when the input model lacks a product-form solution and contains only closed classes; in this case the model is converted on the fly and dispatched to `LQNS`.

6.5.2 From `LayeredNetwork` to Network

The reverse converter `LQN2QN` maps a `LayeredNetwork` into an extended queueing network in which synchronous calls are realised through reply signal classes (`SignalType.REPLY`). The caller class is blocked at the calling station until the reply class returns from the callee, and a class switch at the leaf node restores the caller class so that the request cycle can repeat. Phase-2 activities, when present, are mapped onto fork-join constructs so that the post-reply work is executed in parallel with the released caller. The conversion is invoked as

```
qnmodel = LQN2QN(lqnmodel);
AvgTable = LDES(qnmodel).getAvgTable();
```

Beyond synchronous calls, the converter maps the remaining call and task constructs of the layered model. Asynchronous calls become ordinary routing without a reply signal, since the caller does not block. Forwarding calls are translated into the caller-side pseudo rendezvous described in the forwarding section, so that the original client stays blocked until the last entry of the forwarding chain replies. Fork calls and their joins are mapped onto the corresponding `Fork` and `Join` nodes. Cache tasks are mapped onto a `Cache` node with the item entries as read classes and the hit and miss branches as the corresponding outgoing classes. The multiplicity of a task is enforced as a thread pool rather than as a plain multi-server station: a task with multiplicity m admits at most m concurrent requests across all of its entries, which is expressed as a finite capacity region over the task's stations, so that a task whose entries have different demands cannot exceed its

declared concurrency. Open arrivals declared on an entry become a `Source` and `Sink` pair carrying an open chain into that entry, rather than a closed chain with a large population, so that the arrival rate is honoured exactly. The converter is available in all three codebases.

This route is useful for validating an `LN` or `LQNS` solution against discrete-event simulation, since the resulting network can be analysed with any solver that supports signal classes (currently `LDES` and `JMT`).

6.6 Model import and export

A `LayeredNetwork` can be easily read from, or written to, a XML file based on the LQNS meta-model format[1]. The read operation can be done using a static method of the `LayeredNetwork` class, i.e.,

```
model = LayeredNetwork.parseXML(filename)
```

Conversely, the write operation is invoked directly on the model object

```
model.writeXML(filename)
```

In both examples, `filename` is a string including both file name and its path.

Finally, we point out that it is possible to export a LQN to an XML file in the LQNX format by means of the `writeXML(filename)` function.

6.6.1 Ensemble merging

The `Ensemble.merge` function combines multiple independent models into a single `Network` containing disconnected subnetworks. Node and class names are prefixed with their model name to prevent conflicts, while all `Source` and `Sink` nodes are merged into single `MergedSource` and `MergedSink` nodes.

```
mergedModel = Ensemble.merge({model1, model2});
```

[1] https://raw.githubusercontent.com/layeredqueuing/V5/master/xml/lqn.xsd

Chapter 7

Random environments

Random environments provide a framework for modeling systems whose input parameters, such as service rates, scheduling weights, or number of stations, vary according to the state of an external environment. We refer to these environmental states as *stages*. For instance, a system that alternates between normal-load and peak-load conditions can be represented by a two-stage environment, with each stage determining the number of servers available at a queueing station. Systems of this type arise in many settings, including servers subject to breakdown and repair, auto-scaling applications, and reliability models.

In LINE, an environment is described as a continuous-time Markov chain (CTMC) or as a semi-Markov process (SMP). Compared to CTMCs, SMPs relax the restriction of having exponential transitions between stages, but they are computationally harder to evaluate. In first approximation, one may simply solve independent models for each stage and weight their solution. However, depending on the frequency of transitions between stages, this incur significant errors [21]. LINE automates the evaluation of such scenarios, applying the appropriate corrections based on the environment and system characteristics.

7.1 Environment definition

7.1.1 Specifying the environment

To specify an environment, we first create an `Environment` object

```
E = 2;
envModel = Environment('UnreliableEnv', E);
```

where the second parameter indicates the number of stages in the environment. We then add two stages

```
envModel.addStage('Online', 'UP', network1);
envModel.addStage('Offline', 'DOWN', network2);
```

where the constructor specifies the stage name, an arbitrary string to classify the stage semantics, followed by the system model used while that environment stage is active.

We now describe that the transitions between stages are both exponential, with different rates

```
envModel.addTransition('Online', 'Offline', Exp(1));
envModel.addTransition('Offline', 'Online', Exp(2));
```

Self-loops are allowed. When multiple transitions are possible, the timer that elapses first determines the next stage.

The `getStageTable` method summarizes the properties of an environment:

```
>> envModel.getStageTable
ans =
  2x6 table
    Stage       Name         Type       Prob        HoldT           Model
    _____    __________    _______    _______    ___________    _____________
      1      'Online'      'Up'       0.83333    {1x4 cell}     [1x1 Network]
      2      'Offline'     'Down'     0.16667    {1x4 cell}     [1x1 Network]
```

In the table, the `Stage` column gives a numerical identifier for each stage, followed by its name, type classification, and a pointer to the sub-model associated to that stage.

7.1.2 Specifying system models for each stage

LINE places loose assumptions in the way the system should be described in each stage. It is just expected that the user supplies a model object, either a Network or a `LayeredNetwork`, in each stage, and that a transient analysis method is available in the chosen solver, a requirement fulfilled for example by `FLD`. When the stage models are `LayeredNetwork` objects, each stage is analysed by `LN` and the blending iteration is carried out over the layered ensembles, so that a layered application whose demands or multiplicities differ between environment stages can be studied without flattening it first. A worked two-stage layered example is provided in `renv_lqn_twostages` in the random-environment examples folder.

7.1.3 Specifying a reset policy

A reset policy specifies whether an environment transition instantaneously brings a redistribution of the jobs across the network. When the environment transitions, by default jobs in execution at a server are required all to restart execution at that server upon occurrence of a transition. This may not be appropriate in some models, for example when a station is removed from the model and its jobs can therefore no longer execute at that station. For such cases, one may define a custom reset policy, e.g.,

```
resetRule = @(QExit) [sum(QExit,1); zeros(size(QExit,1)-1,size(QExit,2))]; % ...
move all jobs into station 1, without changing their classes
envModel.addTransition('Online', 'Offline', Exp(1), resetRule);
```

In the above code, `QExit(i,r)` is the queue-length of class-r jobs observed at node i upon exiting the online state. The `resetRule` must produce in output a vector of the same size of `QExit`. The reset policy in

this example moves instantaneously all jobs in the network into station 1 upon entering into the offline state. Note that `resetRule` can be configured differently with each stage transition and the default value is simply `resetRule = @(QExit) QExit`.

State-dependent environment rates

In addition to the queue-length reset rule, the `addTransition` method accepts an optional sixth argument, a *reset rule for the outgoing environment-transition distribution*. This produces a state-dependent semi-Markov environment, in which the timing of stage transitions adapts to the system state at the moment of the previous stage change. The signature is

```
envModel.addTransition(from, to, distrib, resetFun, resetEnvRatesFun);
```

where `resetEnvRatesFun` is a function handle of the form

```
resetEnvRatesFun = @(originalDist, QExit, UExit, TExit) newDist;
```

that returns a new distribution `newDist` given the originally configured `originalDist` and the queue lengths `QExit`, utilizations `UExit` and throughputs `TExit` measured at the instant of leaving the source stage. The default is the identity `@(originalDist, QExit, UExit, TExit) originalDist`, which leaves the originally specified distribution unchanged. When at least one transition supplies a non-trivial `resetEnvRatesFun`, `ENV` automatically switches to the state-dependent solution mode (see `options.method='statedep'` in the `ENV` configuration table below); this mode is incompatible with the semi-Markov mode `options.method='smp'`.

7.2 Solvers

The steady-state analysis of a system in a random environment is carried out in LINE using the blending method [21], which is an iterative algorithm leveraging the transient solution of the model. In essence, the model looks at the *average* state of the system at the instant of each stage transition, and upon restarting the system in the new stage re-initializes it from this average value. This algorithm is implemented in LINE by the `ENV` class, which is described next.

7.2.1 ENV

The `ENV` class (alias: `SolverENV`) applies the blending algorithm by iteratively carrying out a transient analysis of each system model in each environment stage, and probabilistically weighting the solution to extract the steady-state behavior of the system.

The solver offers two analyzer modes, selected through `options.method`. The default `'meanfield'` mode couples stages through the marginal mean queue length (the classic blending method). The alternative `'statevec'` mode carries the full per-stage state-probability vector across stage transitions instead of only

its mean, yielding higher accuracy at the cost of an explicit state space; both are described in the LINE internals manual (`LINE-internals.pdf`, Chapter `ENV`). The `'statevec'` analyzer supports a CTMC backend for closed models and a MAM/LDQBD backend for open and load-dependent queues, and it accepts the additional option `options.sojourn='deterministic'`, which models each stage sojourn as lasting exactly its mean (exact via uniformization) rather than as an exponential holding time.

As in the transient analysis of Network objects, LINE does not supply a method to obtain mean response times, since Little's law does not hold in the transient regime. To obtain the mean queue-length, utilization and throughput of the system one can call as usual the `avg` method on the `ENV` object, e.g.,

```
envSolver = ENV(envModel, @FLD, options);
[QN,UN,TN] = envSolver.avg()
```

Note that as model complexity grows, the number of iterations required by the blending algorithm to converge may grow large. In such cases, the `options.iter_max` option may be used to bound the maximum analysis time.

Per-stage results

While the `avg` family of methods returns environment-averaged metrics, the underlying per-stage results computed by the blending algorithm remain accessible through the `ENV` solver. The full per-stage solver result for stage `e` is available at `envSolver.results{e}`, with the corresponding stage-local child solver retrievable via `envSolver.solvers{e}`. The steady-state probability of being in each environment stage is exposed as `envObj.probEnv`; the same values appear as the `Prob` column of the table returned by `getStageTable`. The environment-averaged metrics returned by `getAvg` are the weighted combination $\sum_e \pi_e X_e$ where X_e is the per-stage queue-length, utilization or throughput and π_e is the corresponding entry of `probEnv`.

Stage holding-time inspection

The semi-Markov holding time in each stage is precomputed when the environment is initialized. Stage `e`'s holding-time MMAP representation is stored as `envObj.holdTime{e}` and is also reported in the `HoldT` column of `getStageTable`, with the convenience alias `getStageT` producing the same table. When using the semi-Markov method (`options.method='smp'`), the per-stage sojourn-time CDF is exposed by the `computeSojournCdf(e, t)` method of `ENV`, which evaluates $\Pr\{S_e \leq t\}$ for the chosen stage `e`.

Decomposition/Aggregation Methods

When the environment has many states, the `ENV` solver can use decomposition/aggregation methods to reduce computational complexity. The decomposition method can be configured using `options.config.da`. The available methods are based on nearly completely decomposable (NCD) Markov chain analysis:

- *Courtois decomposition* (`'courtois'`): The default method, based on [30]. It partitions the environment state space into macro-states and computes approximate steady-state probabilities.
- *Koury-McAllister-Stewart* (`'kms'`): An iterative aggregation-disaggregation method [48] that refines the Courtois solution through successive approximations.
- *Takahashi's method* (`'takahashi'`): Another iterative aggregation-disaggregation approach [81] with different convergence properties than KMS.
- *Multigrid* (`'multi'`): A multi-level decomposition method that applies hierarchical aggregation.

For iterative methods (`'kms'` and `'takahashi'`), the number of iterations can be controlled using `options.config.da_iter` (default: 10).

The configuration options are summarized in Table 7.2.1.

Table 7.1: `ENV` configuration options (MATLAB)

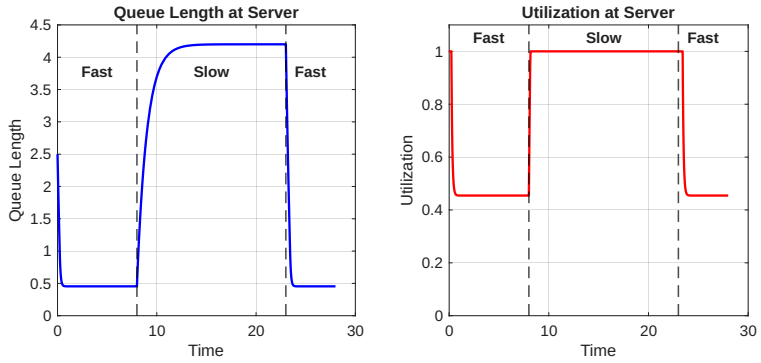

Figure 7.1: Transient queue length and utilization evolution along a sample path.

The scenario models a server that alternates between "Fast" and "Slow" modes. In Fast mode, the service rate is 10.0 (low utilization). In Slow mode, the service rate is 0.8 (high utilization, near saturation). The environment switches from Fast→Slow at rate 0.5 (mean time in Fast mode = 2.0) and from Slow→Fast at rate 1.0 (mean time in Slow mode = 1.0).

We start by creating a base closed queueing network with a delay and a queue:

```
baseModel = Network('BaseModel');
delay = Delay(baseModel, 'ThinkTime');
queue = Queue(baseModel, 'Fast/Slow Server', SchedStrategy.FCFS);

% Closed class with 5 jobs
N = 5;
jobclass = ClosedClass(baseModel, 'Jobs', N, delay);
delay.setService(jobclass, Exp(1.0));  % Think time = 1.0
queue.setService(jobclass, Exp(2.0));  % Placeholder

% Connect nodes in a cycle
baseModel.link(Network.serialRouting(delay, queue));
```

Next, we create the random environment with two stages, each having a different service rate:

```
env = Environment('ServerModes');

% Stage 1: Fast mode (service rate = 10.0, low utilization)
fastModel = baseModel.copy();
fastQueue = fastModel.getNodeByName('Fast/Slow Server');
fastQueue.setService(fastModel.classes{1}, Exp(10.0));
env.addStage('Fast', 'operational', fastModel);

% Stage 2: Slow mode (service rate = 0.8, high utilization)
slowModel = baseModel.copy();
slowQueue = slowModel.getNodeByName('Fast/Slow Server');
```

```
slowQueue.setService(slowModel.classes{1}, Exp(0.8));
env.addStage('Slow', 'degraded', slowModel);

% Define transitions between stages
env.addTransition('Fast', 'Slow', Exp(0.5));  % Fast->Slow
env.addTransition('Slow', 'Fast', Exp(1.0));  % Slow->Fast
```

Finally, we solve the model using `ENV` with the Fluid solver (`FLD`) as the sub-solver for each stage:

```
% Initialize the environment
env.init();
stageTable = env.getStageTable()

% Solve using ENV with Fluid sub-solver
fldOptions = FLD.defaultOptions;
fldOptions.timespan = [0, 100];
solverFactory = @(m) FLD(m, fldOptions);

envSolver = ENV(env, solverFactory);
[QN, UN, TN] = envSolver.getAvg();

% Display environment-averaged results
envAvgTable = envSolver.getAvgTable()
```

The output shows the environment stages and performance metrics:

```
stageTable =
  2x6 table
    Stage    Name       Type          Prob        HoldT           Model
    _____    ____    ___________    _______    __________    _____________
      1      Fast    operational    0.66667    {1x4 cell}    {1x1 Network}
      2      Slow    degraded       0.33333    {1x4 cell}    {1x1 Network}

envAvgTable =
  2x6 table
     Station      JobClass      QLen      Util      RespT      Tput
    _________    _________    ______    ______    _______    ______
    ThinkTime      Jobs       2.9961    2.9961          1    2.9961
    Server         Jobs       2.0039         1    0.66796         3
```

The results show that the server spends approximately 66.67% of time in Fast mode and 33.33% in Slow mode. The environment-averaged queue length at the server is approximately 2.0 jobs, which is a weighted combination of the queue lengths in the Fast and Slow stages.

Analyzing sample paths with getSamplePathTable

In addition to computing steady-state metrics averaged over the environment distribution, `ENV` provides a method to analyze specific sample paths through the environment states. This is useful when you want to

understand the transient behavior of the system as it transitions through a known sequence of environment states.

The `getSamplePathTable` method accepts a specification of a sample path—a list of (stage, duration) pairs—and computes the transient performance metrics for each segment. For each segment, the method:

1. Initializes the model from the previous segment's final queue lengths (or uniform distribution for the first segment)

2. Runs transient analysis for the specified duration using the Fluid solver

3. Extracts the initial and final values of queue length, utilization, and throughput

The output is a table with columns for segment index, stage name, duration, station, job class, and the initial/final values of each metric. An example of the transient behavior along a sample path is shown in Figure 7.1, which depicts a sample path transitioning through environment states Fast → Slow → Fast. In the Fast stage (service rate = 10.0), the server operates at low utilization (≈45%), while in the Slow stage (service rate = 0.8), the server becomes saturated (≈100% utilization), causing queue buildup as arrivals accumulate faster than they can be processed.

```
% Define a sample path: Fast for 8 time units, Slow for 15, Fast for 5
samplePath = {'Fast', 8.0; 'Slow', 15.0; 'Fast', 5.0};

% Compute transient metrics along the sample path
[T, segmentResults] = solver.getSamplePathTable(samplePath);

% Display the results table
disp(T);

% Access detailed transient trajectories for plotting
seg1_QLen = segmentResults{1}.QNt{2,1}.metric;  % Server queue length in ...
segment 1
seg1_time = segmentResults{1}.QNt{2,1}.t;
```

The sample path can be specified using either stage names (strings) or stage indices (integers). Stages are 1-indexed in MATLAB and 0-indexed in Java/Python. The duration for each segment must be positive.

7.3.2 Example 2: Breakdown and repair

LINE provides convenience methods to simplify the common case of modeling node failures and repairs in random environments. These methods automatically create UP and DOWN stages for nodes that can break down, configure service rates appropriately, and set up breakdown and repair transitions.

The primary method is `addNodeFailureRepair`, which creates both breakdown and repair transitions in a single call:

```
env = Environment('ServerEnv');
env.addNodeFailureRepair(baseModel, 'Server1', Exp(0.1), Exp(1.0), Exp(0.5));
env.init();
```

where `baseModel` is the network with normal (UP) service rates, `'Server1'` is the node name, `Exp(0.1)` is the breakdown distribution (mean time to failure = 10 time units), `Exp(1.0)` is the repair distribution (mean time to repair = 1 time unit), and `Exp(0.5)` is the service distribution when the node is down.

This method automatically:

- Creates an UP stage with the service rates of the base model
- Creates a DOWN stage with the specified reduced service rate for the failed node
- Adds a breakdown transition (UP $\rightarrow$ DOWN) with the given breakdown distribution
- Adds a repair transition (DOWN $\rightarrow$ UP) with the given repair distribution

Alternative methods for more control include `addNodeBreakdown` and `addNodeRepair`, which can be called separately. Their full signatures are

```
env.addNodeBreakdown(baseModel, nodeOrName, breakdownDist, downServiceDist, ...
resetFun);
env.addNodeRepair(nodeOrName, repairDist, resetFun);
```

where `nodeOrName` is either a `Node` object or a string node name, `breakdownDist` and `repairDist` are Markovian distributions controlling the timing of the corresponding stage transitions, and `downServiceDist` is the service distribution applied to all classes at the failed node while it is in the DOWN stage. The first call to `addNodeBreakdown` also creates the UP stage from `baseModel` (if it does not yet exist), and each invocation creates a DOWN stage with auto-generated name `DOWN_<nodeName>`, so that several nodes can break down independently within the same environment. The optional `resetFun` argument follows the same convention as in the previous section and defaults to the identity `@(q) q`, leaving queue lengths unchanged on the corresponding transition. Custom reset policies can also be supplied at the convenience entry point `addNodeFailureRepair` as optional parameters:

```
resetBreakdown = @(q) 0*q;  % Clear queues on breakdown
resetRepair = @(q) q;       % Keep jobs on repair
env.addNodeFailureRepair(baseModel, 'Server1', Exp(0.1), Exp(1.0), ...
    Exp(0.5), resetBreakdown, resetRepair);
```

Reset policies can also be modified after environment creation using `setBreakdownResetPolicy` and `setRepairResetPolicy` methods.

Retrieving reliability metrics

After solving an environment model with node breakdown and repair, reliability metrics can be retrieved using the `getReliabilityTable` method (aliases: `relT`, `getRelT`, `relTable`, `getRelTable`):

```
>> env.getReliabilityTable
ans =
  4x4 table
      Metric           Value           Unit                  Description
    _________    _______________    ___________    ____________________________________
    MTTF         10                 time units     Mean time to failure (UP -> DOWN)
    MTTR         1                  time units     Mean time to repair (DOWN -> UP)
    MTBF         11                 time units     Mean time between failures (MTTF + MTTR)
    Availability 0.90909            probability    Steady-state probability of UP state
```

The reliability metrics returned are:

- *MTTF* (Mean Time To Failure): Expected time until breakdown from the UP state. For multiple failure modes, this uses the combined failure rate: $\lambda_{\text{total}} = \sum_i \lambda_i$.
- *MTTR* (Mean Time To Repair): Expected time to restore from the DOWN state. For multiple DOWN states, this is a weighted average based on steady-state probabilities.
- *MTBF* (Mean Time Between Failures): Sum of MTTF and MTTR.
- *Availability*: Steady-state probability of the system being in the UP state.

7.3.3 Example 3: Markov-modulated service

This example shows how to convert a queueing network with MMPP2 (Markov-Modulated Poisson Process) service into an equivalent random environment model using the `MAPQN2RENV` utility. MMPP2 is a 2-phase service process where the service rate switches between two values according to an underlying Markov chain. The `MAPQN2RENV` function automatically transforms such a model into a random environment with two stages, each having exponential service at the corresponding MMPP rate.

We create a closed queueing network with an MMPP2 service process:

```
model = Network('ClosedQN');
queue1 = Queue(model, 'Queue1', SchedStrategy.FCFS);
queue2 = Queue(model, 'Queue2', SchedStrategy.FCFS);

% Closed class with 3 jobs
N = 3;
jobClass = ClosedClass(model, 'Class1', N, queue1);

% MMPP2 service: slow_rate=2.0, fast_rate=5.0, slow_to_fast=0.5, fast_to_slow=0.3
queue1.setService(jobClass, MMPP2(2.0, 5.0, 0.5, 0.3));
queue2.setService(jobClass, Exp(3.0));
```

```
model.link(Network.serialRouting(queue1, queue2));

% Convert to random environment
env = MAPQN2RENV(model);
env.printStageTable();
```

The resulting environment can then be analyzed using `ENV` as shown in Example 1.

7.3.4 Example 4: semi-Markov process environments

This example demonstrates the analysis of random environments with non-Markovian transition distributions using semi-Markov Processes. When environment transitions follow general distributions (e.g., Lognormal), the ENV solver should be run with the `'smp'` method.

Consider a system that alternates between two operational modes with log-normally distributed holding times in each stage.

```
% Create two network models for different modes
model1 = Network('Mode1');
source1 = Source(model1, 'Source');
queue1 = Queue(model1, 'Queue1', SchedStrategy.PS);
sink1 = Sink(model1, 'Sink');
jobClass1 = OpenClass(model1, 'Jobs');
source1.setArrival(jobClass1, Exp(1.0));
queue1.setService(jobClass1, Exp(2.0));
model1.link(Network.serialRouting(source1, queue1, sink1));

model2 = Network('Mode2');
source2 = Source(model2, 'Source');
queue2 = Queue(model2, 'Queue1', SchedStrategy.PS);
sink2 = Sink(model2, 'Sink');
jobClass2 = OpenClass(model2, 'Jobs');
source2.setArrival(jobClass2, Exp(1.0));
queue2.setService(jobClass2, Exp(0.8));
model2.link(Network.serialRouting(source2, queue2, sink2));

% Create environment with LogNormal transitions (non-Markovian)
envModel = Environment('SemiMarkovEnv');
envModel.addStage('Stage1', 'UP', model1);
envModel.addStage('Stage2', 'UP', model2);
envModel.addTransition('Stage1', 'Stage2', Lognormal(1.0, 0.5));  % Non-exponential
envModel.addTransition('Stage2', 'Stage1', Lognormal(0.5, 0.3));  % Non-exponential

% Enable SMP method for non-Markovian transitions
options = SolverOptions();
options.method = 'smp';
solver = ENV(envModel, @FLD, options);
solver.runAnalyzer();
```

```
solver.result
```

The ENV solver correctly handles this by using numerical integration to compute transition probabilities from the non-Markovian distribution functions, rather than assuming Markovian (memoryless) transitions.

7.3.5 Example 5: MMAP-modulated cache workload

A random environment can also drive a multiclass workload generated by a marked Markovian arrival process (MMAP). We feed a small Round-Robin cache with a marked MMPP2 whose two marks are bound, via `setMarkedArrival`, to two open read classes that share the modulating chain: phase 1 is bursty and emits mostly class-1 references at a high rate, phase 2 is calm and emits mostly class-2 references at a low rate. Viewing the modulating phase as the environment turns the MMAP into two phase-conditional Poisson streams (the diagonal D_1 blocks), so the cache can be solved by the ENV meta-solver. We use the state-dependent method (`options.method='blend'`), which carries the cache-state distribution across phase switches and averages each phase's sojourn-weighted distribution; for a Markovian environment it recovers the exact joint (cache $\times$ phase) solution, i.e., it matches CTMC on the true MMAP-fed cache.

```
n = 4; m = 2;                                       % 4 items, cache capacity 2
pAccess1 = DiscreteSampler([8 4 2 1]/15);           % class 1 favors low-index items
pAccess2 = DiscreteSampler([1 2 4 8]/15);           % class 2 favors high-index items

% Marked MMPP2 (M3A layout D = {D0, D11, D12}); D1 = D11 + D12 is diagonal.
D0  = [-4.5, 0.5; 0.5, -1.5];
D11 = [3.6, 0; 0, 0.2];                             % class-1 arrivals per phase
D12 = [0.4, 0; 0, 0.8];                             % class-2 arrivals per phase
mmap = MarkedMAP({D0, D11, D12}, 2);

% buildCacheModel / setStageRates mirror the helper functions in the example
% file: buildCacheModel wires Source -> RR Cache -> Sink with two read classes
% (setRead/setHitClass/setMissClass); setStageRates overrides the per-class
% Source arrival rates of a copy of that model for one environment stage.
buildModel = @() buildCacheModel(n, m, pAccess1, pAccess2);

% Exact reference: CTMC of the true MMAP-fed cache
trueModel = buildModel();
trueModel.getNodeByName('Source').setMarkedArrival(mmap, ...
    {trueModel.classes{1}, trueModel.classes{2}});
CTMC(trueModel, 'cutoff', 1).getAvg();
hitCTMC = trueModel.getNodeByName('Cache').getHitRatio;

% Random environment: MMPP2 phase modulates phase-conditional Poisson arrivals
setRates = @(lam1, lam2) setStageRates(buildModel(), lam1, lam2);
env = Environment('MMPPphase');
env.addStage('Phase1', 'bursty', setRates(D11(1,1), D12(1,1)));
env.addStage('Phase2', 'calm',   setRates(D11(2,2), D12(2,2)));
env.addTransition('Phase1', 'Phase2', Exp(-D0(1,1) - (D11(1,1)+D12(1,1))));
env.addTransition('Phase2', 'Phase1', Exp(-D0(2,2) - (D11(2,2)+D12(2,2))));
env.init();

% State-dependent ('blend') ENV solver over a finite-timespan CTMC inner solver
blendFactory = @(mdl) CTMC(mdl, 'cutoff', 1, 'timespan', [0, 1e3]);
```

```
opt = Solver.defaultOptions; opt.method = 'blend';
opt.iter_max = 100; opt.iter_tol = 1e-4;
envSolver = ENV(env, blendFactory, opt);
envSolver.getAvg();
hitBLEND = envSolver.ensemble{1}.getNodeByName('Cache').getHitRatio;
```

The per-read-class hit ratios returned by the state-dependent `ENV` solver (`hitBLEND`) match those of the exact `CTMC` on the true MMAP-fed cache (`hitCTMC`). The faster `'avg'` (fast-environment, rate-averaged) and `'dec'` (slow-environment, quasi-stationary decomposition) methods discard the within-phase temporal correlation of references and are therefore only approximate. A complete runnable version, including the `LDES` simulation and the two approximate methods, is in `examples/basic/cacheModel/cache_mmap_rr_env.m`.

Mean-field blending. When the per-phase cache state space is too large for the exact `'blend'` inner solver, the default (mean-field) `ENV` analyzer provides a fully fluid alternative driven by an `FLD` inner solver. Its refined mean-field (RMF) algorithm analyses each phase's cache by integrating the mean-field occupancy drift over the phase sojourn; the mean cache occupancy is then carried across phase switches (the cache analog of the queue-length hand-off performed by `initFromMarginal`), and the per-read-class hit ratio is the `probEnv`-weighted, sojourn-averaged product of arrival rate and hit probability. This trades the exact joint (cache $\times$ phase) distribution of `'blend'` for a cheaper fluid approximation, and is available identically in the MATLAB, Java, and Python codebases.

```
% Mean-field alternative: default ENV analyzer with an FLD inner solver
fldFactory = @(mdl) FLD(mdl, 'method', 'rmf', 'timespan', [0, 50]);
opt = Solver.defaultOptions; opt.iter_max = 100; opt.iter_tol = 1e-4;
envMF = ENV(env, fldFactory, opt);
envMF.getAvg();
hitMF = envMF.ensemble{1}.getNodeByName('Cache').getHitRatio;
```

Figure 7.2 shows the mean-field blending iteration converging, from an initially empty cache, to the fluid equilibrium `hitMF`; the residual gap to the exact `CTMC` blend is the mean-field error of the small (n=4, m=2) cache.

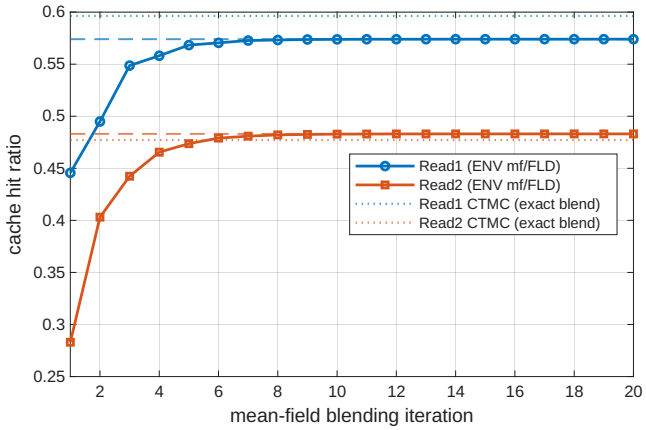

Figure 7.2: Mean-field ENV blending for the MMAP-modulated cache of Example 7.3.5. Starting from an empty cache, the per-read-class hit ratios produced by the default (mean-field) ENV analyzer with an FLD inner solver converge over the blending iteration to the environment-blended equilibrium (dashed), obtained by carrying each phase's mean cache occupancy across phase switches. The exact CTMC blend (dotted) is approached up to the mean-field error of the small (n=4, m=2) cache.

Chapter 8

Parameter Inference

LINE supports parameter inference, i.e., the estimation of queueing network model parameters from empirical data. Given observed system measurements such as response times, throughputs, queue lengths, and utilizations, the inference methods recover the model parameters—primarily service demands—that best explain the data. While the LINE solvers perform *forward analysis* (computing performance metrics from model parameters), the inference functions address the *inverse problem*: recovering model parameters from observed metrics.
The parameter inference methods additionally require the *Optimization Toolbox*.
To cite the parameter inference methods, we recommend referencing [38, 53, 63, 78, 85, 86].

8.1 Quick start

The typical workflow for parameter estimation is:

1. Define a model: create a LINE `Network` with known structure but unknown service demands (set to `NaN`).
2. Collect measurements: wrap observed data as `SampledMetric` objects, specifying the metric type, timestamps, values, node, and (optionally) job class.
3. Configure the estimator: create a `ParamEstimator` with the model and desired method.
4. Add data and estimate: add samples to the estimator, interpolate, and call `estimateAt` to obtain the estimated service demands.

The following example estimates service demands at a PS queue in a closed network using the UBO method:

```
% Step 1: Define the queueing network model
model = Network('model');
node{1} = Delay(model, 'Delay');
```

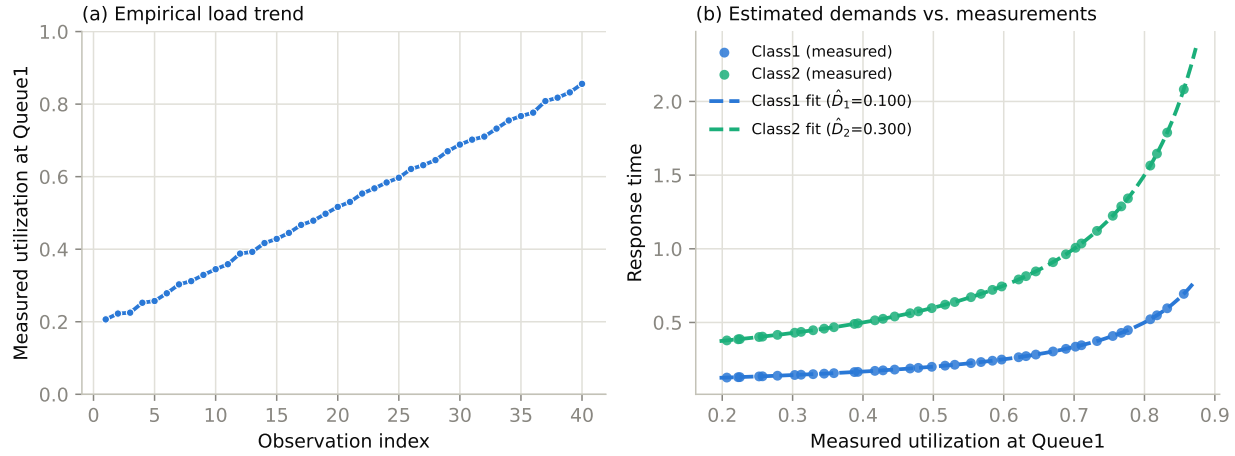

Figure 8.1: Quick-start example: (a) the empirical utilization trend as the workload ramps up over the observation window; (b) the measured response times against utilization, together with the M/G/1 curves $\hat{D}_r/(1-u)$ reconstructed from the demands $\hat{D}_1, \hat{D}_2$ estimated by the UBO method.

```
node{2} = Queue(model, 'Queue1', SchedStrategy.PS);
jobclass{1} = ClosedClass(model, 'Class1', 1, node{1}, 0);
jobclass{2} = ClosedClass(model, 'Class2', 2, node{1}, 0);

% Set known service times at the delay station
node{1}.setService(jobclass{1}, Exp.fitMean(1.0));
node{1}.setService(jobclass{2}, Exp.fitMean(1.0));

% Mark queue service demands as unknown (to be estimated)
node{2}.setService(jobclass{1}, Exp(NaN));
node{2}.setService(jobclass{2}, Exp(NaN));

% Define routing: each class cycles between delay and queue
P = model.initRoutingMatrix();
P{jobclass{1},jobclass{1}}(node{1},node{2}) = 1;
P{jobclass{1},jobclass{1}}(node{2},node{1}) = 1;
P{jobclass{2},jobclass{2}}(node{1},node{2}) = 1;
P{jobclass{2},jobclass{2}}(node{2},node{1}) = 1;
model.link(P);

% Step 2: Generate synthetic measurement data from a ramping workload
% True demands are D1=0.1 and D2=0.3; the arrival rate ramps up over the
% observation window, driving utilization from ~0.2 to ~0.85
n = 40; ts = (1:n)';
```

```
ramp = linspace(0.8, 3.4, n)';
arvr1 = ramp + (rand(n,1) - 0.5)*0.10;
arvr2 = 0.5*ramp + (rand(n,1) - 0.5)*0.05;
util = 0.1*arvr1 + 0.3*arvr2;    % U = D1*lam1 + D2*lam2
respt1 = 0.1 ./ (1 - util);      % M/G/1 response time formula
respt2 = 0.3 ./ (1 - util);

% Step 3: Configure the estimator with the UBO method
options = ParamEstimator.defaultOptions();
options.method = 'ubo';
se = ParamEstimator(model, options);

% Step 4: Add observed samples and run estimation
se.addSamples(SampledMetric(MetricType.ArvR, ts, arvr1, node{2}, jobclass{1}));
se.addSamples(SampledMetric(MetricType.ArvR, ts, arvr2, node{2}, jobclass{2}));
se.addSamples(SampledMetric(MetricType.RespT, ts, respt1, node{2}, jobclass{1}));
se.addSamples(SampledMetric(MetricType.RespT, ts, respt2, node{2}, jobclass{2}));
se.addSamples(SampledMetric(MetricType.Util, ts, util, node{2}));
se.interpolate();  % align all samples to common timestamps
estVal = se.estimateAt(node{2});
disp(estVal); % close to [0.1; 0.3]

% Step 5: Plot the empirical trend against the recovered M/G/1 fit
uFit = linspace(min(util)*0.95, max(util)*1.02, 200)';
subplot(1,2,1);
plot(ts, util, '-o'); xlabel('Observation index'); ylabel('Measured utilization');
subplot(1,2,2);
scatter(util, respt1); hold on; scatter(util, respt2);
plot(uFit, estVal(1)./(1-uFit), '--');
plot(uFit, estVal(2)./(1-uFit), '--');
xlabel('Measured utilization'); ylabel('Response time');
legend('Class1 (measured)','Class2 (measured)','Class1 fit','Class2 fit');
```

8.2 Specifying measurement data

Parameter estimation requires observed performance measurements to be provided as `SampledMetric` objects. Each `SampledMetric` encapsulates a single time series and is constructed from the following arguments:

1. A metric type (one of `MetricType.ArvR`, `MetricType.RespT`, `MetricType.Util`, `MetricType.QLen`, or `MetricType.Tput`).
2. A vector of timestamps `ts`, of length n.
3. A vector of observed values `data`, also of length n, where `data[k]` is the measurement taken at time `ts[k]`.

4. The node at which the measurements were taken (e.g., a `Queue` or `Delay` station).

5. Optionally, the job class to which the measurements refer. If omitted, the metric is treated as an aggregate measurement across all classes.

The five metric types are summarized below:

Metric type	Description	Unit / range
`ArvR`	Arrival rate at the node	jobs / time unit
`RespT`	Response time (waiting + service)	time units
`Util`	Utilization of server capacity	$[0, 1]$
`QLen`	Queue length (jobs present at the node)	jobs (non-negative)
`Tput`	Throughput (completions at the node)	jobs / time unit

Timestamps and values must be numeric vectors of the same length. The timestamps should be monotonically increasing and represent the instants at which each measurement was collected. The values represent the observed metric at each timestamp. The following example creates a `SampledMetric` containing 30 per-class arrival rate observations at a queue:

```
ts = (1:30)';                                % 30 timestamps: [1; 2; ...; 30]
data = 2*ones(30,1) + rand(30,1);            % 30 observed arrival rates
sample = SampledMetric(MetricType.ArvR, ts, data, node{2}, jobclass{1});
```

When the job class argument is omitted, the metric is treated as an aggregate measurement across all classes. This is common for utilization data, which monitoring tools typically report at the station level rather than per class:

```
util_sample = SampledMetric(MetricType.Util, ts, util_data, node{2});
```

Different estimation methods require different combinations of metric types (see Table 8.1). For example, UBR requires per-class arrival rates and station utilization, while MCMC requires per-class queue-length samples.

8.2.1 Periodic samples and trace data

By default, a `SampledMetric` represents periodic samples: the values are averages or snapshots collected at regular intervals. Some estimation methods (MLPS, FMLPS, Gibbs) instead require individual job traces, where each entry records a single job event. In a trace, timestamps are irregular (one per event rather than one per interval), and values represent instantaneous observations for that event. To indicate that a `SampledMetric` contains trace data, call `set_trace()`:

```
% arrival_times(k): time of the k-th arrival
% rates(k): instantaneous arrival rate at that time
arvr_trace = SampledMetric(MetricType.ArvR, arrival_times, rates, node{2}, ...
jobclass{1});
arvr_trace.setTrace();
```

8.2.2 Conditional metrics

Some estimation methods require measurements that were recorded conditionally upon a specific event. For example, the ERPS estimator uses queue-length observations collected only at the instants when a job of a given class arrives. This conditional relationship is expressed using the `Event` class:

```
evt = Event(EventType.ARV, node{2}, jobclass{1}); % arrivals of class1
ql = SampledMetric(MetricType.QLen, ts, ql_data, node{2});
ql.setConditional(evt);                          % queue length seen by ...
arriving class1 jobs
```

8.2.3 Configuring and running the estimator

The `ParamEstimator` takes a LINE `Network` model in which the unknown service demands are set to `Exp(NaN)`. After adding all measurement samples, a call to `interpolate()` aligns the time series onto common timestamps (using linear interpolation so that all metrics share the same time grid), and `estimateAt()` runs the estimation:

```
options = ParamEstimator.defaultOptions();
options.method = 'ubo';                    % choose estimation method
se = ParamEstimator(model, options);
se.addSamples(arvr_sample);               % add each SampledMetric
se.addSamples(respt_sample);
```

Table 8.1: Estimation methods for `ParamEstimator`.

Method	Required Data	Description	Refs.
`ubr`	ArvR, Util	Utilization-based non-negative least squares regression	[78]
`ubo`	ArvR, RespT, Util	Utilization-based constrained quadratic optimization	[53]
`erps`	RespT (trace), QLen (cond.)	Extended regression for processor sharing stations	[63]
`ekf`	ArvR, RespT, Util	Sequential estimation via Extended Kalman Filter	[78]
`mle`	ArvR, RespT, Util	Maximum likelihood via bounded optimization with forward solver	[63]
`mcmc`	QLen (aggregate)	Gibbs sampling with product-form queueing network likelihood	[85]
`qmle`	QLen (per-class)	Closed-form quick maximum likelihood estimation	[86]
`mlps`	ArvR (trace), RespT (trace)	Exact CTMC-based maximum likelihood for PS stations	[63]
`fmlps`	ArvR (trace), RespT (trace)	Fluid-limit maximum likelihood for PS stations	[63]
`gibbs`	ArvR (trace), RespT (trace), Tput	Gibbs sampling from individual job trace data	[85]
`rnn`	QLen (per-class, trace)	Physics-informed recurrent neural network	[38]

8.4 Inference examples

The table below lists the inference example scripts available under the `examples/inference/` folder.

Table 8.2: Inference examples

Example	Problem
Utilization-Based Regression (UBR)	
`est_ubr_closed`	UBR estimation in a closed queueing network
`est_ubr_open`	UBR estimation in an open queueing network
`est_ubr_changepoint`	UBR estimation with workload change-point detection
Utilization-Based Optimization (UBO)	
`est_ubo_closed`	UBO estimation in a closed queueing network
`est_ubo_open`	UBO estimation in an open queueing network
`est_ubo_changepoint`	UBO estimation with workload change-point detection
`est_ubo_variants_closed`	Comparison of UBO and MLE estimators in a closed network
Extended Kalman Filter (EKF)	
`est_ekf_closed`	EKF estimation in a closed queueing network
`est_ekf_open`	EKF estimation in an open queueing network
`est_ekf_changepoint`	EKF estimation with workload change-point detection
Maximum Likelihood (MLE)	
`est_mle_open`	MLE estimation in an open queueing network
Markov Chain Monte Carlo (MCMC)	
`est_mcmc_closed`	MCMC estimation in a closed queueing network
Quick Maximum Likelihood (QMLE)	
`est_qmle_closed`	QMLE estimation in a closed queueing network
Extended Regression for PS (ERPS)	
`est_erps_closed`	ERPS estimation in a closed queueing network
`est_erps_open`	ERPS estimation in an open queueing network
Recurrent Neural Network (RNN)	
`est_rnn_closed`	RNN estimation from queue-length traces
Trace-Based Estimation	
`est_trace_closed`	Trace-based estimation (MLPS/FMLPS) in a closed network

8.5 Layered network parameter identification

The methods described so far estimate service demands of flat queueing networks. LINE additionally provides `infer_lqn`, which identifies hidden parameters of a *layered* queueing network—activity host demands and task think times—from measured performance data (response times, utilizations, throughputs). This addresses the inverse problem for `LayeredNetwork` models, using the Extended Kalman Filter (EKF) tracking scheme of Zheng et al. [92].

The unknown parameter vector $\mathbf{a}$ is modelled as a zero-mean random walk $\mathbf{a}_k = \mathbf{a}_{k-1} + \mathbf{w}_k$, and the measurement as $\mathbf{z}_k = \mathbf{h}(\mathbf{a}_k) + \mathbf{v}_k$, where $\mathbf{h}$ is the layered solver (`SolverLN` by default, configurable). At each step the filter forms the prediction error $\mathbf{e}_k = \mathbf{z}_k - \mathbf{h}(\mathbf{a}_{k-1})$ and updates $\mathbf{a}_k = \mathbf{a}_{k-1} + \mathbf{K}_k\mathbf{e}_k$, where the gain $\mathbf{K}_k = \mathbf{P}'_k\mathbf{H}_k^\top(\mathbf{H}_k\mathbf{P}'_k\mathbf{H}_k^\top + \mathbf{R})^{-1}$ uses the sensitivity matrix $\mathbf{H}_k = \partial\mathbf{h}/\partial\mathbf{a}$ obtained by finite differences. The drift covariance $\mathbf{Q}$ and measurement covariance $\mathbf{R}$ are diagonal, tuned by the factors `QFac` and `RFac` respectively.

Each estimated parameter is named by a specification giving its kind (`hostdem` for an activity host demand, `think` for a task think time) and the element name; each observation by a metric (`RespT`, `Util`, `Tput` or `QLen`) and an element name. The measurements are passed as a matrix `Z` of size (number of observations) $\times$ (number of time steps), one column per step. The estimated parameter trajectory is returned in `ahat`, whose last column is the final estimate; the model is updated in place. In the following, `model` is a `LayeredNetwork` whose parameters are set to an initial guess.

```
% hidden parameters: a reference-task think time and an activity host demand
paramSpec = struct('type', {'think','hostdem'}, 'name', {'T1','AS3'});
% observed metrics: a response time and two processor utilizations
obsSpec   = struct('metric', {'RespT','Util','Util'}, 'name', {'E1','P1','P2'});
% Z: an (nobs x nsteps) matrix of measurements, one column per step
opt.QFac = 0.1; opt.RFac = 0.2; opt.gammaT = 1.0;
[model, info] = infer_lqn(model, paramSpec, obsSpec, Z, opt);
disp(info.ahat(:, end));              % final parameter estimate
```

A complete, runnable version that synthesises a measurement sequence with a workload change and recovers the parameter trajectory is provided as `examples/basic/layeredModel/lqn_paramident.m`.

Chapter 9

Optimization

9.1 Overview

Besides evaluating a given model, LINE can solve design problems on queueing networks. While the solvers answer evaluation questions (given a model, what are its performance metrics?), the optimization layer answers design questions: how many servers, what service rates, which routing probabilities, or what job populations optimize a cost or performance objective subject to service-level constraints.

An optimization problem couples a LINE `Network` model with three ingredients:

- *Decision variables*, declaring which model parameters are free to change and over what ranges;
- An *objective*, such as cost minimization or performance maximization;
- *Constraints*, bounding performance metrics such as response times or utilizations.

Each candidate configuration is evaluated by applying the decision variable values to a copy of the model and solving it with `SolverAuto`, which automatically selects an appropriate analysis algorithm (e.g., MVA). The optimization layer never inspects solver internals: all metrics are read from the standard `getAvg` and `getAvgSys` result tables.

The optimization layer is available in the MATLAB, Java and Python codebases with the same class names, options and default values. The differential evolution solver reproduces the same pseudo-random sequence in all three, so a seeded run explores an identical trajectory and returns identical results across languages.

9.2 Requirements

The optimization classes ship with LINE in the `opt` package under `src/opt/` and are on the path after `lineStart`. They are referenced with the package prefix, e.g. `opt.OptimizationProblem`, and require no toolbox beyond the ones already needed by the solvers.

9.3 Examples

In this section, we present some examples that illustrate the optimization features. The relevant scripts are included under the `examples/opt/` and each runs in seconds. The general structure of an optimization script consists of four blocks:

1. Definition of the LINE network model
2. Declaration of the decision variables
3. Specification of the objective and constraints
4. Solution

Cost coefficients are supplied as `containers.Map` objects keyed by station name, and solver options are collected in an `opt.LineOptSolverOptions` object whose setters return the object itself.

9.3.1 Tutorial 1: Sizing an M/M/c queue

The M/M/c queue is a classic model of a service center with c parallel servers fed by a common infinite-capacity buffer. In this example, we wish to find the minimum number of servers such that the utilization does not exceed 50%. Jobs arrive at rate $\lambda = 3$ job/s and each server works at rate $\mu = 1$ job/s, hence the utilization with c servers is $\rho = \lambda/(c\mu) = 3/c$. It is known from theory that the optimum is therefore $c^* = 6$; we wish to recover this value. The following script (`opt_server_sizing.m`) solves the problem with the default differential evolution solver, at a cost of 10 units per server:

```
%% Block 1: LINE model (Source -> Queue -> Sink)
model = Network('MMc');
source = Source(model, 'Arrivals');
queue = Queue(model, 'Server', SchedStrategy.FCFS);
sink = Sink(model, 'Departures');
jobs = OpenClass(model, 'Jobs');
source.setArrival(jobs, Exp(3.0));    % lambda = 3
queue.setService(jobs, Exp(1.0));     % mu = 1 per server
model.link(Network.serialRouting(source, queue, sink));

%% Block 2: decision variables
problem = opt.OptimizationProblem(model);
problem.addVariable(opt.ServerAllocation(queue, [1 10]));

%% Block 3: objective and constraints
serverCost = containers.Map('KeyType','char','ValueType','double');
serverCost('Server') = 10.0;
problem.setObjective(opt.MinimizeCost(serverCost, [], [], {}));
problem.addConstraint(opt.UtilizationConstraint(queue, 0.5));
```

```
%% Block 4: solution
options = opt.LineOptSolverOptions(); options.setSeed(42);
result = problem.solve(options);

fprintf('Optimal servers : %d\n', result.getVariableValue('Server_servers'));
fprintf('Cost            : %.1f\n', result.objectiveValue);
```

Obtaining the following output:

```
Optimal servers : 6
Cost            : 60.0
Feasible        : True
LINE evaluations: 10
```

Note that only 10 distinct configurations required a LINE solve: the evaluator caches results at the level of the decoded configuration, so the hundreds of candidate vectors explored by differential evolution collapse onto the 10 possible server counts.

9.3.2 Tutorial 2: Exact sizing by bisection

The sizing problem of Tutorial 1 has a special structure: feasibility is monotone in the decision variable, since adding servers can only decrease utilization. In such cases `BisectionSolver` finds the exact optimum in $O(\log n)$ LINE evaluations. Model and problem definition are identical to Tutorial 1 (`opt_bisection_sizing.m`); only the solution block changes:

```
result = opt.BisectionSolver(problem).solve();
```

Obtaining the following output:

```
Optimal servers : 6
Cost            : 60.0
LINE evaluations: 4 (vs hundreds for DE)
```

9.3.3 Tutorial 3: Continuous service rate tuning

We now choose the cheapest service rate, a continuous decision, meeting a response time SLA on an M/M/1 queue. With $\lambda = 3$, the response time is $R = 1/(\mu - 3)$, so $R \leq 0.5$ requires $\mu \geq 5$; the rate is billed at 20 cost units per unit, hence the optimum is $\mu^* = 5$ at cost 100. In the script (`opt_service_rate.m`), the model is an M/M/1 queue built as in Tutorial 1 with initial rate $\mu = 4$, and the problem is declared as:

```
problem = opt.OptimizationProblem(model);
problem.addVariable(opt.ServiceRate(queue, jobs, [3.5 8.0]));
rateCost = containers.Map('KeyType','char','ValueType','double');
rateCost('Server') = 20.0;
problem.setObjective(opt.MinimizeCost([], rateCost, [], {}));
```

```
problem.addConstraint(opt.ResponseTimeConstraint(queue, jobs, 0.5));

options = opt.LineOptSolverOptions();
options.setSeed(42); options.setMaxIterations(60);
result = problem.solve(options);
```

Obtaining the following output:

```
Optimal rate    : 5.001 (theory: 5.0)
Cost            : 100.0
Feasible        : True
```

9.3.4 Tutorial 4: Load balancing across heterogeneous servers

An arrival stream of rate $\lambda = 2$ must be split between a fast queue ($\mu_1 = 4$) and a slow queue ($\mu_2 = 2.5$) so that the mean end-to-end response time is minimized. Modeling each queue as M/M/1, the mean number of jobs in the system is $N(p) = \frac{\rho_1}{1-\rho_1} + \frac{\rho_2}{1-\rho_2}$ with $\rho_1 = p\lambda/\mu_1$ and $\rho_2 = (1-p)\lambda/\mu_2$, and by Little's law the system response time is $N(p)/\lambda$. Setting $dN/dp = 0$ yields $p^* \approx 0.743$ with response time 0.4264: the optimal policy does not saturate the fast server. The script (`opt_load_balancing.m`) declares the routing split as the decision variable:

```
problem = opt.OptimizationProblem(model);
problem.addVariable(opt.RoutingProbabilities(jobs, source, {fast, slow}));
problem.setObjective(opt.MinimizeSystemResponseTime(jobs));

options = opt.LineOptSolverOptions();
options.setSeed(42); options.setMaxIterations(60);
result = problem.solve(options);
probs = result.getVariableValue('Jobs_routing_from_S');
```

Obtaining the following output:

```
Fraction to Fast: 0.736 (theory: 0.743)
Fraction to Slow: 0.264
System RT       : 0.4248 (theory: 0.4264)
```

The system response time is computed by Little's law from the chain queue lengths and throughput, which remains correct on parallel topologies (Section 9.5).

9.3.5 Tutorial 5: Population sizing in a closed network

An interactive system is modeled as a closed network with a Delay station (mean think time 1) and a processor sharing server (rate 2). We seek the largest number of concurrent users the system can sustain while the server response time stays within an SLA of 2 time units. Feasibility is monotone non-increasing in the population, so bisection applies with the `max_feasible` direction (`opt_population_sizing.m`):

```
model = Network('Interactive');
think = Delay(model, 'Think');
server = Queue(model, 'AppServer', SchedStrategy.PS);
users = ClosedClass(model, 'Users', 1, think);
think.setService(users, Exp(1.0));
server.setService(users, Exp(2.0));
model.link(Network.serialRouting(think, server));

problem = opt.OptimizationProblem(model);
problem.addVariable(opt.JobPopulation(users, [1 50]));
problem.setObjective(opt.MinimizeCost([], [], [], {}));  % feasibility sizing
problem.addConstraint(opt.ResponseTimeConstraint(server, users, 2.0));

result = opt.BisectionSolver(problem, 'max_feasible').solve();
```

Obtaining the following output:

```
Max users       : 5
Feasible        : True
LINE evaluations: 6
```

9.3.6 Tutorial 6: Robust sizing under workload scenarios

Capacity decisions should often hold under load uncertainty. Here the server pool of Tutorial 1 is sized so that the utilization SLA holds both under the average load ($\lambda = 3$) and under a peak-load scenario ($\lambda = 4.5$). Scenario models share the node and class names of the base model and are registered with `addScenario`; the solver enforces every constraint on all scenarios (`opt_robust_sizing.m`):

```
base = buildMMc(3.0);      % average load
peak = buildMMc(4.5);      % peak load scenario
nodes = base.getNodes(); queue = nodes{2};

problem = opt.OptimizationProblem(base);
problem.addVariable(opt.ServerAllocation(queue, [1 12]));
serverCost = containers.Map('KeyType','char','ValueType','double');
serverCost('Server') = 10.0;
problem.setObjective(opt.MinimizeCost(serverCost, [], [], {}));
problem.addConstraint(opt.UtilizationConstraint(queue, 0.5));
problem.addScenario(peak, 1.0);

result = opt.BisectionSolver(problem).solve();
```

Obtaining the following output:

```
Robust servers  : 9 (6 for average load only)
Cost            : 90.0
```

```
Feasible        : True
```

The average load alone needs 6 servers; the peak scenario pushes the answer to $\lceil 4.5/0.5 \rceil = 9$.

9.3.7 Tutorial 7: Cost-performance tradeoff frontier

How much does tightening the utilization SLA cost? `ParetoSweep` re-solves the sizing problem for a sweep of utilization bounds (epsilon-constraint method) and reports the non-dominated cost frontier, here with each point solved exactly by bisection (`opt_pareto_frontier.m`):

```
factory = @(eps) opt.UtilizationConstraint(queue, eps);
sweep = opt.ParetoSweep(problem, factory, ...
    [0.3 0.4 0.5 0.6 0.75 0.9], 'bisection');
sweep.solve();
frontier = sweep.getFrontier();
for i = 1:numel(frontier)
    p = frontier{i};
    fprintf('  util <= %-4g -> %6.1f  (%d)\n', p.epsilon, p.objectiveValue, ...
        p.result.getVariableValue('Server_servers'));
end
```

Obtaining the following output:

```
Utilization bound -> optimal cost (servers)
  util <= 0.3  ->  100.0  (10)
  util <= 0.4  ->   80.0  (8)
  util <= 0.5  ->   60.0  (6)
  util <= 0.6  ->   50.0  (5)
  util <= 0.75 ->   40.0  (4)
```

The bound 0.9 is absent from the frontier: it requires the same 4 servers as 0.75, so its point is dominated. Frontier points are returned as a cell array of `opt.ParetoPoint` objects carrying the bound (`epsilon`), the optimal objective (`objectiveValue`) and the underlying `opt.OptimizationResult` (`result`), from which a plot can be drawn with the standard MATLAB `stairs` function.

9.3.8 Tutorial 8: Decomposing a multi-variable problem

Jointly choosing the number of servers and the service rate couples two variable types. The decomposition workflow splits them into blocks (servers first, then rates), solves the blocks in sequence with the other block's values held fixed, and cycles until the full penalized objective stabilizes, i.e., block coordinate descent (`opt_decomposition.m`):

```
problem = opt.OptimizationProblem(model);
problem.addVariable(opt.ServerAllocation(queue, [1 10]));
problem.addVariable(opt.ServiceRate(queue, jobs, [1.0 4.0]));
serverCost = containers.Map('KeyType','char','ValueType','double');
```

```
serverCost('Server') = 10.0;
rateCost = containers.Map('KeyType','char','ValueType','double');
rateCost('Server') = 20.0;
problem.setObjective(opt.MinimizeCost(serverCost, rateCost, [], {}));
problem.addConstraint(opt.ResponseTimeConstraint(queue, jobs, 0.5));

workflow = problem.decompose();
workflow.autoDecompose();
options = opt.LineOptSolverOptions(); options.setSeed(42);
workflow.setSolverOptions(options);
result = workflow.solveSequential(4, 1e-3);
```

Obtaining the following output:

```
Converged       : True after 1 cycle(s)
Servers         : 10
Rate            : 2.045
Objective       : 140.90
```

As with any coordinate descent scheme on a non-convex problem, the workflow converges to a block-wise optimum that may differ from the joint optimum; a joint solve on this instance finds cost 76. Decomposition remains preferable when the joint dimensionality is intractable.

9.4 Decision variables

Decision variables encode a model parameter into one or more continuous values in $[0, 1]$, which the solvers explore; each variable decodes candidate values back to its native domain and applies them to a per-evaluation model copy, resolving stations and classes by name.

Variable	**Meaning**
`ServerAllocation`	Integer number of servers of a station
`StationReplicas`	Number of identical station replicas, represented as a pooled multiserver station
`ServiceRate`	Exponential service rate of a class at a station
`RoutingProbabilities`	Outgoing routing probabilities of a class at a node (stick-breaking encoding, probabilities sum to one)
`ClassServiceMapping`	Which station among a candidate set serves a class
`JobPopulation`	Population of a closed class
`ClassPriority`	Priority levels or a priority permutation over classes

Variables are constructed with the base model's objects, e.g. `opt.ServerAllocation(queue, [1 20])` or `opt.ServiceRate(queue, jobclass, [0.5 5.0])`. Integer variables use rounding of the continuous encoding; the solver caches evaluations at the level of the decoded configuration, so candidate vectors that round to the same configuration cost a single LINE solve. Each variable exposes its decoded value in the result under a canonical name, such as `Server_servers`, `Server_Jobs_rate`, `Users_population` or `Jobs_routing_from_S`.

9.5 Objectives and constraints

Three objective families are provided. `MinimizeCost` minimizes a linear cost in the decision variables, with per-server, per-rate and per-replica coefficients, passed as `containers.Map` objects keyed by station name in the order `(serverCost, rateCost, replicaCost, subjectTo)`. `MaximizePerformance` maximizes a weighted combination of throughput and inverse response time, optionally under a budget. `MinimizeSystemResponseTime` minimizes the mean end-to-end response time of a chain, which is the natural objective for load balancing and routing problems. Constraints may be attached to the objective (its last argument) or added to the problem with `addConstraint`; both are enforced identically through a penalty method with weight `penaltyWeight` (default 10^6).

Constraint	Meaning
`ResponseTimeConstraint`	Station response time $\leq$ bound
`SystemResponseTimeConstraint`	End-to-end (chain) response time $\leq$ bound
`ThroughputConstraint`	Throughput $\geq$ bound
`UtilizationConstraint`	Utilization $\leq$ bound
`BudgetConstraint`	Linear cost of the variables $\leq$ budget

Station-level constraints reference stations and classes by object or name. `SystemResponseTimeConstraint` and `MinimizeSystemResponseTime` use the mean end-to-end response time of the chain; for open chains this is computed by Little's law (jobs in system divided by system throughput), which coincides with the chain sojourn time on serial paths and remains correct on parallel topologies.

9.6 Solvers

9.6.1 Differential evolution

The default solver, `LineOptSolver`, implements differential evolution in `opt.de.DifferentialEvolution`, a self-contained implementation that reproduces the `scipy` trajectory bit for bit given the same seed. Options are passed to `problem.solve(...)` in an `opt.LineOptSolverOptions` object:

Option	Default	Meaning
`strategy`	`'best1bin'`	DE mutation strategy
`popsize`	`15`	Population size multiplier
`maxIterations`	`100`	Maximum DE generations
`tol`	`0.01`	Convergence tolerance
`timeLimit`	`300.0`	Wall-clock limit in seconds, enforced per evaluation; the best solution found so far is returned
`seed`	`[]`	Random seed for reproducibility
`penaltyWeight`	`1e6`	Constraint violation penalty
`scenarioAggregation`	`'worst'`	Objective aggregation across scenarios
`verbose`	`false`	Progress reporting

Each option has a setter of the form `setMaxIterations`, `setSeed`, and so on; setters return the options object, and `setMutation(low, high)` sets the dithering range.

9.6.2 Analytic gradients for product-form networks

The optimizer option selects the backend: 'evolution' (the default, differential evolution), 'gradient' (projected gradient descent with a deterministic multistart controlled by gradientRestarts, default 4), or 'auto', which takes the gradient path only when every decision variable is continuous, that is of type service rate or routing. The gradient path minimizes the same penalized scalar objective as the evolutionary path, so constraints need no separate handling.

The Jacobian first tries an analytic assembly and otherwise falls back to central finite differences in the encoded space with step fdStep (default 10^{-6}), reverting to a one-sided difference at a bound where the two-sided value is not finite. The analytic path costs no extra LINE solves: it composes three exact pieces, namely the derivative of each performance measure with respect to the service rates, the derivative of the penalized scalar with respect to each metric (obtained by arithmetic-only finite differences in metric space, with no solving), and the derivative of the rate with respect to the encoded variable, which is linear in the decode.

The performance derivatives come from opt.sens.computeModelSensitivities, which is invoked once per model evaluation and attached to the EvaluationResult as its sensitivities field. It maps each metric kind ('RespT', 'QLen', 'Tput', 'Util') and metric key (a station-class pair, a station name alone for 'Util') to the derivative with respect to each parameter key, the rate of a class at a station. Two model families are covered:

- Open product-form networks of single-server queueing stations and infinite-server delay stations, which decouple. The derivatives follow in closed form from $R_r = D_r/(1 - U)$ and $Q_r = \lambda_r/(1 - U)$ by the quotient rule, and the routine declines when a station is unstable or multiserver.
- Closed single-server product-form networks with unit visit ratios, through the exact differentiated MVA primitive pfqn_sens, which yields the full cross-station Jacobian since closed networks couple the stations. Utilization derivatives are aggregated per station to match the utilizations reported by the evaluation result.

Mixed, multiserver, and non-unit-visit topologies are declined, as is any failure of the sensitivity computation, in which case the optimizer silently uses finite differences. The analytic assembly is additionally declined, and finite differences used instead, when the problem carries more than one workload scenario or when any decision variable is not a service-rate variable.

9.6.3 Exact sizing by bisection

For the common sizing pattern (a single integer variable whose feasibility is monotone, such as adding servers to meet an SLA), BisectionSolver finds the exact optimum in $O(\log n)$ LINE solves instead of hundreds of DE evaluations:

```
result = opt.BisectionSolver(problem).solve();      % smallest feasible value
result = opt.BisectionSolver(problem, 'max_feasible').solve();
```

be solved in LINE using at least `JMT`. A feature with "Partial" support implies that some core aspects of this feature available in JSIM are not available in LINE.

A few notes are needed to clarify the entries with partial support:

- `Fork` and `Join` are supported with their default policies. Advanced policies, such as partial joins or setting a distribution for the forked tasks on each output link, are not supported yet.
- a single `Sink` and a single `Source` can be instantiated in a LINE model, whereas there is no such constraint in JMT.
- `Finite Capacity Region` is supported on export (LINE to JMT), but the `JMT2LINE` import does not yet parse JSIM blocking regions.

Table 10.1: Supported JSIM features for automated model import and analysis

JMT Feature	**Support**	**Notes**
Distributions	Full	Phase-Type, Burst (MAP), Burst (MMPP2), Deterministic, Disabled, Exponential, Erlang, Gamma, Hyperexponential, Coxian, Lognormal, Pareto, Uniform, Zero Service Time, Replayer, Trace, Weibull
Classes	Full	Open class, Closed class, Class priorities
Metrics	Full	Number of customers, Residence Time, Throughput, Response Time, Throughput per sink, Utilization, Arrival Rate
Nodes	Full	ClassSwitch, Place, Delay, Queue, Router, Transition
Routing	Full	Random, Probabilities, Round Robin, Weighted Round Robin, Power of k, Join the Shortest Queue
Mechanisms	Full	Polling, Setup times, Delay-off times, Switchover times, Impatience, Load Dependence, Retrial, Soft deadlines, Heterogeneous servers, Server Compatibilities
Scheduling	Full	FCFS, FCFSPRIO, LCFS, LCFS-PR, SIRO (Random), SJF, SEPT, LJF, LEPT, PS, DPS, GPS, PS Priority, DPS Priority, GPS Priority, LPS, EDD, EDF, SRPT, FSP
Nodes	Partial	Fork, Join, Source, Sink, Finite Capacity Region
Distributions	No	Burst (General), Normal
Nodes	No	Scaler, Semaphore
Routing	No	Shortest Response Time, Least Utilization, Fastest Service, Load Dependent, Class Switch Routing
Metrics	No	Drop rate, Response time per sink, Power
Scheduling	No	TBS, QBPS
Mechanisms	No	Parallelism

10.2 JSON model format

LINE defines a portable JSON format for saving and loading queueing network models. The format is supported across the MATLAB, Java and Python-native codebases and enables model interchange without loss of information. A formal JSON Schema specification is available at `doc/line-model.schema.json` in the LINE distribution.

The reference for this format is the implementation, that is the four bridges

- `matlab/src/io/linemodel_save.m`
- `matlab/src/io/linemodel_load.m`
- `python/line_solver/io/linemodel_io.py`
- `jar/src/main/java/jline/io/LineModelIO.java`

Where a reader accepts more than one spelling of a key, the spelling emitted by the writers is described below as *canonical* and the remaining ones as *legacy aliases*.

10.2.1 API

Models are saved and loaded using each codebase's JSON I/O functions:

```
% Save
linemodel_save(model, 'mymodel.json');
% Load
model = linemodel_load('mymodel.json');
```

10.2.2 Top-level structure

Every LINE JSON file has the following envelope:

```
{
  "format": "line-model",
  "version": "1.0",
  "model": { ... }
}
```

The `model` object must contain a `type` field set to one of `Network`, `LayeredNetwork`, `Workflow`, or `Environment`. Any other value is rejected by the loaders.

Models held in a foreign format (`.lqnx`, `.jmva`, `.jsimg`) are not referenced from within a LINE JSON file; they are passed directly to the tool that reads them, which dispatches on the file extension. See Section 10.3 for the `line-cli` behaviour.

10.2.3 Network models

A `Network` model describes an extended queueing network. Its fields are:

Table 10.2: Network model fields

Field	Type	Description
`type`	`"Network"`	Discriminator of the model kind. Required.
`name`	`string`	Model name. Node and class names must be unique within it. Required.
`nodes`	`array of Node`	Node definitions, in the order that fixes the node indices used by the routing matrix. Required.

Field	Type	Description
classes	array of JobClass	Job class definitions, in the order that fixes the class indices. Required.
routing	RoutingMatrix	Routing probability matrix, given per class pair over the nodes. Required.
routingStrategies	{key: {key: RoutingStrategy}}	Per-node, per-class routing strategy: {nodeName: {className: strategy}}. Written at model level; the readers also accept it nested inside the 'routing' object and prefer the nested copy. Only strategies other than PROB and RAND are emitted, since the routing matrix already captures those.
routingWeights	{key: {key: {key: number}}}	Per-node, per-class, per-destination routing weights for WRROBIN: {nodeName: {className: {destNode: weight}}}.
routingParams	{key: {key: SQParams or RLParams}}	Per-node, per-class parameters of the parameterized routing strategies: {nodeName: {className: params}}. Absent params make a SQ node fall back to d=2, and an RL node degrade to the JSQ fallback; both cases warn.
finiteCapacityRegions	array of FiniteCapacityRegion	Finite capacity regions constraining groups of stations jointly.
rewards	array of Reward	Reward definitions evaluated on the state space by the CTMC solver.

The routingStrategies, routingWeights and routingParams objects are written at model level. The readers also accept them nested inside the routing object, and prefer the nested copy when both are present.

Nodes

Each node is an object with at least name and type fields. Table 10.3 lists the node types emitted by the writers. Additional fields depend on the node type.

Table 10.3: Node types

Type	Description
Source	Generates arrivals for open classes
Sink	Absorbs departing jobs
Queue	Queueing station with one or more servers
Delay	Infinite-server station
Fork	AND-fork for parallel branching
Join	AND-join for synchronization; references a paired Fork via forkNode
Router	Probabilistic routing node (no queueing)
Cache	Cache node with hit/miss class transformation
ClassSwitch	Deterministic class-switching node
Place	Place node for stochastic Petri nets
Transition	Transition node for stochastic Petri nets
Logger	Logging/monitoring node

A node of any other class is serialized under its own class name, obtained by reflection; such a node is not

restored by the readers.

Table 10.4 lists the fields available on station nodes (`Queue`, `Delay`, `Source`, `Place`).

Table 10.4: Station node fields

Field	Type	Description
`name`	`string`	Node name, unique within the model. Routing and per-class maps key on it. Required.
`type`	`Source, Sink, Queue, Delay, Fork, Join, ...`	Node type. A node of any other class is serialized under its own class name, obtained by reflection, and is not restored by the readers. Required.
`scheduling`	`SchedStrategy`	Scheduling strategy of the station. Omitted for nodes that do not serve jobs.
`servers`	`integer`	Number of servers (default 1). There is no infinite sentinel: use a Delay node for an infinite-server station.
`buffer`	`integer`	Total buffer capacity; omit for infinite.
`classCap`	`{key: integer}`	Per-class buffer capacity: {className: capacity}.
`dropRule`	`{key: DropRule}`	Per-class drop rule: {className: rule}.
`service`	`{key: Distribution}`	Per-class service/arrival distribution: {className: Distribution}.
`markedClasses`	`array of string`	Source only: marked (MMAP) arrival binding; class names ordered by mark index, so mark k emits jobs of the k-th listed class. All listed classes share one MMAP arrival entry in 'service'.
`schedParams`	`{key: number}`	Per-class scheduling parameters (DPS weights, GPS shares, priorities): {className: value}.
`serverTypes`	`array of ServerType`	Heterogeneous server types for this node.
`heteroSchedPolicy`	`ORDER, ALIS, ALFS, FAIRNESS, FSF, RAIS`	Heterogeneous server scheduling policy.
`switchoverTimes`	`array of object`	Class-to-class switchover times: [{from, to, distribution}]. A per-class (rather than class-pair) switchover omits 'to'.
`pollingType`	`GATED, EXHAUSTIVE, KLIMITED`	Polling strategy for POLLING scheduling. Crosses as the uppercase enum name.
`pollingPar`	`integer`	K limit for KLIMITED polling.
`loadDependence`	`LoadDependence`	Load-dependent scaling factor f(n) at this station.
`classDependence`	`ClassDependence`	Class-dependent scaling beta_{i,r}(n) at this station. A sibling of loadDependence, not one of its modes.
`jointDependence`	`JointDependence`	Joint-dependent (non-product-form) scaling eta_i(n) at this station. A sibling of classDependence; the handle may read the joint per-class vector arbitrarily, so the demands are not product form.
`oiServiceRate`	`{key: {key: number}}`	Order-independent per-state service rate table: {className: {state: rate}}.
`oiCutoffs`	`array of integer`	Per-class cutoffs of the oiServiceRate table.
`swapGraph`	`array of array of number`	Pass-and-swap graph, read only for a station scheduled PAS.
`departureDiscipline`	`{key: N, FIFO}`	Place only: per-class departure discipline {className: rule}.
`balking`	`{key: object}`	Per-class balking configuration. Keys are class names.

Field	Type	Description
retrial	{key: object}	Per-class retrial configuration. Keys are class names.
patience	{key: object}	Per-class patience (reneging) configuration. Keys are class names. A node-scoped patience overrides the class-scoped one declared on JobClass.
orbitImpatience	{key: Distribution}	Per-class orbit impatience distribution: {className: Distribution}.
immediateFeedback	{key: boolean}	Per-class immediate feedback flag: {className: bool}.
setupTime	{key: Distribution}	Per-class server setup time: {className: Distribution}.
delayOffTime	{key: Distribution}	Per-class server delay-off time: {className: Distribution}.
initialState	number or array of number	Initial state vector. Read for Place nodes, closed pass-and-swap (PAS) queues, and Cache nodes.
stateSpace	array of array of number	Explicit state space of a stateful node. Emitted together with statePrior.
statePrior	array of number	Prior probability over the rows of stateSpace.

Table 10.5 lists the fields of the remaining node types.

Table 10.5: Fields of non-station nodes

Field	Type	Description
classSwitchMatrix	{key: {key: number}}	For ClassSwitch nodes: explicit switching probability matrix. {fromClass: {toClass: probability}}.
csMatrix	array of array of number	For ClassSwitch nodes: legacy 2D array form of classSwitchMatrix, indexed by class order.
tasksPerLink	integer	For Fork nodes: number of tasks generated per outgoing link.
forkNode	string	For Join nodes: name of the paired Fork node.
joinStrategy	STD, PARTIAL, Quorum, QUORUM, Guard	For Join nodes: join strategy. The writers emit PARTIAL for a quorum join; the readers accept QUORUM as an alias of Quorum. An unrecognized value falls back to STD.
joinQuorum	integer	Quorum count for a quorum join.
modes	array of TransitionMode	For Transition (SPN) nodes: firing modes.

Cache node fields are given in Section 10.2.3.

Table 10.6: Scheduling strategies

Strategy	Description
INF	Infinite servers (Delay)
FCFS	First Come First Served
FCFSPR	FCFS with preemptive resume
FCFSPI	FCFS with preemptive restart
LCFS	Last Come First Served
LCFSPR	LCFS with preemptive resume
LCFSPI	LCFS with preemptive restart

Strategy	Description
SIRO	Service In Random Order
SJF	Shortest Job First
LJF	Longest Job First
PS	Processor Sharing
DPS	Discriminatory Processor Sharing (uses schedParams for weights)
GPS	Generalized Processor Sharing
SEPT	Shortest Expected Processing Time
LEPT	Longest Expected Processing Time
SRPT	Shortest Remaining Processing Time
SRPTPRIO	SRPT with priorities
HOL	Head-Of-Line priority
FCFSPRIO	FCFS with priorities
FORK	Fork node pseudo-strategy
EXT	External world pseudo-strategy
REF	Reference station pseudo-strategy
POLLING	Polling discipline (uses pollingType, pollingPar)
PSPRIO	PS with priorities
DPSPRIO	DPS with priorities
GPSPRIO	GPS with priorities
LCFSPRIO	LCFS with priorities
LCFSPRPRIO	LCFS preemptive resume with priorities
LCFSPIPRIO	LCFS preemptive restart with priorities
FCFSPRPRIO	FCFS preemptive resume with priorities
FCFSPIPRIO	FCFS preemptive restart with priorities
EDD	Earliest Due Date
EDF	Earliest Deadline First
LPS	Limited Processor Sharing
PSJF	Preemptive Shortest Job First
FB	Foreground-Background
LRPT	Longest Remaining Processing Time
SETF	Shortest Elapsed Time First
PAS	Pass and swap; the swap relation travels in swapGraph

Strategy names cross as uppercase enum names.

Cache nodes

Cache configuration is written as flat node-level keys. The readers accept both these keys and a nested cache object using the alternative names given in the last column of Table 10.7, and prefer the nested object when both are present.

Table 10.7: Cache node fields

Field	Type	Description
numItems	integer	Cache nodes: total number of cacheable items. Also readable as cache.items.
itemLevelCap	integer or array of integer	Cache nodes: capacity per list level. Also readable as cache.capacity.
replacementStrategy	ReplacementStrategy	Cache nodes: replacement policy. Also readable as cache.replacement.

Field	Type	Description
admissionProb	number	Cache nodes: probability of admitting a missed item.
hitClass	{key: string}	Cache nodes: hit class mapping {inputClassName: outputClassName}.
missClass	{key: string}	Cache nodes: miss class mapping {inputClassName: outputClassName}.
popularity	{key: Distribution}	Cache nodes: per-class item popularity distributions {className: Distribution}.
accessGraph	array of array of number	Cache nodes: item access graph.
accessProb	array	Cache nodes: explicit per-class item access probabilities.
cache	CacheConfig	Nested alternative to the flat cache keys above. The writers emit the flat keys; the readers accept both and prefer this nested object when present.

Table 10.8: Cache replacement policies

Policy	Description
RR	Random replacement
FIFO	First-in first-out
SFIFO	Strict first-in first-out
LRU	Least recently used
HLRU	h-LRU
CLIMB	CLIMB
QLRU	q-LRU

Load and class dependence

The loadDependence object specifies scaling of the service rate in the total station population n:

```
"loadDependence": { "type": "loadDependent", "scaling": [1.0, 1.8, 2.4] }
```

where scaling holds $f(n)$ for $n = 1, 2, \ldots$

The classDependence object is a sibling key of loadDependence, not one of its modes. It specifies the class-dependent scaling $\beta_{i,r}(\boldsymbol{n})$ in the per-class population vector $\boldsymbol{n}$ at the station. The underlying object is a function handle, which cannot be serialized, so the writer materializes it over the per-class box lattice $0 \leq n_r \leq c_r$, where c_r is the r-th entry of cutoffs:

Table 10.9: Class dependence fields

Field	Type	Description
type	classDependent	Shape of the class-dependent rate scaling.
cutoffs	array of integer	Per-class cutoffs, ordered by class index. A closed class is bounded by its population; an open class uses a saturation cutoff beyond which beta is constant.

Field	Type	Description
scaling	{key: array of number}	Lattice table keyed by the comma-joined 0-based per-class counts ("n1,n2,...,nR"). Each value is the length-R vector [beta_1(n), ..., beta_R(n)]; a scalar-valued handle is broadcast to R equal entries by the writer.
peak	array of number	Declared peak (maximum) rate scaling per class [p_1,...,p_R], used to normalize utilization as U_r = T_r*S_r/p_r (the same T*S/c convention as multiserver stations). A scalar peak is broadcast to R entries by the writer. When absent (legacy models) the reader derives it as the lattice peak of scaling.

A reader rebuilds a handle that clamps the population to the cutoffs, so β saturates beyond the tabulated range. When `peak` is absent (legacy models) the reader derives it as the lattice peak of the scaling.

The `jointDependence` object is the non-product-form twin of `classDependence`, storing the joint-dependent scaling $\eta_i(\boldsymbol{n})$ (see §3.1.6). It is materialized over the same per-class box lattice and rebuilt with clamping; unlike `classDependence`, the handle may read foreign class marginals, so the demands are not product form.

Table 10.10: Joint dependence fields

Field	Type	Description
type	jointDependent	Shape of the joint-dependent rate scaling.
cutoffs	array of integer	Per-class cutoffs, ordered by class index. A closed class is bounded by its population; an open class uses a saturation cutoff beyond which eta is constant.
scaling	{key: array of number}	Lattice table keyed by the comma-joined 0-based per-class counts ("n1,n2,...,nR"). Each value is the length-R vector [eta_1(n), ..., eta_R(n)]; a scalar-valued handle is broadcast to R equal entries by the writer.
peak	array of number	Declared peak (maximum) rate scaling per class [p_1,...,p_R], used to normalize utilization as U_r = T_r*S_r/p_r (the same T*S/c convention as multiserver stations). A scalar peak is broadcast to R entries by the writer. When absent the reader derives it as the lattice peak of scaling.

Impatience, balking and retrials

Three node-level objects, each keyed by class name, describe customer impatience:

```
"balking":  { "Class1": { "strategy": "QUEUE_LENGTH",
                          "thresholds": [ { "minJobs": 5, "maxJobs": -1,
                                            "probability": 0.5 } ] } },
"retrial":  { "Class1": { "delay": { "type": "Exp", "params": { "lambda": 2.0 } },
                          "maxAttempts": -1 } },
"patience": { "Class1": { "distribution": { "type": "Exp",
                                            "params": { "lambda": 0.1 } },
```

```
                                "impatienceType": "reneging" } }
```

The balking `strategy` is one of `QUEUE_LENGTH`, `EXPECTED_WAIT`, or `COMBINED`; a `maxJobs` of -1 denotes infinity. The `maxAttempts` field is -1 when the number of retrials is unlimited. The `impatienceType` is one of `reneging`, `balking`, or `retrial`.

A patience distribution may also be attached to a job class rather than to a station; see Section 10.2.3. A node-scoped patience overrides the class-scoped one.

Petri net nodes

A `Place` node is a station: it carries `scheduling`, `servers`, `service`, `departureDiscipline` and `initialState`. A `Transition` node carries a `modes` array:

Table 10.11: Transition mode fields

Field	Type	Description
`name`	`string`	Mode name, unique within its transition. Required.
`distribution`	`Distribution`	Firing time distribution.
`timingStrategy`	`TIMED, IMMEDIATE`	Whether the mode fires after a delay or immediately.
`numServers`	`integer or "Infinity"`	Number of firing servers; the string "Infinity" denotes an infinite-server mode.
`firingPriority`	`integer`	Integer priority breaking ties among simultaneously enabled modes; higher fires first.
`firingWeight`	`number`	Weight randomizing the choice among enabled modes of equal highest priority.
`enablingConditions`	`array of Arc`	Token multiplicity required on each input place for the mode to be enabled.
`inhibitingConditions`	`array of Arc`	Token multiplicity on each input place above which the mode is inhibited.
`firingOutcomes`	`array of Arc`	Tokens deposited on each output place when the mode fires.

Every arc object has the same three fields: `node` (name of the `Place`), `class` (name of the job class), and `count` (number of tokens).

```
{ "name": "T1", "type": "Transition",
  "modes": [
    { "name": "mode1", "timingStrategy": "TIMED", "numServers": 1,
      "distribution": { "type": "Exp", "params": { "lambda": 1.0 } },
      "enablingConditions": [ { "node": "P1", "class": "Class1", "count": 1 } ],
      "firingOutcomes":     [ { "node": "P2", "class": "Class1", "count": 1 } ] }
  ] }
```

Job classes

Each class is an object with at least `name` and `type` fields.

Table 10.12: Job class fields

Field	Type	Description
name	string	Class name, unique within the model. Per-class maps key on it. Required.
type	Open, Closed, SelfLooping, Signal	Class type. SelfLooping specializes Closed, so the writers test for it first. Required.
population	integer	For Closed and SelfLooping classes: number of jobs.
refNode	string	Reference node name (Source for Open, home station for Closed/SelfLooping/ClosedSignal).
priority	integer	Class priority (0=highest).
deadline	number	Class deadline (Inf = no deadline).
isReferenceClass	boolean	True if the class is the reference class of its chain.
patience	Distribution	Class-scoped patience distribution. A node-scoped patience overrides it.
impatienceType	ImpatienceType	Qualifies the class-scoped patience.
replySignalClass	string	Name of the class carrying the reply of a reply signal. Without it, REPLY signals are inert after a round trip.
signalType	negative, reply, catastrophe	For Signal classes: signal type.
openOrClosed	Open, Closed	For Signal classes: whether the signal is open or closed.
targetClass	string	For Signal classes: name of the associated job class.
removalDistribution	Distribution	For Signal classes: distribution for batch removal count.
removalPolicy	random, fcfs, lcfs	For Signal classes: policy for selecting jobs to remove.

Since `SelfLooping` classes specialize `Closed` ones, the writers test for them first; a reader that ignores the `SelfLooping` value would silently obtain an ordinary closed class.

Routing

Routing is specified by a probability matrix. The `routing` object has its `type` field set to `matrix` and its `matrix` object uses composite keys of the form `"fromClass,toClass"`, each mapping to a nested dictionary `{fromNode: {toNode: probability}}`.

```
"routing": {
  "type": "matrix",
  "matrix": {
    "Class1,Class1": {
      "Source": { "Queue1": 0.5, "Queue2": 0.5 },
      "Queue1": { "Sink": 1.0 },
      "Queue2": { "Sink": 1.0 }
    },
    "Class1,Class2": {
      "Queue1": { "Queue2": 0.3 }
    }
  }
}
```

Only non-zero probabilities need to be specified. Same-class routing (e.g., `"Class1,Class1"`) describes routing without class switching; cross-class entries (e.g., `"Class1,Class2"`) describe class-switching transitions.

Routing strategies other than `PROB` and `RAND`, which the matrix already captures, are carried by the model-level `routingStrategies` object:

```
"routingStrategies": { "Router1": { "Class1": "RROBIN" } }
```

Table 10.13: Routing strategies

Strategy	**Description**
`RAND`	Random selection with equal probability
`PROB`	Probabilistic routing, given by the routing matrix
`RROBIN`	Round-robin over the destinations
`WRROBIN`	Weighted round-robin; uses `routingWeights`
`JSQ`	Join the shortest queue
`FIRING`	Firing routing, used by Petri net transitions
`SQ`	Shortest queue of d, SQ(d); uses `routingParams`
`RL`	Reinforcement-learning routing; uses `routingParams`
`DISABLED`	Class may not depart the node

`DISABLED` is emitted explicitly because the routing matrix records only positive probabilities and therefore cannot express it.

Weights for `WRROBIN` are given per node, class and destination:

```
"routingWeights": { "Router1": { "Class1": { "Queue1": 2.0, "Queue2": 1.0 } } }
```

The `routingParams` object mirrors that shape and carries the parameters of the two parameterized strategies. For `SQ` the entry is `{"d": int}`; for `RL` it is:

Table 10.14: RL routing parameters

Field	**Type**	**Description**
`valueFunction`	`array of number`	Flattened value function. Required.
`vfShape`	`array of integer`	Shape of the value function array.
`stateSize`	`integer`	Size of the state descriptor.
`actionNodes`	`array of string`	Names of the nodes on which the policy acts.

Omitting `routingParams` for a `SQ` node makes the reader warn and fall back to $d = 2$; omitting it for an `RL` node makes the routing degrade to the `JSQ` fallback.

Finite capacity regions

A finite capacity region groups stations under a shared capacity constraint.

Table 10.15: Finite capacity region fields

Field	Type	Description
name	string	Region name, unique within the model. Required.
stations	array of object	Names of the stations the region spans; the constraint applies to their joint population. Required.
globalMaxJobs	integer	Maximum total jobs in the region.
globalMaxMemory	integer	Maximum total memory in the region.
classMaxJobs	{key: integer}	Per-class maximum jobs: {className: maxJobs}.
classMaxMemory	{key: integer}	Per-class maximum memory: {className: maxMemory}.
constraintA	array of array of number	Linear constraint matrix A of A*x <= b.
constraintB	array of number	Linear constraint capacity vector b of A*x <= b.
dropRule	{key: DropRule}	Per-class drop rule: {className: rule}.

Rewards

The `rewards` array carries reward definitions for CTMC analysis in a declarative form:

```
"rewards": [ { "name": "q1", "type": "QLen", "node": "Queue1", "class": "Class1" } ]
```

The `type` field is one of `QLen`, `Util`, or `Blocking`, matching the `Reward.queueLength`, `Reward.utilization` and `Reward.blocking` templates. The `class` field is optional; when omitted the reward aggregates over all classes.

Only a reward created through a `Reward.*` template carries the structural metadata needed to reproduce it. A reward defined from a bare function handle, or through `Reward.custom`, is not reproducible from JSON: the writer warns and omits it rather than emit a reward that would be wrong on reload.

10.2.4 Distributions

Distributions are used throughout the format for service times, arrival processes, think times, and host demands. A distribution object always contains a `type` field and one of several parameterization modes.

Direct parameterization

The `params` object provides distribution-specific parameters.

Table 10.16: Distribution types and parameters

Type	Parameters	Description
Exp	lambda	Exponential of rate λ. `rate` is accepted as a read-side alias
Det	value	Deterministic
Erlang	lambda, k	Erlang (per-phase rate λ, k phases)
HyperExp	p: [ ], lambda: [ ]	Hyperexponential (weights and rates, n phases)

Type	Parameters	Description
`Coxian`	`mu: [ ], phi: [ ]`	Coxian (phase rates μ and completion probabilities ϕ)
`Gamma`	`alpha, beta`	Gamma (`alpha` shape, `beta` scale)
`Lognormal`	`mu, sigma`	Lognormal
`Normal`	`mu, sigma`	Normal
`Uniform`	`a, b`	Uniform over $[a, b]$
`Pareto`	`alpha, scale`	Pareto (`alpha` shape, `scale` scale)
`Weibull`	`alpha, beta`	Weibull. Note the roles: `alpha` is the *scale* and `beta` the *shape*
`Zipf`	`s, n`	Zipf (exponent, items)
`Geometric`	`p`	Geometric
`Binomial`	`p, n`	Binomial
`Poisson`	`lambda`	Poisson
`Bernoulli`	`p`	Bernoulli
`Discrete Uniform`	`min, max`	Discrete uniform
`Discrete Sampler`	`p: [ ], x: [ ]`	Empirical discrete (probs and values)
`EmpiricalCDF`	`x: [ ], F: [ ]`	Empirical CDF (support and cdf values)
`NHPP`	`breakpoints: [ ], rates: [ ], cyclic`	Non-homogeneous Poisson process, piecewise-constant intensity (cyclic when `cyclic` is true)
`MMPP2`	`lambda0, lambda1, sigma0, sigma1`	Two-state MMPP
`MMDP2`	`r0, r1, sigma0, sigma1`	Two-state MMDP: deterministic rates in the two states, and the two switching rates
`ME`	`alpha: [ ], A: [[ ]]`	Matrix-exponential
`RAP`	`H0: [[ ]], H1: [[ ]]`	Rational arrival process
`DMAP`	`D0: [[ ]], D1: [[ ]]`	Discrete MAP
`BMAP`	`D: [[[ ]]]`	Batch MAP; `D[0]`$= \boldsymbol{D}_0$, `D[1]`$= \boldsymbol{D}_1, \ldots$
`MarkedMMPP`	`D: [[[ ]]], K`	Marked MMPP (M3PP) with K marking types
`Replayer`	`fileName, mean`	Trace replay; see Section 10.2.4
`Immediate`	`(none)`	Zero service time
`Disabled`	`(none)`	No service (class disabled at this node)

Distributions carrying no `params` block, namely `PH`, `APH`, `MAP`, `MMAP` and `Expolynomial`, are described in Sections 10.2.4 and 10.2.4.

Parameterization of `MMDP2`. The two-state Markov-modulated deterministic process is parameterized by deterministic *rates* in every codebase: `r0` and `r1` are the rates in the two states, and `sigma0`, `sigma1` the switching rates. The stationary mean rate is $(r_0\sigma_1 + r_1\sigma_0)/(\sigma_0 + \sigma_1)$ and the reported mean is its reciprocal. The Python-native class formerly took inter-arrival *times* `d0`, `d1`; that spelling is no longer written or read.

Fit specification

Instead of direct parameters, a distribution can be constructed by fitting from summary statistics via the `fit` object. This is the preferred mode when the user specifies a distribution by its moments rather than its native parameters.

Table 10.17: Fit methods

Field	Type	Description
method	fitMean, fitMeanAndSCV, fitMeanAndOrder, fitCentral, fitRawMoments	Fitting method: which moments the parameters are derived from. Required.
mean	number	Target mean, matched exactly by every method.
scv	number	Squared coefficient of variation.
order	integer	Number of phases/stages.
moments	array of number	The three moments for fitCentral (central) or fitRawMoments (raw). Must have length 3. Alternatively give the named fields: mean/var/skew for fitCentral, m1/m2/m3 for fitRawMoments. Both methods resolve to Cox2 when type is Coxian or Cox2, and to APH otherwise.
var	number	Variance, for fitCentral given by named fields.
skew	number	Skewness, for fitCentral given by named fields.
m1	number	First raw moment, for fitRawMoments given by named fields.
m2	number	Second raw moment, for fitRawMoments given by named fields.
m3	number	Third raw moment, for fitRawMoments given by named fields.

The moments array must have length 3 for fitCentral and fitRawMoments; a shorter array is rejected. Both methods resolve to Cox2 when the type is Coxian or Cox2, and to APH otherwise.
Example:

```
{ "type": "Erlang", "fit": { "method": "fitMeanAndOrder", "mean": 2.0, "order": 3 } }
```

Phase-type and MAP representations

Phase-type distributions (PH, APH, Coxian, Cox2) can be specified via a ph object containing:

- alpha: initial probability row vector α
- T: sub-generator matrix T (negative diagonal, non-negative off-diagonal)

Example:

```
{ "type": "PH", "ph": { "alpha": [1.0, 0.0], "T": [[-2.0, 2.0], [0.0, -1.0]] } }
```

Markovian Arrival Processes are specified via a map object containing D0, the background transition matrix D_0, and D1, the arrival transition matrix D_1.

Marked MAPs (MMAP) instead use an mmap object containing D0 and D1k, an array holding one arrival matrix per mark. The aggregate $D_1 = \sum_k D_{1k}$ is rebuilt on load. A 1×1 matrix may be encoded as a bare scalar.

Special distributions

- `Replayer`: replays inter-arrival times from a trace file, given by `params.fileName`. The writer also emits `params.mean` and, when it can fit one, a `ph` block; the reader uses the file when it resolves and otherwise falls back to the fitted phase-type. A trace built from in-memory samples lives in a generated temporary file whose path does not survive a round trip, and the writer warns in that case.
- `Expolynomial`: specified via an `expolynomial` object with `density` (a density expression in Sirio format, such as `"0.5 * x^2 * Exp[-0.3 x]"`), `eft` (earliest firing time) and `lft` (latest firing time, with the string `"Inf"` allowed for an unbounded support).
- `Prior`: a weighted mixture of alternative distributions for posterior or uncertainty analysis, specified by `distributions` (an array of distribution objects) and `probabilities` (an array of weights).
- `Workflow`: an activity graph that is compiled to a phase-type distribution, specified by a `workflow` object holding `activities` (each with a `name` and a nested `distribution`) and `precedences` between them. The compiled phase-type law is what the solvers see; the graph is retained so that a round trip does not lose the structure.

A distribution for which no branch matches is written as its own type name carrying `params.mean` and `params.scv`, with a warning. On reading, an unrecognized `type` carrying a mean and an SCV is reconstructed as an `APH` matching those two moments, again with a warning.

10.2.5 LayeredNetwork models

A `LayeredNetwork` model describes a layered queueing network (LQN). It is organized hierarchically into hosts (processors), tasks, entries, and activities.

Table 10.18: LayeredNetwork model fields

Field	**Type**	**Description**
`type`	`"LayeredNetwork"`	Discriminator of the model kind. Required.
`name`	`string`	Model name. Required.
`hosts`	`array of LHost`	Host (processor) definitions. Canonical key; the MATLAB and Python readers accept 'processors' as a legacy alias, the Java reader does not. Required.
`processors`	`array of LHost`	Legacy alias of 'hosts'. Read by MATLAB and Python only.
`tasks`	`array of LTask`	Task definitions; each names the host it runs on. Required.
`entries`	`array of LEntry`	Entry definitions; each names the task that offers it. Required.
`activities`	`array of LActivity`	Activity definitions; each names the task it belongs to. Required.
`precedences`	`array of LPrecedence`	Precedence relations over the activities of a task.

The canonical key for the host array is `hosts`; the MATLAB and Python readers accept `processors` as a legacy alias, the Java reader does not.

Hosts

Table 10.19: Host fields

Field	Type	Description
name	string	Host (processor) name, unique within the model. Required.
multiplicity	integer	Number of processors (default 1).
scheduling	FCFS, PS, INF, HOL, RAND	Host scheduling (default INF).
quantum	number	Scheduling quantum (time slice) for PS.
speedFactor	number	Processor speed factor (default 1.0).
replication	integer	Number of replicas.

Tasks

Table 10.20: Task fields

Field	Type	Description
name	string	Task name, unique within the model. Required.
host	string	Name of the host. Canonical key; the MATLAB and Python readers accept 'processor' as a legacy alias.
processor	string	Legacy alias of 'host'. Read by MATLAB and Python only.
multiplicity	integer	Number of task threads/copies (default 1).
scheduling	REF, FCFS, INF, HOL, POLL	Task scheduling strategy.
priority	integer	Task priority at its host, 0 being the highest.
thinkTime	Distribution	Think time distribution (for REF tasks). Alternative to the thinkTimeMean/thinkTimeSCV pair.
thinkTimeMean	number	Mean think time (for REF tasks).
thinkTimeSCV	number	Think time squared coefficient of variation.
replication	integer	Number of replicas.
fanIn	object	Fan-in from the named task.
fanOut	object	Fan-out to the named task.
taskType	FunctionTask, CacheTask	Specialized task kind.
setupTime	Distribution	Setup time distribution (FunctionTask). Alternative to the setupTimeMean/setupTimeSCV pair.
setupTimeMean	number	Mean setup time paid when the task resumes from an idle period.
setupTimeSCV	number	Squared coefficient of variation of the setup time.
delayOffTime	Distribution	Delay-off time distribution (FunctionTask). Alternative to the delayOffTimeMean/delayOffTimeSCV pair.

Field	Type	Description
delayOffTimeMean	number	Mean idle time the task waits before powering off.
delayOffTimeSCV	number	Squared coefficient of variation of the delay-off time.
totalItems	integer	Number of cacheable items (CacheTask).
cacheCapacity	integer or array of integer	Cache capacity per list level (CacheTask).
replacementStrategy	ReplacementStrategy	Replacement policy (CacheTask).

A think time may also be given as a `thinkTime` distribution object instead of the `thinkTimeMean`/`thinkTimeSCV` pair; likewise `setupTime` and `delayOffTime` accept distribution objects.

Entries

Table 10.21: Entry fields

Field	Type	Description
name	string	Entry name, unique within the model. Required.
task	string	Name of parent task. Required.
arrival	Distribution	Open arrival distribution for this entry.
forwarding	array of object	Forwarding to other entries.
entryType	ItemEntry	Specialized entry kind.
totalItems	integer	Number of items (ItemEntry).
accessProb	Distribution	Item popularity distribution (ItemEntry).

Activities

Table 10.22: Activity fields

Field	Type	Description
name	string	Activity name, unique within its task. Required.
task	string	Name of parent task. Required.
hostDemand	Distribution	Host demand distribution.
boundToEntry	string	Entry name this activity is bound to. Canonical key; the MATLAB and Python readers accept 'boundTo' as a legacy alias.
boundTo	string	Legacy alias of 'boundToEntry'. Read by MATLAB and Python only.
repliesTo	string	Entry name this activity replies to.
callOrder	string	Call order for this activity's calls.
synchCalls	array of LCall	Synchronous (blocking) calls to entries.
asynchCalls	array of LCall	Asynchronous (non-blocking) calls to entries.

Each call object has an `entry` field (target entry name) and an optional `mean` field (mean number of calls, default 1).

Activity precedences

Precedence relations define control flow within a task. Two schemas are accepted. The first is the *typed* form, written by the MATLAB and Python bridges, in which the pre/post split is implied by the type: a fork names its predecessor first, a join names its successor last.

Table 10.23: Typed precedence fields

Field	Type	Description
task	string	Parent task name.
type	Serial, AndFork, AndJoin, OrFork, OrJoin, Loop, ...	Kind of precedence relation between the pre and post activity sets. Required.
activities	array of string	Ordered list of activity names.
probabilities	array of number	For OrFork: branch probabilities. Defaults to uniform.
loopCount	number	For Loop: mean number of iterations.
preActivity	string	For Loop: the loop trigger activity; the loop body is then carried whole in 'activities'.

The second is the *explicit* form, written by the Java bridge, which carries the precedence types verbatim and so needs no shape inference. A precedence object carrying `preActs` or `postActs` is read in this form.

Table 10.24: Explicit precedence fields

Field	Type	Description
task	string	Parent task name. Required.
preActs	array of string	Predecessor activity names.
postActs	array of string	Successor activity names.
preType	pre, pre-AND, pre-OR	Pre-condition type (default 'pre').
postType	post, post-AND, post-OR, post-LOOP, post-CACHE	Post-condition type (default 'post').
preParams	array of number	Parameters of the pre-condition, such as an AND-join quorum. Defaults to all predecessors required.
postParams	array of number	Parameters of the post-condition, such as OR-fork probabilities or a loop count.

10.2.6 Workflow models

A `Workflow` model describes an activity graph that compiles to a phase-type distribution. Its `type` field is set to `"Workflow"` and it carries a `name`, an `activities` array of {`name`, `distribution`} objects, and a `precedences` array using the typed precedence schema of Table 10.23.

A workflow may equally appear as a distribution, through a `workflow` object with the same `activities` and `precedences` fields.

10.2.7 Environment models

An `Environment` model describes a queueing network operating in a random environment. Each environment stage is associated with a `Network` sub-model.

Table 10.25: Environment model fields

Field	Type	Description
`type`	`"Environment"`	Discriminator of the model kind. Required.
`name`	`string`	Model name. Required.
`numStages`	`integer`	Number of environment stages.
`stages`	`array of EnvStage`	Environment stages, in the order that fixes the 0-based stage indices used by the transitions. Required.
`transitions`	`array of object`	Transitions between stages. 'from' and 'to' are 0-based stage indexes, not names. There is no scalar rate field: the stage holding time is a full Distribution object.

A transition object holds `from` and `to` as 0-based *stage indexes*, not stage names, and a `distribution` object (Section 10.2.4) giving the stage holding time. There is no scalar rate field: an exponential holding time of rate λ is written `{"type": "Exp", "params": {"lambda": λ}}`.

Environment state not carried by the format

The reset policy of an environment transition (Section 7.1.3) and its state-dependent rate rule are function-valued and are not serialized. The consequences are described in Section 10.2.9.

10.2.8 Examples

M/M/1 queue

```
{
  "format": "line-model",
  "version": "1.0",
  "model": {
    "type": "Network",
    "name": "M/M/1",
    "nodes": [
      { "name": "Source", "type": "Source",
        "service": {
          "Class1": { "type": "Exp",
                      "fit": { "method": "fitMean", "mean": 1.0 } } } },
      { "name": "Queue", "type": "Queue", "scheduling": "FCFS",
        "service": {
          "Class1": { "type": "Exp",
                      "fit": { "method": "fitMean", "mean": 0.5 } } } },
      { "name": "Sink", "type": "Sink" }
    ],
    "classes": [
      { "name": "Class1", "type": "Open" }
```

```
      ],
      "routing": {
        "type": "matrix",
        "matrix": {
          "Class1,Class1": {
            "Source": { "Queue": 1.0 },
            "Queue": { "Sink": 1.0 }
          }
        }
      }
    }
  }
```

Two-task layered queueing network

```
  {
    "format": "line-model",
    "version": "1.0",
    "model": {
      "type": "LayeredNetwork",
      "name": "lqn_twotasks",
      "hosts": [
        { "name": "P1", "scheduling": "PS" },
        { "name": "P2", "scheduling": "PS" }
      ],
      "tasks": [
        { "name": "T1", "host": "P1", "multiplicity": 100,
          "scheduling": "REF",
          "thinkTime": { "type": "Erlang",
            "fit": { "method": "fitMeanAndOrder",
                     "mean": 10, "order": 1 } } },
        { "name": "T2", "host": "P2", "multiplicity": 1,
          "scheduling": "INF" }
      ],
      "entries": [
        { "name": "E1", "task": "T1" },
        { "name": "E2", "task": "T2" }
      ],
      "activities": [
        { "name": "A1", "task": "T1",
          "hostDemand": { "type": "Exp", "params": { "lambda": 1.0 } },
          "boundToEntry": "E1",
          "synchCalls": [ { "entry": "E2" } ] },
        { "name": "A2", "task": "T2",
          "hostDemand": { "type": "Exp", "params": { "lambda": 1.0 } },
          "boundToEntry": "E2", "repliesTo": "E2" }
      ]
    }
  }
```

Random environment

The model below alternates between an `Up` stage and a `Down` stage in which the queue serves at half its nominal rate. Each stage carries a complete `Network` sub-model, and the two stages are connected by exponential transitions.

```
{
  "format": "line-model",
  "version": "1.0",
  "model": {
    "type": "Environment",
    "name": "server_breakdown",
    "numStages": 2,
    "stages": [
      { "name": "Up",
        "model": {
          "type": "Network", "name": "up",
          "nodes": [
            { "name": "Delay", "type": "Delay",
              "service": { "Class1": { "type": "Exp",
                "params": { "lambda": 0.2 } } } },
            { "name": "Queue1", "type": "Queue", "scheduling": "FCFS",
              "service": { "Class1": { "type": "Exp",
                "params": { "lambda": 1.0 } } } }
          ],
          "classes": [
            { "name": "Class1", "type": "Closed",
              "population": 3, "refNode": "Delay" }
          ],
          "routing": {
            "type": "matrix",
            "matrix": {
              "Class1,Class1": {
                "Delay": { "Queue1": 1.0 },
                "Queue1": { "Delay": 1.0 }
              }
            }
          }
        }
      },
      { "name": "Down",
        "model": {
          "type": "Network", "name": "down",
          "nodes": [
            { "name": "Delay", "type": "Delay",
              "service": { "Class1": { "type": "Exp",
                "params": { "lambda": 0.2 } } } },
            { "name": "Queue1", "type": "Queue", "scheduling": "FCFS",
              "service": { "Class1": { "type": "Exp",
                "params": { "lambda": 0.5 } } } }
          ],
          "classes": [
            { "name": "Class1", "type": "Closed",
              "population": 3, "refNode": "Delay" }
          ],
```

```
            "routing": {
              "type": "matrix",
              "matrix": {
                "Class1,Class1": {
                  "Delay": { "Queue1": 1.0 },
                  "Queue1": { "Delay": 1.0 }
                }
              }
            }
          }
        }
      ],
      "transitions": [
        { "from": 0, "to": 1,
          "distribution": { "type": "Exp", "params": { "lambda": 0.1 } } },
        { "from": 1, "to": 0,
          "distribution": { "type": "Exp", "params": { "lambda": 1.0 } } }
      ]
    }
  }
```

10.2.9 State not carried by the format

Some model state is function-valued and therefore has no JSON representation. A round trip through the format silently drops it, and the reloaded model reverts to the corresponding default. Users must reapply such state programmatically after loading.

Table 10.26: Model state not carried by the JSON format

State	Consequence of a round trip
`Environment reset policy` (Section 7.1.3)	Reverts to the identity rule, so jobs keep their positions across a stage transition instead of being redistributed
`Environment state-dependent rate rule` (Section 7.1.3)	Reverts to the identity rule. `ENV` then no longer selects `options.method='statedep'`, so the reloaded model is solved as a plain random environment
`State-dependent routing functions`	Reverts to the routing matrix
`Custom rewards` (Section 10.2.3)	Omitted with a warning at save time

The reset policy is a per-transition function, supplied as the fourth argument of `addTransition`, and may differ on each transition. It is not a model-level enumeration and there is no field in the JSON format that selects between clearing and keeping the network state.

10.2.10 Serialization conventions

The following conventions apply to the JSON serialization:

- Matrices are stored as nested JSON arrays, e.g., `[[1,2],[3,4]]` for a 2×2 matrix. A 1×1 matrix may be encoded as a bare scalar.
- Enumerations cross as names, never as ordinal numbers, since the numeric values of the enumerations differ across codebases.
- `NaN` values are represented as JSON `null`.
- Infinity values are clamped to $\pm 10^{308}$, except where a key documents the string `"Infinity"`.
- All identifiers (node names, class names) are strings and must be unique within their scope.
- ClassSwitch nodes that are auto-generated by `link()` are excluded from serialization.
- An absent key means the corresponding feature is not configured.
- Optional fields may be omitted; the loader applies default values as documented in the schema.

10.3 Command-line interface (line-cli)

The LINE command-line interface (`line-cli`) provides a standalone tool for solving queueing network models directly from the terminal. This interface is useful for batch processing, integration with external tools, scripting, and environments where interactive use of MATLAB, Python, or Java is not desirable.

10.3.1 Installation

The CLI tool requires:

- Java 8+: The Java Runtime Environment (JRE) version 8 or later.
- Python 3.11+: Python 3.11 or later.
- jline.jar: The LINE solver JAR file, typically located in the `common/` directory.

The CLI is included in the standard LINE distribution. After extracting the release archive, verify the installation by running:

```
python line-cli.py info
```

If the JAR file is not automatically detected, set the `LINE_JAR_PATH` environment variable:

```
export LINE_JAR_PATH=/path/to/jline.jar
```

10.3.2 Basic Usage

The CLI supports several commands for solving models and querying information.

Solving a Model. To solve a queueing network model, use the `solve` command with the optional `-s` (or `--solver`) option to specify the solver algorithm (when omitted, the solver defaults to `auto`):

```
python line-cli.py solve model.jsimg -s mva
```

If you are unsure of which solver to use, the `auto` solver provides an automatic choice. To list all available solvers:

```
python line-cli.py list solvers
```

The following table summarizes the solvers presently interfaced with the cli tool:

Solver	Name	Formats	Notes
`auto`	Automatic Selection	All	Selects best solver for model
`ctmc`	CTMC	jsim/jsimg/jsimw	Exact, small models only
`des`	LDES (LINE Discrete Event Simulator)	jsim/jsimg/jsimw	Simulation-based
`fld`	Fluid ODE	jsim/jsimg/jsimw	Fast, approximate
`jmt`	Java Modelling Tools	jsim/jsimg/jsimw	External simulation
`ln`	Layered Network	lqnx/xml	For LQN models
`lqns`	LQN Solver	lqnx/xml	External LQNS tool
`mam`	Matrix Analytic	jsim/jsimg/jsimw	Supports Fork-Join percentiles
`mva`	Mean Value Analysis	jsim/jsimg/jsimw	Fast, general purpose
`nc`	Normalizing Constant	jsim/jsimg/jsimw	Exact analysis
`qns`	QNS	jsim/jsimg/jsimw	External QNSolver integration
`ssa`	Stochastic Simulation	jsim/jsimg/jsimw	State-space sampling

The input format is automatically detected from the file extension. Supported formats include:

- `.jsimg`, `.jsim`, `.jsimw`: JMT simulation model formats
- `.lqnx`, `.xml`: Layered Queueing Network Solver XML format

Models can also be piped via stdin when the input format is specified:

```
cat model.jsimg | python line-cli.py solve -i jsimg -s mva
```

The CLI also supports multiple output formats via the `-o` or `--output-format` option:

```
python line-cli.py solve model.jsimg -s mva -o table   # Human-readable table
python line-cli.py solve model.jsimg -s mva -o json    # JSON format
python line-cli.py solve model.jsimg -s mva -o csv     # Comma-separated values
python line-cli.py solve model.jsimg -s mva -o raw     # Raw solver output
```

If unspecified, the table output format is used by default.

To save results to a file, further add the `-O` option:

```
python line-cli.py solve model.jsimg -s mva -o json -O results.json
```

Analysis Types. The `-a` or `--analysis` option specifies which metrics to compute:

```
python line-cli.py solve model.jsimg -s mva -a all   # Average and system ...
metrics (default)
python line-cli.py solve model.jsimg -s mva -a avg   # Average metrics only
python line-cli.py solve model.jsimg -s mva -a sys   # System metrics only
```

Advanced analysis types include:

- `stage`, `chain`, `node`, `nodechain`: stage-, chain-, node-, and node-chain-level averages
- `cdf-respt`, `cdf-passt`: Response/passage time CDFs
- `perct-respt`: Response time percentiles (MAM solver)
- `tran-avg`, `tran-cdf-respt`, `tran-cdf-passt`: transient averages and transient CDFs
- `prob`, `prob-aggr`, `prob-marg`: State probabilities (CTMC/SSA)
- `prob-sys`, `prob-sys-aggr`: System state probabilities
- `sample`, `sample-aggr`, `sample-sys`, `sample-sys-aggr`: State trajectory sampling (SSA)
- `reward`, `reward-steady`, `reward-value`: Reward metrics (CTMC)

To list all analysis types:

```
python line-cli.py list analysis
```

10.3.3 Server Modes

The CLI can run as a server to accept solve requests over the network.

WebSocket Server. Start a WebSocket server for real-time communication:

```
python line-cli.py server -p 5863
```

This starts a WebSocket server on port 5863 (default). Clients can connect to `ws://localhost:5863` to submit models and receive results.

REST API Server. Start a REST API server for HTTP-based integration:

```
python line-cli.py rest -p 8080
```

Available endpoints:

- `POST /solve`: Submit a model for solving
- `GET /health`: Health check endpoint

10.3.4 Advanced Options

Stochastic Solver Options. For stochastic solvers (`des`, `ssa`, `jmt`), a random seed can be specified for reproducibility:

10.3.5 Command Reference

Command/Option	Description
`solve <file>`	Solve a queueing model
`info`	Display system information
`list solvers`	List available solvers
`list formats`	List supported formats
`list analysis`	List analysis types
`server`	Start WebSocket server
`rest`	Start REST API server
`-s, -solver`	Solver algorithm
`-i, -input-format`	Input format (auto-detected)
`-o, -output-format`	Output format (table/json/csv/raw)
`-a, -analysis`	Analysis type(s)
`-d, -seed`	Random seed
`--events`	Maximum simulation events (`des`, `ssa`)
`-n, -node`	Node index (for prob/sample)
`-c, -class-idx`	Class index (for prob-marg)
`-v, -verbose`	Verbose output
`-q, -quiet`	Suppress status messages
`-O, -output-file`	Write results to file
`-p, -port`	Server port
`-H, -host`	Server host address

10.3.6 Examples

Sample Output. Running the CLI on a sample open queueing network model:

```
python line-cli.py solve example.jsimg -s mva
```

produces the following output:

```
Solving example.jsimg...

Average Metrics
===============
Station   Class    Metric  Value    Unit
----------------------------------------
Source 1  Class A  QLen    0
Source 1  Class A  Util    0
Source 1  Class A  RespT   0
Source 1  Class A  Tput    2
Source 1  Class B  QLen    0
Source 1  Class B  Tput    1
Queue 1   Class A  QLen    1.33318
Queue 1   Class A  Util    0.4
Queue 1   Class A  RespT   0.66659
Queue 1   Class A  Tput    2
Queue 1   Class B  QLen    0.99989
Queue 1   Class B  Util    0.3
Queue 1   Class B  RespT   0.99989
Queue 1   Class B  Tput    1
Queue 2   Class C  QLen    0.81818
Queue 2   Class C  Util    0.45
Queue 2   Class C  RespT   0.27273
```

```
Queue 2   Class C  Tput    3

System Metrics
==============
Station  Class                    Metric    Value    Unit
------------------------------------------------------------
Chain1   Class A Class B Class C  SysRespT  0.77769
Chain1   Class A Class B Class C  SysTput   3

Execution time: 0.252s
```

Bibliography

[1] M. Ajmone Marsan, G. Conte, and G. Balbo. A class of generalized stochastic petri nets for the performance evaluation of multiprocessor systems. *ACM Trans. Comput. Syst.*, 2(2):93–122, May 1984.

[2] D. F. Anderson. A modified next reaction method for simulating chemical systems with time dependent propensities and delays. *The Journal of chemical physics*, 127(21), 2007.

[3] S. Asmussen and M. Bladt. Point processes with finite-dimensional conditional probabilities. *Stochastic Processes and their Applications*, 82(1):127–142, 1999.

[4] S. Balsamo. Product form queueing networks. In Günter Haring, Christoph Lindemann, and Martin Reiser, editors, *Performance Evaluation: Origins and Directions*, volume 1769 of *Lecture Notes in Computer Science*, pages 377–401. Springer, 2000.

[5] M. Bertoli, G. Casale, and G. Serazzi. The JMT simulator for performance evaluation of non-product-form queueing networks. In *Proc. of the 40th Annual Simulation Symposium (ANSS)*, pages 3–10, 2007.

[6] A. L. Bertozzi and J. McKenna. Multidimensional residues, generating functions, and their application to queueing networks. *SIAM Review*, 35(2):239–268, 1993.

[7] D. Bini, B. Meini, S. Steffé, J. F. Pérez, and B. Van Houdt. Smcsolver and q-mam: tools for matrix-analytic methods. *SIGMETRICS Performance Evaluation Review*, 39(4):46, 2012.

[8] Mogens Bladt and Bo Friis Nielsen. *Matrix-Exponential Distributions in Applied Probability*. Springer, New York, 2017.

[9] G. Bolch, S. Greiner, H. de Meer, and K. S. Trivedi. *Queueing Networks and Markov Chains*. Wiley, 2006.

[10] A. B. Bondi and W. Whitt. The influence of service-time variability in a closed network of queues. *Perform. Eval.*, 6:219–234, 1986.

[11] S. C. Bruell, G. Balbo, and P. V. Afshari. Mean value analysis of mixed, multiple class BCMP networks with load dependent service stations. *Performance Evaluation*, 4:241–260, 1984.

[12] G. Casale. CoMoM: Efficient class-oriented evaluation of multiclass performance models. *IEEE Trans. Software Engineering*, 35(2):162–177, 2009.

[13] G. Casale. Accelerating performance inference over closed systems by asymptotic methods. In *Proc. of ACM SIGMETRICS*. ACM Press, 2017.

[14] G. Casale. Integrated Performance Evaluation of Extended Queueing Network Models with Line. In *2020 Winter Simulation Conference (WSC)*, pages 2377–2388. IEEE, dec 2020.

[15] G. Casale, V. De Nitto Personé, and E. Smirni. QRF: An optimization-based framework for evaluating complex stochastic networks. *ACM Transactions on Modeling and Computer Simulation*, 26(3):15:1–15:24, 2016.

[16] G. Casale and P. G. Harrison. AutoCAT: Automated product-form solution of stochastic models. In *Matrix-Analytic Methods in Stochastic Models*, volume 27 of *Springer Proceedings in Mathematics & Statistics*, pages 57–85. Springer, 2013.

[17] G. Casale, P.G. Harrison, and O.W. Hong. Facilitating load-dependent queueing analysis through factorization. *Perform. Eval.*, 2021.

[18] G. Casale, Richard R. Muntz, and Giuseppe Serazzi. Geometric bounds: A noniterative analysis technique for closed queueing networks. *IEEE Trans. Computers*, 57(6):780–794, 2008.

[19] G. Casale, J. F. Pérez, and W. Wang. QD-AMVA: Evaluating systems with queue-dependent service requirements. In *Proceedings of IFIP PERFORMANCE*, 2015.

[20] G. Casale and M. Tribastone. Fluid analysis of queueing in two-stage random environments. In *Proceedings of QEST 2011*, 2011.

[21] G. Casale, M. Tribastone, and P. G. Harrison. Blending randomness in closed queueing network models. *Perform. Eval.*, 82:15–38, 2014.

[22] G. Casale, E. Z. Zhang, and E. Smirni. KPC-Toolbox: Best recipes for automatic trace fitting using Markovian arrival processes. *Performance Evaluation*, 67(9):873–896, 2010.

[23] K. M. Chandy and D. Neuse. Linearizer: A heuristic algorithm for queuing network models of computing systems. *Commun. ACM*, 25(2):126–134, 1982.

[24] G. L. Choudhury, K. K. Leung, and W. Whitt. Calculating normalization constants of closed queuing networks by numerically inverting their generating functions. *J. ACM*, 42(5):935–970, 1995.

[25] W.-M. Chow. Approximations for large scale closed queueing networks. *Perform. Eval*, 3(1):1–12, 1983.

[26] A. Chydzinski. Per-flow throughput of a FIFO buffer. *Applied System Innovation*, 9(5):112, 2026.

[27] C. Comte and J.-P. Dorsman. Pass-and-swap queues. *Queueing Systems*, 98(3):275–331, 2021.

[28] A. E. Conway. Fast Approximate Solution of Queueing Networks with Multi-Server Chain-Dependent FCFS Queues, pages 385–396. Springer US, Boston, MA, 1989.

[29] A. E. Conway and N. D. Georganas. RECAL - A new efficient algorithm for the exact analysis of multiple-chain closed queueing networks. *J. ACM*, 33(4):768–791, 1986.

[30] P. J. Courtois. *Decomposability: queueing and computer system applications*. Academic Press, New York, 1977.

[31] E. de Souza e Silva and R. R. Muntz. A note on the computational cost of the linearizer algorithm for queueing networks. *IEEE Trans. Computers*, 39(6):840–842, 1990.

[32] R.-A. Dobre, Z. Niu, and G. Casale. Approximating fork-join systems via mixed model transformations. In *Companion of the 15th ACM/SPEC International Conference on Performance Engineering*, ICPE '24 Companion, page 273–280, New York, NY, USA, 2024. Association for Computing Machinery.

[33] D. L. Eager and J. N. Lipscomb. The AMVA priority approximation. *Perform. Eval.*, 8(3):173–193, 1988.

[34] G. Franks. *Performance Analysis of Distributed Server Systems*. PhD thesis, Carleton, 1996.

[35] G. Franks, T. Al-Omari, M. Woodside, O. Das, and S. Derisavi. Enhanced modeling and solution of layered queueing networks. *IEEE Trans. Software Engineering*, 35(2):148–161, 2009.

[36] G. Franks, P. Maly, C. M. Woodside, D. C. Petriu, A. Hubbard, and M. Mroz. *Layered Queueing Network Solver and Simulator User Manual*, 2012.

[37] G. Franks, P. Maly, M. Woodside, D. C. Petriu, Alex Hubbard, and Martin Mroz. *Layered Queueing Network Solver and Simulator User Manual*. Carleton University, January 2013.

[38] Giulio Garbi, Emilio Incerto, and Mirco Tribastone. Learning queuing networks by recurrent neural networks. In *Proc. of ACM/SPEC ICPE*, pages 56–66. ACM, 2020.

[39] N. Gast and B. Van Houdt. Transient and steady-state regime of a family of list-based cache replacement algorithms. *Queueing Syst.*, 83(3-4):293–328, 2016.

[40] D. T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *J. Phys. Chem.*, 81(25):2340–2361, 1977.

[41] A. Harel, S. Namn, and J. Sturm. Simple bounds for closed queueing networks. *Queueing Systems*, 31(1-2):125–135, 1999.

[42] P. Heidelberger and K. Trivedi. Queueing network models for parallel processing with asynchronous tasks. *IEEE Trans. Computers*, 100(11):1099–1109, 1982.

RQNA (Robust Queueing Network Analyzer), *see* Network solvers

SJF (Shortest Job First), *see* Network models
SQ (Shortest Queue of d), *see* Network models
SSA (Stochastic Simulation Algorithms), *see* Network solvers
State-dependent routing, *see* Network models

Utilization (Util), 11

Zipf distribution, *see* Network models